Databases and PostgreSQL From the Ground Up

Relations, 8 KB Slotted Pages, B-Link Trees, Write-Ahead Logging, MVCC, and Production Query Execution

An Entry Path into Real Database Internals

November 2026

Preface

Every backend engineer eventually meets the day when the database stops pretending to be an infinite hash map in the sky.

Perhaps a query that ran in four milliseconds on Friday suddenly takes nineteen seconds on Monday morning, even though nobody deployed code over the weekend. Perhaps two concurrent requests check that a customer has $100 available, both pass the check inside separate `BEGIN ... COMMIT` blocks, both deduct $80, and the account settles at −$60. Or perhaps someone adds a harmless-looking `WHERE id NOT IN (SELECT blocked_id FROM blocks)` filter, a single row in `blocks` happens to have `blocked_id = NULL`, and the entire query returns zero rows without raising a warning.

When those moments arrive, guessing at index combinations or copying configuration snippets from blog posts feels terrible. Underneath every surprise sits a deterministic machine built out of concrete data structures: 8,192-byte slotted pages, line-pointer arrays, B-link trees with right-sibling pointers, append-only Write-Ahead Log segments, and 32-bit transaction IDs compared on a modular circle. Once you can picture those bytes and trace the invariants that govern them, PostgreSQL stops feeling mysterious. Every behavior has a physical cause you can inspect, predict, and measure.

I wrote this book because most treatments of databases give you only half the picture. Academic textbooks teach relational algebra, functional dependencies, and serializability graphs cleanly, yet rarely show how a tuple header is padded to an 8-byte boundary or why a 4 KB operating-system write tearing an 8 KB database page forces full-page logging after a checkpoint. Operational manuals list parameters like `shared_buffers`, `work_mem`, and `autovacuum_vacuum_scale_factor`, yet seldom derive from first principles what breaks when you set them wrong. Here we keep the mathematical model, the byte-level PostgreSQL implementation, and production engineering on the very same page.

What this book is, and who it is for

You know Java 6 (classes, `null`, `synchronized`, `ArrayList`, `HashMap`, the garbage collector) and basic C (structs, pointers, `malloc`, `write` and `fsync`). You may have written SQL through JDBC or Hibernate, or you may be meeting SQL seriously for the first time. We assume no prior knowledge of database internals. Each new mechanism is first tied to the Java or C construct it resembles — an object graph, a `TreeMap`, a C struct with padding, a `FileOutputStream` that has not been synced — and then we show exactly where PostgreSQL differs and why.

By the end of these eight chapters, you will be able to reason like a systems engineer who genuinely understands PostgreSQL from the query parser down to NVMe flash:

- read a complex SQL query in the order the engine logically evaluates it (`FROM` → `JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `WINDOW` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`), and spot three-valued `NULL` traps before they ship;
- inspect a raw 8 KB PostgreSQL heap file in your head or with `pageinspect`, read `PageHeaderData`, trace the `ItemIdData` array growing downward and heap tuples growing upward, and compute exact row sizes including `MAXALIGN` padding;
- design composite, covering (`INCLUDE`), partial, GIN, GiST, and BRIN indexes from their physical tree geometry rather than superstition, and explain how Lehman–Yao high keys let readers traverse splitting B-tree pages without holding parent locks;
- trace a `COMMIT` through `shared_buffers`, `pg_wal`, and `fsync`, state the Write-Ahead Logging rule precisely, and replay crash recovery by hand from a checkpoint's redo point;
- determine exactly which tuple versions a transaction snapshot ($x_{\min}, x_{\max}, \text{xip}$) can see, why `UPDATE` creates dead tuples, when Heap-Only Tuple (HOT) chains avoid index bloat, and why 32-bit transaction ID wraparound requires freezing;
- construct two-transaction timelines for lost updates and write skew, choose deliberately between `SELECT ... FOR UPDATE`, `REPEATABLE READ`, and `SERIALIZABLE` (SSI), and read every counter in an `EXPLAIN (ANALYZE, BUFFERS)` plan;
- design schemas for high-integrity financial ledgers — pairing naturally with Rust, `Axum`, and `SQLx` — backed by streaming replication and transaction-mode connection pooling.

How to read this book

Every major mechanism in this book follows the same four-step rhythm: a concrete problem, a small paper-and-pencil trace, the exact PostgreSQL data structure, and a hands-on inspection query.

We always start with *the failure or bottleneck that forced the design.* Why not store pointers from a customer record directly to their order records on disk? Why not overwrite an updated integer in place inside its 8 KB page? Why not flush dirty table pages straight to disk at `COMMIT`? When you feel the constraint first, the invention that resolves it reads like common sense rather than arbitrary complexity.

Next we work through *tiny numbers by hand.* Before stating general formulas, we pack three rows into a miniature slotted page and watch byte offsets move when the middle row is deleted and compacted. We split a B-tree leaf node holding four keys while a concurrent reader is halfway down the tree. We trace crash recovery across six Log Sequence Numbers (`0/1000` through `0/1260`) and check every page header's `pd_lsn`. Doing the arithmetic on six items gives you complete command over sixty million.

Then we look directly at *PostgreSQL's real C structures and extensions.* PostgreSQL is open-source C written with unusual clarity, and its built-in extensions (`pageinspect`, `pg_walinspect`, `pg_buffercache`, `pg_stat_statements`) let you inspect live pages, line pointers, hint bits, and WAL records from an ordinary `psql` prompt. Whenever we introduce an internal structure, we name the real fields (`pd_lower`, `pd_upper`, `t_xmin`, `t_xmax`, `t_infomask`) so that PostgreSQL's source tree and documentation feel immediately familiar.

What the recurring callouts mean

Throughout each chapter, shaded callout blocks signal shifts in perspective so you always know what kind of thinking a passage asks of you:

Intuition

The geometric or physical mental picture — the one clear visual model that makes all the details snap into place. If you revisit a chapter months later, reading these boxes will reconstruct the whole story in minutes.

Formal

Exact definitions, invariants, and correctness rules — relational algebra operators, the Write-Ahead Logging invariant, or the MVCC snapshot visibility predicate — stated without hand-waving.

Why it works

The engineering and mathematical proof for *why* a mechanism preserves correctness under concurrency or power loss, or why an alternative design fails.

Common trap

Subtle mistakes that bite production systems every week: `NOT IN` against a nullable column, leading range columns in composite B-trees, `TIMESTAMP WITHOUT TIME ZONE`, or `REPEATABLE READ` silently permitting write skew.

Under the hood

Direct looks inside PostgreSQL's C structs (`PageHeaderData`, `HeapTupleHeaderData`, `BufferDesc`) or compact reference code in Rust, C, and Python showing how the mechanism operates at the byte level.

Try it in PostgreSQL

Self-contained SQL sessions you can paste straight into a local PostgreSQL 16+ container to watch raw pages, indexes, locks, and execution plans with your own eyes:

```
-- Start an ephemeral container with:
-- docker run --rm -it -e POSTGRES_PASSWORD=postgres postgres:17 psql -U postgres
SELECT version();
```

One end-to-end trace, so you see how the layers connect

Before we begin Chapter 1, let us trace one tiny SQL statement all the way from text to silicon. Watching all the layers cooperate on a single row gives you a map of the entire book.

Suppose a ledger table `accounts` holds rows with `(id BIGINT, balance_cents BIGINT)`, and account `7` currently sits on heap block `0` at slot `1` with a balance of `25000` cents ($250.00). A backend session executes:

```
BEGIN;
UPDATE accounts
   SET balance_cents = balance_cents - 5000
 WHERE id = 7;
COMMIT;
```

A naive mental model imagines PostgreSQL seeking to byte offset 8160 of the table file, overwriting `25000` with `20000`, and returning `UPDATE 1`. PostgreSQL does *none* of that. Overwriting those eight bytes in place would violate two foundational requirements at once:

1. **Concurrent readers without read locks:** Another session running an analytical report that started ten milliseconds ago must still see `25000`, without blocking your `UPDATE` and without being blocked by it.

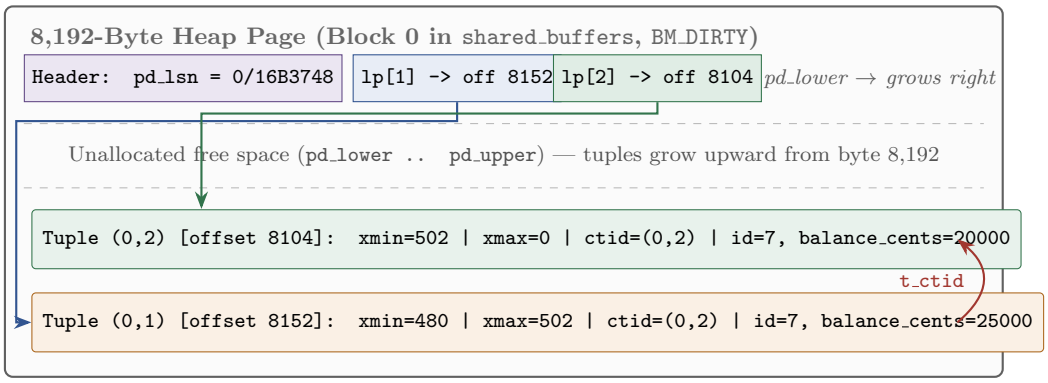
2. **Atomic rollback and crash safety:** If your transaction aborts on the next statement — or the server loses power mid-flight — `25000` must still be intact.

Instead, suppose your transaction has been assigned Transaction ID (`xid`) `502`, while the original row was inserted earlier by committed transaction `480`. Here is what PostgreSQL actually does inside RAM and on disk:

1. **Index lookup and buffer pin (Chapters 2–3):** Using the primary-key B-tree on `id`, the executor walks from the root page down to a leaf entry mapping `id = 7` → `ctid = (0, 1)` (block `0`, line pointer `1`). It looks up block `0` in `shared_buffers`, pins the 8 KB frame in RAM so clock-sweep eviction cannot steal it, and locates tuple `(0, 1)` via line pointer `1` at byte offset `8152`.

2. **Multi-version tuple creation in RAM (Chapter 5):** Rather than overwriting `25000`, PostgreSQL stamps the existing tuple's `t_xmax` field with `502` (marking it deleted by transaction `502`) and allocates a brand-new tuple in the free space at the top of the tuple area (say, byte offset `8104`, pointed to by new line pointer `2`). The new tuple gets `t_xmin = 502`, `t_xmax = 0`, and `balance_cents = 20000`. The old tuple's `t_ctid` pointer is updated from `(0, 1)` to `(0, 2)`, linking the version chain forward.

3. **Write-Ahead Logging before dirty page flush (Chapter 4):** Modifying the 8 KB buffer in RAM marks the buffer dirty, but PostgreSQL does *not* rush to write that 8 KB heap page to disk. Instead, it appends a compact sequential record describing the heap update to the Write-Ahead Log buffer, say at Log Sequence Number (LSN) `0/16B3748`, and stamps `pd_lsn = 0/16B3748` at byte offset `0` of the 8 KB heap page. When `COMMIT` runs, PostgreSQL appends a tiny commit record at `0/16B3780` and calls `fdatasync` on the sequential WAL file *only*. Once those few dozen sequential bytes hit non-volatile flash, your transaction is durable — even while the dirty 8 KB heap page sits lazily in RAM for minutes until a background writer or checkpoint flushes it.



Every piece of that story raises the exact engineering questions the rest of this book answers. What happens when old dead tuples like `(0, 1)` fill up the page? (Page pruning, HOT updates, and `VACUUM` in Chapter 5.) Did every secondary index on `accounts` have to add a new entry pointing to `(0, 2)`? (Not if no indexed column changed and `(0, 2)` fit on the same 8 KB page.) What if power failed after `COMMIT` flushed WAL record `0/16B3780`, before block `0` ever left RAM? (Crash recovery replays the WAL stream forward from the redo point in Chapter 4.)

How the book is organized

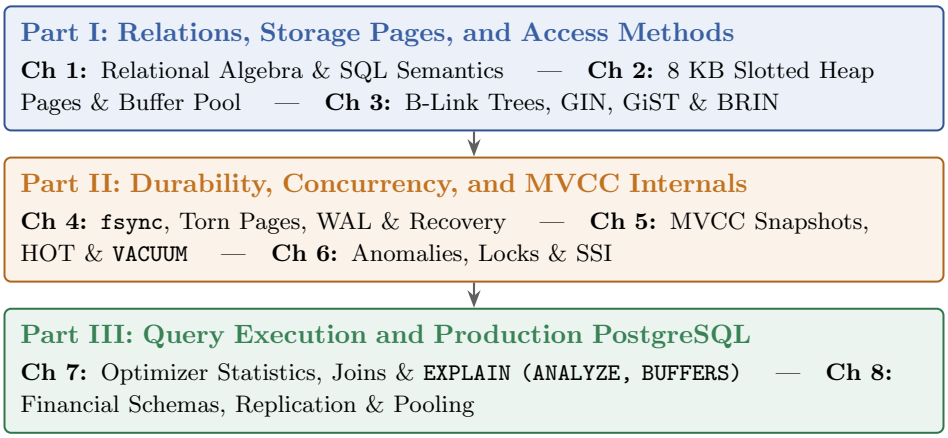
The eight chapters divide into three parts that climb from logical meaning down to storage bytes and back up to distributed production architecture:

Part I — Relations, Storage Pages, and Access Methods (Chapters 1–3). Chapter 1 starts with Codd's relational model, translating relational algebra ($\sigma, \pi, \bowtie, \gamma$) into SQL's true logical evaluation pipeline, unpacking three-valued `NULL` logic, window frames, and recursive fixpoint queries. Chapter 2 opens `$PGDATA/base` on disk, dissects the 8,192-byte slotted heap page, aligns tuple columns to hardware boundaries, explains TOAST chunking for large attributes, and traces clock-sweep eviction in `shared_buffers`. Chapter 3 builds indexes from B+tree fanout math up to PostgreSQL's Lehman–Yao high-concurrency B-link trees, composite index geometry, covering `INCLUDE` indexes with the Visibility Map, partial/expression indexes, and Hash, GIN, GiST, and BRIN access methods.

Part II — Durability, Concurrency, and MVCC Internals (Chapters 4–6). Chapter 4 confronts hardware failure: volatile caches, torn 4 KB writes inside 8 KB pages, Write-Ahead Logging, Log Sequence Numbers, `full_page_writes`, group commit, checkpoints, and step-by-step crash recovery. Chapter 5 builds Multi-Version Concurrency Control from `t_xmin`/`t_xmax`, the `pg_xact` commit log, and tuple hint bits up to snapshots, HOT chains, `VACUUM`, and 32-bit transaction wraparound. Chapter 6 maps every concurrency anomaly — dirty reads, non-repeatable reads, phantoms, lost updates (including `EvalPlanQual`), and write skew — onto concrete two-transaction timelines, contrasting row locks (`FOR UPDATE`, `SKIP LOCKED`) with Serializable Snapshot Isolation's `rw`-antidependency detection.

Part III — Query Execution and Production PostgreSQL (Chapters 7–8). Chapter 7 steps inside the cost-based query planner: `pg_stats` histograms and correlation, sequential vs. bitmap vs. index scans, nested-loop vs. Grace hash vs. merge joins, and reading `EXPLAIN (ANALYZE, BUFFERS)` plans without guesswork. Chapter 8 brings every piece together into production engineering for financial-grade systems: integer minor units, `TIMESTAMPTZ`, time-ordered `UUIDv7` keys, exclusion

and deferred balance constraints, declarative partitioning, physical/logical WAL replication, and transaction-mode connection pooling with `PgBouncer` and Rust's `SQLx`.



Connection

This book pairs directly with two companions in the curriculum. *Algorithms From the Ground Up* develops asymptotic cost analysis, hashing, dynamic programming, and textbook B-trees, which we extend in Chapters 3 and 7 into concurrent B-link trees and relational join planning. *Programming Languages From the Ground Up* builds an asynchronous financial ledger service in Rust with `Axum`, `Tokio`, and `SQLx`; Chapter 8 of this book designs the exact PostgreSQL storage, isolation, replication, and pooling layer that backs that service.

Exercises

Every chapter closes with concrete exercises — calculations, trace tables, and failure scenarios. Work them with a pen before moving on; calculating one page layout or race condition yourself teaches more than skimming ten pages of prose. These five require nothing beyond what we just traced.

Exercise 0.1 (Counting rows per 8 KB page)**.** An empty PostgreSQL heap page is 8,192 bytes long and begins with a 24-byte `PageHeaderData`. Each row added to the page consumes one 4-byte `ItemIdData` line pointer at the front of the page plus the padded tuple bytes at the back. Suppose each tuple in a narrow ledger table occupies exactly 40 bytes (a 24-byte tuple header plus two 8-byte `BIGINT` columns). What is the maximum number of live tuples that can fit on a single 8,192-byte page, and how many bytes of free space remain unused?

Exercise 0.2 (Why in-place overwrite breaks crash atomicity)**.** Suppose a naive storage engine has no Write-Ahead Log and updates rows by overwriting bytes directly inside heap pages on disk. A transaction transfers $50 from Account A (stored on block 12) to Account B (stored on block 904). Describe the exact disk state after power fails halfway between flushing block 12 and flushing block 904, and explain why restarting the server cannot tell whether to undo block 12 or finish block 904.

Exercise 0.3 (Log Sequence Number byte arithmetic)**.** PostgreSQL writes a 64-bit Log Sequence Number as `X/YYYYYYYY`, where `X` is the upper 32 bits in hexadecimal and `YYYYYYYY` is the lower 32 bits in hexadecimal. In our trace above, the heap update record began at `0/16B3748` and the commit record began at `0/16B3780`. How many bytes long was the heap update WAL record?

Exercise 0.4 (Reading through a version chain)**.** Look back at the 8 KB heap-page diagram in this preface, where tuple `(0, 1)` has `xmin = 480, xmax = 502` and tuple `(0, 2)` has `xmin = 502, xmax = 0`. Suppose Transaction `501` started *before* Transaction `502` committed, while Transaction `505` started *after* Transaction `502` committed. Which tuple should Transaction `501` read, which tuple should Transaction `505` read, and why can neither transaction safely delete `(0, 1)` while Transaction `501` is still running?

Exercise 0.5 (Sequential WAL versus random heap I/O)**.** A busy payment checkout inserts one row into an `orders` table that has four secondary B-tree indexes (on `customer_id`, `merchant_id`, `created_at`, and `status`). How many distinct 8 KB pages in `shared_buffers` may be dirtied by this single `INSERT`? Compare the I/O cost of calling `fdatasync` on five scattered files across a 2 TB table space at `COMMIT` versus appending 300 bytes to one sequential `pg_wal` file.

Contents

Preface **iii**

I Relations, Storage Pages, and Access Methods **1**

1 The Relational Model and True SQL Semantics **3**

1.1 Codd's idea: physical data independence 4

1.2 Relations versus SQL tables: sets, bags, and keys 5

1.2.1 Why normalization matters: functional dependencies in one picture 7

1.3 Relational algebra: the engine underneath SQL 8

1.3.1 Why algebraic laws make declarative SQL fast 9

1.4 SQL's true logical evaluation pipeline 10

1.4.1 The `LEFT JOIN`: `ON` predicate versus `WHERE` predicate 12

1.5 `NULL` and Kleene's three-valued logic 13

1.5.1 Why `NOT IN` returns nothing when the subquery contains `NULL` 15

1.6 Window functions: aggregating without collapsing rows 16

1.6.1 The silent `RANGE` versus `ROWS` trap on duplicate timestamps 17

1.7 CTEs and fixpoint iteration with `WITH RECURSIVE` 18

1.7.1 Non-recursive `WITH`, planner inlining, and data-modifying CTEs 19

1.7.2 How `WITH RECURSIVE` actually executes: two tables in a loop 19

1.8 Exercises 21

2 Inside an 8 KB Heap Page and the Buffer Pool **25**

2.1 Where a Table Actually Lives on Disk 26

2.1.1 Why `oid` and `relfilenode` Split: Transactional File Swaps 26

2.1.2 1 GB Segment Splitting 27

2.1.3 Companion Forks: Free Space Map (_fsm) and Visibility Map (_vm) 28
2.2 Anatomy of an 8,192-Byte Slotted Heap Page 29
2.2.1 Byte-Offset Tour of PageHeaderData (Bytes 0..24) 30
2.2.2 The Line Pointer Array: ItemIdData (4 Bytes per Slot) 32
2.2.3 Step-by-Step Trace: Inserting, Deleting, and Compacting Without Breaking Indexes 32
2.3 Inside a Heap Tuple: Headers, Null Bitmaps, and Column Tetris 34
2.3.1 Why a 23-Byte Struct Produces a 24-Byte Header (and Free 8-Column Null Bitmaps) 35
2.3.2 CPU Alignment Padding and "Column Tetris" 36
2.3.3 Decoding Real Disk Bytes with pageinspect 38
2.4 TOAST: The Oversized-Attribute Storage Technique 39
2.4.1 Inside varlena Headers: Saving 3 to 6 Bytes on Every Short String 40
2.4.2 The Four TOAST Strategies and pg_toast Chunk Slicing 40
2.5 Double Buffering, shared_buffers, Pins, and Clock-Sweep 42
2.5.1 Why Not Just Rely on the Linux Page Cache? (Double Buffering) 42
2.5.2 Inside shared_buffers: Hash Partitions and BufferDesc.state 43
2.5.3 Buffer Pins (refcount) vs. Content Locks (LWLock) 44
2.5.4 Clock-Sweep Eviction: Step-by-Step Trace on a 6-Frame Pool 45
2.5.5 Buffer Access Strategies: Why a 500 GB Seq Scan Doesn't Wipe Out RAM . 46
2.6 Exercises 48

3 Indexes: Lehman-Yao B-Link Trees, GIN, GiST, and BRIN 51
3.1 Why Secondary Indexes Exist and B+Tree Branching Math 52
3.1.1 Why Binary Search Trees Lose Badly on Block Devices 52
3.1.2 Inside an 8 KB nbtree Page: IndexTupleData and BTPageOpaqueData 54
3.2 From Textbook B+Trees to PostgreSQL's Lehman–Yao B-Link Tree 55
3.2.1 Why Naive Top-Down Latch Crabbing Limits Multi-Core Scaling 55
3.2.2 The Lehman–Yao Insight: P_HIKEY and Right-Links (btpo_next) 56
3.2.3 Step-by-Step Trace: A Concurrent Reader Crossing an Unfinished Split 57
3.2.4 Modern nbtree Engineering: Tie-Breaking, Deduplication, and Bottom-Up Deletion 58
3.3 Composite Index Lexicographic Geometry and Column Order 60
3.3.1 The Phonebook Ribbon: Equality Pins a Slice, Range Scatters Jumps 60

3.3.2 Sort Elimination and Backward Leaf Scans . . . 62
3.4 Covering Indexes (INCLUDE), the Visibility Map, and True Index-Only Scans . . . 63
3.4.1 Why PostgreSQL Index Tuples Carry Zero MVCC Visibility Metadata . . . 63
3.4.2 How the Visibility Map (_vm) Rescues Index-Only Scans . . . 64
3.4.3 Why INCLUDE Beats Appending Payload Columns to the Key . . . 65
3.5 Partial Indexes (WHERE ...) and Expression Indexes . . . 65
3.5.1 Partial Indexes: Shrinking a 4 GB Queue Index to 2 MB . . . 66
3.5.2 Expression Indexes and Syntactic Predicate Alignment . . . 67
3.6 Beyond B-Trees: Hash, GIN, GiST, and BRIN . . . 67
3.6.1 Hash Indexes (USING hash): 32-Bit Codes Only . . . 68
3.6.2 GIN (Generalized Inverted Index): Multi-Element Containment at Scale . . . 68
3.6.3 GiST (Generalized Search Tree): Bounding Hierarchies and Exclusion Constraints . . . 69
3.6.4 BRIN (Block Range Index): Indexing a Terabyte in 30 Megabytes . . . 70
3.7 Exercises . . . 72

II Durability, Concurrency, and MVCC Internals 77

4 Write-Ahead Logging, Checkpoints, and Crash Recovery 79
4.1 The Physics of write(), fsync(), and Torn 8 KB Pages . . . 80
4.1.1 Why Direct Heap Flushing on COMMIT Fails Twice . . . 81
4.2 Write-Ahead Logging (pg_wal), LSNs, and the Fundamental WAL Invariant . . . 83
4.2.1 Segment Files in $PGDATA/pg_wal/ and pg_lsn Byte Arithmetic . . . 83
4.2.2 Inside an XLogRecord and Resource Managers . . . 84
4.2.3 pd_lsn and the Fundamental WAL Invariant . . . 85
4.3 Torn Pages Solved: full_page_writes After a Checkpoint . . . 87
4.3.1 Hole-Stripped Full Page Images (FPIs) vs. Doublewrite Buffers . . . 88
4.4 Commit Durability Tradeoffs, Group Commit, and wal_writer . . . 90
4.4.1 wal_buffers, Parallel Insertion Slots, and Group Commit . . . 90
4.4.2 The synchronous_commit Spectrum and wal_writer . . . 92
4.4.3 UNLOGGED Tables: Trading Crash Survival for Raw RAM-to-Disk Speed . . . 93
4.5 Checkpoints, Background Writer, and Step-by-Step Crash Recovery . . . 94

4.5.1 global/pg_control, Fuzzy Checkpoints, and Why Recovery Rewinds to the Redo Point . . . 94
4.5.2 Three Buffer Writers, Three Distinct Jobs . . . 96
4.5.3 Classic ARIES (3 Passes) vs. PostgreSQL's 1-Pass Redo-Only Recovery . . . 96
4.5.4 Worked LSN-by-LSN Crash Recovery Trace . . . 98
4.6 Exercises . . . 102

5 MVCC, Snapshots, VACUUM, and Transaction ID Wraparound 105
5.1 Two Ways to Build a Time Machine: Undo Logs vs. In-Heap Versions . . . 106
5.2 Inside HeapTupleHeaderData: xmin, xmax, cid, and ctid . . . 107
5.2.1 t_xmin and Lazy Transaction ID Assignment . . . 108
5.2.2 t_xmax: Deletion, Update Chains, and Row Locks . . . 108
5.2.3 t_cid and the Halloween Problem . . . 109
5.3 Commit Status (pg_xact) and Why a SELECT Can Dirty Disk Pages . . . 110
5.3.1 The Commit Log (pg_xact): Two Bits per Transaction . . . 110
5.3.2 Tuple Hint Bits: Caching Commit Status on the Heap Tuple . . . 111
5.4 Snapshots and the Exact Visibility Formula . . . 112
5.5 Write Amplification and HOT (Heap-Only Tuple) Updates . . . 114
5.5.1 How HOT Decouples B-Trees from Unindexed Updates . . . 115
5.5.2 Single-Page Pruning (On-the-Fly Defragmentation Without VACUUM) . . . 116
5.6 Table Bloat, VACUUM, autovacuum, and VACUUM FULL . . . 117
5.6.1 The Three Phases of Concurrent ("Lazy") VACUUM . . . 117
5.6.2 Why VACUUM Does Not Shrink Disk Usage (vs. VACUUM FULL) . . . 118
5.7 32-Bit Transaction ID Wraparound and Tuple Freezing . . . 119
5.7.1 Modular Circle Comparison (TransactionIdPrecedes) . . . 119
5.7.2 The Cure: Tuple Freezing, relfrozenxid, and the Safety Shutdown . . . 121
5.8 Exercises . . . 122

6 Isolation Anomalies, Row Locking, and Serializable Snapshot Isolation 125
6.1 What "Isolation" Actually Promises (and Why ANSI SQL-92 Missed Half the Story) . 126
6.1.1 The ANSI SQL-92 Table vs. PostgreSQL Reality . . . 126
6.2 Read Anomalies: Dirty Reads, Read Skew, and Phantoms . . . 127
6.2.1 Dirty Read (P_1): Reading Uncommitted Bytes . . . 127

6.2.2 Non-Repeatable Read (P_2) and Read Skew Under `READ COMMITTED` . . . 127
6.2.3 Phantom Reads (P_3) . . . 128
6.3 Lost Updates and `EvalPlanQual` Re-Evaluation . . . 129
6.3.1 Case 1: Read-Modify-Write in Application Code Under `READ COMMITTED` . . . 129
6.3.2 Case 2: Atomic SQL `SET bal = bal - 8000` and `EvalPlanQual` (`EPQ`) . . . 129
6.4 Write Skew: Why `REPEATABLE READ` Still Breaks Multi-Row Invariants . . . 131
6.5 Serializable Snapshot Isolation (SSI): True `SERIALIZABLE` Without Read Locks . . . 132
6.5.1 Conflict Edges and the "Dangerous Structure" Theorem . . . 132
6.5.2 How `SIRead` Predicate Locks Work Inside PostgreSQL . . . 133
6.6 Explicit Row Locks, Work Queues, Deadlocks, and Advisory Locks . . . 134
6.6.1 PostgreSQL's Four Row-Level Lock Strengths . . . 135
6.6.2 Non-Blocking Modifiers: `NOWAIT` and `SKIP LOCKED` Job Queues . . . 135
6.6.3 Deadlocks (`deadlock_timeout`) and Deterministic Lock Ordering . . . 136
6.6.4 Transactional Advisory Locks (`pg_advisory_xact_lock`) . . . 137
6.7 Exercises . . . 138

III Query Execution and Production PostgreSQL 141

7 The Query Planner, Join Algorithms, and `EXPLAIN ANALYZE` 143
7.1 From SQL Text to a Running Plan . . . 144
7.2 What the Planner Knows: `ANALYZE` and `pg_stats` . . . 144
7.2.1 Sampling a Table of Any Size . . . 144
7.2.2 Reading One Row of `pg_stats` . . . 145
7.2.3 Turning Statistics into Selectivity . . . 146
7.2.4 When Two Predicates Are Not Independent . . . 147
7.3 The Planner's Currency: Cost Units . . . 148
7.3.1 Pricing a Sequential Scan by Hand . . . 149
7.4 Four Ways to Read One Table . . . 149
7.4.1 Seq Scan . . . 149
7.4.2 Index Scan . . . 150
7.4.3 Bitmap Index Scan plus Bitmap Heap Scan . . . 151
7.4.4 Index-Only Scan . . . 152

7.5 Three Ways to Join Two Tables . . . 153
7.5.1 Nested Loop . . . 153
7.5.2 Hash Join . . . 154
7.5.3 Merge Join . . . 155
7.6 Searching Join Orders . . . 156
7.6.1 Dynamic Programming over Subsets . . . 156
7.6.2 Limits and the Genetic Optimizer . . . 156
7.7 Reading `EXPLAIN (ANALYZE, BUFFERS)` Line by Line . . . 157
7.7.1 A Complete Plan . . . 157
7.7.2 Diagnosing a Bad Plan . . . 159
7.8 Exercises . . . 161

8 Financial Schemas, Replication, and Connection Pooling 163
8.1 Data Types That Never Lie About Money, Time, or Identity . . . 164
8.1.1 Money: Why Binary Floats Fail, and `BIGINT` vs. `NUMERIC` . . . 164
8.1.2 Time: `TIMESTAMPTZ` and the Daylight-Saving Trap . . . 165
8.1.3 Primary Keys at Scale: Why `UUIDv4` Keys Thrash B-Trees and `UUIDv7` Flies . 166
8.2 Encoding Domain Invariants Directly in the Schema . . . 167
8.2.1 `NOT NULL`, `CHECK`, Domains, and Atomic Idempotency Keys . . . 167
8.2.2 Temporal Non-Overlapping Invariants with `EXCLUDE USING gist` . . . 168
8.3 A Double-Entry Financial Ledger in PostgreSQL and Rust . . . 169
8.3.1 Conservation of Value and `DEFERRABLE INITIALLY DEFERRED` Triggers . . . 169
8.3.2 Deadlock-Free Transfers in Rust (`Axum` + `SQLx`) . . . 171
8.4 Declarative Partitioning for Billion-Row Ledgers . . . 175
8.5 High Availability: Physical Streaming and Logical Replication . . . 176
8.5.1 Physical Streaming WAL Replication and Standby Conflicts . . . 176
8.5.2 The `synchronous_commit` Durability and Visibility Ladder . . . 177
8.5.3 Logical Replication for Zero-Downtime Upgrades and Event Streaming . . . 178
8.6 Connection Pooling at Scale: Process Architecture and PgBouncer . . . 179
8.6.1 Why Five Thousand Backend Processes Melt PostgreSQL . . . 179
8.6.2 `PgBouncer` Transaction Multiplexing and the Prepared Statement Trap . . . 180
8.7 Exercises . . . 182

Part I

Relations, Storage Pages, and Access Methods

Chapter 1

The Relational Model and True SQL Semantics

Suppose you are building the storage engine for a bank in 1968. You have magnetic disks divided into fixed-size blocks, a C or COBOL compiler, and three kinds of facts to store: customers, accounts, and ledger postings. How do you organize those records on disk so you can find all postings belonging to a customer?

If you think like a systems programmer accustomed to in-memory structs, the natural answer is *pointers*. Inside each customer record on disk, you store the disk address (cylinder, track, and record offset) of that customer's first account. Inside each account record, you store a pointer to the customer's next account, plus a pointer to that account's first ledger posting. Inside each posting, you store a pointer to the next posting in chronological order, and perhaps a back-pointer to the owning account. To read a customer's monthly statement, your application program chases disk pointers around a linked ring, one block seek at a time.

If you have written Java 6, you have written this kind of program. A loop such as

```
for (Account a : customer.getAccounts())
    for (Posting p : a.getPostings())
        total += p.getCents();
```

follows object references, and the JVM stores each reference as a pointer to a heap address. The 1968 databases did the same thing, but on disk, and the pointers were part of the file format. Any query that did not follow the chains the designer had built in advance needed new chains, a file reorganization, and new application code.

For twenty years, that was how commercial databases worked. IBM's Information Management System (IMS, built for the Apollo space program in 1966) arranged records into physical parent–child trees. Charles Bachman's Integrated Data Store (IDS) and the CODASYL standard generalized trees into arbitrary pointer networks, earning Bachman the 1973 Turing Award for casting the programmer as a "navigator" steering a cursor through a web of disk addresses.

Then, in June 1970, an IBM mathematician named Edgar F. Codd published an eleven-page paper titled *A Relational Model of Data for Large Shared Data Banks*, and argued that application programs should never follow disk pointers at all.

This chapter builds Codd's alternative from the ground up and connects it directly to the SQL that

PostgreSQL parses and executes today. We will trace relational algebra on small ledger tables, map those operators onto SQL's true ten-stage logical evaluation order, uncover the traps hidden inside Kleene's three-valued `NULL` logic (including why a single `NULL` turns a `NOT IN` query into an empty result), and master window frames and recursive fixpoint queries before descending into physical 8 KB pages in Chapter 2.

1.1 Codd's idea: physical data independence

To see why Codd wanted to ban user-visible pointers from databases, watch what happens to a pointer-based banking system the moment requirements change.

In our 1968 pointer network, every posting chain runs forward from `Account` → `Posting_1` → `Posting_2` → ⋯ → `Posting_k`. Answering "list all postings for Account 102" is fast: jump to Account 102 and follow `next_posting` links until the chain loops back to the owner.

Now Monday morning arrives, and the fraud desk asks the inverse question: "Which accounts sent a posting of more than $5,000 to Merchant 88 yesterday?"

Nothing in your disk layout wires Merchant 88 to its incoming postings, and nothing wires yesterday's date to a starting address. Worse, the application code that reads existing reports is coupled line-by-line to the physical pointer topology:

1. **Asymmetric query cost and brittle code:** Every query has to be hand-written as imperative nested loops (`FIND FIRST`, `FIND NEXT WITHIN SET`) that know which pointer chains exist. If an engineer later replaces a linked list of postings with a sorted B-tree array or partitions postings by month, every existing application program that called `FIND NEXT` breaks and must be rewritten.
2. **Dangling and corrupt pointer chains:** Inserting one posting requires updating up to four physical disk pointers across different blocks (previous sibling, next sibling, account head/tail, merchant ring). If a program bug or crash leaves one pointer half-updated, traversing the ring loops forever or lands inside an unrelated account's records.

History

Edgar F. Codd joined IBM after serving as a flight lieutenant in the Royal Air Force and completing a doctorate in mathematics at the University of Michigan. Watching IBM customers rewrite thousands of lines of application code every time a file layout or index changed on System/360 mainframes, Codd proposed separating the *logical representation of facts* completely from *physical access paths*. Inside IBM, defenders of IMS initially dismissed Codd's model as an academic toy that could never match hand-tuned pointer navigation. Two research prototypes proved otherwise: IBM's System R in San Jose (which invented the SEQUEL language — later shortened to SQL — and cost-based dynamic programming query optimization) and Michael Stonebraker's Ingres project at UC Berkeley (whose successor, Postgres, became modern PostgreSQL). Codd received the 1981 Turing Award once it became clear that an algebraic optimizer choosing indexes automatically beat hand-navigated pointer rings on maintainability and, soon enough, throughput as well.

Codd's design rests on one clean separation, known as **physical data independence**:

- **Logical layer (what your SQL sees):** All information in the database is represented in exactly one way — as explicit *values* sitting inside two-dimensional mathematical *relations*. An account does not hold a disk pointer to its owner; it holds the value `customer_id = 42`, matching the value `id = 42` in the `customers` relation.
- **Physical layer (what the storage engine hides):** Underneath the logical interface, PostgreSQL is completely free to store rows in unsorted 8 KB heap pages (Chapter 2), maintain Lehman–Yao B-link trees or BRIN block summaries (Chapter 3), compress oversized columns out-of-line, or partition a billion-row ledger across monthly files (Chapter 8). You can add or drop five indexes at noon without changing a single character of any `SELECT` query, and without changing a single answer those queries return.

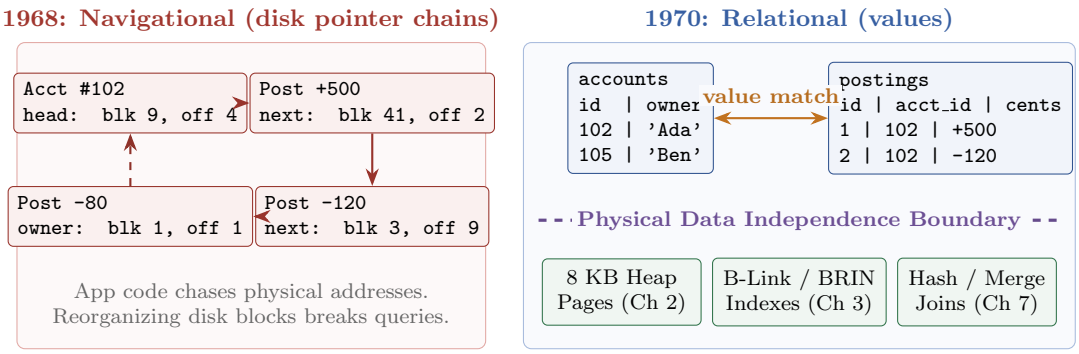


Figure 1.1: Navigational databases expose physical disk pointer rings directly to application code (left). Codd's relational model represents relationships purely through matching data values above a strict data-independence boundary (right), freeing PostgreSQL to swap physical storage layouts and indexes underneath unchanged SQL queries.

Intuition

Pointers still exist inside PostgreSQL — millions of them. Every B-tree leaf entry holds a 6-byte `ctid` pointing to an 8 KB heap block and slot, and every updated heap tuple points forward to its newer version. Codd did not abolish pointers from the *machine*; he moved them *below the abstraction line* where the storage engine maintains them automatically and the query optimizer chooses whether to follow them or scan sequentially.

1.2 Relations versus SQL tables: sets, bags, and keys

To reason sharply about queries, we need both Codd's mathematical definition of a relation and the one pragmatic compromise that real SQL engines make for speed.

Formal

Let $D_1, D_2, \ldots, D_k$ be sets of atomic values called **domains** (for instance, 64-bit integers $\mathbb{Z}_{2^{64}}$, UTF-8 text strings, or UTC microsecond timestamps).

1. A **relation schema** $R(A_1 : D_1, \ldots, A_k : D_k)$ names a relation R together with k distinct **attributes** (columns) A_i, each constrained to domain D_i. The integer k is the **degree**

(or **arity**) of R.

2. A **tuple** t conforming to R assigns each attribute A_i an element $t[A_i] \in D_i$ (or, in SQL, the missing-information marker `NULL`).
3. A **pure mathematical relation** (or **relation instance**) over R is a finite subset

$$r \subseteq D_1 \times D_2 \times \cdots \times D_k.$$

Because r is a *set*, two consequences hold immediately: tuples have **no inherent top-to-bottom order**, and **duplicate tuples cannot exist** in r.

If you open `psql`, create a table without a `PRIMARY KEY`, and run `INSERT INTO logs VALUES ('login')` twice, PostgreSQL stores two identical rows. And if you run `SELECT currency FROM accounts` on a table with ten thousand `'USD'` accounts, PostgreSQL returns `'USD'` ten thousand times rather than collapsing it to a single row. Why does SQL depart from pure set theory here?

The answer is pure algorithmic cost. Suppose a table has $N = 50{,}000{,}000$ rows and you project two non-unique columns: `SELECT merchant_id, amount_cents FROM postings`. Eliminating duplicates across 50 million projected pairs requires either sorting all 50 million tuples in $O(N \log N)$ time across gigabytes of temporary disk spill or building a hash table with one entry per distinct pair in RAM. Most callers streaming rows into an application aggregate or report do not want to pay an $O(N \log N)$ deduplication tax on every projection unless they explicitly ask for it.

In Java terms, Codd's relation behaves like a `HashSet<Row>`: adding an equal row a second time changes nothing, and iteration order is unspecified. A SQL table without a key behaves like an `ArrayList<Row>` whose order you are not allowed to rely on: duplicates are kept, and `SELECT` hands them all back.

Consequently, an unconstrained SQL table or query result is not a mathematical set; it is a **bag** (a **multiset**), modeled formally by attaching a non-negative integer multiplicity $\mu_R(t) \in \mathbb{N}$ to every possible tuple t:

SQL Operator	Multiset Multiplicity	Physical Consequence in PostgreSQL
`SELECT a, b FROM R`	Sums $\mu_R(t)$ over matches	$O(N)$ streaming pass; zero memory overhead
`SELECT DISTINCT a, b`	$\min(1, \sum \mu_R(t))$	Requires `HashAggregate` or `Sort + Unique`
`R UNION ALL S`	$\mu_R(t) + \mu_S(t)$	$O(1)$-memory `Append` streaming R then S
`R UNION S`	$\min(1, \mu_R(t) + \mu_S(t))$	Concatenates, then hashes or sorts to deduplicate
`R INTERSECT ALL S`	$\min(\mu_R(t), \mu_S(t))$	Pairs up matching multiplicities via hash counters
`R EXCEPT ALL S`	$\max(0, \mu_R(t) - \mu_S(t))$	Subtracts S's count of t from R's count of t

Common trap

Writing `UNION` when you actually know the two branches are disjoint (for example, combining active orders from 2025 with archived orders from 2024) is one of the most common self-inflicted performance wounds in SQL. Bare `UNION` forces PostgreSQL to buffer and hash or sort the combined output just to prove there are no duplicates. Whenever two subqueries are already disjoint — or duplicates are harmless — write `UNION ALL` so PostgreSQL can stream both inputs directly through a zero-overhead `Append` operator.

How do we bring a base SQL table back into Codd's clean set world where every row represents a distinct, addressable fact? By declaring a **key**:

- A **superkey** of schema R is any subset of attributes $K \subseteq \{A_1, \ldots, A_k\}$ such that no two legal tuples in R may agree on all attributes of K: if $t_1[K] = t_2[K]$, then $t_1 = t_2$.
- A **candidate key** is a *minimal* superkey — removing any attribute from K destroys uniqueness. For example, on an `accounts` table, {`id`} might be one candidate key and {`iban`} a second candidate key, whereas {`id`, `currency`} is a superkey that is not minimal.
- A **primary key** (`PRIMARY KEY`) is the candidate key chosen as the canonical identifier of rows in R, with the additional SQL guarantee that none of its columns may ever be `NULL`. PostgreSQL enforces every `PRIMARY KEY` and `UNIQUE` constraint physically by maintaining a unique B-tree index (Chapter 3).
- A **foreign key** (`REFERENCES`) on child relation S asserts *referential integrity* against a candidate key of parent relation R: every non-`NULL` key value appearing in S must currently exist in R.

1.2.1 Why normalization matters: functional dependencies in one picture

Even with a primary key on every table, a poorly factored schema can rot from the inside. Suppose an engineer designs one wide ledger table `postings_denorm` with columns `(posting_id, account_id, owner_email, account_tier, amount_cents)`, where `posting_id` is the primary key. Because `account_id = 102` belongs to `'ada@example.org'` on tier `'pro'`, those two strings are repeated across all 40,000 postings for Account 102.

Mathematically, the table contains a **functional dependency**, written

$$\texttt{account_id} \rightarrow \{\texttt{owner_email}, \texttt{account_tier}\},$$

meaning that knowing `account_id` uniquely determines `owner_email` and `account_tier`, even though `account_id` is *not* a superkey of `postings_denorm`. Whenever a non-superkey attribute determines another attribute, three classic anomalies appear immediately:

1. **Update anomaly (physical write amplification + inconsistency):** When Ada changes her email address, PostgreSQL cannot update one row; it must rewrite 40,000 heap tuples across hundreds of 8 KB pages (Chapter 2). Worse, without a separate `accounts` table, nothing stops a bug from updating 39,999 rows and leaving one row with her old email.
2. **Insertion anomaly:** You cannot open Account 109 for a brand-new customer who has zero postings yet without inventing a fake dummy posting or permitting `posting_id` to be `NULL`.

3. **Deletion anomaly:** If a test account's lone posting is reversed and deleted, the fact that the account ever existed — along with its owner's email and tier — vanishes with it.

Why it works

Boyce–Codd Normal Form (BCNF) in one sentence. A relation schema R is in Boyce–Codd Normal Form if and only if for every non-trivial functional dependency $X \to Y$ holding on R (meaning $Y \not\subseteq X$), X is a *superkey* of R. Colloquially: *every fact in R depends on the key, the whole key, and nothing but the key.* Splitting `postings_denorm` into `accounts(account_id, owner_email, account_tier)` and `postings(posting_id, account_id, amount_cents)` puts both relations in BCNF: each fact lives in one place, Ada's email update touches exactly one 8 KB page, and no update, insertion, or deletion anomaly can ever occur.

1.3 Relational algebra: the engine underneath SQL

When PostgreSQL receives a SQL query, it does not execute the English words directly. Its parser and analyzer translate your SQL text into a tree of algebraic operators whose inputs are relations and whose outputs are relations. Because every operator produces a relation, operators *compose* cleanly — the output of a join feeds straight into a filter, which feeds into a grouping operator, which feeds into a projection.

Let us fix a concrete miniature dataset that we can trace by hand across the entire chapter: a four-row `accounts` table A and a six-row `postings` table P, with amounts stored in integer cents.

Relation A (`accounts`)

id	owner	tier
101	'Ada'	'pro'
102	'Ben'	'free'
103	'Cara'	'pro'
104	'Dan'	'pro'

Relation P (`postings`)

pid	acct_id	cents	posted_on
1	101	+10000	'2026-10-01'
2	101	−2500	'2026-10-02'
3	102	+4000	'2026-10-02'
4	103	+15000	'2026-10-01'
5	103	−3000	'2026-10-03'
6	103	+2000	'2026-10-03'

Notice Account `104` (`'Dan'`): he opened a `'pro'` account, so he sits in A, yet he has zero rows in P. Watching what happens to Dan across joins and filters will expose half of the bugs people write in production SQL.

Five core relational operators (plus outer, semi-, and anti-join variants) form the algebra we need:

1. **Selection $\sigma_\theta(R)$ (horizontal slice):** Keeps every tuple $t \in R$ satisfying predicate θ, preserving R's exact column schema. On our dataset, $\sigma_{\texttt{tier = 'pro'}}(A)$ keeps rows `101`, `103`, and `104`, dropping Ben (`102`).

2. **Projection $\pi_{e_1 \to b_1, \ldots, e_m \to b_m}(R)$ (vertical slice & column expressions):** Evaluates expressions $e_1, \ldots, e_m$ on each tuple of R to produce an m-column output schema. In pure set algebra π deduplicates automatically; in SQL's bag algebra (often written π^b), multiplicities are preserved unless `DISTINCT` (δ) is applied on top.

3. **Cartesian Product $R \times S$ and Join $R \bowtie_\theta S$:** The cross product $A \times P$ pairs *every* row of A with *every* row of P, yielding $|A| \cdot |P| = 4 \times 6 = 24$ wide tuples of $3 + 4 = 7$ columns. An **inner theta-join** is defined logically as a cross product followed by selection:

$$A \bowtie_\theta P \;\coloneqq\; \sigma_\theta(A \times P).$$

When θ is an equality such as $A.\texttt{id} = P.\texttt{acct_id}$, we have an **equi-join**: of the 24 possible pairs in $A \times P$, only the 6 pairs where `id` matches `acct_id` survive (two for Ada, one for Ben, three for Cara, and *zero* for Dan, whose account `104` has no matching rows in P).

4. **Outer, Semi-, and Anti-Joins:**
   - **Left Outer Join** (A `LEFT JOIN` P): Returns all pairs from $A \bowtie_\theta P$, *plus* one tuple for each unmatched row of A padded with `NULL` in every attribute of P. Here Dan (`104`) survives as (`104`, `'Dan'`, `'pro'`, `NULL`, `NULL`, `NULL`, `NULL`).
   - **Semi-Join** ($A \ltimes_\theta P$, SQL `WHERE EXISTS (...)`): Returns tuples of A that match *at least one* tuple in P, without duplicating A's row when P has multiple matches. Whereas $A \bowtie P$ produces three rows for Cara (`103`), $A \ltimes P$ returns Cara *once* (`101`, `102`, `103`).
   - **Anti-Join** ($A \triangleright_\theta P$, SQL `WHERE NOT EXISTS (...)`): Returns tuples of A that match *zero* tuples in P — here returning solely (`104`, `'Dan'`, `'pro'`).
5. **Grouping and Aggregation $\gamma_{G;\, f_1(c_1)\to s_1,\ldots}(R)$:** Partitions the tuples of R into equivalence classes that agree on all grouping attributes G, then collapses each bucket into a *single* output tuple carrying G and the aggregate results $f_j(c_j)$ (`COUNT`, `SUM`, `MIN`, `MAX`, `AVG`). Applying $\gamma_{\texttt{acct_id};\,\texttt{SUM(cents)}\to\texttt{bal}}(P)$ partitions P's six rows into three buckets:

$$(101,\ 7500), \qquad (102,\ 4000), \qquad (103,\ 14000).$$

1.3.1 Why algebraic laws make declarative SQL fast

Why did defining queries in this small algebra defeat hand-coded pointer loops? Because relational operators obey exact **algebraic identities** that let an optimizer transform a slow expression into an equivalent fast one before touching a single disk block.

Suppose a user asks for the posting amounts of all `'pro'` accounts on or after October 2:

$$E_{\text{naive}} \;=\; \pi_{\texttt{owner},\texttt{cents}}\Big(\sigma_{A.\texttt{tier}=\texttt{'pro'} \,\wedge\, P.\texttt{posted_on}\geq\texttt{'2026-10-02'} \,\wedge\, A.\texttt{id}=P.\texttt{acct_id}}(A \times P)\Big).$$

Executed as written on a production bank with $|A| = 10^6$ accounts and $|P| = 10^8$ postings, forming $A \times P$ first would materialize 10^{14} intermediate tuples — crashing the server long before finishing.

Instead, PostgreSQL's planner applies three equivalence laws automatically:

$$\begin{aligned}
\sigma_{p_1 \wedge p_2}(R) &= \sigma_{p_1}(\sigma_{p_2}(R)) && \text{(cascade of selections)},\\
\sigma_{p_A}(A \bowtie_\theta P) &= \big(\sigma_{p_A}(A)\big) \bowtie_\theta P && \text{(selection pushdown when } p_A \text{ mentions only } A),\\
\sigma_\theta(A \times P) &= A \bowtie_\theta P && \text{(cross product + equality} \to \text{hash/index equi-join)}.
\end{aligned}$$

Pushing $\sigma_{\texttt{tier='pro'}}$ down onto A and $\sigma_{\texttt{posted_on}\geq\texttt{'2026-10-02'}}$ down onto P *before* joining shrinks both inputs dramatically, and replacing $\times$ with an indexed or hash equi-join drops the work from 10^{14} comparisons to linear in the surviving rows.

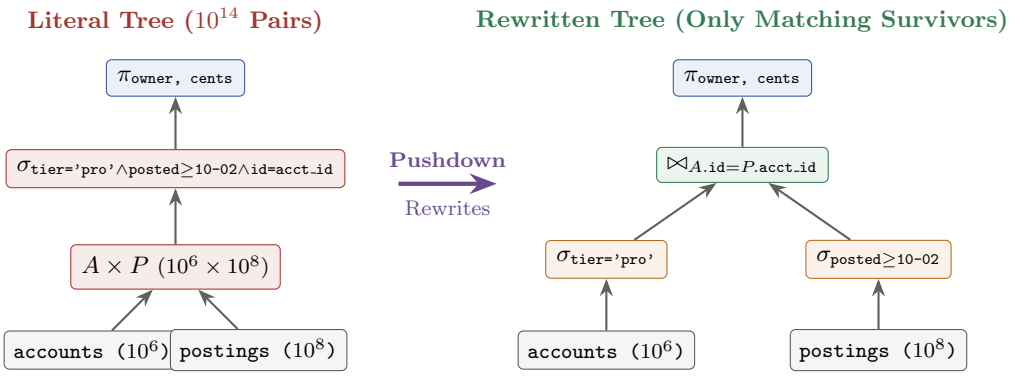


Figure 1.2: Relational algebra optimization in miniature: pushing single-table selections below the join and converting $\sigma_{A.\texttt{id}=P.\texttt{acct_id}}(A \times P)$ into an equi-join produces the exact same output relation while avoiding trillions of useless tuple pairs.

1.4 SQL's true logical evaluation pipeline

If you ask a programmer why the query below fails in PostgreSQL with an unknown-column error (`column "net_cents" does not exist`), they are often momentarily baffled:

```
-- BROKEN: net_cents was written on line 1, so why can't line 3 see it?
SELECT acct_id, SUM(cents) AS net_cents
  FROM postings
 WHERE net_cents > 5000
 GROUP BY acct_id;
```

PostgreSQL rejects it at parse time:

```
ERROR:  column "net_cents" does not exist
LINE 3:  WHERE net_cents > 5000
               ^

```

After all, `net_cents` is defined right there on line 1, two lines *above* `WHERE`. Why can `WHERE` not see it, while `ORDER BY net_cents DESC` at the bottom of the query *can* see it?

Because the *lexical order* in which SQL clauses are written on the page (designed in the 1970s to mimic an English sentence: "Select these columns from these tables where…") is **not** the order in which SQL defines the meaning of a query. Logically, every `SELECT` block is a ten-stage pipeline, and `SELECT` itself does not run until **Stage 7**.

Formal

The Ten-Stage Logical Evaluation Pipeline of a SQL Query. Given a single query block, its exact output bag of rows is defined by executing these ten transformations in sequence, where each stage consumes the virtual table produced by the stage immediately before it:

1. `FROM`: Materialize base tables, views, CTE references, or `LATERAL` table functions into initial virtual relations.
2. `JOIN / ON`: Combine input relations via cross product, filter pairs by the `ON` predicate,

and (for `LEFT`, `RIGHT`, or `FULL OUTER` joins) add back unmatched preserved-side rows padded with `NULL`s.

3. `WHERE`: Apply row-level selection σ_θ; keep *only* rows for which θ evaluates to `TRUE` (discarding both `FALSE` and `UNKNOWN`).
4. `GROUP BY`: Partition surviving rows into buckets sharing equal values across the grouping columns (treating two `NULL`s as belonging to the same bucket), and collapse each bucket into a group representative ready for aggregation.
5. `HAVING`: Filter the collapsed groups; keep *only* groups whose aggregate predicate evaluates to `TRUE`.
6. `WINDOW`: Evaluate every `OVER (...)` window function across the surviving rows (or collapsed groups) *without* collapsing rows further.
7. `SELECT`: Project output expressions and bind column aliases (`AS alias`). Only from this stage onward do `SELECT`-list aliases exist.
8. `DISTINCT` (or `DISTINCT ON (...)`): Deduplicate projected rows.
9. `ORDER BY`: Sort the resulting rows (may reference `SELECT` aliases, and if `DISTINCT` was not used, may even reference input columns not projected in `SELECT`).
10. `LIMIT / OFFSET` (`FETCH FIRST ... ROWS ONLY`): Discard leading `OFFSET` rows and truncate after `LIMIT` rows.

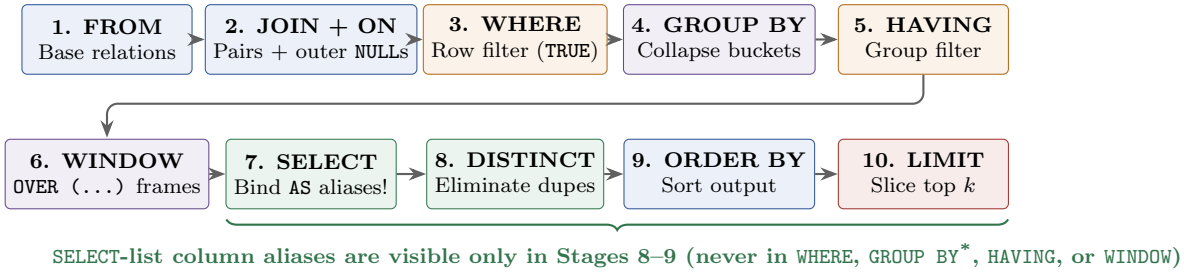


Figure 1.3: SQL's ten-stage logical evaluation pipeline. Notice why `WHERE` cannot reference aggregates (Stage 3 runs before Stage 4), why `WHERE` cannot filter on `ROW_NUMBER()` (Stage 3 runs before Stage 6), and why `SELECT` aliases appear only at Stage 7. (*PostgreSQL extends the SQL standard slightly by letting `GROUP BY` and `ORDER BY` refer to simple `SELECT` output column names when unambiguous, but never inside expressions or `WHERE`/`HAVING`.)

Once you know this order, several confusing SQL rules follow directly from the stage numbers. Each comes with the error PostgreSQL actually prints:

1. **Why `WHERE` rejects `SUM(cents) > 5000` and requires `HAVING`:** At Stage 3 (`WHERE`), rows have not been grouped yet — PostgreSQL is looking at one individual posting row at a time, so `SUM(cents)` does not exist yet. Individual postings are filtered at Stage 3 (`WHERE`); collapsed account buckets are filtered at Stage 5 (`HAVING`). Writing `WHERE SUM(cents) > 5000` fails with:

```
ERROR:  aggregate functions are not allowed in WHERE
```

2. **Why `WHERE rn = 1` fails on `ROW_NUMBER() OVER (...) AS rn`:** Window functions are not computed until Stage 6, three stages *after* `WHERE`. To keep only the latest posting per account using `ROW_NUMBER()`, wrap the query in a CTE or subquery so Stage 6 finishes inside the inner block, and filter `WHERE rn = 1` in the outer block's Stage 3 (or use PostgreSQL's `DISTINCT ON (acct_id) ... ORDER BY acct_id, posted_on DESC`). Referring to the alias gives `column "rn" does not exist`; inlining the window call into `WHERE` gives:

```
ERROR:  window functions are not allowed in WHERE
```

3. **Why `ORDER BY random()` after `SELECT DISTINCT` errors out:** At Stage 8, `DISTINCT` collapses multiple input rows into one output row. If Stage 9 (`ORDER BY`) then asks to sort by a column or expression that was *not* part of the deduplicated `SELECT` list (say, `SELECT DISTINCT tier FROM accounts ORDER BY owner`), which `owner` among Ada, Cara, and Dan should represent the single `'pro'` row? The question is ill-defined, so PostgreSQL rejects it at parse time:

```
ERROR:  for SELECT DISTINCT, ORDER BY expressions must appear in select list
```

1.4.1 The `LEFT JOIN`: `ON` predicate versus `WHERE` predicate

Nothing illustrates Stage 2 versus Stage 3 more starkly than filtering the nullable side of a `LEFT JOIN`. Suppose you want a report listing *every* `'pro'` account — including zero-activity accounts like Dan (`104`) — together with how many postings that account made on or after `'2026-10-02'`. Compare these two queries that differ by a single keyword (`AND` versus `WHERE`):

```
-- Query A (WRONG for zero-activity accounts): puts date filter in Stage 3 (WHERE)
SELECT a.id, a.owner, COUNT(p.pid) AS oct2_or_later_cnt
  FROM accounts a
  LEFT JOIN postings p ON p.acct_id = a.id
 WHERE a.tier = 'pro'
   AND p.posted_on >= DATE '2026-10-02'
 GROUP BY a.id, a.owner;

-- Query B (CORRECT): puts right-table date filter in Stage 2 (JOIN ... ON)
SELECT a.id, a.owner, COUNT(p.pid) AS oct2_or_later_cnt
  FROM accounts a
  LEFT JOIN postings p
    ON p.acct_id = a.id
   AND p.posted_on >= DATE '2026-10-02'
 WHERE a.tier = 'pro'
 GROUP BY a.id, a.owner;
```

Trace Dan (`id = 104`) through both pipelines by hand:

- In **Query A**, Stage 2 (`LEFT JOIN`) finds no postings for Dan and emits the padded row (`104`, `'Dan'`, `'pro'`, `NULL`, `NULL`, `NULL`, `NULL`). Then Stage 3 (`WHERE`) evaluates `NULL >= DATE '2026-10-02'`, which yields `UNKNOWN`. Because `WHERE` keeps only `TRUE`, **Dan's row is discarded** — your `LEFT JOIN` silently turned into an `INNER JOIN`.
- In **Query B**, Stage 2 evaluates `p.acct_id = a.id AND p.posted_on >= '2026-10-02'` *during the join match itself*. Dan matches zero rows under that combined condition, so the `LEFT`

JOIN's outer-padding step emits Dan padded with NULLs. Stage 3 checks only a.tier = 'pro' (TRUE), Stage 4 groups Dan's single padded row, and COUNT(p.pid) (which counts non-NULL values of p.pid, unlike COUNT(*)) returns 0.

Common trap

Whenever you write A LEFT JOIN P, check two things before hitting enter:

1. Any filter on columns of the optional table P belongs in ON, never in WHERE (unless your explicit goal is an anti-join WHERE p.pid IS NULL to find unmatched rows).
2. In SELECT, write COUNT(p.pid), *never* COUNT(*). For Dan's NULL-padded outer-join row, COUNT(*) counts rows in the bucket and reports 1 posting for an account with zero postings, whereas COUNT(p.pid) counts non-NULL pids and accurately reports 0.

Intuition

Logical order is a specification, not a physical straitjacket. PostgreSQL promises that the *result* of every query equals what the ten-stage pipeline produces. Physically, its planner (Chapter 7) reorders aggressively whenever the output is guaranteed identical: in Query B above, PostgreSQL pushes WHERE a.tier = 'pro' all the way down into the scan of accounts *before* Stage 2 ever runs, because filtering the preserved left side of a LEFT JOIN early never changes which right-side rows match. By contrast, PostgreSQL knows it *cannot* push WHERE p.posted_on >= ... in Query A into ON without noticing that the NULL-rejecting WHERE clause actually permits simplifying the entire LEFT JOIN into a faster INNER JOIN.

1.5 NULL and Kleene's three-valued logic

Codd's 1970 paper assumed every tuple cell contained a known element of its attribute's domain. Real databases quickly ran into messy facts: a pending wire transfer does not have a settled_at timestamp yet; a cash deposit has no counterparty_iban. SQL models missing or inapplicable data with a special marker called NULL.

Start with one rule: **NULL is not a value**. It is a *marker* meaning "unknown or missing."

Suppose Alice's age is unknown (NULL) and Bob's age is also unknown (NULL). Are Alice and Bob the same age? *You do not know.* Alice might be 29 and Bob 44, or both might be 31. Asking NULL = NULL cannot honestly return TRUE, and asking NULL <> NULL cannot honestly return TRUE either. Both comparisons must return a third truth value: UNKNOWN (which PostgreSQL displays in BOOLEAN output columns as blank/NULL).

Java 6 programmers need to unlearn one habit here. In Java, null == null is true, and a Boolean field holding null throws NullPointerException the moment you unbox it in if (flag). SQL does neither. A comparison involving NULL does not throw and does not return FALSE; it returns UNKNOWN, and the clause that consumes it decides what UNKNOWN means.

Arithmetic and scalar functions propagate unknownness immediately: NULL + 500 is NULL, 'USD-' || NULL is NULL, and ABS(NULL) is NULL, because adding $5.00 to an unknown balance produces an unknown balance.

What happens when UNKNOWN enters boolean connectives NOT, AND, and OR? SQL uses the three-

valued logic formulated by logician Stephen Cole Kleene in 1938. You never need to memorize Kleene's tables by rote; you can derive every single cell in two seconds by substituting "maybe `TRUE`, maybe `FALSE`" for `UNKNOWN`:

- `NOT UNKNOWN` is `UNKNOWN`: flipping an unknown coin is still unknown.
- `FALSE AND UNKNOWN` is `FALSE`: if the first half of an `AND` is already `FALSE`, the whole conjunction is guaranteed `FALSE` regardless of what the unknown half turns out to be. By contrast, `TRUE AND UNKNOWN` is `UNKNOWN`, because the outcome still hinges on the missing fact.
- Symmetrically, `TRUE OR UNKNOWN` is `TRUE` (one `TRUE` suffices for `OR`), whereas `FALSE OR UNKNOWN` is `UNKNOWN`.

Negation		p AND q				p OR q			
p	NOT p	AND	T	U	F	OR	T	U	F
T	F	T	T	U	F	T	T	T	T
U	U	U	U	U	F	U	T	U	U
F	T	F	F	F	F	F	T	U	F

Kleene's logic breaks the classical law of the excluded middle ($p \vee \neg p$): when `fee_cents` is `NULL`, the tautology-looking filter

```
SELECT * FROM postings WHERE fee_cents = 100 OR fee_cents <> 100;
```

evaluates to `UNKNOWN OR UNKNOWN`, which is `UNKNOWN` — and `WHERE` drops the row. To test for `NULL` explicitly, SQL provides two null-safe predicates that *always* return two-valued `TRUE` or `FALSE` (never `UNKNOWN`):

- `x IS NULL` and `x IS NOT NULL`;
- `x IS DISTINCT FROM y` and `x IS NOT DISTINCT FROM y`, which treat `NULL` as equal to `NULL` and distinct from every non-`NULL` value (`NULL IS NOT DISTINCT FROM NULL` evaluates to `TRUE`).

Common trap

`WHERE` discards `UNKNOWN`, whereas `CHECK` constraints accept `UNKNOWN`. This asymmetry catches experienced engineers off guard. In `WHERE` and `HAVING`, a row survives only if the predicate evaluates to `TRUE`. In a table `CHECK` constraint, however, the SQL standard specifies that a row is rejected only if the constraint evaluates to `FALSE`. Suppose you write:

```
CREATE TABLE transfers (
  id           BIGINT PRIMARY KEY,
  amount_cents BIGINT NOT NULL,
  fee_cents    BIGINT,                  -- oops: forgot NOT NULL!
  CHECK (fee_cents >= 0 AND fee_cents <= amount_cents)
);
```

A negative fee is rejected as you would expect:

```
INSERT INTO transfers VALUES (1, 5000, -10);
ERROR:  new row for relation "transfers" violates check constraint "transfers_check"
```

But if a client inserts `(1, 5000, NULL)`, the expression `NULL >= 0 AND NULL <= 5000` evaluates to `UNKNOWN`. Because `UNKNOWN` is not `FALSE`, **PostgreSQL accepts the row**. Always mark constrained columns `NOT NULL` unless missingness is genuinely intended.

1.5.1 Why `NOT IN` returns nothing when the subquery contains `NULL`

The `AND` table explains a bug that reaches production regularly. Suppose you maintain a watchlist table `sanctioned_watchlist` with columns `(id, acct_id)`, where `acct_id` is nullable because some entries come from external name-only feeds not yet matched to an internal account ID:

$$\texttt{sanctioned_watchlist.acct_id} = \{102, \texttt{NULL}\}.$$

You want to list every clean account in `accounts` (`101`, `103`, `104`) that is *not* on the watchlist. A developer writes:

```
-- BUG: returns zero rows whenever the subquery yields even one NULL
SELECT a.id, a.owner
  FROM accounts a
 WHERE a.id NOT IN (SELECT w.acct_id FROM sanctioned_watchlist w);
```

What should this return? Accounts `101`, `103`, and `104`. What does PostgreSQL actually return? **Zero rows.**

Let us expand SQL's definition of `IN` and `NOT IN` on Ada (`a.id = 101`) to see why:

$$\begin{aligned} \texttt{101 IN (102, NULL)} &\equiv (\texttt{101 = 102}) \texttt{ OR } (\texttt{101 = NULL}) \\ &\equiv \texttt{FALSE OR UNKNOWN} = \texttt{UNKNOWN}. \end{aligned}$$

Now negate that expression with `NOT` (or equivalently apply De Morgan's law):

$$\begin{aligned} \texttt{101 NOT IN (102, NULL)} &\equiv \texttt{NOT}\big(\texttt{101 IN (102, NULL)}\big) = \texttt{NOT UNKNOWN} = \texttt{UNKNOWN} \\ &\equiv (\texttt{101 <> 102}) \texttt{ AND } (\texttt{101 <> NULL}) \\ &\equiv \texttt{TRUE AND UNKNOWN} = \texttt{UNKNOWN}. \end{aligned}$$

Because Stage 3 (`WHERE`) keeps *only* `TRUE`, `UNKNOWN` eliminates Ada (`101`), Cara (`103`), and Dan (`104`). Meanwhile, for Ben (`102`), evaluating `(102 <> 102) AND (102 <> NULL)` yields `FALSE AND UNKNOWN` = `FALSE`, eliminating Ben too. A single `NULL` anywhere inside a 10-million-row `NOT IN` subquery empties the entire output set without warning.

Now rewrite the exact same question using `NOT EXISTS`:

```
-- SAFE & IDIOMATIC ANTI-JOIN: immune to NULLs and plans as a true Hash/Index Anti-Join
SELECT a.id, a.owner
  FROM accounts a
 WHERE NOT EXISTS (
   SELECT 1
     FROM sanctioned_watchlist w
    WHERE w.acct_id = a.id
 );
```

Trace Ada (`a.id = 101`) through `NOT EXISTS`:

1. Inside the correlated subquery, `WHERE w.acct_id = 101` tests `102 = 101` (`FALSE`, dropped) and `NULL = 101` (`UNKNOWN`, *also dropped by the inner `WHERE`*).
2. The inner subquery therefore returns **zero rows**.
3. `EXISTS (empty set)` is a two-valued predicate that always returns boolean `FALSE` (never `UNKNOWN`).
4. In the outer `WHERE`, `NOT FALSE` evaluates to clean boolean `TRUE`, so Ada (`101`), Cara (`103`), and Dan (`104`) are all returned.

Better still, PostgreSQL's query planner recognizes `NOT EXISTS` directly as the relational anti-join operator $A \triangleright P$ (`Hash Anti Join` or `Nested Loop Anti Join`), whereas `NOT IN` forces awkward null-tracking `hashed SubPlan` execution paths because of its weird `NULL` semantics. Make it a reflex: **never write `NOT IN (SELECT ...)`; always write `NOT EXISTS` (or `LEFT JOIN ... WHERE right.pk IS NULL`).**

Why it works

Where SQL quietly switches from comparison equality to "grouping equality." If `NULL = NULL` is `UNKNOWN`, what happens when two rows with `tier = NULL` hit `GROUP BY tier` or `SELECT DISTINCT tier`? If SQL used `WHERE` comparison rules, `NULL`s could never group together. Instead, the SQL standard defines a second notion — *grouping distinctness* (`IS NOT DISTINCT FROM`) — for `GROUP BY`, `DISTINCT`, `UNION`, and window `PARTITION BY`: all `NULL`s land in a **single bucket**. Curiously, for `UNIQUE` indexes, standard SQL goes the opposite way: two `NULL`s are treated as distinct so a `UNIQUE(phone)` column allows multiple users without phone numbers (though PostgreSQL 15+ lets you override this via `UNIQUE NULLS NOT DISTINCT`). Finally, standard SQL aggregates (`SUM`, `AVG`, `MIN`, `MAX`, `COUNT(col)`) skip `NULL` inputs before folding — with one classic edge case: `COUNT` of zero non-null values returns `0`, whereas `SUM` of zero non-null values returns `NULL` (wrap financial sums in `COALESCE(SUM(cents), 0)`).

1.6 Window functions: aggregating without collapsing rows

Suppose an auditor asks for a statement of Cara's postings (`acct_id = 103`) showing each individual posting amount *alongside the running account balance after that posting*, plus each posting's share of Cara's total volume.

Look at why ordinary `GROUP BY` (Stage 4) hits a wall here: `GROUP BY acct_id` crushes Cara's three posting rows into a single summary row, destroying the individual `pid`, `cents`, and `posted_on` columns. Before SQL:2003 introduced window functions, engineers had to join `postings` back to a grouped subquery *and* run a quadratic $O(N^2)$ triangular self-join (`p2.pid <= p1.pid`) just to compute a running sum.

In Java you would write the running balance as a loop over postings sorted by account and date, keeping a `long balance` that you reset to zero whenever `accountId` changes and that you add each posting to before printing the row. A window function is that loop written declaratively: `PARTITION BY` is the reset rule, `ORDER BY` is the sort, and the frame says which earlier rows feed the running total.

Window functions (evaluated at Stage 6 of the logical pipeline) solve this in a single $O(N \log N)$ sort pass (or $O(N)$ streaming pass over an index). Every window call attaches an `OVER (...)` clause with three optional parts:

```
function_name(args) OVER (
  PARTITION BY acct_id                                  -- 1. Independent sub-tables
  ORDER BY posted_on ASC, pid ASC                       -- 2. Sort order inside partition
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW      -- 3. Sliding frame bounds
)
```

1. `PARTITION BY`: Splits Stage 5's surviving rows into independent horizontal groups (just like `GROUP BY`), except *zero rows are collapsed*. Window calculations never cross a partition boundary: Ada's running balance resets cleanly when the scan advances to Ben's partition.
2. `ORDER BY`: Defines the sequence of rows within each partition for ranking functions (`ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`), offset navigation (`LAG(col, 1)`, `LEAD(col, 1)`), and cumulative frames.
3. **Frame specification (`ROWS`, `RANGE`, or `GROUPS`)**: For each current row r, defines the exact sliding subset of rows inside r's partition fed into the aggregate.

1.6.1 The silent `RANGE` versus `ROWS` trap on duplicate timestamps

The default frame produces a wrong running balance on Cara's postings (`pid = 4, 5, 6`) in our miniature dataset. Look back at Cara's three rows in P:

- `pid = 4` on `2026-10-01`: $+15000$ cents;
- `pid = 5` on `2026-10-03`: -3000 cents;
- `pid = 6` on `2026-10-03`: $+2000$ cents.

Notice that postings `5` and `6` occurred on the *exact same date* (`2026-10-03`). Now suppose an engineer writes a running balance query ordering only by `posted_on`, omitting an explicit frame clause:

```
SELECT pid, posted_on, cents,
       -- Subtle trap: omits ROWS/RANGE, so SQL defaults to RANGE!
       SUM(cents) OVER (
         PARTITION BY acct_id
         ORDER BY posted_on
       ) AS range_running_bal,
       -- Explicit physical row-by-row running balance with deterministic tie-breaker:
       SUM(cents) OVER (
         PARTITION BY acct_id
         ORDER BY posted_on, pid
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS exact_running_bal
  FROM postings
 WHERE acct_id = 103
 ORDER BY posted_on, pid;
```

What is SQL's default window frame when you omit the `ROWS/RANGE` clause?

- If `OVER (...)` has **no** `ORDER BY`, the default frame spans the **entire partition** (`RANGE` from `UNBOUNDED PRECEDING` to `UNBOUNDED FOLLOWING`), so `SUM(cents) OVER (PARTITION BY acct_id)` gives Cara's grand total (14000) on all three rows.
- If `OVER (...)` **includes an** `ORDER BY`, the default frame silently switches to `RANGE` from `UNBOUNDED PRECEDING` to `CURRENT ROW`.

And what does `RANGE ... CURRENT ROW` mean? In `RANGE` mode, `CURRENT ROW` does *not* stop at the physical row currently being visited; it includes **all peer rows tied on the** `ORDER BY` **key**. Because `pid = 5` and `pid = 6` tie on `posted_on = '2026-10-03'`, `range_running_bal` sums $\{+15000, -3000, +2000\} = 14000$ for *both* row 5 and row 6 — Cara's intermediate balance of 12000 after posting 5 completely disappears.

pid	posted_on	cents	Default RANGE	Explicit ROWS
4	2026-10-01	+15000	15000	15000
5	2026-10-03	-3000	14000 (skips 12000!)	12000 (4..5 only)
6	2026-10-03	+2000	14000	14000 (4..6)

Tied `posted_on` peers merged by `RANGE`, separated by `ROWS`

Figure 1.4: Why explicit `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` plus a unique tie-breaker (`pid`) matters for financial running balances: default `RANGE` merges tied timestamps into a single peer step, hiding the intermediate balance of `12000` after posting `5`.

Common trap

The exact same default-frame rule explains why `LAST_VALUE(cents) OVER (PARTITION BY acct_id ORDER BY pid)` surprises most people the first time they run it: instead of returning the final posting of the account, it returns *each row's own* `cents`. Why? Because the default frame stops at `CURRENT ROW`, so the "last value" inside `UNBOUNDED PRECEDING .. CURRENT ROW` is always the current row itself. Whenever you use `LAST_VALUE` (or `NTH_VALUE`), widen the frame explicitly:

```
LAST_VALUE(cents) OVER (
  PARTITION BY acct_id ORDER BY pid
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
)
```

1.7 CTEs and fixpoint iteration with `WITH RECURSIVE`

Pure relational algebra $(\sigma, \pi, \times, \cup, -)$ has a celebrated theoretical boundary proved by Alfred Aho and Jeffrey Ullman in 1979: **relational algebra cannot express transitive closure**. If a table `account_hierarchy(id, parent_id, name)` stores an arbitrarily deep tree of corporate sub-accounts (or a graph of wire-transfer hops between accounts), no fixed number of self-joins $H \bowtie H \bowtie \cdots \bowtie H$ can find all descendants when the depth is not known in advance.

SQL bridges that expressiveness gap with **Common Table Expressions (CTEs)**, written with `WITH` for modular non-recursive subqueries and `WITH RECURSIVE` for iterative fixpoint computation.

1.7.1 Non-recursive `WITH`, planner inlining, and data-modifying CTEs

In ordinary queries, `WITH step1 AS (...), step2 AS (...) SELECT ...` lets you write complex pipelines top-to-bottom in the exact order you think about them, rather than burying subqueries four levels deep inside `FROM`.

Under the hood

How PostgreSQL plans `WITH` (`MATERIALIZED` vs. inlined). Before PostgreSQL 12, every CTE acted as a strict *optimization fence*: PostgreSQL materialized the full CTE output into an in-memory/temp-file `Tuplestore` before running the outer query, preventing outer `WHERE` filters from pushing down inside the CTE. Starting in PostgreSQL 12, a non-recursive, side-effect-free CTE referenced at most once is **inlined automatically** into the outer query tree (just like a view) so predicates push straight down to indexes. When you *want* single-evaluation materialization (for instance, calling an expensive function once and joining its result twice), write `WITH summary AS MATERIALIZED (...)`; to force pushdown even when referenced multiple times, write `WITH summary AS NOT MATERIALIZED (...)`.
PostgreSQL also supports **data-modifying CTEs** (`INSERT`/`UPDATE`/`DELETE ... RETURNING` inside `WITH`), all executing atomically under a single statement snapshot — ideal for moving archived rows from an active queue into an audit log in one round-trip:

```
WITH archived AS (
  DELETE FROM outbox_events
   WHERE processed_at < NOW() - INTERVAL '30 days'
  RETURNING id, payload, processed_at
)
INSERT INTO outbox_events_archive (id, payload, processed_at)
SELECT id, payload, processed_at FROM archived;
```

1.7.2 How `WITH RECURSIVE` actually executes: two tables in a loop

The keyword `RECURSIVE` is slightly misleading to programmers trained in Java or C: PostgreSQL does *not* push stack frames onto a call stack. Instead, PostgreSQL's `RecursiveUnion` executor node evaluates a **bottom-up breadth-first fixpoint loop** using two temporary buffers: a **Result Table** R and a **Working Table** W. If you have written breadth-first search in Java 6 with an `ArrayDeque<Long>` frontier and a `HashSet<Long>` of visited nodes, you already know this loop: W is the frontier, R is everything found so far, and `UNION` (as opposed to `UNION ALL`) plays the role of the visited set.

Every `WITH RECURSIVE` definition has the shape B `UNION [ALL]` $F(W)$, where B is the non-recursive *base query* (anchor) and $F(W)$ is the *recursive step* referencing the CTE's own name:

1. **Step 0 (Seed):** Evaluate the base query B once. Copy its rows into both the Result Table R and the Working Table W.
2. **Loop while W is non-empty:**
   (a) Evaluate the recursive query $F(W)$, substituting the *current Working Table* W (not all of R!) wherever the CTE's self-reference appears. Put the newly produced rows into an Intermediate Table I.

(b) If the connector is `UNION` (rather than `UNION ALL`), discard from I any row already present in R (and deduplicate I itself).

(c) Append all surviving rows of I to R, replace $W \leftarrow I$, and repeat.

3. **Terminate:** As soon as W becomes empty ($I = \emptyset$), return R. Mathematically, $R = \bigcup_{k \geq 0} F^k(B)$ is the *least fixpoint* of the monotone relational operator $T(X) = B \cup F(X)$.

Let us trace a concrete four-node corporate ledger hierarchy starting from root treasury account `1` (`'Treasury'`), with children `10` (`'US Ops'`) and `20` (`'EU Ops'`), and grandchild `101` (`'US Payroll'` under `10`):

```
1  WITH RECURSIVE acct_tree AS (
2    -- Base term B: start at the root Treasury account
3    SELECT id, parent_id, name, 0 AS depth, ARRAY[id] AS path
4      FROM ledger_accounts
5     WHERE id = 1
6    UNION ALL
7    -- Iterative step F(W): join current Working Table W to child rows
8    SELECT c.id, c.parent_id, c.name, w.depth + 1, w.path || c.id
9      FROM acct_tree w
10     JOIN ledger_accounts c ON c.parent_id = w.id
11    WHERE NOT (c.id = ANY(w.path))    -- cycle guard!
12 )
13 SELECT id, depth, name, path FROM acct_tree ORDER BY path;
```

Watch the contents of W and R across each iteration:

- **Iteration 0 (Base B):** $W_0 = \{(1, \texttt{NULL}, \texttt{'Treasury'}, 0, [1])\}$. $R = W_0$.
- **Iteration 1 ($F(W_0)$):** Joining W_0 (`id = 1`) to `ledger_accounts.parent_id` finds children `10` and `20`. Now $W_1 = \{(10, \ldots, 1, [1, 10]),\ (20, \ldots, 1, [1, 20])\}$, and R holds $\{1, 10, 20\}$.
- **Iteration 2 ($F(W_1)$):** Joining W_1 (`10` and `20`) finds `101` under `10` and nothing under `20`. Now $W_2 = \{(101, \ldots, 2, [1, 10, 101])\}$, and R holds $\{1, 10, 20, 101\}$.
- **Iteration 3 ($F(W_2)$):** Joining W_2 (`101`) finds zero children. $W_3 = \emptyset$, the loop halts, and PostgreSQL outputs all four rows of R!

Line 11, `WHERE NOT (c.id = ANY(w.path))`, is the cycle guard. What if corrupt upstream data accidentally set `parent_id = 101` on root account `1`, creating a cycle $1 \to 10 \to 101 \to 1$? Without a cycle guard under `UNION ALL` (where `depth` increments every turn, making every row distinct even under `UNION`), W_k would never empty. PostgreSQL does not detect this as an error: the query keeps appending rows to temporary files until `statement_timeout` cancels it (`ERROR: canceling statement due to statement timeout`) or the disk fills. Tracking `ARRAY[id] AS path` — or using PostgreSQL 14+'s built-in SQL standard clause `CYCLE id SET is_cycle USING path` right after the `UNION ALL` body — guarantees termination on arbitrary directed graphs.

Connection

How Rust's `SQLx` turns SQL's three-valued `NULL`s into compile-time types. In the companion *Programming Languages From the Ground Up* reader, we connect a Rust financial backend to PostgreSQL using `sqlx::query`. Rather than silently deserializing SQL `NULL` into a null pointer reference (Tony Hoare's "billion-dollar mistake" in Java 6), `SQLx` asks Post-

greSQL's own query analyzer at *Rust compile time* which output columns of your `SELECT` pipeline can ever be `NULL`. Non-null columns become plain `i64` or `String`; columns from the nullable right side of a `LEFT JOIN` or an unguarded `SUM(cents)` automatically become `Option<i64>`, forcing your Rust code to handle missingness before the binary will even compile.

Try it in PostgreSQL

Paste this self-contained session into `psql` to verify all three subtleties from this chapter — `LEFT JOIN` filter placement, the `NOT IN NULL` wipeout versus `NOT EXISTS`, and `RANGE` versus `ROWS` window running balances — on our exact miniature ledger:

```
CREATE TEMP TABLE accounts (id BIGINT PRIMARY KEY, owner TEXT NOT NULL, tier TEXT
    NOT NULL);
CREATE TEMP TABLE postings (pid BIGINT PRIMARY KEY, acct_id BIGINT REFERENCES
    accounts(id),
                            cents BIGINT NOT NULL, posted_on DATE NOT NULL);
CREATE TEMP TABLE watchlist (id INT PRIMARY KEY, acct_id BIGINT);

INSERT INTO accounts VALUES (101,'Ada','pro'), (102,'Ben','free'), (103,'Cara','pro'
    ), (104,'Dan','pro');
INSERT INTO postings VALUES
  (1,101, 10000,'2026-10-01'), (2,101,-2500,'2026-10-02'), (3,102,4000,'2026-10-02')
    ,
  (4,103, 15000,'2026-10-01'), (5,103,-3000,'2026-10-03'), (6,103,2000,'2026-10-03')
    ;
INSERT INTO watchlist VALUES (1, 102), (2, NULL);

-- 1. Watch NOT IN return 0 rows while NOT EXISTS returns 101, 103, 104:
SELECT id FROM accounts WHERE id NOT IN (SELECT acct_id FROM watchlist);
SELECT id FROM accounts a WHERE NOT EXISTS (SELECT 1 FROM watchlist w WHERE w.
    acct_id = a.id);

-- 2. Compare default RANGE (14000 on pid 5!) vs explicit ROWS (12000 then 14000):
SELECT pid, posted_on, cents,
       SUM(cents) OVER (PARTITION BY acct_id ORDER BY posted_on) AS range_bal,
       SUM(cents) OVER (PARTITION BY acct_id ORDER BY posted_on, pid
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
    rows_bal
  FROM postings WHERE acct_id = 103 ORDER BY posted_on, pid;
```

1.8 Exercises

Exercise 1.1 (Bag multiplicities by hand)**.** Suppose single-column bag R contains `'USD'`$\times 3$, `'EUR'`$\times 2$, and `'GBP'` $\times 1$, while bag S contains `'USD'` $\times 1$, `'EUR'` $\times 4$, and `'JPY'` $\times 2$. Compute the exact row count and per-currency multiplicity of: (a) `R UNION ALL S`, (b) `R UNION S`, (c) `R INTERSECT ALL S`, (d) `R EXCEPT ALL S`, and (e) `R EXCEPT S`.

Exercise 1.2 (Relational algebra pushdown cost)**.** Relation A has 200,000 account tuples (of which

5,000 have `country = 'NG'`) and relation P has 40,000,000 posting tuples (of which 80,000 occurred today, 2,000 belonging to `'NG'` accounts). Count how many tuple pairs enter the join comparison in:

$$\text{(a)}\quad \sigma_{A.\texttt{country}=\texttt{'NG'} \,\wedge\, P.\texttt{day}=\texttt{today}}(A \times P),$$
$$\text{(b)}\quad \big(\sigma_{\texttt{country}=\texttt{'NG'}}(A)\big) \;\bowtie_{A.\texttt{id}=P.\texttt{acct_id}}\; \big(\sigma_{\texttt{day}=\texttt{today}}(P)\big),$$

assuming the equi-join builds an in-memory hash table on the filtered A rows and probes once per filtered P row.

Exercise 1.3 (Diagnosing the ten-stage pipeline)**.** For each SQL query below over our `accounts` and `postings` tables, state whether PostgreSQL executes it or rejects it with an error; if rejected, identify the exact two stage numbers from Figure 1.3 that are out of order, and rewrite the query cleanly:

```
-- (1)
SELECT acct_id, COUNT(*) AS cnt FROM postings GROUP BY acct_id HAVING cnt >= 2;
-- (2)
SELECT pid, cents, LAG(cents) OVER (PARTITION BY acct_id ORDER BY pid) AS prev
  FROM postings WHERE prev < 0;
-- (3)
SELECT DISTINCT acct_id FROM postings ORDER BY posted_on DESC;
```

Exercise 1.4 (Outer join predicate placement)**.** Using our four-row `accounts` table A and six-row `postings` table P, write down the exact set of rows returned by:

```
SELECT a.id, COUNT(*), COUNT(p.pid), COALESCE(SUM(p.cents), 0)
  FROM accounts a
  LEFT JOIN postings p ON p.acct_id = a.id AND p.cents < 0
 GROUP BY a.id
 ORDER BY a.id;
```

Explain why `COUNT(*)` and `COUNT(p.pid)` disagree on two of the four accounts.

Exercise 1.5 (Three-valued truth table proof)**.** In classical two-valued boolean algebra, De Morgan's law $\neg(p \vee q) \equiv (\neg p) \wedge (\neg q)$ and the distributive law $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$ hold identically, as does $p \vee \neg p \equiv \texttt{TRUE}$. Check all three identities across Kleene's three-valued truth tables $\{\texttt{T}, \texttt{U}, \texttt{F}\}$: prove that De Morgan's law and distributivity *still hold identically* for every combination of $(\texttt{T}, \texttt{U}, \texttt{F})$, whereas $p \vee \neg p$ fails specifically when $p = \texttt{U}$.

Exercise 1.6 (Designing a leak-proof `CHECK` constraint)**.** A payments table `refunds` with columns `(id BIGINT PRIMARY KEY, charge_cents BIGINT NOT NULL, refund_cents BIGINT, reason_code TEXT)` must enforce two business rules: (1) if `refund_cents` is present, it must satisfy $0 <$ `refund_cents` $\leq$ `charge_cents` *and* `reason_code` must be non-`NULL`; (2) if `refund_cents` is `NULL`, `reason_code` must also be `NULL`. Write a single `CHECK (...)` expression that cannot be bypassed by `UNKNOWN`, and verify all four null/non-null combinations of `(refund_cents, reason_code)`.

Exercise 1.7 (Sliding 7-day rolling volume vs. 7-row rolling volume)**.** An account has postings on October 1 (+100), October 2 (+200), October 2 (+50), and October 10 (+300), with zero postings on October 3 through 9. Contrast what PostgreSQL computes on each of the four rows for:

```
-- Window 1: physical row count frame
SUM(cents) OVER (ORDER BY posted_on, pid ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
-- Window 2: calendar interval range frame (PostgreSQL 11+!)
SUM(cents) OVER (ORDER BY posted_on RANGE BETWEEN INTERVAL '7 days' PRECEDING AND CURRENT
    ROW)
```

When do you want `ROWS`, and when is `RANGE BETWEEN INTERVAL ...` genuinely the right tool?

Exercise 1.8 (Tracing `WITH RECURSIVE` on a cyclic transfer graph)**.** Suppose `wires(from_acct, to_acct)` contains edges $(1 \to 2)$, $(2 \to 3)$, $(3 \to 1)$, and $(2 \to 4)$. Trace the contents of the Working Table W_k and Result Table R_k at steps $k = 0, 1, 2, 3$ starting from `from_acct = 1` for: (a) a query using `UNION` projecting only `to_acct`, and (b) a query using `UNION ALL` projecting `(to_acct, depth)` *without* a cycle guard. Why does (a) terminate while (b) runs forever, and how does adding `CYCLE to_acct SET is_cycle USING path` stop (b) cleanly?

In this chapter we stayed deliberately above Codd's data-independence line: relations were sets and bags of abstract tuples, and PostgreSQL promised to evaluate our ten-stage pipeline faithfully. Now it is time to cross below that line, open `$PGDATA/base` on an actual Linux filesystem, and look inside the 8,192-byte slotted pages and `shared_buffers` frames where every tuple physically lives.

Chapter 2

Inside an 8 KB Heap Page and the Buffer Pool

In Chapter 1, a table was a clean mathematical object: a relation (or a SQL bag of rows) governed by set algebra and three-valued logic. When you write `INSERT INTO accounts VALUES (1001, 'Ada', 50000, true)`, SQL lets you picture a neat rectangular spreadsheet gaining one horizontal line.

The Linux kernel knows nothing about relations, columns, or `BIGINT`s. An NVMe solid-state drive only knows how to read and write flat arrays of bytes at numeric block addresses, and C pointers in RAM only know byte offsets and CPU alignment boundaries. Bridging that gap is the job of PostgreSQL's storage manager and buffer manager. Every design choice we will meet later in this book—why B-tree indexes store 6-byte tuple identifiers (Chapter 3), why Write-Ahead Logging stamps an 8-byte log sequence number at byte 0 of every block (Chapter 4), and why `UPDATE` writes a brand-new row version instead of overwriting bytes in place (Chapter 5)—is physically anchored inside an **8,192-byte (8 KB) slotted heap page**.

Focus

By the end of this chapter, you will be able to:

1. Locate the exact Linux files backing any PostgreSQL table under `$PGDATA/base/`, explain why `pg_class.oid` and `relfilenode` diverge after `TRUNCATE` or `VACUUM FULL`, and trace how the `_fsm` and `_vm` companion forks accelerate inserts, index-only scans, and vacuuming.
2. Read an 8,192-byte slotted heap page byte by byte (`PageHeaderData`, downward-growing `ItemIdData` line pointers, and upward-growing heap tuples) and prove why slotted indirection lets PostgreSQL defragment a page without rewriting external indexes.
3. Compute exact tuple sizes down to a single byte—accounting for the 23-byte `HeapTupleHeaderData`, the optional null bitmap, `MAXALIGN` padding, and CPU type alignment—and reorder columns to save hundreds of megabytes on large tables.
4. Explain `varlena` 1-byte versus 4-byte headers and trace how TOAST compresses and slices oversized values into `pg_toast` chunks.
5. Trace how `shared_buffers` maps block tags to frames, why buffer pins differ from

content locks, how the clock-sweep hand chooses eviction victims, and how a 256 KB ring buffer stops a 500 GB sequential scan from evicting hot OLTP data.

2.1 Where a Table Actually Lives on Disk

Let us create a small ledger table in `psql` and ask PostgreSQL a direct physical question: *which file on the Linux filesystem holds these rows?*

```
CREATE TABLE accounts (
    account_id    BIGINT  PRIMARY KEY,
    owner_name    TEXT    NOT NULL,
    balance_cents BIGINT  NOT NULL,
    is_active     BOOLEAN NOT NULL
);

SELECT oid, relname, relfilenode
FROM pg_class
WHERE relname = 'accounts';
--   oid  | relname  | relfilenode
-- -------+----------+-------------
--  24576 | accounts |       24576

SELECT pg_relation_filepath('accounts');
--  pg_relation_filepath
-- ----------------------
--  base/16384/24576
```

Relative to your PostgreSQL data directory (`$PGDATA`, such as `/var/lib/postgresql/17/main`), the table `accounts` is an ordinary regular file at `$PGDATA/base/16384/24576`. Every piece of that path has a specific catalog meaning:

- `base/` holds all default-tablespace databases (the `pg_default` tablespace). Shared cluster-wide catalogs like `pg_database` and `pg_authid` live next door in `global/`.
- `16384` is the object identifier (`oid`) of the current database in `pg_database` (`SELECT oid, datname FROM pg_database`). Every database inside the cluster gets its own subdirectory under `base/`.
- `24576` is `pg_class.relfilenode`—not `pg_class.oid`. Right after `CREATE TABLE`, the two integers happen to match, which tempts many engineers into thinking they are the same thing. Watch what happens the moment we truncate the table.

2.1.1 Why `oid` and `relfilenode` Split: Transactional File Swaps

Suppose a transaction runs `TRUNCATE accounts;` and then, three statements later, hits an error and executes `ROLLBACK;`. Unlike many older database engines, PostgreSQL makes `TRUNCATE`, `CLUSTER`, and `VACUUM FULL` **fully transactional**.

Think about what would go wrong if `TRUNCATE` simply called the Linux `ftruncate(fd, 0)` system

call on file `24576` in place. The instant the kernel zeroes the file's length, the old 8 KB blocks are gone. If the transaction rolls back ten milliseconds later, there is nothing left to restore.

Intuition

PostgreSQL solves transactional table rewrites by separating **logical identity** (`pg_class.oid`) from **physical storage file** (`pg_class.relfilenode`):

- Views, foreign keys, triggers, and privilege grants all point to `oid = 24576`. That number never changes for the lifetime of the table.
- When you run `TRUNCATE accounts` (or rewrite the table with `VACUUM FULL` or `CLUSTER`), PostgreSQL allocates a fresh file number—say `24589`—creates a brand-new file `base/16384/24589`, and updates the `pg_class` catalog row for `oid = 24576` so its `relfilenode` column now reads `24589`.

If your transaction executes `ROLLBACK`, the catalog update becomes invisible, PostgreSQL unlinks the unused file `24589`, and file `24576` is still sitting intact on disk without a single byte harmed. If your transaction executes `COMMIT`, the catalog update becomes visible atomically, new queries open `24589`, and PostgreSQL unlinks the old file `24576` as part of post-commit cleanup.

Let us verify this split directly in SQL:

```
BEGIN;
TRUNCATE accounts;

SELECT oid, relfilenode, pg_relation_filepath('accounts') AS path
FROM pg_class
WHERE relname = 'accounts';
--   oid  | relfilenode |       path
-- -------+-------------+------------------
--  24576 |       24589 | base/16384/24589

ROLLBACK;

SELECT oid, relfilenode, pg_relation_filepath('accounts') AS path
FROM pg_class
WHERE relname = 'accounts';
--   oid  | relfilenode |       path
-- -------+-------------+------------------
--  24576 |       24576 | base/16384/24576
```

Inside the C source code (modern PostgreSQL 16+ in `src/include/storage/relfilelocator.h`), the physical address of a relation on disk is a three-integer triple called `RelFileLocator`: `(spcOid, dbOid, relNumber)`. Whenever you inspect files on disk after a maintenance rewrite, always query `pg_relation_filepath(oid)` rather than grepping for `pg_class.oid`.

2.1.2 1 GB Segment Splitting

A busy production table easily grows past 100 GB, yet if you run `ls -lh $PGDATA/base/16384/` you will never see a file larger than **1 GiB** (1,073,741,824 bytes).

PostgreSQL slices every relation fork into fixed segments of `RELSEG_SIZE` = 131,072 blocks of 8,192 bytes (131,072 × 8,192 = 1,073,741,824 bytes = 1 GiB). Originally introduced to avoid 2 GB signed `off_t` limits on 32-bit Unix filesystems, segment splitting remains valuable on 64-bit Linux because incremental backup tools and parallel file copiers can process independent 1 GiB files cleanly.

For a 2.6 GiB table with `relfilenode = 24589`, the directory contains three segment files:

- `24589` (blocks 0 … 131,071, bytes 0 … 1 GiB − 1)
- `24589.1` (blocks 131,072 … 262,143, the second GiB)
- `24589.2` (blocks 262,144 … 340,786, the remaining 0.6 GiB)

Locating any logical 32-bit `BlockNumber` B on disk is pure integer division and modulo arithmetic:

$$\text{segment_index} = \left\lfloor \frac{B}{131{,}072} \right\rfloor, \qquad \text{byte_offset_in_segment} = (B \bmod 131{,}072) \times 8{,}192.$$

For instance, block B = 300,000 lives in segment $\lfloor 300{,}000/131{,}072 \rfloor = 2$ (`24589.2`) at local block 300,000 − 262,144 = 37,856, starting at byte 37,856 × 8,192 = 310,116,352. Because `BlockNumber` is a 32-bit unsigned integer (0 … 4,294,967,294, with `0xFFFFFFFF` reserved as `InvalidBlockNumber`), a single standard PostgreSQL table can hold up to $(2^{32} - 1) \times 8$ KiB ≈ **32** TiB before requiring partitioning.

2.1.3 Companion Forks: Free Space Map (_fsm) and Visibility Map (_vm)

Look again in `$PGDATA/base/16384/` after inserting rows and running `VACUUM accounts;`, and two sibling files appear alongside `24589`: `24589_fsm` and `24589_vm`. In PostgreSQL terminology (`ForkNumber` in `src/include/common/relpath.h`), one relation owns up to four physical **forks**:

1. **Main Fork (`MAIN_FORKNUM = 0`, file `24589`)**: Holds the 8 KB heap data pages (or index pages, if the relation is an index).
2. **Free Space Map (`FSM_FORKNUM = 1`, file `24589_fsm`)**: Suppose `accounts` has 1,000,000 pages (≈ 8 GB) with deleted gaps scattered throughout, and a backend wants to `INSERT` a 110-byte tuple. Checking heap pages one by one to find 110 free bytes would turn every `INSERT` into a linear scan.

   Instead, the Free Space Map quantizes each heap page's largest contiguous free region into **a single byte** (`0..255`) measured in 32-byte chunks ($\lfloor \text{free_bytes}/32 \rfloor$, since 8,192/256 = 32). A leaf byte of `4` guarantees at least 4 × 32 = 128 free bytes on that heap page. Inside each 8 KB `_fsm` block, PostgreSQL arranges roughly 4,000 leaf bytes at the bottom of an **array-backed binary max-heap tree**, where every internal node stores max(left_child, right_child):

   - If the root node at the top of an `_fsm` page reads `100` (3,200 bytes), at least one heap page underneath has 3,200+ free bytes.
   - Finding a page for our 110-byte tuple ($\lceil 110/32 \rceil = 4$ units) walks downward from the root in $O(\log N)$ byte comparisons, descending into whichever child has value ≥ 4. Across a multi-terabyte table, stacked upper-level `_fsm` pages form a 3-level tree of binary trees, locating an eligible target block in at most **three 8 KB buffer lookups**.
   - Notice that `_fsm` is an *advisory* hint structure, not ground truth: if a concurrent backend fills the page a microsecond earlier, the inserter inspects the real heap page header, updates the `_fsm` leaf downward, and asks the tree for the next candidate.

3. **Visibility Map (`VM_FORKNUM = 2`, file `24589_vm`)**: Stores just **2 bits per heap page**:
   - Bit 0 (`VISIBILITYMAP_ALL_VISIBLE = 0x01`): Every tuple on that heap page is committed and visible to *all* current and future transactions, with zero dead tuples awaiting cleanup.
   - Bit 1 (`VISIBILITYMAP_ALL_FROZEN = 0x02`): Every tuple on that heap page has also been frozen against 32-bit transaction ID wraparound (Chapter 5).

   Four heap pages pack into a single byte of the Visibility Map. After subtracting the 24-byte page header (8,192−24 = 8,168 payload bytes), **one 8 KB `_vm` page tracks** 8,168×4 = 32,672 **heap pages**—roughly **255 MiB** of table data. Those two tiny bits unlock two major production speedups: **Index-Only Scans** (Chapter 3) consult the `_vm` page in RAM and skip reading the heap block entirely whenever `ALL_VISIBLE` is `1`; and **aggressive `VACUUM`** (Chapter 5) skips scanning any heap block whose `ALL_FROZEN` bit is already `1`. Any `INSERT`, `UPDATE`, or `DELETE` on heap block B clears both bits for block B.

4. **Initialization Fork (`INIT_FORKNUM = 3`, file `24589_init`)**: Created only for `UNLOGGED` tables and their indexes. It holds an empty template image of the relation; if the server crashes, PostgreSQL wipes the corrupted main fork during startup and copies `_init` over it to reset the unlogged table to a clean empty state (Chapter 4).

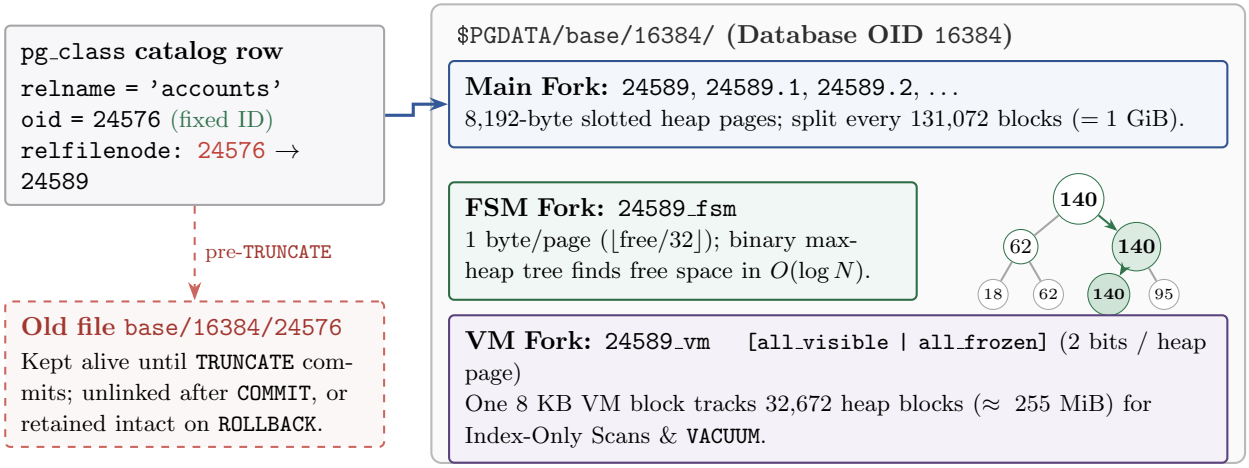


Figure 2.1: Physical file layout under `$PGDATA/base/16384/`. Transactional rewrites (`TRUNCATE`, `CLUSTER`, `VACUUM FULL`) preserve `pg_class.oid` while swapping `relfilenode` to a new file family (`24589`, `24589_fsm`, `24589_vm`).

2.2 Anatomy of an 8,192-Byte Slotted Heap Page

Open `base/16384/24576` in a hex editor and slice it every `BLCKSZ` = 8,192 bytes. Each 8,192-byte chunk (offsets 0 . . . 8,191) is a self-contained **slotted page**.

Walking question

Try designing the page layout yourself first. You have an 8,192-byte array in RAM, and SQL rows are **variable-length** (`'Ada'` is 3 characters; `'Grace Hopper'` is 12; another row has a

400-byte note). Worse, rows get deleted out of order, leaving variable-sized holes in the middle of the page, while three separate B-tree indexes outside this page hold pointers to your live rows.
If a B-tree index stored a row's raw byte offset—say `(block 0, byte 8072)`—what happens the moment you slide rows together to reclaim a deleted 64-byte gap? Every moved row's byte offset changes, forcing you to hunt down and rewrite entries across every index on the table. How can we pack variable-length tuples into 8,192 bytes, coalesce free space into one contiguous region, *and* move tuples inside the page without invalidating a single external pointer?

The slotted-page architecture (pioneered in System R and refined across four decades of Postgres) solves all three problems at once with two tricks:

1. **Two-way growth toward a central free gap**: Small fixed-size 4-byte **line pointers** (`ItemIdData`) grow *downward* from byte 24 toward higher addresses, while variable-length **heap tuples** grow *upward* from byte 8,192 toward lower addresses. All unallocated space sits in a single contiguous gap `[pd_lower .. pd_upper)` in the middle—neither side ever wastes space guessing a fixed boundary in advance.

2. **One level of slot indirection**: External B-tree indexes never store a tuple's byte offset. They store `ctid = (block_number, slot_number)`, where `slot_number` $(1, 2, 3, \dots)$ indexes into the 4-byte line pointer array at the top of the page. When PostgreSQL compacts a fragmented page, it slides tuple bytes upward in RAM and updates the 15-bit byte offset *inside that page's own 4-byte slot*—leaving `(block_number, slot_number)` untouched everywhere else in the database.

2.2.1 Byte-Offset Tour of `PageHeaderData` (Bytes `0..24`)

Every 8 KB page—whether heap, B-tree, GIN, GiST, FSM, or VM—begins with the exact 24-byte C struct `PageHeaderData` defined in `src/include/storage/bufpage.h`:

```
/* Simplified from src/include/storage/bufpage.h */
typedef struct PageHeaderData
{
    /* [0..8)   LSN of last WAL record modifying this page */
    PageXLogRecPtr pd_lsn;

    /* [8..10)  16-bit page checksum (if data_checksums is enabled) */
    uint16         pd_checksum;

    /* [10..12) Flag bits: PD_HAS_FREE_LINES, PD_PAGE_FULL, PD_ALL_VISIBLE */
    uint16         pd_flags;

    /* [12..14) Byte offset to START of free space (end of line pointers) */
    LocationIndex  pd_lower;

    /* [14..16) Byte offset to END of free space (start of lowest tuple) */
    LocationIndex  pd_upper;

    /* [16..18) Byte offset to start of special access-method space */
```

```
    LocationIndex  pd_special;

    /* [18..20) Page size (upper 8 bits) + layout version (lower 8 bits) */
    uint16         pd_pagesize_version;

    /* [20..24) Oldest unpruned XMAX on page (hint for in-page pruning) */
    TransactionId  pd_prune_xid;

    /* [24..)   1-indexed array of 4-byte line pointers growing downward */
    ItemIdData     pd_linp[FLEXIBLE_ARRAY_MEMBER];
} PageHeaderData;
```

Let us inspect every field and verify that the byte offsets sum to exactly **24 bytes**:

- **Bytes** `[0..8)` — `pd_lsn` **(8 bytes)**: Stores the 64-bit Write-Ahead Log Sequence Number (`XLogRecPtr`) of the most recent WAL record that mutated this page. (In C, `PageXLogRecPtr` stores the 64-bit integer as two consecutive `uint32` halves so the struct remains 4-byte aligned even on legacy 32-bit platforms.) In Chapter 4 we will see how the buffer manager compares `pd_lsn` against the durable WAL flush pointer before allowing any dirty page onto the NVMe drive.
- **Bytes** `[8..10)` — `pd_checksum` **(2 bytes)**: When the cluster is initialized with `initdb --data-checksums` (the default in PostgreSQL 18, and strongly recommended on all earlier versions), PostgreSQL computes a fast SIMD-vectorized FNV-1a checksum over the 8,192 bytes (temporarily zeroing `pd_checksum` and mixing in the `BlockNumber` so a page accidentally written to the wrong disk sector is caught too) right before flushing the page out of `shared_buffers`. If silent storage corruption flips a bit on disk, the next read immediately raises `ERROR: invalid page in block`.
- **Bytes** `[10..12)` — `pd_flags` **(2 bytes)**: Three bitflags live here:
  - `PD_HAS_FREE_LINES (0x0001)`: At least one existing `ItemIdData` slot is marked `LP_UNUSED`, so an `INSERT` can recycle an old slot number instead of extending `pd_lower`.
  - `PD_PAGE_FULL (0x0002)`: Not enough free space remained for a recent update, warning future updates not to waste effort trying in-page placement until pruning runs.
  - `PD_ALL_VISIBLE (0x0004)`: Page-local mirror of the Visibility Map's `ALL_VISIBLE` bit.
- **Bytes** `[12..14)` **and** `[14..16)` — `pd_lower` **and** `pd_upper` **(2 bytes each)**: Two unsigned 16-bit byte offsets (`LocationIndex`) framing the central free gap. On a freshly initialized empty heap page, `pd_lower` = 24 and `pd_upper` = 8,192, leaving 8,192 − 24 = **8,168** contiguous free bytes. Checking whether a new tuple fits requires one subtraction: `pd_upper` − `pd_lower`.
- **Bytes** `[16..18)` — `pd_special` **(2 bytes)**: Index access methods need a small trailer at the very end of each 8 KB page for metadata like left/right sibling page pointers in a B-link tree (`BTPageOpaqueData`, 16 bytes, setting `pd_special` = 8,176 in Chapter 3). Heap pages need no trailer at all, so on every heap page `pd_special` = 8,192.
- **Bytes** `[18..20)` — `pd_pagesize_version` **(2 bytes)**: Packs the compile-time block size and page layout version: `8192 | 4 = 0x2004` = 8,196 (version 4 has been PostgreSQL's page format since PostgreSQL 8.3).

- **Bytes `[20..24)` — `pd_prune_xid` (4 bytes)**: Records the oldest deleting/updating transaction ID (`xmax`) among dead or updated tuples on this page. Before scanning the page, a reader can compare `pd_prune_xid` against its snapshot horizon; if `pd_prune_xid` has already finished behind every active transaction, the backend knows immediately that a quick in-page defragmentation pass (*HOT pruning*, Chapter 5) will reclaim dead space.

2.2.2 The Line Pointer Array: `ItemIdData` (4 Bytes per Slot)

Starting right at byte 24 is `pd_linp[]`, an array of 32-bit (**4-byte**) line pointers defined in `src/include/storage/itemid.h`. Note a classic PostgreSQL convention: in C, `pd_linp[0]` sits at byte 24, yet SQL and `ctid` number slots **starting at 1** (`OffsetNumber = 1` means bytes `[24..28)`, `OffsetNumber = 2` means bytes `[28..32)`, and slot k sits at byte $24 + 4(k-1)$).

```
/* From src/include/storage/itemid.h */
typedef struct ItemIdData
{
    unsigned lp_off:15,   /* byte offset to tuple (0..32767) */
             lp_flags:2,  /* state of line pointer, see below */
             lp_len:15;   /* byte length of tuple (0..32767) */
} ItemIdData;

#define LP_UNUSED    0    /* unused slot (lp_len is always 0) */
#define LP_NORMAL    1    /* used slot pointing to a tuple (lp_len > 0) */
#define LP_REDIRECT  2    /* HOT redirect (lp_off = target slot, lp_len == 0) */
#define LP_DEAD      3    /* dead item, tuple storage already reclaimed */
```

Look how cleanly those 32 bits are budgeted:

- **`lp_off` (15 bits)**: Ranges over $0 \ldots 32{,}767$, enough to address every byte of an 8,192-byte page (and supporting compile-time `--with-blocksize=32` up to 32 KB).
- **`lp_len` (15 bits)**: Exact unpadded byte length of the tuple.
- **`lp_flags` (2 bits)**: Encodes the four-state lifecycle of a slot. Every live or recently deleted tuple starts as `LP_NORMAL (1)`. If an in-page Heap-Only Tuple update occurs (Chapter 5), the root slot can become `LP_REDIRECT (2)`, where `lp_off` holds another *slot number* on the same page. When an ordinary dead tuple's bytes are pruned away before indexes have been cleaned, the slot transitions to `LP_DEAD (3)` (`lp_len` $= 0$): the 4-byte gravestone stays put so index scans arriving with `ctid = (0, 2)` immediately see `LP_DEAD` without reading garbage, and once `VACUUM` removes those index entries, the slot finally flips back to `LP_UNUSED (0)` for reuse by a future `INSERT`.

2.2.3 Step-by-Step Trace: Inserting, Deleting, and Compacting Without Breaking Indexes

Let us trace Page 0 of our `accounts` table with exact numbers from initialization through three inserts, one delete, and an in-page compaction (`PageRepairFragmentation`). In Section 2.3 we will verify why these three rows have raw byte lengths 49, 57, and 49, which round up under 8-byte `MAXALIGN` to **56**, **64**, and **56** bytes when carved from `pd_upper`:

1. **Fresh Empty Page 0**:

   `pd_lower` = 24, `pd_upper` = 8,192, free gap = 8,192 − 24 = 8,168 B.

2. **Insert Row 1** (`1001, 'Ada', 50000, true`; raw `lp_len` = 49 B, aligned 56 B):
   - Allocate Slot 1 at bytes `[24..28)`; advance `pd_lower` : 24 → 28.
   - Pull `pd_upper` down by 56 bytes: 8,192 − 56 = **8,136**. Copy the 49 tuple bytes into `[8136..8185)` (leaving `[8185..8192)` as 7 trailing alignment pad bytes).
   - Write Slot 1 = `(lp_off = 8136, lp_flags = LP_NORMAL, lp_len = 49)`.
   - The primary-key B-tree index `accounts_pkey` records `1001` ↦ `ctid = (0, 1)`.

3. **Insert Row 2** (`1002, 'Grace Hopper', 125000, true`; raw `lp_len` = 57 B, aligned 64 B):
   - Allocate Slot 2 at bytes `[28..32)`; advance `pd_lower` : 28 → 32.
   - Pull `pd_upper` down by 64 bytes: 8,136 − 64 = **8,072**. Copy the 57 tuple bytes into `[8072..8129)`.
   - Write Slot 2 = `(lp_off = 8072, lp_flags = LP_NORMAL, lp_len = 57)`.
   - `accounts_pkey` records `1002` ↦ `ctid = (0, 2)`.

4. **Insert Row 3** (`1003, 'Edsger', 90000, false`; raw `lp_len` = 49 B, aligned 56 B):
   - Allocate Slot 3 at bytes `[32..36)`; advance `pd_lower` : 32 → 36.
   - Pull `pd_upper` down by 56 bytes: 8,072 − 56 = **8,016**. Copy the 49 tuple bytes into `[8016..8065)`.
   - Write Slot 3 = `(lp_off = 8016, lp_flags = LP_NORMAL, lp_len = 49)`.
   - `accounts_pkey` (and any secondary indexes on `owner_name` or `balance_cents`) records `1003` ↦ `ctid = (0, 3)`. Free gap is now 8,016 − 36 = 7,980 B.

5. **Delete Row 2 (`1002`) and Compact Page 0 (`PageRepairFragmentation`)**:
   - Once the `DELETE` commits and no running transaction can see Row 2 anymore, bytes `[8072..8136)` form a **64-byte dead hole** trapped between Tuple 3 (`[8016..8072)`) and Tuple 1 (`[8136..8192)`).
   - Inside RAM, PostgreSQL executes `memmove` to slide Tuple 3 upward by 64 bytes: from `[8016..8072)` into `[8080..8136)`.
   - `pd_upper` jumps upward from 8,016 to **8,080**, merging all 64 reclaimed bytes straight back into the central free gap (8,080 − 36 = 8,044 B).
   - Slot 2 (`[28..32)`) is marked `(lp_off = 0, lp_flags = LP_DEAD, lp_len = 0)`. Notice that **Slot 3 is never renumbered to Slot 2**.
   - Slot 3 (`[32..36)`) simply rewrites its internal offset from `8016` to `8080`: `(lp_off = 8080, lp_flags = LP_NORMAL, lp_len = 49)`.

Why it works

Look at what happened to every external B-tree index pointing to `'Edsger'` at `ctid = (0, 3)`. Not a single index page had to be locked, dirtied, WAL-logged, or rewritten. When an index lookup lands on Page 0 and asks for Slot 3, it reads bytes `[32..36)`, follows the

updated `lp_off = 8080`, and lands on Edsger's relocated tuple in one memory dereference. Four bytes of per-row slot indirection decouple *internal page defragmentation* from *global index maintenance*.

Compare this with two mechanisms you already know. Java's compacting garbage collector also moves objects to close gaps, and it then rewrites every reference to each moved object; it can do that because it stops the application threads and walks the whole heap. PostgreSQL cannot afford to find and rewrite index entries stored in other files every time it tidies one page, so it adds one level of indirection instead: an index stores `(block, slot)`, and only the 4-byte slot inside the page knows the byte offset. In C you see the failure this avoids: a raw pointer into a buffer dangles as soon as `realloc` moves the buffer, whereas an index into the buffer stays valid.

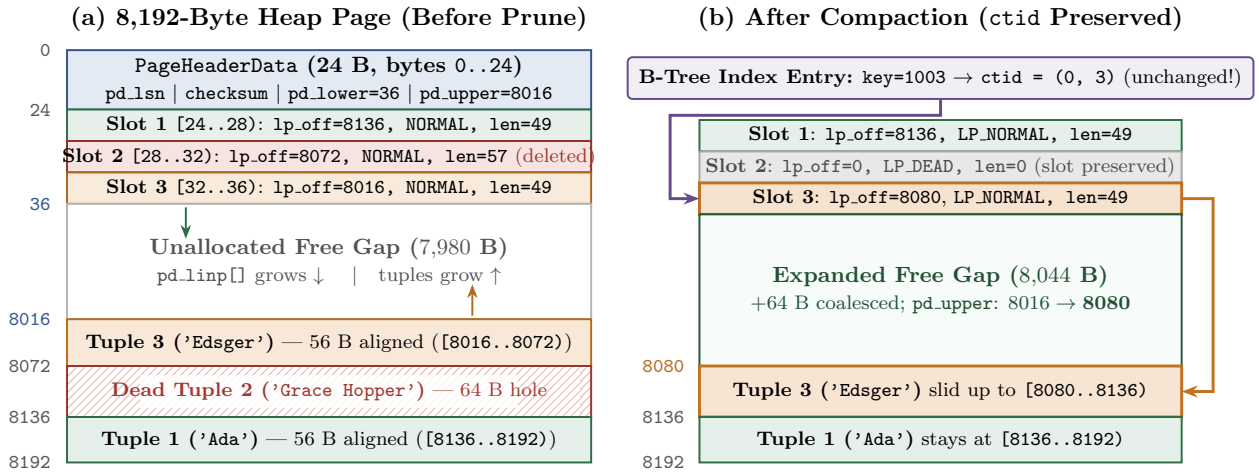


Figure 2.2: Anatomy of an 8,192-byte slotted heap page and in-page defragmentation (`PageRepairFragmentation`). Sliding Tuple 3 upward over Dead Tuple 2 reclaims 64 contiguous bytes into the central gap while updating only Slot 3's internal `lp_off` (8016 → 8080); external index pointers to `ctid = (0, 3)` remain valid with zero index I/O.

2.3 Inside a Heap Tuple: Headers, Null Bitmaps, and Column Tetris

Now let us zoom in on the 49 bytes starting at byte 8,136 of Page 0—Row 1 of `accounts`. Every heap tuple begins with a fixed **23-byte** C structure, `HeapTupleHeaderData` (`src/include/access/htup_details.h`), optionally followed by a variable-length **null bitmap**, alignment padding up to byte `t_hoff`, and finally the serialized user columns.

```
/* Simplified from src/include/access/htup_details.h */
struct HeapTupleHeaderData
{
    union {
        HeapTupleFields t_heap;   /* [0..12) t_xmin (4B), t_xmax (4B), t_cid (4B) */
        DatumTupleFields t_datum; /* used for in-memory composite datums */
    } t_choice;

```

```
    ItemPointerData t_ctid;       /* [12..18) current or newer (block, slot) (6B) */

    uint16          t_infomask2;  /* [18..20) lower 11 bits = natts; upper = HOT flags */
    uint16          t_infomask;   /* [20..22) HASNULL, HASVARWIDTH, XMIN/XMAX hint bits */
    uint8           t_hoff;       /* [22..23) byte offset to user data (MAXALIGN'd!) */

    /* [23..) Optional null bitmap (1 bit per column when HEAP_HASNULL is set) */
    bits8           t_bits[FLEXIBLE_ARRAY_MEMBER];

    /* User column bytes start at exact offset t_hoff from start of header! */
};
```

2.3.1 Why a 23-Byte Struct Produces a 24-Byte Header (and Free 8-Column Null Bitmaps)

A Java 6 object on 64-bit HotSpot also starts with a header: an 8-byte mark word (lock and garbage-collector state) and an 8-byte class pointer, or 12 bytes total with compressed references. A PostgreSQL tuple header plays the same role for a row, but what it records is different: which transactions created and deleted this version, and where the newer version lives. Count its bytes carefully:

- `t_xmin` **([0..4), 4 bytes)** and `t_xmax` **([4..8), 4 bytes)**: The 32-bit transaction IDs that inserted and deleted/updated/locked this tuple version (the heart of MVCC in Chapter 5).
- `t_cid` **([8..12), 4 bytes)**: Command ID tracking which SQL statement inside a multi-statement transaction created or modified the row.
- `t_ctid` **([12..18), 6 bytes)**: On the latest version of a row, `t_ctid` points to *itself* (e.g., `(0, 1)`). When an `UPDATE` creates a newer version of the row, the old version's `t_ctid` is overwritten to point forward to the new version's `(block, slot)`, forming a version chain.

  Under the hood

  Why is `ItemPointerData` 6 bytes instead of 8 bytes when it holds a 32-bit `BlockNumber` plus a 16-bit `OffsetNumber`? In C, a struct containing a `uint32` and a `uint16` has 4-byte alignment and `sizeof` = 8 (wasting 2 tail-padding bytes inside every tuple header and every B-tree index entry). PostgreSQL splits `BlockIdData` into *two consecutive* ***`uint16`*** *integers* (`bi_hi` and `bi_lo`), keeping alignment at 2 bytes and `sizeof(ItemPointerData)` at **6 bytes exact**. Saving 2 bytes per pointer shrinks every heap row and every B-tree leaf entry across the entire database.

- `t_infomask2` **([18..20), 2 bytes)**: Masks the column count `natts` in its lower 11 bits (`0x07FF`) and HOT-update flags (`HEAP_HOT_UPDATED`, `HEAP_ONLY_TUPLE`) in its upper 5 bits.
- `t_infomask` **([20..22), 2 bytes)**: Tracks structural flags—`HEAP_HASNULL` (`0x0001`), `HEAP_HASVARWIDTH` (`0x0002`), and `HEAP_HASEXTERNAL` (`0x0004`, TOAST pointer present)—plus commit/abort **hint bits** (`HEAP_XMIN_COMMITTED` `0x0100`, `HEAP_XMAX_INVALID` `0x0800`) cached directly on the tuple so readers do not repeatedly consult the commit log.
- `t_hoff` **([22..23), 1 byte)**: Byte offset where user column 1 begins.

That brings us to offset **23**. Why can't user column 1 (`account_id BIGINT`) start right at byte 23? Because on 64-bit CPUs, reading an 8-byte `int64` or `double` expects an 8-byte aligned memory address (`MAXALIGN` = 8); since every tuple starts at an 8-byte aligned `pd_upper`, user data must also start at a multiple of 8 relative to the tuple header.

Watch the arithmetic across three cases:

1. **Row has zero `NULL` values (`HEAP_HASNULL = 0`)**: PostgreSQL omits `t_bits[]` completely. Rounding 23 up to the next multiple of 8 yields `t_hoff` = **24** (byte 23 is a single zero-padding byte).
2. **Row has at least one `NULL`, and $1 \leq$ natts ≤ 8 columns (`HEAP_HASNULL = 1`)**: The null bitmap needs $\lceil \text{natts}/8 \rceil = 1$ byte, which drops neatly into **byte 23**—the exact byte that was previously wasted on alignment padding. Now $23 + 1 = 24$, which is already divisible by 8, so `t_hoff` **is still** 24. Even better, any column whose bit in `t_bits[]` is `0` (*null*) writes **0 data bytes** into the tuple payload. (In PostgreSQL's bitmap convention, bit `1` means *present / NOT NULL* and bit `0` means *NULL*.)
3. **Row has at least one `NULL`, and natts = 9 columns**: Now the null bitmap needs $\lceil 9/8 \rceil = 2$ bytes (`[23..25)`), bringing the unaligned header to $23 + 2 = 25$ bytes. Rounding 25 up to the next multiple of 8 jumps `t_hoff` from 24 all the way to **32 bytes** (7 padding bytes at `[25..32)`).

Common trap

Adding a 9th column (or a 73rd column, where $23 + 10 = 33 \rightarrow$ `t_hoff` = 40) to a wide table that frequently contains `NULL`s adds an invisible **8-byte cliff** to every nullable row's header. Conversely, contrary to a widespread myth, `NULL` columns in PostgreSQL do *not* reserve their full column width on disk: a row with twenty `BIGINT` columns all set to `NULL` stores just the 23-byte header + 3-byte bitmap + 6 pad bytes = **32 bytes total**, not $20 \times 8 = 160$ bytes.

2.3.2 CPU Alignment Padding and "Column Tetris"

Once PostgreSQL reaches `t_hoff`, it serializes each non-`NULL` column in exact `CREATE TABLE` order. Before writing column i, PostgreSQL advances the current offset to the alignment boundary dictated by `pg_type.typalign`:

`typalign`	Boundary	Common PostgreSQL Types	Width (`typlen`)
`'d'` (double)	8 bytes	`BIGINT` (`int8`), `DOUBLE PRECISION`, `TIMESTAMPTZ`, `TIMESTAMP`	8 B (`INTERVAL`: 16 B)
`'i'` (int)	4 bytes	`INTEGER` (`int4`), `REAL` (`float4`), `DATE`, 4B-header `varlena`	4 B (or variable)
`'s'` (short)	2 bytes	`SMALLINT` (`int2`)	2 B
`'c'` (char)	1 byte	`BOOLEAN`, `"char"`, `UUID` (16 B), short 1B-header `varlena`	1 B, 16 B, or var.

Note `UUID` in the bottom row: even though a `UUID` is 16 bytes wide (`unsigned char data[16]`), its C representation is an array of single bytes, so `typalign = 'c'` (1-byte aligned, zero pre-padding ever required). By contrast, `BIGINT` and `TIMESTAMPTZ` demand `'d'` (8-byte) alignment: if the previous

column ended at offset 25, PostgreSQL inserts **7 dead zero bytes** (`[25..32)`) so the `BIGINT` starts cleanly at offset 32.

This is the same rule your C compiler applies to struct members. On x86-64, `struct { char a; int64_t b; }` has `sizeof` 16, not 9, because `b` must start at offset 8. The C compiler pays that padding once per struct in RAM. PostgreSQL pays it once per row on disk, again in WAL, and again in `shared_buffers`. Java hides the issue because HotSpot is free to reorder an object's fields by size; the order you declare fields in a Java class does not decide its memory layout.

PostgreSQL *never* reorders columns behind your back to save padding; physical tuple order matches logical `pg_attribute.attnum` order. Let us calculate what happens on a 50-million-row payment-events table when an engineer alternates narrow and 8-byte columns versus ordering them from widest alignment downward:

```
-- Layout A: Alternating 1B/2B and 8B columns ("Swiss-cheese tuple")
CREATE TABLE events_bad (
    is_settled   BOOLEAN     NOT NULL, -- 1B at [24..25) + 7B pad [25..32)
    account_id   BIGINT      NOT NULL, -- 8B at [32..40)
    region_code  SMALLINT    NOT NULL, -- 2B at [40..42) + 6B pad [42..48)
    created_at   TIMESTAMPTZ NOT NULL, -- 8B at [48..56)
    retry_flag   BOOLEAN     NOT NULL, -- 1B at [56..57) + 7B pad [57..64)
    amount_cents BIGINT      NOT NULL, -- 8B at [64..72)
    channel_id   INTEGER     NOT NULL, -- 4B at [72..76)
    fee_bps      SMALLINT    NOT NULL  -- 2B at [76..78) -> lp_len=78, MAXALIGN=80B!
);

-- Layout B: Identical 8 columns ordered 8B -> 4B -> 2B -> 1B ("Column Tetris")
CREATE TABLE events_good (
    account_id   BIGINT      NOT NULL, -- 8B at [24..32)
    created_at   TIMESTAMPTZ NOT NULL, -- 8B at [32..40)
    amount_cents BIGINT      NOT NULL, -- 8B at [40..48)
    channel_id   INTEGER     NOT NULL, -- 4B at [48..52)
    region_code  SMALLINT    NOT NULL, -- 2B at [52..54)
    fee_bps      SMALLINT    NOT NULL, -- 2B at [54..56)
    is_settled   BOOLEAN     NOT NULL, -- 1B at [56..57)
    retry_flag   BOOLEAN     NOT NULL  -- 1B at [57..58) -> lp_len=58, MAXALIGN=64B!
);
```

Check the math. Both tables store the exact same $8+8+8+4+2+2+1+1 = \mathbf{34}$ bytes of user payload plus a 24-byte header (58 useful bytes total):

- In `events_bad`, three alignment gaps ($7+6+7 = 20$ B) inflate `lp_len` to 78 B, which `MAXALIGN` rounds to **80 bytes** on the page (+4 B `ItemIdData` = **84 B/row**). Each 8 KB heap page (8,168 usable bytes) holds $\lfloor 8{,}168/84 \rfloor = \mathbf{97}$ rows. Across 50,000,000 rows, `events_bad` consumes $\lceil 50{,}000{,}000/97 \rceil = 515{,}464$ blocks $\approx \mathbf{4.22}$ GB (4,027 MiB).
- In `events_good`, every single column starts naturally on its required boundary with **zero internal padding**: `lp_len` = 58 B, `MAXALIGN` = **64** bytes (+4 B `ItemIdData` = **68 B/row**). Each 8 KB page now holds $\lfloor 8{,}168/68 \rfloor = \mathbf{120}$ rows! Across 50,000,000 rows, `events_good` consumes 416,667 blocks $\approx \mathbf{3.41}$ GB (3,255 MiB).

Reordering eight lines of DDL saves **16 bytes per row** and **810 MB** of NVMe storage, WAL traffic, and `shared_buffers` RAM without changing a single query. Better yet, if you also have `TEXT` or

`JSONB` columns, place all fixed-width `NOT NULL` columns *first* (ordered 8B → 4B → 2B → 1B) and variable-width or nullable columns *last*: PostgreSQL caches the fixed byte offset (`attcacheoff`) of leading fixed-width non-null columns so the executor can jump straight to `amount_cents` at offset 40 in $O(1)$ CPU instructions instead of walking earlier variable-length headers.

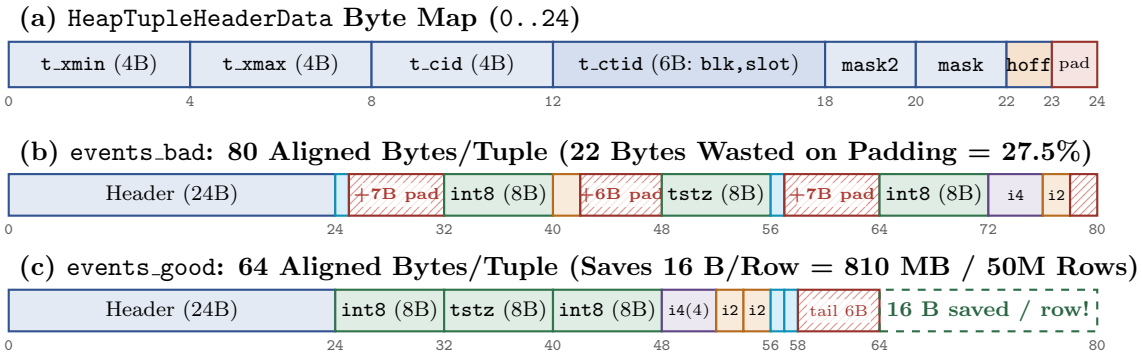


Figure 2.3: Byte-by-byte tuple layout and "Column Tetris." Ordering columns from 8-byte alignment downward eliminates all 20 bytes of internal alignment holes in `events_bad`, shrinking the `MAXALIGN`ed footprint from 80 bytes to 64 bytes.

2.3.3 Decoding Real Disk Bytes with `pageinspect`

Nothing cements this byte arithmetic like inspecting a live PostgreSQL heap page yourself. The built-in `pageinspect` contrib extension exposes raw 8 KB blocks directly as SQL rows.

Try it in PostgreSQL

Run the following session in any local PostgreSQL scratch database (superuser required for `get_raw_page`):

```
CREATE EXTENSION IF NOT EXISTS pageinspect;

INSERT INTO accounts VALUES
    (1001, 'Ada',          50000,  true),
    (1002, 'Grace Hopper', 125000, true),
    (1003, 'Edsger',       90000,  false);

SELECT lower, upper, special, pagesize
FROM page_header(get_raw_page('accounts', 0));
--  lower | upper | special | pagesize
-- -------+-------+---------+----------
--     36 |  8016 |    8192 |     8192

SELECT lp, lp_off, lp_flags, lp_len, t_xmin, t_xmax, t_ctid,
       t_infomask2, t_infomask, t_hoff, t_attrs
FROM heap_page_item_attrs(get_raw_page('accounts', 0), 'accounts'::regclass);
-- lp | lp_off | lp_flags | lp_len | t_xmin | t_ctid | mask2 | mask | hoff
-- ---+--------+----------+--------+--------+--------+-------+------+-----
--  1 |   8136 |        1 |     49 |    742 | (0,1)  |     4 | 2306 |   24
--      t_attrs: {"\\xe903000000000000","\\x09416461","\\x50c3000000000000","\\x01"}
```

```
-- 2 |   8072 |        1 |     57 |    742 | (0,2)  |     4 | 2306 |   24
--       t_attrs: {"\\xea03000000000000","\\x1b477261636520486f70706572","\\
    x48e8010000000000","\\x01"}
-- 3 |   8016 |        1 |     49 |    742 | (0,3)  |     4 | 2306 |   24
--       t_attrs: {"\\xeb03000000000000","\\x0f456473676572","\\x905f010000000000
    ","\\x00"}
```

Every single number from our hand calculation in Section 2.2 appears verbatim. Look closer at Row 1's fields on an x86-64 (little-endian) Linux machine:

- `t_infomask2 = 4`: 4 columns (`natts = 4`), zero HOT flags.
- `t_infomask = 2306 = 0x0902`: Bitwise OR of `0x0002` (`HEAP_HASVARWIDTH`, because `owner_name` is `TEXT`), `0x0100` (`HEAP_XMIN_COMMITTED`), and `0x0800` (`HEAP_XMAX_INVALID`, meaning nobody has deleted Row 1).
- `t_attrs[1] = \xe903000000000000`: Little-endian 8-byte integer `0x03e9` $= 3 \times 256 + 233 =$ **1001**.
- `t_attrs[2] = \x09416461`: Why does the 3-byte string `'Ada'` (`0x41 0x64 0x61`) start with the single header byte `0x09`? Because $1 + 3 = 4$ total bytes encoded as a **short 1-byte `varlena` header** is $(4 \ll 1) \mid 1 = 9 =$ `0x09`. And notice the hidden Column Tetris flaw inside `accounts`: placing `owner_name` (4 B ending at byte 36) *before* `balance_cents BIGINT` forced 4 wasted alignment padding bytes (`[36..40)`) that brought `lp_len` to $24+8+4+4+8+1 =$ **49** B. Moving `owner_name` to the end of `accounts` drops `lp_len` to $24 + 8 + 8 + 1 + 4 =$ **45** B, shrinking the aligned page footprint of Row 1 from **56 bytes down to 48 bytes**.

2.4 TOAST: The Oversized-Attribute Storage Technique

Stop and ask an uncomfortable question about fixed 8,192-byte pages. Every `ItemIdData` slot points to a single tuple inside a single 8 KB block, and after subtracting the 24-byte page header plus one 4-byte line pointer, at most `MaxHeapTupleSize` = 8,160 bytes exist on an empty page. Yet PostgreSQL lets you insert a **400 KB** JSON document or a **50 MB** `TEXT` report into a single column. Where do those bytes go if a heap tuple is forbidden from spanning two 8 KB pages?

First, why *does* PostgreSQL forbid tuples from straddling page boundaries? Because keeping every heap tuple self-contained inside one 8 KB frame keeps the entire storage engine simple and fast:

- Reading a row by `ctid = (block, slot)` pins **one** buffer frame and acquires **one** content lock—no multi-page latch deadlocks.
- Every 8 KB page carries a single `pd_lsn` for Write-Ahead Logging and crash recovery (Chapter 4).
- In-page compaction (`PageRepairFragmentation`) is a local `memmove` inside a single 8,192-byte C array.

In fact, PostgreSQL goes further: to keep sequential scans and page packing healthy, it wants *at least four tuples* to fit on every 8 KB heap page whenever possible. That sets the threshold `TOAST_TUPLE_THRESHOLD` $\approx$ **2,000** bytes (specifically 2,032 bytes on 64-bit builds with 8 KB pages:

(8,192 − 40)/4 = 2,038, rounded down to a multiple of 8). Whenever an inserted or updated row would exceed roughly 2 KB, PostgreSQL invokes **TOAST**—*The Oversized-Attribute Storage Technique* (`src/backend/access/common/toast_internals.c`).

2.4.1 Inside `varlena` Headers: Saving 3 to 6 Bytes on Every Short String

Before TOAST even touches a large string, PostgreSQL optimizes *small* variable-length values (`TEXT`, `VARCHAR`, `BYTEA`, `NUMERIC`, `JSONB`) using a polymorphic C header family called `varlena` (*variable-length array*, defined in `src/include/varatt.h`).

Naive implementations prepend a 4-byte (`uint32`) length word to every string. On a 3-letter string like `'USD'` or `'Ada'`, a 4-byte header plus up to 3 bytes of 4-byte CPU alignment padding would burn **7 overhead bytes** just to store 3 ASCII characters. On little-endian architectures (x86-64 and ARM64 Linux), PostgreSQL inspects the **lowest two bits of the very first byte** in memory to distinguish four physical layouts:

First-Byte Tag	Variant (`varatt.h`)	Align	Payload & Max Capacity
`xxxxxx00` (2b)	4B uncompressed (`va_4byte`)	4 B (`'i'`)	Upper 30 bits = total len up to $2^{30}-1$ (**1 GB**)
`xxxxxx10` (2b)	4B compressed (`va_compressed`)	4 B (`'i'`)	+4B `tcinfo` (raw size + `pglz`/`lz4` codec ID)
`xxxxxxx1` (≠ `0x01`)	**1B short** (`va_1byte`)	**1 B (`'c'`)**	Upper 7 bits = total len 1…127 (**0..126 data B**)
`00000001` (`0x01`)	**18B TOAST pointer**	**1 B (`'c'`)**	Byte 1 = tag (`18`); bytes `2..18` = `varatt_external`

Note the 1-byte short header (`xxxxxxx1`). Any `TEXT`, `VARCHAR`, or `NUMERIC` value up to **126 bytes** long needs only **1 byte of length header** *and* drops from 4-byte alignment to **1-byte alignment** (zero pre-padding). That is why `VARCHAR(20)` is never physically smaller on disk than `TEXT` in PostgreSQL: both use the exact same 1-byte `varlena` representation.

And look at the fourth row: when byte 0 equals exact `0x01` and byte 1 equals `VARTAG_ONDISK (18)`, the entire giant column inside the main heap tuple is replaced by a compact **18-byte TOAST pointer** (`struct varatt_external`):

```
/* From src/include/varatt.h -- 16-byte payload inside an 18-byte TOAST pointer */
typedef struct varatt_external
{
    int32    va_rawsize;     /* Original uncompressed size (incl. 4B header) */
    uint32   va_extinfo;     /* Lower 30 bits: stored size; upper 2: codec   */
    Oid      va_valueid;     /* Unique chunk_id inside the TOAST table       */
    Oid      va_toastrelid;  /* OID of pg_toast.pg_toast_<table_oid>         */
} varatt_external;
```

2.4.2 The Four TOAST Strategies and `pg_toast` Chunk Slicing

Every column in `pg_attribute` carries a 1-character `attstorage` policy telling TOAST what transformations are permitted when a tuple crosses `TOAST_TUPLE_THRESHOLD` (≈ 2,000 B):

1. `PLAIN ('p')`: Used by fixed-width types (`BIGINT`, `BOOLEAN`, `TIMESTAMPTZ`, `UUID`). Never compressed, never moved out-of-line.

2. `EXTENDED ('x', the default for TEXT, BYTEA, JSONB, NUMERIC)`: Permits *both* inline compression (`pglz` by default, or much faster `lz4` when `default_toast_compression = 'lz4'`) *and* out-of-line storage in the companion TOAST table.

3. `EXTERNAL ('e')`: Permits out-of-line TOAST storage while **disabling compression**. Why disable compression? Two practical reasons: if a `BYTEA` column already holds pre-compressed PNGs, Zstandard archives, or random ciphertext, running `pglz` wastes CPU cycles for zero byte savings; and if queries extract slices of a large string via `substring(payload, 50000, 80)`, an uncompressed `EXTERNAL` column lets PostgreSQL compute the exact chunk number and read **one 2 KB chunk** instead of fetching and decompressing all chunks from byte 0.

4. `MAIN ('m')`: Permits inline compression and fights hard to keep the attribute **inside the main heap page**, evicting it out-of-line only as a last resort when the row still exceeds `MaxHeapTupleSize` (8,160 B) after all `EXTENDED` and `EXTERNAL` columns have already been moved out.

If no column is allowed to leave the page, an oversized row is rejected rather than split across two pages:

```
CREATE TABLE notes (body TEXT);
ALTER TABLE notes ALTER COLUMN body SET STORAGE PLAIN;
INSERT INTO notes SELECT repeat('x', 9000);
-- ERROR:  row is too big: size 9028, maximum size 8160
```

The size is the 24-byte tuple header, the 4-byte `varlena` length word, and the 9,000 payload bytes; 8,160 is `MaxHeapTupleSize`, the 8,192-byte page minus its 24-byte header, minus one 4-byte line pointer, rounded down to a multiple of 8.

When a variable-length value is evicted out-of-line, where does it land? Whenever you create a table with at least one TOASTable column, PostgreSQL automatically creates a hidden companion heap table `pg_toast.pg_toast_`⟨oid⟩ (linked via `pg_class.reltoastrelid`) with three columns and a unique B-tree index on `(chunk_id, chunk_seq)`:

```
-- Conceptual schema of pg_toast.pg_toast_24576:
--   chunk_id   OID    (matches varatt_external.va_valueid)
--   chunk_seq  INT4   (0, 1, 2, ... in byte order)
--   chunk_data BYTEA  (up to TOAST_MAX_CHUNK_SIZE ~= 1,996 bytes each)
--   PRIMARY KEY (chunk_id, chunk_seq)
```

Example 2.1 (Counting TOAST Chunks by Hand)**.** Suppose we add `audit_json JSONB` (`EXTENDED`, `lz4`) to `accounts` (`oid = 24576`) and insert a row whose `audit_json` serializes to 30,720 bytes (30 KiB).

1. Because $30{,}720 + 49 > 2{,}032$, `toast_insert_or_update` first compresses `audit_json` with `lz4`, shrinking it to, say, 9,000 **bytes**.

2. $9{,}000 + 49 = 9{,}049$ B still exceeds `TOAST_TUPLE_TARGET` (2,032 B), so PostgreSQL moves the compressed blob out-of-line.

3. Each TOAST chunk holds at most `TOAST_MAX_CHUNK_SIZE` = 1,996 bytes (sized so 4 TOAST chunk rows plus their headers and line pointers pack cleanly onto one 8,192-byte TOAST page). Slicing 9,000 bytes produces:

$$\left\lceil \frac{9{,}000}{1{,}996} \right\rceil = \mathbf{5} \text{ chunks} \quad (\texttt{chunk_seq} = 0, 1, 2, 3 \text{ of } 1{,}996 \text{ B each, plus } \texttt{chunk_seq} = 4 \text{ of } 1{,}016 \text{ B}).$$

4. Those 5 rows are inserted into `pg_toast.pg_toast_24576` under a fresh `chunk_id`, while the `accounts` tuple in the main heap shrinks `audit_json` down to an **18-byte TOAST pointer** (total main tuple `lp_len` = 49 + 18 = 67 B, aligned to 72 B, setting `HEAP_HASEXTERNAL` in `t_infomask`).

In production

Understanding that 18-byte pointer explains two production behaviors:

- **Why `SELECT *` on wide tables is slow**: If an ORM runs `SELECT * FROM accounts WHERE is_active`, PostgreSQL must *detoast* every matching row—probing `pg_toast_24576_index`, fetching 5 chunk rows, and decompressing 30 KB of JSON that your application code immediately throws away. Projecting only the columns you need (`SELECT account_id, balance_cents FROM accounts WHERE is_active`) scans 110+ compact rows per 8 KB main heap page and **never opens the TOAST table at all**.
- **Why `UPDATE` on narrow columns stays cheap**: When you run `UPDATE accounts SET balance_cents = balance_cents + 500 WHERE account_id = 1001`, PostgreSQL writes a new version of the main heap tuple (Chapter 5), sees that `audit_json` was untouched, and **copies the 18-byte `varatt_external` pointer verbatim** into the new tuple without duplicating or rewriting a single byte in `pg_toast_24576`. (Conversely, an `UPDATE` that appends one key to a 10 MB `JSONB` column must rewrite all 10 MB of TOAST chunks.)

2.5 Double Buffering, `shared_buffers`, Pins, and Clock-Sweep

So far we have inspected 8 KB pages on disk. Now let us trace how a running PostgreSQL server moves those 8 KB blocks between NVMe storage and RAM.

PostgreSQL uses a multi-process architecture: each client connection talks to a dedicated `postgres` backend OS process, and all backend processes attach to a single large shared-memory region allocated at server startup called `shared_buffers` (`src/backend/storage/buffer/bufmgr.c`). Every read or write of a heap, index, FSM, or VM block goes through `shared_buffers` first.

2.5.1 Why Not Just Rely on the Linux Page Cache? (Double Buffering)

When a backend misses in `shared_buffers`, it calls `pread()` on the segment file descriptor. Notice what Linux does inside `pread()`: the kernel reads the NVMe sectors into the **Linux OS page cache** first, and then copies those 8,192 bytes into PostgreSQL's `shared_buffers` frame. For a while, the exact same 8 KB block sits in RAM *twice*—once in kernel page cache and once in `shared_buffers`. This phenomenon is called **double buffering**, and it explains why seasoned PostgreSQL DBAs

typically set `shared_buffers` to **25%–40% of total machine RAM** (leaving the remaining RAM to double as an effective secondary victim cache in the Linux kernel plus working memory for sorts and hashes) rather than 90% of RAM.

Why doesn't PostgreSQL abandon `shared_buffers` and simply `mmap()` its heap files straight out of the Linux page cache? Two fundamental reasons forbid it:

1. **Strict Write-Ordering for Crash Safety (The WAL Rule)**: In Chapter 4 we will prove the iron law of durability: *a dirty 8 KB heap page must never reach disk before the WAL record describing its mutation (`pd_lsn`) has been `fsync`ed to disk.* If PostgreSQL mutated `mmap`ped kernel pages directly, Linux's background writeback threads could flush a half-committed dirty page to NVMe behind PostgreSQL's back before its WAL record hit disk, causing unrecoverable database corruption on a power outage. Keeping dirty pages inside user-space `shared_buffers` guarantees the OS kernel *never even sees* the modified bytes until PostgreSQL explicitly verifies `FlushedWAL` $\geq$ `pd_lsn` and issues `pwrite()`.

2. **Zero-Syscall Shared-Memory Reads**: Hitting a hot root B-tree block inside `shared_buffers` executes entirely in user space—one hash-table probe and one atomic CPU instruction, with zero user-to-kernel context switches and zero 8 KB `memcpy` overhead. (Modern PostgreSQL 17 and 18 are going a step further by adding asynchronous Direct I/O and `io_uring` paths that eventually let large installations bypass double buffering on the OS side too.)

2.5.2 Inside `shared_buffers`: Hash Partitions and `BufferDesc.state`

Think of `shared_buffers` with N frames (for instance, `shared_buffers = 16GB` $\implies$ N = 2,097,152 frames of 8 KB each) as three parallel structures in shared memory (`src/include/storage/buf_internals.h`):

1. `BufferBlocks`: A contiguous array of N raw 8,192-byte page frames (`buf_id = 0 .. N-1`).

2. **Shared Buffer Lookup Hash Table**: Maps a 20-byte `BufferTag` = `(RelFileLocator, ForkNumber, BlockNumber)` to its integer frame slot `buf_id`. To prevent 64 CPU cores from bottlenecking on one global hash-table lock, PostgreSQL splits the hash bucket space across `NUM_BUFFER_PARTITIONS = 128` **independent `LWLock` partitions**: two backends looking up different blocks hash to different lock partitions almost 99.2% of the time.

3. `BufferDescriptors`: An array of N compact **64-byte** metadata headers (`BufferDesc`, padded to match one CPU L1 cache line so adjacent frames never suffer false sharing).

Look inside `BufferDesc` at how PostgreSQL packs a buffer's reference count, clock-sweep popularity score, and status flags into a **single 32-bit atomic integer** (`pg_atomic_uint32 state`):

```
/* Simplified from src/include/storage/buf_internals.h */
typedef struct BufferDesc
{
    BufferTag       tag;          /* (RelFileLocator, ForkNumber, BlockNumber) */
    int             buf_id;       /* Frame index 0 .. NBuffers-1               */

    /* Single 32-bit atomic word: [31..22 flags | 21..18 usage | 17..0 refcount] */
    pg_atomic_uint32 state;

```

```
    int             wait_backend_pgprocno; /* For cleanup-lock waiters         */
    int             freeNext;              /* Link in initial free list        */
    LWLock          content_lock;          /* Shared/Exclusive page byte lock  */
} BufferDesc;

/* Exact bit layout of BufferDesc.state:
 *   Bits  0..17 (18 bits): refcount (pin count, up to 262,143 concurrent pins!)
 *   Bits 18..21 ( 4 bits): usage_count (clock-sweep popularity, capped 0..5)
 *   Bits 22..31 (10 bits): BM_LOCKED, BM_DIRTY, BM_VALID, BM_TAG_VALID,
 *                          BM_IO_IN_PROGRESS, BM_PERMANENT, ...
 */
```

Why pack `refcount`, `usage_count`, and `BM_DIRTY` into one 32-bit word? Because in older PostgreSQL releases, pinning or unpinning a buffer had to acquire a per-buffer spinlock first; on multi-socket servers hammering a hot index root page, spinlock cache-line bouncing dominated CPU profiles. With the packed 32-bit `state` word, pinning a buffer and bumping its `usage_count` executes in **one lock-free compare-and-swap (CAS) instruction** (and if `usage_count` is already at its cap of `BM_MAX_USAGE_COUNT` = **5**, `PinBuffer` only adds +1 to `refcount`).

2.5.3 Buffer Pins (`refcount`) vs. Content Locks (`LWLock`)

Two different mechanisms protect a buffer: **pinning** the buffer and **locking** its contents.

Mechanism	Where Stored	What It Protects & How Long It Is Held
Pin (`refcount > 0`)	`state[17:0]`	Prevents *eviction* of the frame. Held across tuple evaluation; **never blocks** other readers or writers.
Shared Lock (`LW_SHARED`)	`content_lock`	Protects `pd_linp[]` / tuple bytes from concurrent `memmove`; held for **microseconds** while locating tuples.
Exclusive Lock (`LW_EXCLUSIVE`)	`content_lock`	Acquired to insert/update tuples or modify header offsets; blocks other `LW_SHARED`/`LW_EXCLUSIVE` holders briefly.
Cleanup Lock	`LW_EXCLUSIVE` + `ref` = 1	Required by `PageRepairFragmentation` / `VACUUM` before sliding tuple bytes so no C pointer goes stale.

Trace what happens during a table scan to see why both mechanisms are indispensable:

1. Your backend hashes `(accounts, MAIN_FORK, block 0)` $\mapsto$ `buf_id = 42`, increments `refcount` ($0 \to 1$, **pinned**), and acquires `content_lock` in `LW_SHARED` mode.
2. Under `LW_SHARED`, it quickly checks MVCC visibility of the slots on Page 0, records the visible slots, and **immediately releases** `content_lock`—while *keeping its pin* (`refcount` = 1).
3. Now your backend evaluates expensive SQL `WHERE` functions or streams rows up the executor plan tree. Because `content_lock` was released, another backend can simultaneously acquire `LW_EXCLUSIVE` on Page 0, `INSERT` Row 4 into the central free gap, and commit without waiting for your scan. Meanwhile, because your pin keeps `refcount` ≥ 1, neither the clock-sweep

evictor nor `VACUUM`'s byte-sliding compaction is allowed to overwrite or shift the live tuple bytes your C pointer is reading.

4. Once your scan steps to Page 1, it decrements Page 0's `refcount` ($1 \to 0$, **unpinned**).

2.5.4 Clock-Sweep Eviction: Step-by-Step Trace on a 6-Frame Pool

What happens when every frame in `shared_buffers` is occupied and a backend requests a block not in RAM?

Textbooks teach doubly-linked **Least Recently Used (LRU)** queues: every time a page is read, unlink its node and move it to the head of the global queue. On a 64-core database serving 2,000,000 buffer hits per second, a textbook LRU list does not scale: every read hit mutates the global list head pointers, serializing all 64 cores on a single cache line. You may have built this textbook LRU in Java 6 as `new LinkedHashMap<K,V>(16, 0.75f, true)` with an overridden `removeEldestEntry`. With access order turned on, every `get()` relinks the entry to the tail, so a read is a write; wrapping the map in `Collections.synchronizedMap` makes every cache hit take the same lock. That lock is exactly the bottleneck clock-sweep avoids.

PostgreSQL instead uses **Clock-Sweep** (a generalized Second-Chance / NFU clock algorithm in `StrategyGetBuffer`, `src/backend/storage/buffer/freelist.c`):

- **On a buffer hit (the 99% fast path)**: Touch *nothing* global. Locally increment that frame's 4-bit `usage_count` up to the ceiling of **5** ($\min(\text{usage_count} + 1, 5)$). Why cap at 5 instead of 15? If a hot page accumulated `usage_count` = 15, a sudden workload shift would force the clock hand to spin around the entire buffer pool 15 times before evicting old pages.
- **On a buffer miss (eviction path)**: Advance the shared clock hand `nextVictimBuffer` modulo N, inspecting `BufferDescriptors[hand]` at each step:
  1. **If** `refcount` > 0 **(currently pinned)**: Cannot evict. Leave `usage_count` untouched and advance `hand` $\leftarrow$ (`hand` $+ 1$) mod N.
  2. **If** `refcount` $= 0$ **and** `usage_count` > 0: Give the frame another life by decrementing `usage_count` $\leftarrow$ `usage_count` $- 1$, and advance `hand` $\leftarrow$ (`hand` $+ 1$) mod N.
  3. **If** `refcount` $= 0$ **and** `usage_count` $= 0$: **Victim found.** Pin the frame (`refcount` $= 1$), advance `hand` for the next caller, flush the old page if `BM_DIRTY` is set (after enforcing `FlushedWAL` $\geq$ `pd_lsn`; normally PostgreSQL's background writer `bgwriter` pre-cleans upcoming low-usage dirty buffers ahead of the clock hand so user backends rarely pay this write cost), replace the hash-table `BufferTag`, and `pread()` the new 8 KB block into the frame with `usage_count` $= 1$.

Example 2.2 (Tracing the Clock Hand Across 6 Buffer Frames)**.** Consider a 6-frame pool (`F0..F5`) with `nextVictimBuffer` starting at **Frame 0** and initial states `(refcount, usage)`:

`F0:(1, 3)`, `F1:(0, 2)`, `F2:(0, 1)`, `F3:(2, 0)`, `F4:(0, 1)`, `F5:(0, 2)`.

Watch how `StrategyGetBuffer` finds a victim when a backend misses on `accounts` block 9:

1. **Tick 1 (Frame 0)**: `refcount` $= 1 > 0$ (pinned). Skip immediately; `F0` stays `(1, 3)`. Hand $\to$ `F1`.
2. **Tick 2 (Frame 1)**: `refcount` $= 0$, `usage` $= 2 > 0$. Decrement `usage` : $2 \to 1$. Hand $\to$ `F2`.

3. **Tick 3 (Frame 2)**: `refcount` $= 0$, `usage` $= 1 > 0$. Decrement `usage` : $1 \to$ **0**. Hand $\to$ `F3`.
4. **Tick 4 (Frame 3)**: Even though `usage` $= 0$, `refcount` $= 2 > 0$ (two backends hold pins on `F3`). Pinned frames are immune to eviction; skip `F3`. Hand $\to$ `F4`.
5. **Tick 5 (Frame 4)**: `refcount` $= 0$, `usage` $= 1 > 0$. Decrement `usage` : $1 \to 0$. Hand $\to$ `F5`.
6. **Tick 6 (Frame 5)**: `refcount` $= 0$, `usage` $= 2 > 0$. Decrement `usage` : $2 \to 1$. Hand wraps $\to$ `F0`.
7. **Tick 7 (Frame 0)**: Still pinned (`refcount` $= 1$). Skip. Hand $\to$ `F1`.
8. **Tick 8 (Frame 1)**: `refcount` $= 0$, `usage` $= 1 > 0$. Decrement `usage` : $1 \to 0$. Hand $\to$ `F2`.
9. **Tick 9 (Frame 2)**: `refcount` $= 0$ **and** `usage` $= 0$! **Frame 2 is evicted.** It loads `accounts` block 9 with state `(refcount=1, usage=1)`, and `nextVictimBuffer` parks at **Frame 3** ready for the next miss.

2.5.5 Buffer Access Strategies: Why a 500 GB `Seq Scan` Doesn't Wipe Out RAM

Example 2.2 has a weakness. Suppose your production server has `shared_buffers = 16 GB` (2,097,152 frames) holding your entire hot OLTP working set of customer accounts and B-tree indexes at `usage_count` $= 5$. At 2:00 PM, an analyst runs an unindexed reporting query:

```
SELECT count(*), sum(amount_cents) FROM historical_ledger; -- 500 GB cold table!
```

Reading 500 GB streams 65,536,000 cold blocks through the buffer manager—more than **31 times larger** than the entire 16 GB pool. If every cold block used the global clock-sweep hand, the hand would spin around `shared_buffers` dozens of times, decrementing every hot OLTP buffer's `usage_count` from $5 \to 4 \to 3 \to 2 \to 1 \to 0$ and **evicting your entire hot OLTP cache** for blocks the reporting query reads once and never touches again.

PostgreSQL prevents this cache pollution with **Buffer Access Strategies** (*Ring Buffers*, `GetAccessStrategy` in `freelist.c`):

- `BAS_BULKREAD` **(32-frame = 256 KB ring)**: Whenever a sequential scan targets a table larger than `shared_buffers`/4, the executor allocates a tiny private array of **32 buffer IDs** (32×8 KB $=$ **256** KB). As the 500 GB scan streams blocks $0, 1, 2, \ldots$, it cycles strictly around those 32 frames, immediately reusing the frame it read 32 blocks earlier (since its `usage_count` $= 1$ and `refcount` $= 0$). What if a concurrent OLTP backend *also* happened to read one of those 32 frames, bumping its `usage_count` > 1 or pinning it? Then PostgreSQL *graduates* that genuinely popular frame out of the ring into the main pool and pulls one fresh replacement frame. Thus a 500 GB sequential scan touches only 256 KB of `shared_buffers`, leaving **99.998%** of your hot OLTP cache untouched.
- `BAS_BULKWRITE` **(2,048-frame = 16 MB ring)**: Used by `COPY FROM`, `CREATE TABLE AS`, and `SELECT INTO` so bulk data loads stream through a 16 MB ring instead of flooding `shared_buffers` with newly written pages.
- `BAS_VACUUM` **(default 2 MB ring via `vacuum_buffer_usage_limit`)**: Keeps background autovacuum workers inside a compact ring so routine dead-tuple cleanup never evicts user query caches.

Connection

Notice an unexpected bonus of `BAS_BULKREAD` called **synchronized sequential scans** (`synchronize_seqscans = on`): if Analyst B launches a second `Seq Scan` on that same 500 GB `historical_ledger` table halfway through Analyst A's scan, PostgreSQL starts Analyst B *at the exact block Analyst A is currently reading* (wrapping around to block 0 at the end) so both queries ride inside the same 256 KB cache window and cut disk I/O in half—which is also why Chapter 1 warned that `SELECT` without `ORDER BY` makes zero promise of returning rows starting from block 0.

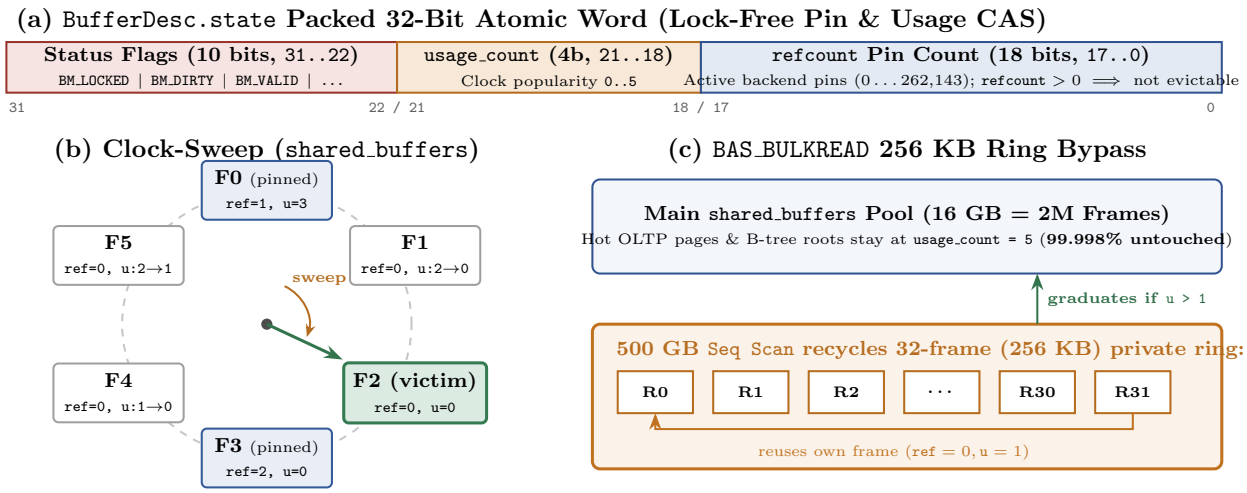


Figure 2.4: Shared buffer management in PostgreSQL: (a) the lock-free 32-bit `BufferDesc.state` word; (b) Clock-Sweep eviction skipping pinned frames (`F0`, `F3`) and decrementing `usage_count` until `F2` hits $(0, 0)$; (c) the 256 KB `BAS_BULKREAD` ring buffer isolating a 500 GB sequential scan from the hot OLTP working set.

Cost summary

Physical Numbers Worth Memorizing:

- **Page size (`BLCKSZ`)**: 8,192 B (24 B `PageHeaderData` $\implies$ 8,168 B usable; 4 B per `ItemIdData` slot).
- **Segment size (`RELSEG_SIZE`)**: 131,072 blocks = 1 GiB per physical file; max standard table $\approx$ 32 TiB.
- **Companion forks**: `_fsm` = 1 B/heap page in a binary max-heap tree; `_vm` = 2 bits/heap page (32,672 heap blocks $\approx$ 255 MiB per 8 KB VM block).
- **Tuple header**: 23 B struct $\rightarrow$ `t_hoff` = 24 B (with 0 nulls, or with nulls and ≤ 8 columns); +4 B slot + 8-byte `MAXALIGN` means **even a single-boolean row costs $4 + 32 = 36$ bytes** on a page.
- **TOAST**: Triggers at $\approx$ 2,000 B; short `varlena` uses 1 B header up to 126 B; out-of-line pointer is 18 B pointing to $\approx$ 1,996 B chunks in `pg_toast`.
- `shared_buffers`: 128 hash lock partitions; `usage_count` $\in \{0 \ldots 5\}$; `BAS_BULKREAD` sequential-scan ring = 32 frames = 256 KB.

2.6 Exercises

Exercise 2.1 (Segment File and Byte-Offset Arithmetic)**.** A production ledger table has `pg_class.oid = 41200`, `pg_class.relfilenode = 58910` inside database `oid = 16384`, and currently holds 420,000 heap blocks.

(a) List the exact filenames (and exact byte sizes) of all main-fork segment files on disk under `$PGDATA/base/16384/`.

(b) An index lookup returns `ctid = (275000, 14)`. Which exact segment file does PostgreSQL open, at what byte offset inside that file does the 8 KB page begin, and at what exact byte offset inside that 8 KB page does the 4-byte `ItemIdData` for slot 14 sit?

(c) How many 8 KB blocks does the Visibility Map file `58910_vm` require to track all 420,000 heap blocks?

Exercise 2.2 (Tracing Slotted-Page Offsets and In-Page Compaction)**.** Start with a freshly initialized empty 8,192-byte heap page (`pd_lower` = 24, `pd_upper` = 8,192, `pd_special` = 8,192). Four tuples with unpadded lengths `lp_len` = $52, 75, 40, 61$ bytes are inserted in order as Slots 1, 2, 3, and 4.

(a) Accounting for 8-byte `MAXALIGN` when carving tuple space from `pd_upper`, give the exact values of `pd_lower`, `pd_upper`, contiguous free bytes, and each slot's `(lp_off, lp_flags, lp_len)` after all four inserts.

(b) Suppose Tuples 1 and 3 are deleted, their deleting transactions commit behind every active snapshot, and PostgreSQL runs `PageRepairFragmentation` (before index vacuuming). Give the new `pd_lower`, `pd_upper`, and all four slots' `(lp_off, lp_flags, lp_len)`.

(c) Next, `VACUUM` removes the index entries for Slots 1 and 3, marking those slots `LP_UNUSED` and setting `PD_HAS_FREE_LINES` in `pd_flags`. A fifth tuple of length `lp_len` = 46 bytes is now inserted. Which slot number does it receive, and what are the resulting `pd_lower` and `pd_upper`?

Exercise 2.3 (Null Bitmaps and the 9th-Column `t_hoff` Cliff)**.** Consider two schemas where every column is `SMALLINT` (2 bytes, 2-byte aligned): `t8` has 8 `SMALLINT` columns, and `t9` has 9 `SMALLINT` columns. Compute `t_hoff`, raw `lp_len`, and the `MAXALIGN`ed page footprint (including the 4-byte `ItemIdData` slot) for:

(a) A row in `t8` with zero `NULL`s versus a row in `t8` where columns 2 and 7 are `NULL`.

(b) A row in `t9` with zero `NULL`s versus a row in `t9` where column 9 alone is `NULL`. Explain why setting column 9 to `NULL` actually makes the row *8 bytes larger* on the page than storing a concrete integer in column 9.

Exercise 2.4 (Column Tetris on a 100-Million-Row Sensor Table)**.** An IoT team proposes the following schema for a 100-million-row table (all columns `NOT NULL`):

```
CREATE TABLE telemetry (
    online       BOOLEAN,
    device_id    BIGINT,
    firmware_ver SMALLINT,
    recorded_at  TIMESTAMPTZ,
    battery_pct  SMALLINT,
    reading_val  DOUBLE PRECISION,
```

```
    sensor_slot  BOOLEAN,
    seq_no       INTEGER,
    trace_uuid   UUID
);
```

(a) Compute the exact offset of every column, total internal + trailing padding bytes, `lp_len`, `MAXALIGN`ed tuple size, rows per 8 KB page, and total table size in GiB for 100,000,000 rows. (Watch `UUID`'s alignment carefully.)

(b) Write an optimal `CREATE TABLE telemetry_packed` ordering that minimizes tuple size without changing any column name or data type. Compute the new bytes saved per row, rows per 8 KB page, and total GiB saved across 100,000,000 rows.

Exercise 2.5 (Decoding Raw Hex Bytes from `pageinspect`)**.** On an x86-64 Linux PostgreSQL server, `heap_page_item_attrs` on a 3-column table `(id INT4, code TEXT, price_cents INT8)` returns a tuple with `t_hoff` = 24, `t_infomask` = `0x0903`, `t_bits` = `'10100000'`, and `t_attrs` = {''\x2a000000'', NULL, ''\x1027000000000000''}.

(a) Decode the exact SQL values of `id`, `code`, and `price_cents`, and explain every set bit of `t_infomask = 0x0903`.

(b) Compute the exact `lp_len` of this tuple, paying close attention to alignment padding before `price_cents` when `code` is `NULL`.

(c) If `code` is updated from `NULL` to `'EUR'`, write the exact hex bytes of `t_attrs[2]` (including its 1-byte `varlena` header) and compute the new `lp_len`.

Exercise 2.6 (TOAST Compression, Chunk Counts, and `EXTERNAL` Slicing)**.** A table `documents` with three `NOT NULL` columns (`doc_id BIGINT`, `title TEXT`, `body TEXT`) stores a row with a 20-byte ASCII `title` and a 120,000-byte `body`.

(a) Under default `EXTENDED` storage with `lz4`, `body` compresses by 3× to 40,000 bytes. How many rows are inserted into the `pg_toast` table (assuming `TOAST_MAX_CHUNK_SIZE` = 1,996 B), how many 8 KB TOAST heap pages do they occupy, and what is the exact `lp_len` of the main heap row in `documents`?

(b) Suppose a hot API endpoint runs `SELECT substring(body, 110000, 64) FROM documents WHERE doc_id = $1` ten thousand times per second. Compare the exact number of TOAST chunks fetched and decompressed per query under `EXTENDED` versus `ALTER TABLE documents ALTER COLUMN body SET STORAGE EXTERNAL`. When does `EXTERNAL` win despite using 3× more disk space?

Exercise 2.7 (Clock-Sweep Hand Simulation)**.** A 5-frame toy buffer pool (`F0..F4`) starts with `nextVictimBuffer = F1` and states `(refcount, usage_count)`:

`F0:(0, 3)`, `F1:(1, 1)`, `F2:(0, 2)`, `F3:(0, 0)`, `F4:(0, 4)`.

(a) Backend A misses on block 10 and calls `StrategyGetBuffer`. Trace every tick of the clock hand, give the victim frame, the new state of all 5 frames (assuming Backend A keeps block 10 pinned for a moment), and the new position of `nextVictimBuffer`.

(b) Immediately afterward, Backend A unpins its buffer (`refcount` : 1 → 0) while Backend B unpins `F1` (`refcount` : 1 → 0), and Backend C misses on block 11. Trace `StrategyGetBuffer`

again from the current `nextVictimBuffer` until the second victim is chosen, listing the final `(refcount, usage_count)` of every frame.

Exercise 2.8 (Sizing Ring Buffers vs. Cache Pollution)**.** A server has `shared_buffers` = 32 GiB. Two workloads run concurrently: 4,000 OLTP point lookups per second hitting a 20 GiB hot index/heap working set, and a batch ETL job running `Seq Scan` over a 2 GiB staging table `raw_imports` ten times per minute.

(a) Using PostgreSQL's `shared_buffers`/4 threshold for `BAS_BULKREAD`, determine whether `Seq Scan` on `raw_imports` uses the 256 KB ring buffer or allocates straight from the main pool, and explain why.

(b) Now suppose `raw_imports` grows from 2 GiB to 12 GiB. How does `Seq Scan`'s buffer allocation behavior suddenly change, and what happens to `EXPLAIN (ANALYZE, BUFFERS) shared hit` vs. `shared read` counts across consecutive scans of `raw_imports`?

Chapter 3

Indexes: Lehman-Yao B-Link Trees, GIN, GiST, and BRIN

At the end of Chapter 2, we left our data sitting inside an array of 8,192-byte slotted heap pages. Every row lives at a physical address `ctid` = (block, offset): a 32-bit block number naming the 8 KB page inside the relation fork, paired with a 16-bit line-pointer slot inside that page's `ItemIdData` array. If someone hands you `ctid` = (8412, 9), PostgreSQL can pin block 8,412 in `shared_buffers`, read slot 9 at a known byte offset, jump to `lp_off`, and hand you the tuple in microseconds.

The trouble is that SQL queries never hand you a `ctid`. An API request arrives asking for every ledger posting belonging to `account_id = 4291804` since October 1, or the single user row matching `email = 'ada@lovelace.org'`. Because heap pages accept new tuples wherever the Free Space Map (`_fsm`) finds room, rows have no guaranteed physical order on disk. Without an auxiliary directory mapping logical column values to physical `ctid`s, the executor has only one possible plan: pin block 0, inspect every live tuple, pin block 1, inspect every live tuple, and keep walking until `EOF`.

Focus

In this chapter we build PostgreSQL's indexing subsystem from physical byte layouts up to production query geometry:

1. **Fanout arithmetic and 8 KB `nbtree` pages:** why binary trees fail on block storage, how branching factor $B \approx 350$ compresses 100 million rows into three cached hops, and how `BTPageOpaqueData` links leaves into a doubly-linked ribbon.
2. **Lehman–Yao high-concurrency B-link trees:** why textbook top-down latch crabbing stalls multi-core servers, and how PostgreSQL's `P_HIKEY` slot plus right-sibling link (`btpo_next`) lets readers traverse unfinished page splits without holding two page locks during descent.
3. **Composite index geometry:** how multi-column keys flatten 2D predicates into a 1D lexicographic ribbon, why equality-before-range reads one contiguous leaf slice while range-before-equality scatters reads, and when `ORDER BY ... LIMIT` eliminates sorting altogether.
4. **Covering indexes (`INCLUDE`) and the Visibility Map (`_vm`):** why PostgreSQL index tuples store zero MVCC visibility metadata, and how a 2-bit-per-page bitmap rescues

true Index-Only Scans.

5. **Partial, expression, and specialized access methods:** shrinking queue indexes by 2,000× with `WHERE`, aligning functional predicates, and choosing cleanly among **Hash**, **GIN** (inverted trees with `fastupdate`), **GiST** (bounding hierarchies and exclusion constraints), and **BRIN** (min/max block-range summaries).

3.1 Why Secondary Indexes Exist and B+Tree Branching Math

Put concrete numbers on that sequential heap scan first. Suppose our `postings` table holds $N = 100{,}000{,}000$ (10^8) rows. Each row stores a `BIGINT` primary key, an `account_id`, an `amount_cents`, a `TIMESTAMPTZ`, a 3-byte currency code, and status flags—roughly 72 bytes of payload plus the 24-byte `HeapTupleHeaderData`, or 96 bytes per tuple. Add the 4-byte `ItemIdData` line pointer from Chapter 2 and each row consumes 100 bytes inside an 8,192-byte page. Subtracting the 24-byte `PageHeaderData` leaves 8,168 usable bytes, packing $\lfloor 8168/100 \rfloor = 81$ rows per page.

Across 10^8 rows, the heap spans

$$\left\lceil \frac{100{,}000{,}000}{81} \right\rceil = 1{,}234{,}568 \text{ pages} \times 8{,}192 \text{ bytes} \approx 10.11 \text{ GB}.$$

Even if a sequential scan streams through Chapter 2's 256 KB `BAS_BULKREAD` ring buffer at 2.5 GB/s on a fast NVMe array, reading 10.11 GB to answer one point lookup takes four full seconds and burns 2.5 GB/s of storage bandwidth. At 500 user requests per second, those scans would need 2,000 seconds of disk time every second, so the system cannot keep up.

3.1.1 Why Binary Search Trees Lose Badly on Block Devices

Your algorithms instinct says: keep a balanced binary search tree over the keys. For $N = 10^8$, an AVL or red-black tree has height $\lceil \log_2(10^8) \rceil = 27$. Twenty-seven comparisons in CPU registers take less than a microsecond. This is Java's `TreeMap`: a red-black tree in which every entry is its own heap object holding `left`, `right`, and `parent` references. In RAM that design works well, because following a reference costs a few nanoseconds.

On block storage, however, comparisons are free and *page misses* dominate everything. Storage hardware and `shared_buffers` move data in 8,192-byte pages, never 32-byte tree nodes. If you allocate binary tree nodes inside 8 KB pages as rows arrive over months of random traffic, parent and child nodes land on unrelated pages. Descending 27 pointer hops where the top 10 levels sit in RAM still triggers ≈ 17 sequential dependent random 8 KB reads from NVMe. At 70 μs per random NVMe read, 17×70 μs ≈ 1.2 ms of pure I/O stall—and every 8 KB transfer hauled 8,160 bytes of unrelated neighbors across the PCIe bus just to read one 8-byte key and two child pointers.

The remedy, introduced by Rudolf Bayer and Edward McCreight in 1970 and refined into the **B+tree** showcased in Douglas Comer's 1979 survey, is to design the tree node around the 8,192-byte page itself. Instead of 2 children per node, pack *hundreds* of separator keys and child block numbers into a single 8 KB internal page, and push all heap `ctid` payloads down to the bottom leaf level.

Example 3.1 (Exact Fanout Arithmetic on an 8 KB `nbtree` Page)**.** Let us count the exact bytes inside an 8,192-byte PostgreSQL B-tree (`nbtree`) internal page indexing an 8-byte `BIGINT` column such as `account_id`:

- **Fixed page overhead:** 24-byte `PageHeaderData` at the top of the page, plus a 16-byte `BTPageOpaqueData` trailer in the special space at the bottom (`pd_special` $= 8192-16 = 8176$). That leaves $8192 - 40 = 8{,}152$ bytes for line pointers and index tuples.
- **Per-entry byte cost:** Each entry requires a 4-byte `ItemIdData` line pointer plus an `IndexTupleData` struct. As we will inspect shortly in C, `IndexTupleData` has an 8-byte header (`t_tid` storing 6 bytes of block/offset metadata plus 2 bytes of `t_info` length and null flags), followed immediately by the 8-byte `BIGINT` key. Because $8 + 8 = 16$ is already a multiple of `MAXALIGN` (8 bytes on 64-bit Linux), zero padding is added. Each entry costs $4 + 16 = 20$ bytes.
- **Theoretical and practical branching factor:** Dividing 8,152 usable bytes (minus one 20-byte slot reserved for the page's high key) by 20 bytes yields $\lfloor 8132/20 \rfloor = 406$ child downlinks on a completely full internal page. In real production indexes, internal pages sit between 70% and 95% occupancy after splits, and composite or `UUID` keys are slightly wider (24–28 bytes per entry). That puts the everyday internal branching factor right around $B \approx 350$.

Now watch what $B = 350$ does to tree height as we descend from the root:

Tree Level	Pages	Size (8 KB/pg)	Where It Lives in Production
Level 2 (Root page)	1	8 KB	Permanently hot in `shared_buffers`
Level 1 (Internal)	350	2.73 MB	Permanently hot in `shared_buffers`
Level 0 (Leaves, 350^2)	122,500	957.03 MB	Hot subset in RAM; cold on NVMe

At default `fillfactor = 90`, each leaf page uses $0.90 \times 8132 \approx 7{,}318$ bytes, holding $\lfloor 7318/20 \rfloor = 365$ `(account_id, heap_ctid)` entries. Multiply 122,500 leaf pages by 365 entries per leaf: **three index levels cover** 44,712,500 **rows.** Add just one more internal level ($350^3 = 42{,}875{,}000$ leaf pages, with levels 3, 2, and 1 totaling $1 + 350 + 122{,}500 = 122{,}851$ internal pages ≈ 960 MB of RAM) and a four-level B+tree addresses **over 15.6 billion rows**.

Pause on the RAM column of Example 3.1. To find a single row out of 44.7 million:

1. Pin the root page (in RAM, 0 disk reads) and binary-search 350 keys ($\lceil \log_2 350 \rceil = 9$ CPU comparisons) to pick 1 of 350 level-1 child blocks.
2. Pin that level-1 internal page (part of a 2.73 MB working set that never leaves `shared_buffers`, 0 disk reads) and run 9 CPU comparisons to pick 1 of 122,500 leaf blocks.
3. Pin the leaf page (at most 1 NVMe read if cold) and binary-search its 365 entries to extract `ctid` $= (8412, 9)$.
4. Pin heap block 8,412 (at most 1 NVMe read if cold) and read tuple slot 9.

Instead of reading 1,234,568 pages (10.11 GB), PostgreSQL pins **4 buffers total**, performing **0 to 2 physical NVMe reads** (16 KB worst case)—a 630,000× reduction in I/O.

3.1.2 Inside an 8 KB `nbtree` Page: `IndexTupleData` and `BTPageOpaqueData`

Because PostgreSQL reuses the exact same 8,192-byte slotted page container from Chapter 2 for both tables and indexes, an `nbtree` page starts with the familiar 24-byte `PageHeaderData` and an array of 4-byte `ItemIdData` line pointers growing downward toward `pd_lower`. Two things change inside an index page: the tuple format at `pd_upper`, and the 16-byte trailer sitting at `pd_special`. Here are the core structures from `src/include/access/itup.h` and `src/include/access/nbtree.h`:

```
/* 8-byte header prepended to every index entry (src/include/access/itup.h) */
typedef struct IndexTupleData {
    ItemPointerData t_tid;   /* 6 bytes: heap ctid on leaf, or child blkno on internal */
    unsigned short  t_info;  /* 2 bytes: bit 15 = has nulls, bit 14 = has varlena,
                                         bits 12..0 = tuple byte length (max 8191) */
} IndexTupleData;            /* Key column bytes follow immediately after t_info */

/* 16-byte trailer at offset 8176..8192 of every nbtree page (nbtree.h) */
typedef struct BTPageOpaqueData {
    BlockNumber btpo_prev;   /* 4 bytes: left sibling block (P_NONE = 0 if leftmost) */
    BlockNumber btpo_next;   /* 4 bytes: right sibling block (P_NONE = 0 if rightmost) */
    uint32      btpo_level;  /* 4 bytes: tree level (0 = leaf, >= 1 = internal;
                                         reused for xact epoch on deleted pages) */
    uint16      btpo_flags;  /* 2 bytes: BTP_LEAF, BTP_ROOT, BTP_META,
                                         BTP_DELETED, BTP_HALF_DEAD, BTP_INCOMPLETE_SPLIT
    */
    BTCycleId   btpo_cycleid;/* 2 bytes: vacuum cycle ID preventing split-scan races */
} BTPageOpaqueData;
```

Three details in those 24 bytes of C declarations matter:

- **Dual use of `t_tid`:** On a leaf page (`btpo_level` == 0), `t_tid` holds the 6-byte heap `ctid` (block, offset) of the table row. On an internal branch page (`btpo_level` ≥ 1), there is no heap row to point to—so `t_tid`'s 4-byte block field stores the *child index page's block number*, while its 2-byte offset field is repurposed to pack metadata flags (such as `BT_STATUS_OFFSET_MASK` and column count overrides during suffix truncation).
- **Dynamic root via Block 0's metapage:** Block 0 of every `nbtree` file is never a tree node; it carries `BTP_META` and stores `BTMetaPageData`, including `btm_root` (the current root block number) and `btm_level` (its height). When a single-leaf index splits for the first time, PostgreSQL allocates a second leaf and a new root page at block 3, updating `btm_root = 3` rather than copying tuples into block 1. Backends cache `btm_root` in backend-local `relcache` memory so normal lookups jump straight to the root without even pinning block 0.
- **Bidirectional sibling chains (`btpo_prev` and `btpo_next`):** Every level of the tree—especially leaf level 0—forms a doubly-linked horizontal list of sorted pages. Once a range query descends from the root to the starting key `account_id` = 42, `created_at` = '2026-10-01', it *never climbs back up to internal parent pages*. Forward scans (`ASC`) walk horizontally via `btpo_next`; backward scans (`DESC`) walk via `btpo_prev`.

Intuition

Why B+trees keep heap pointers strictly at the leaves. In Rudolf Bayer's original 1970 B-tree, internal nodes also carried data pointers alongside separator keys. In a B+tree,

internal nodes act purely as *road signs* (copied or truncated separator keys + 4-byte child block numbers), and *every* key-to-`ctid` entry appears in the sorted leaf chain. That tiny duplication buys two huge payoffs: first, internal nodes stay narrow (especially when payload columns are wide or `INCLUDE`d), maximizing fanout B; second, a range scan across 5,000 rows simply marches straight across adjacent leaves instead of performing an in-order tree traversal up and down internal levels.

3.2 From Textbook B+Trees to PostgreSQL's Lehman–Yao B-Link Tree

If you implement a textbook B+tree and run it single-threaded on your laptop, it flies. Now run 128 concurrent OLTP worker processes inserting orders and reading balances at 80,000 transactions per second. Within milliseconds, a naive textbook concurrency protocol grinds all 128 cores to a halt on a single 64-byte CPU cache line: the `LWLock` inside the root page's `BufferDesc`.

3.2.1 Why Naive Top-Down Latch Crabbing Limits Multi-Core Scaling

To see why, trace what can go wrong if two backends touch a B+tree without synchronization:

1. **Reader vs. concurrent leaf split:** Reader Q reads internal parent R, learns that keys in $(40, 100]$ live on child leaf P (block 17), and pauses for 2 μs on an OS context switch before pinning block 17. During those 2 μs, Writer W fills block 17, splits it into block 17 covering $(40, 60]$ and block 41 covering $(60, 100]$, and moves keys $\{71, 82, 85, 94\}$ off block 17 onto block 41. When Reader Q wakes up and pins block 17 looking for key $k = 85$, key 85 is gone. A naive reader concludes 85 does not exist and returns zero rows for a committed account.
2. **Cascading splits upward:** When Writer W descends to insert a key into leaf P, P might be full. Splitting P inserts a new separator key into parent R—which might *also* be full, splitting R and inserting into the grandparent, all the way up to the root.

The classic textbook answer is **pessimistic latch crabbing** (lock coupling):

- A *reader* holds a shared latch on parent R, acquires a shared latch on child P, and *only then* releases R.
- A *writer* descending the tree holds **exclusive latches** on its ancestor chain all the way down until it reaches a "safe" internal node (one with enough free space that a child split cannot force *it* to split). Even under optimistic retry protocols, any split near full internal nodes forces writers to re-descend acquiring exclusive upper latches while readers constantly toggle atomic reference counts on the root's `LWLock`.

Holding two latches during every parent-to-child step sounds harmless until you count cache-coherency traffic: even shared read latches execute atomic `fetch_add` instructions on `BufferDesc.state`, bouncing the root page's cache line across CPU sockets twice per query. Worse, holding a parent latch while waiting for a child page to be read from NVMe stalls every concurrent writer whose subtree passes through that parent.

In Java 6 terms, latch crabbing is hand-over-hand locking: `synchronized (parent) { synchronized (child) { ... } }`, releasing the parent once the child is held. Java 6 also shipped the alternative in `java.util.concurrent.ConcurrentSkipListMap`: readers traverse without taking any lock, and writers publish new links with a single compare-and-swap so a reader always sees either the old or the new structure. Lehman and Yao applied the same principle to disk pages, as the next subsection shows.

3.2.2 The Lehman–Yao Insight: `P_HIKEY` and Right-Links (`btpo_next`)

In 1981, Philip Lehman and S. Bing Yao published *Efficient Locking for Concurrent Operations on B-Trees* (later extended to deletions by Vladimir Lanin and Dennis Shasha in 1986), introducing the **B-link tree**. PostgreSQL's `src/backend/access/nbtree/` is a production implementation of Lehman–Yao and Lanin–Shasha, documented in one of the clearest design essays in open-source systems software (`src/backend/access/nbtree/README`).

Lehman and Yao asked a deceptively simple question: *why did Reader Q get lost when Leaf P split?* Two things lacked local witnesses on P itself:

1. Looking only at block 17, Q had *no way to tell* that key 85 used to belong on block 17 and had just migrated rightward.
2. Even if Q knew 85 had moved off block 17, Q had no direct pointer to the new sibling page because the parent R might not even be updated yet.

The B-link tree adds two structural invariants to every page at every level of the tree:

Definition 3.1 (Lehman–Yao B-Link Page Invariants in PostgreSQL)**.** Every live `nbtree` page P (leaf or internal) maintains two local fields inside its own 8 KB image:

1. **Right-sibling link (`btpo_next`):** Every non-rightmost page stores the block number of its immediate right neighbor at the same tree level (`btpo_next` $\neq$ `P_NONE`). When P splits into P and P', P.`btpo_next` points to P' *before* P's exclusive lock is released.
2. **Explicit High Key (`P_HIKEY`, slot 1):** On every non-rightmost page ($\neg$`P_RIGHTMOST`(opaque)), line-pointer slot 1 (`P_HIKEY = 1`) stores an upper-bound key K_{high} rather than data. Every key k currently residing on P satisfies $k \leq K_{\text{high}}$, and any key strictly greater than K_{high} that reached P via an older parent downlink belongs somewhere to the right via `btpo_next`. Real data items begin at `P_FIRSTDATAKEY`: slot 2 on non-rightmost pages, and slot 1 on the rightmost page of each level (whose implicit upper bound is $+\infty$).

History

From Berkeley Postgres to Lehman–Yao. Early university Postgres used a simpler locking discipline over its B-trees that suffered severe contention under multi-user benchmarks. Paul Aoki and Mike Olson replaced it with the Lehman–Yao `nbtree` ("New B-Tree") access method in the early 1990s; Tom Lane later integrated Lanin–Shasha concurrent page deletion, and Peter Geoghegan re-architected tuple uniqueness tie-breaking, suffix truncation, deduplication, and bottom-up deletion across PostgreSQL 12–14. The folder name `nbtree` still preserves that 1990s "New B-Tree" moniker today.

3.2.3 Step-by-Step Trace: A Concurrent Reader Crossing an Unfinished Split

Let us trace every single lock and pointer step with concrete numbers so you can watch Lehman–Yao eliminate downward lock coupling.

Suppose internal parent R (block 4, `btpo_level` = 1) currently routes keys in $(40, 100]$ to leaf P (block 17) and keys > 100 to leaf block 29. Leaf P (block 17) has `P_HIKEY` = 100 at slot 1, data keys $\{42, 49, 55, 60, 71, 82, 94\}$ in slots 2..8, and `btpo_next` = 29.

Now Writer W arrives to insert key $k = 85$ (pointing to heap `ctid` = $(912, 6)$), and Leaf P has zero free bytes left:

1. **Phase 1 of Split (Bottom-Up, Leaf Only):** Writer W holds an exclusive content lock on P (block 17) and allocates a fresh page P' at block 41, locking block 41 exclusively as well. Notice that W holds **zero locks on parent R (block 4)**. Choosing split point 60, W rebuilds both pages in memory:
   - Right sibling P' (block 41) inherits the old high key (`P_HIKEY` = 100), receives upper keys $\{71, 82, 85, 94\}$, sets `btpo_prev` = 17, and sets `btpo_next` = 29 (updating block 29's `btpo_prev` to 41).
   - Original leaf P (block 17) keeps lower keys $\{42, 49, 55, 60\}$, lowers its own high key to `P_HIKEY` = 60, sets `btpo_next` = 41, and sets the `BTP_INCOMPLETE_SPLIT` flag in `btpo_flags` as a breadcrumb that parent R does not yet know about block 41.

   W writes a single atomic WAL record covering the split and **releases its exclusive locks on both block 17 and block 41**.
2. **The Half-Split Window (Reader Q Descends Lock-Couple-Free):** At this exact microsecond, Writer W is preempted by the OS kernel before touching parent R (block 4). Reader Q begins executing `SELECT * FROM accounts WHERE account_id = 85`:
   - Q acquires a shared lock on root/parent R (block 4), binary-searches R's stale separator keys, sees $40 < 85 \leq 100 \implies$ child block 17, **releases the shared lock on block 4 completely**, and then acquires a shared lock on block 17 (P). At the instant between releasing block 4 and locking block 17, Q holds **zero page latches anywhere in the tree**.
3. **Horizontal Recovery via `_bt_moveright`:** Reader Q inspects block 17. Before binary-searching data slots 2..5, PostgreSQL's `_bt_moveright()` routine checks one condition: is block 17 non-rightmost, and is our target key strictly past slot 1 (`P_HIKEY`)?

   $$k = 85 \ > \ P.\texttt{P_HIKEY} = 60 \quad \implies \quad \text{Block 17 split after we read its parent.}$$

   Without ever re-visiting parent R, Q reads `btpo_next` = 41 from block 17's footer, steps rightward (coupling left-to-right: lock block 41 shared, unlock block 17), checks $85 \leq P'.\texttt{P_HIKEY}$ (100), binary-searches slots 2..5 of block 41 via `_bt_binsrch()`, and finds $(85, \texttt{ctid} = (912, 6))$ at slot 4.
4. **Phase 2 of Split (Parent Downlink Insertion):** Writer W wakes up, locks parent R (block 4) and briefly re-locks child P to clear `BTP_INCOMPLETE_SPLIT`, inserts the new separator tuple ($60 \rightarrow$ blk 41) into R, and unlocks. Future readers searching for 85 now jump straight from block 4 to block 41 in one hop.

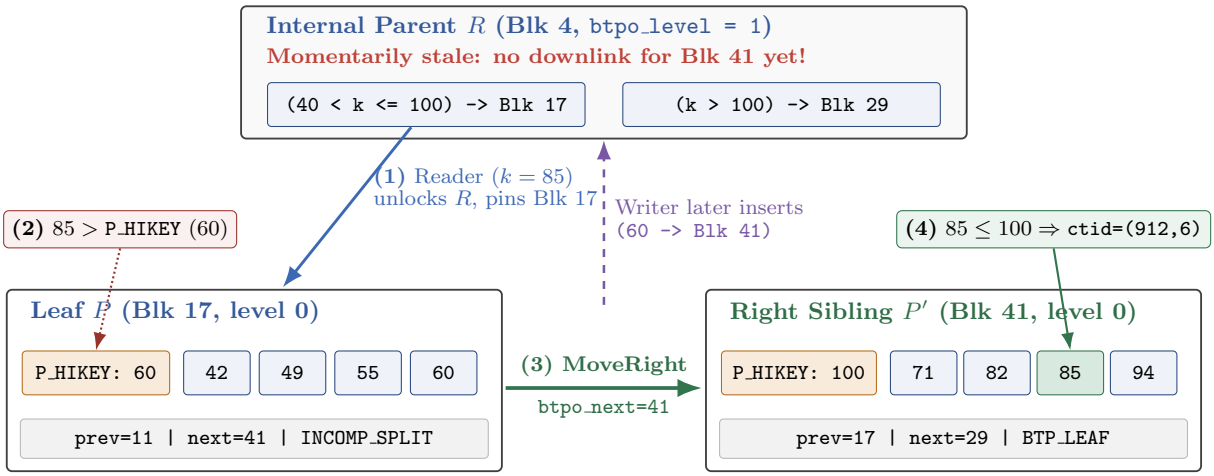


Figure 3.1: Lehman–Yao B-link split in action. Writer splits Leaf P (Block 17) into P and P' (Block 41), installs lowered `P_HIKEY = 60` and `btpo_next = 41`, and unlocks both leaves *before* updating Parent R. A concurrent reader seeking $k = 85$ follows the stale parent pointer to Block 17 without holding two locks during descent, sees $85 > 60$, and steps right via `_bt_moveright` to find 85 on Block 41.

Why it works

Why Lehman–Yao cannot deadlock (even if the writer crashes mid-split). Notice the lock acquisition order across the entire tree:

- Descending readers hold *at most one page latch at a time* (release parent, then lock child). A thread holding zero locks while requesting one cannot participate in a lock-wait cycle.
- Horizontal `_bt_moveright` steps acquire locks strictly *left-to-right* (Blk 17 → Blk 41). Splitting writers and parent insertions also acquire sibling locks strictly left-to-right and climb strictly *bottom-up* (releasing leaf locks before locking the parent, then re-acquiring left-to-right only when clearing `BTP_INCOMPLETE_SPLIT`). Because the dependency graph of simultaneously held latches is a strict DAG, deadlocks among page latches are impossible by construction.
- Even if the power cord is yanked out of the wall between Step 1 and Step 4—so the parent downlink for Block 41 is *never* written before crash recovery ends—the index remains 100% readable and accurate. Every lookup for keys in $(60, 100]$ simply takes one extra `_bt_moveright` hop from Block 17 to Block 41 until the next split on that parent notices `BTP_INCOMPLETE_SPLIT` and finishes inserting the missing downlink.

3.2.4 Modern `nbtree` Engineering: Tie-Breaking, Deduplication, and Bottom-Up Deletion

Three subtle real-world hazards threatened classical B-link trees under heavy OLTP workloads until PostgreSQL 12, 13, and 14 solved them cleanly:

1. **Unique keys in internal nodes via heap `ctid` tie-breaking (PG 12):** What happens if you index a low-cardinality column like `status = 'PENDING'` where 2,000,000 rows share the

exact same key? In older PostgreSQL versions, duplicate keys had arbitrary physical order across thousands of identical leaf pages, turning inserts into linear scans across duplicate runs. Since PostgreSQL 12, `nbtree` appends the 6-byte heap `ctid` as an implicit final tie-breaker attribute during comparisons. Every index entry $(k, \texttt{ctid})$ is now *globally unique*, so binary search lands directly on the exact leaf page where a new duplicate belongs in $O(\log N)$ time.

2. **B-tree deduplication and posting list tuples (PG 13):** Making duplicates fast still left them fat: storing `'USD'` 365 times on a leaf page repeats key headers and string bytes 365 times. With `deduplicate_items = on` (the default since PG 13), when a leaf page fills up, `nbtree` runs a deduplication pass before splitting. Identical keys merge into a single **posting list tuple** flagged with `BT_IS_POSTING`: one copy of the key `'USD'` followed by a packed array of 6-byte heap `ItemPointerData` (`ctid`s), shrinking foreign-key and status indexes by 3× to 4×!

3. **Bottom-up index deletion against version churn (PG 14):** Recall from Chapter 2 (and Chapter 5's upcoming MVCC analysis) that updating a heap row creates a new tuple version at a new `ctid`. Whenever *any* indexed column changes, PostgreSQL cannot use a Heap-Only Tuple (HOT) update—it must insert a fresh index entry into *every* index on the table, even indexes whose own columns did not change. Before PG 14, updating a `last_seen_at` column 50 times on an account forced 50 dead duplicate entries into `idx_accounts_email`, splitting leaf pages irreversibly (index bloat) before `autovacuum` ever arrived. PostgreSQL 14 introduced **bottom-up index deletion**: right before a leaf split caused by duplicate keys, `nbtree` visits the heap pages pointed to by duplicate entries on that leaf, reclaims known-dead `ctid` slots in place, and cancels the page split completely.

Try it in PostgreSQL

Inspect a real `nbtree` metapage, `P_HIKEY` slot, and PG 13+ deduplicated posting list tuple using PostgreSQL's built-in `pageinspect` extension:

```
CREATE EXTENSION IF NOT EXISTS pageinspect;

CREATE TABLE ledger_demo (
    posting_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    currency   CHAR(3) NOT NULL,
    amount     BIGINT  NOT NULL
);

INSERT INTO ledger_demo (currency, amount)
SELECT CASE WHEN mod(g, 5) = 0 THEN 'EUR' ELSE 'USD' END, g * 100
FROM generate_series(1, 5000) AS g;

CREATE INDEX idx_ledger_currency ON ledger_demo (currency);

-- 1. Inspect Block 0 (the metapage) to find root block & tree height
SELECT version, root, level, fastroot, fastlevel
FROM bt_metap('idx_ledger_currency');
--  version | root | level | fastroot | fastlevel
-- ---------+------+-------+----------+-----------
--        4 |    3 |     1 |        3 |         1

-- 2. Inspect Leaf Block 1: slot 1 is P_HIKEY; slots 2+ are posting lists!
```

```
SELECT itemoffset AS off, ctid, itemlen, htid, tids[1:2] AS sample_tids, data
FROM bt_page_items('idx_ledger_currency', 1)
LIMIT 4;
--  off |  ctid   | itemlen |  htid  |     sample_tids     |    data
-- -----+---------+---------+--------+---------------------+------------
--    1 | (0,1)   |      16 | (5,42) | NULL                | 55 53 44 20
--    2 | (16,42) |     808 | (0,5)  | {"(0,5)","(0,10)"}  | 45 55 52 20
--    3 | (16,42) |     808 | (1,28) | {"(1,28)","(1,33)"} | 45 55 52 20
--    4 | (16,22) |     224 | (3,11) | {"(3,11)","(3,16)"} | 45 55 52 20
```

Look at `off = 1`: `tids` is `NULL` because slot 1 on non-rightmost leaf Block 1 is `P_HIKEY` (`'USD'` with tie-breaker `htid` $= (5, 42)$, ASCII hex `55 53 44`). Look at `off = 2`: a single 808-byte posting list tuple packs **132 heap `ctids`** for `'EUR'` (`45 55 52`) at just $808/132 \approx 6.12$ bytes per row.

3.3 Composite Index Lexicographic Geometry and Column Order

Single-column B-trees are straightforward because numbers and timestamps already live on a 1D line. Real production queries, however, almost always filter on two or three dimensions at once:

```
SELECT posting_id, amount_cents, created_at
FROM postings
WHERE account_id = 42
  AND created_at >= '2026-10-01 00:00:00+00'
ORDER BY created_at DESC
LIMIT 20;
```

A B-tree leaf chain is strictly one-dimensional: every tuple sits before or after its neighbor along a single ribbon of pages linked by `btpo_next`. How do you map a 2D plane of (`account_id`, `created_at`) pairs onto a 1D leaf ribbon, and why does declaring `(account_id, created_at DESC)` run in 0.08 ms while declaring `(created_at DESC, account_id)` on the exact same two columns can take 450 ms?

3.3.1 The Phonebook Ribbon: Equality Pins a Slice, Range Scatters Jumps

Suppose we build:

```
CREATE INDEX idx_postings_acct_created
    ON postings (account_id ASC, created_at DESC);
```

PostgreSQL compares two composite keys (a_1, t_1) and (a_2, t_2) in **lexicographic (dictionary) order**, just like sorting a phonebook by (Last Name, First Name):

$$(a_1, t_1) \prec (a_2, t_2) \quad \Longleftrightarrow \quad (a_1 < a_2) \ \vee \ \big(a_1 = a_2 \ \wedge \ t_1 > t_2\big),$$

where the second inequality flips to $t_1 > t_2$ because we specified `created_at DESC`. In Java 6 this is exactly the `Comparator` you would hand to `Collections.sort` or to a `TreeMap`:

```
public int compare(Posting x, Posting y) {
    if (x.accountId != y.accountId)
```

```
        return x.accountId < y.accountId ? -1 : 1;  // ascending
    return y.createdAt.compareTo(x.createdAt);      // descending
}
```

The B-tree keeps its leaf entries in this order, so everything you know about a sorted `TreeMap` (cheap range scans with `subMap`, no help for a lookup on the second field alone) carries over directly.

Picture the resulting leaf pages stretched left-to-right across the floor. First come *all* postings for `account_id = 1`, sorted newest-to-oldest from 2026 back to 2019. Immediately after account 1's oldest 2019 posting sits account 2's newest 2026 posting, and so on.

Now trace two queries across that exact same leaf ribbon (illustrated side by side in Figure 3.2):

1. **Equality on leading column, range on second column (`account_id = 42 AND created_at >= '2026-10-01'`):** Fixing `account_id` $= 42$ collapses the outer dimension to a single contiguous sub-ribbon. Within that sub-ribbon, `created_at` is monotonically ordered from newest to oldest. PostgreSQL descends the tree directly to $(42, +\infty)$, reads consecutive index tuples while `created_at` $\geq$ '2026-10-01', and **halts the instant it encounters the first tuple with `created_at` $<$ '2026-10-01'**. Every single index tuple touched satisfies *both* predicates.

2. **Range on leading column, filter on second column (`account_id BETWEEN 41 AND 43 AND created_at >= '2026-10-25'`, or `amount_cents > 50000 AND created_at >= '2026-10-25'` on an `(amount_cents, created_at)` index):** Because the leading column varies across a range, October 2026 timestamps are *no longer contiguous* in the leaf chain. Account 41's late-October entries sit at the start of sub-ribbon 41, followed by *seven years of historical postings for account 41* (`2026-10-24` down to `2019-01-02`), before account 42's late-October entries appear. A naive index range scan starting at $(41, +\infty)$ and ending at $(43, \text{'2026-10-25'})$ must trudge through hundreds or thousands of irrelevant leaf pages between matching pockets, applying `created_at` $\geq$ '2026-10-25' as an in-index filter on every skipped entry.

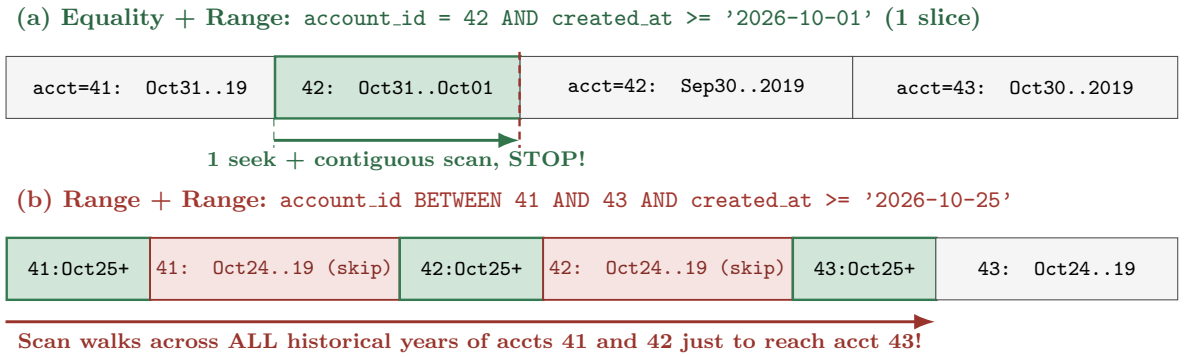


Figure 3.2: Lexicographic ribbon geometry on `(account_id ASC, created_at DESC)`. In (a), fixing `account_id = 42` isolates one sub-ribbon where `created_at` is contiguous. In (b), a leading range scatters recent timestamps into disconnected islands separated by years of cold index entries.

Common trap

`IN (...)` lists vs. true range inequalities. PostgreSQL's `nbtree` executor has a smart trick for small discrete `ScalarArrayOpExpr` predicates like `WHERE account_id IN (41, 42, 43) AND created_at >= '2026-10-25'`: instead of blindly scanning every historical entry of account 41, `_bt_advance_array_keys()` detects the end of $(41, \text{'2026-10-25'})$ and leaps directly to $(42, +\infty)$. However, once your leading predicate is a continuous inequality (`amount_cents > 50000`, `created_at >= ...`, or a wide `BETWEEN`), there is no small list of discrete keys to leap across. Follow the golden ordering rule for composite B-trees:

1. Equality (`=`) $\longrightarrow$ 2. Range / `ORDER BY` $\longrightarrow$ 3. `INCLUDE` payload.

And do not trust the folk myth "always put the highest-cardinality column first" blindly: if every query writes `WHERE tenant_id = $1 AND status = 'OPEN' AND created_at >= $2`, both `tenant_id` and `status` are equality predicates, so *both* belong ahead of the range column `created_at`, regardless of whether `status` only has four distinct values.

3.3.2 Sort Elimination and Backward Leaf Scans

Composite column order also governs whether PostgreSQL can stream `ORDER BY ... LIMIT` k in $O(k)$ page touches without allocating a `Sort` work buffer. Look back at our opening query:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT posting_id, amount_cents, created_at
FROM postings
WHERE account_id = 42
  AND created_at >= '2026-10-01 00:00:00+00'
ORDER BY created_at DESC
LIMIT 20;
-- Limit  (cost=0.56..28.41 rows=20 width=24) (actual time=0.031..0.068 rows=20 loops=1)
--   Buffers: shared hit=22
--   -> Index Scan using idx_postings_acct_created on postings
--        Index Cond: ((account_id = 42) AND (created_at >= '2026-10-01 00:00:00+00'::
    timestamptz))
```

Notice what is *missing* from that `EXPLAIN` output: there is **no `Sort` node**. Even if `account_id = 42` is a mega-merchant with 5,000,000 postings in October, PostgreSQL descends to $(42, +\infty)$, emits the first 20 live rows from the leaf ribbon, and stops after 22 buffer hits in 68 μs.

What if an analyst flips the sort direction and asks for `ORDER BY account_id DESC, created_at ASC` against our `(account_id ASC, created_at DESC)` index? Because *every* column's direction reversed simultaneously, the requested order is the exact mirror image of our leaf ribbon. PostgreSQL executes an `Index Scan Backward`, following `btpo_prev` links right-to-left with zero sorting. By contrast, if a query asks for `ORDER BY account_id ASC, created_at ASC` (matching the first column's direction while flipping only the second), neither forward nor backward traversal visits rows in that order—forcing the planner to insert an explicit `Sort` or PG 13+'s `Incremental Sort` (which sorts one `account_id` group at a time).

3.4 Covering Indexes (INCLUDE), the Visibility Map, and True Index-Only Scans

Look again at the `EXPLAIN (ANALYZE, BUFFERS)` plan at the end of Section 3.3: returning 20 rows from a 3-level B-tree touched **22 buffers**. Where did those 22 buffer hits come from? Three index pages (root, level 1, leaf) plus 19 separate 8 KB heap pages. Even when all 20 index entries sit shoulder-to-shoulder inside a single 8 KB leaf page, their heap `ctid`s point to 19 different historical heap blocks scattered across a 10 GB table file. If those 19 heap blocks are cold on NVMe, a dashboard summing `SUM(amount_cents)` across 10,000 postings triggers up to 10,000 random 8 KB heap reads (≈ 700 ms).

The obvious dream is: *if we store `amount_cents` and `currency` right inside the B-tree leaf tuple alongside `(account_id, created_at)`, can PostgreSQL answer the query from the index alone without ever touching the heap?*

3.4.1 Why PostgreSQL Index Tuples Carry Zero MVCC Visibility Metadata

In storage engines built around **clustered primary indexes** (such as MySQL's InnoDB or SQL Server), the table *is* the primary-key B+tree: leaf pages hold the full rows together with transaction IDs and undo-log pointers, while secondary indexes map secondary keys to the primary key.

PostgreSQL made the opposite architectural bet: tables are unclustered slotted heaps addressed by 6-byte physical `ctid`s, and **an `IndexTupleData` carries zero transaction visibility fields (`t_xmin`, `t_xmax`, `t_cid`)**. Go back and inspect `struct IndexTupleData` in Section 3.1: 6 bytes of `t_tid`, 2 bytes of `t_info`, and raw column bytes. Nothing else.

Intuition

Why omit `xmin` and `xmax` from every index? Count the write amplification if indexes stored MVCC transaction headers. Suppose your `orders` table has 6 secondary indexes. When a transaction runs `DELETE FROM orders WHERE order_id = 991`, heap-only MVCC simply stamps `t_xmax` on *one* 8 KB heap page—zero index pages are dirtied at `DELETE` time. If every index tuple carried its own `t_xmax`, every `DELETE` and every non-HOT `UPDATE` would have to descend and dirty all 6 secondary indexes synchronously inside the foreground transaction, multiplying random write I/O and WAL volume by 6×.

Omitting `xmin` and `xmax` from indexes makes writes fast, but it creates an immediate puzzle for reads: when an index scan sees a leaf tuple `(account_id=42, amount_cents=12500, ctid=(8412, 9))`, how does the executor know whether heap tuple `(8412, 9)` was committed five days ago, inserted two microseconds ago by an uncommitted transaction that is about to `ROLLBACK`, or already deleted by a concurrent session?

Before PostgreSQL 9.2, the answer was blunt: *it didn't*, so PostgreSQL had to visit the heap page on every single index match without exception.

3.4.2 How the Visibility Map (_vm) Rescues Index-Only Scans

PostgreSQL 9.2 solved the puzzle without adding a single byte to `IndexTupleData` by enlisting the **Visibility Map (_vm)** relation fork we met in Chapter 2.

Inside `$PGDATA/base/<dbOid>/<relfilenode>_vm`, PostgreSQL maintains exactly **2 bits per 8 KB heap page**:

- Bit 0 (`VISIBILITYMAP_ALL_VISIBLE`): when set to 1, `VACUUM` has verified that *every live tuple on this entire 8 KB heap block is committed and visible to all current and future database snapshots*, and no transaction has inserted, updated, or deleted any row on that block since.
- Bit 1 (`VISIBILITYMAP_ALL_FROZEN`): used by aggressive `VACUUM` to skip frozen pages entirely during transaction-ID wraparound maintenance (Chapter 5).

Look at the compression ratio of that bitmap: 2 bits per 8,192 bytes means 1 byte of _vm covers 4 heap pages (32 KB), and a single 8 KB _vm page (holding ≈ 8,160 payload bytes) tracks ≈ 32,640 heap blocks (255 MB of table data). For our entire 10.11 GB `postings` heap (1,234,568 pages), the entire _vm fork occupies just $\lceil 1{,}234{,}568/4 \rceil \approx$ **308** KB—roughly 38 pages pinned permanently in `shared_buffers`.

When all columns referenced by a query are available inside the index leaf tuple, PostgreSQL's `IndexOnlyScan` node runs a two-path check for each matching leaf entry `ctid` = (blk, off), shown in Figure 3.3:

1. Probe bit 0 for `blk` inside the in-memory _vm page.
2. **Fast path (`ALL_VISIBLE` == 1):** Every tuple on heap block `blk` is guaranteed visible. Emit the column values straight from the B-tree leaf tuple and advance to the next index entry. **Zero heap buffer pins, zero NVMe reads.**
3. **Fallback path (`ALL_VISIBLE` == 0):** A recent write modified heap block `blk` and cleared its _vm bit. Pin heap block `blk`, inspect slot `off`'s `t_xmin`/`t_xmax` against the query snapshot, emit the row only if visible, and increment the `Heap Fetches` counter reported by `EXPLAIN (ANALYZE, BUFFERS)`.

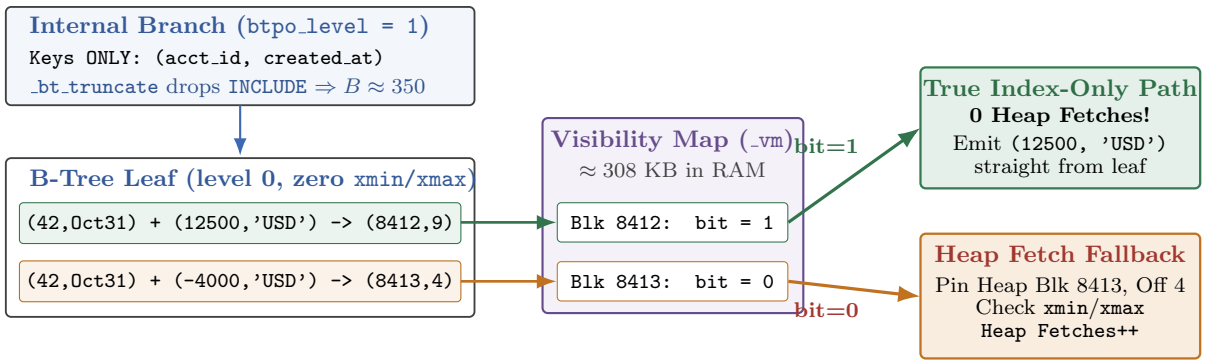


Figure 3.3: Covering index (`INCLUDE`) and Visibility Map (_vm) execution pipeline. Internal branch nodes drop `INCLUDE` attributes via `_bt_truncate` to stay narrow. At the leaf level, the executor probes the cached 2-bit-per-page _vm bitmap: bit 1 emits payload straight from the index with zero heap I/O; bit 0 falls back to pinning the heap block to verify MVCC visibility.

3.4.3 Why `INCLUDE` Beats Appending Payload Columns to the Key

Since PostgreSQL 11, you create a covering index using the `INCLUDE` clause:

```
CREATE INDEX idx_postings_acct_created_cov
    ON postings (account_id ASC, created_at DESC)
    INCLUDE (amount_cents, currency);
```

Why write `INCLUDE (amount_cents, currency)` instead of simply listing all four columns inside the key parentheses `(account_id, created_at, amount_cents, currency)`? Two physical and logical wins separate them:

1. **Suffix truncation keeps upper internal nodes tiny (`_bt_truncate`):** When a leaf page splits, PostgreSQL promotes a separator key into the parent internal page. For an `INCLUDE` index, `_bt_truncate()` unconditionally strips *all* non-key `INCLUDE` columns (and even strips trailing key columns that are unnecessary to distinguish the left leaf's max key from the right leaf's min key) before writing the parent tuple. Upper branch pages store only `(account_id, created_at)`, preserving a high branching factor $B \approx 350$ even if you `INCLUDE` a 64-byte merchant description.

2. **Clean uniqueness constraints:** Suppose `users.email` must be unique and your authentication hot path runs `SELECT user_id, pw_hash FROM users WHERE email = $1`. Writing `CREATE UNIQUE INDEX ON users (email, user_id, pw_hash)` breaks your constraint—it would allow two users to register the same `email` with different `user_id`s. Writing `CREATE UNIQUE INDEX ON users (email) INCLUDE (user_id, pw_hash)` enforces uniqueness strictly on `email` while making every login lookup a single-buffer Index-Only Scan.

Common trap

When "Index Only Scan" in `EXPLAIN` is a mirage. Never stop reading `EXPLAIN` at the node title `Index Only Scan`; always run `EXPLAIN (ANALYZE, BUFFERS)` and read the line right underneath it: `Heap Fetches`. On an insert-heavy or update-heavy table whose `autovacuum_vacuum_scale_factor` (default 0.20 = 20% of the table) waits for 20 million changed rows before running `VACUUM`, vast stretches of heap blocks sit with `ALL_VISIBLE` = 0. Your query plan proudly prints `Index Only Scan`, yet `Heap Fetches: 48219` reveals it read 48,219 heap blocks anyway. Lowering per-table `autovacuum_vacuum_scale_factor = 0.01` (or `autovacuum_vacuum_insert_scale_factor = 0.005` on append-heavy ledgers in PG 13+) keeps `_vm` bits set and turns `Heap Fetches` back to 0.

3.5 Partial Indexes (`WHERE ...`) and Expression Indexes

Every index we have built so far paid storage, `shared_buffers` RAM, and write-ahead log tax for 100% of the rows in the table. Yet many of the hottest queries in production care about a tiny, predictable fraction of rows.

3.5.1 Partial Indexes: Shrinking a 4 GB Queue Index to 2 MB

Consider a transactional outbox or webhook job table `jobs` holding 100,000,000 historical delivery rows. At any moment, 99,950,000 rows (99.95%) have `status = 'processed'`, while only 50,000 rows (0.05%) have `status = 'pending'`. Every millisecond, worker threads poll for the oldest ready jobs:

```
SELECT job_id, payload
FROM jobs
WHERE status = 'pending' AND run_after <= NOW()
ORDER BY run_after ASC
LIMIT 50
FOR UPDATE SKIP LOCKED;
```

Compare a full composite index against a **partial index** carrying a `WHERE` filter:

```
-- Naive full index: indexes all 100,000,000 rows (~3.8 GB on disk!)
CREATE INDEX idx_jobs_full ON jobs (status, run_after);

-- Partial index: indexes ONLY the 50,000 pending rows (~1.9 MB on disk!)
CREATE INDEX idx_jobs_pending ON jobs (run_after ASC)
    WHERE status = 'pending';
```

Count the wins of `idx_jobs_pending`:

- 2,000× **smaller memory footprint:** 50,000 rows × ∼ 28 bytes (plus transient churn slack) fits inside ≈ 240 pages (∼ 1.9 MB), residing 100% inside CPU L3 cache and `shared_buffers` instead of evicting 3.8 GB of heap pages. Notice we did not even need `status` as a key column inside `(run_after ASC)`—every entry in the index already satisfies `status = 'pending'`.
- **Zero index maintenance on excluded writes:** When a historical audit job updates a `'processed'` row's metadata, neither the old row nor the new row satisfies `status = 'pending'`, so PostgreSQL skips touching `idx_jobs_pending` completely (preserving HOT updates on the heap page).
- **Conditional uniqueness:** Partial unique indexes enforce invariants that standard SQL `UNIQUE` constraints cannot express cleanly. The index below guarantees each customer has at most one primary card while allowing any number of non-primary saved cards (`is_primary = FALSE`):

```
CREATE UNIQUE INDEX uq_one_primary_card
    ON payment_methods (customer_id)
 WHERE is_primary = TRUE;
-- A second primary card for customer 42 fails with:
-- ERROR:  duplicate key value violates unique constraint "uq_one_primary_card"
-- DETAIL:  Key (customer_id)=(42) already exists.
```

One rule governs planner eligibility: **the query's `WHERE` clause must logically imply the index's `WHERE` predicate** using constants visible at plan time. A query filtering `WHERE status = 'pending'` matches `WHERE status = 'pending'` immediately. However, a prepared statement parameterized as `WHERE status = $1` cannot use `idx_jobs_pending` under a generic plan (because `$1` might be `'processed'` at execution time), though PostgreSQL 12+'s custom-plan mode can use it when planning with the bound literal `'pending'`.

3.5.2 Expression Indexes and Syntactic Predicate Alignment

What if your login endpoint performs case-insensitive lookups or filters a scalar key extracted from a `JSONB` webhook payload?

```
-- A plain index on (email) CANNOT satisfy LOWER(email) = ...!
CREATE UNIQUE INDEX idx_users_email_lower ON users (LOWER(email));

-- Indexing a scalar TEXT value extracted from a JSONB column
-- Note the mandatory double parentheses around non-function expressions!
CREATE INDEX idx_webhooks_merchant ON webhooks ((payload->>'merchant_id'));
```

In an **expression index**, PostgreSQL evaluates the expression at `INSERT`/`UPDATE` time and stores the *result* inside the `IndexTupleData`—so `idx_users_email_lower` stores `'ada@lovelace.org'` even if the heap row holds `'Ada@Lovelace.org'`. Even better, when you run `ANALYZE`, PostgreSQL computes dedicated histograms and most-common-value statistics on the expression itself in `pg_stats`, giving the query planner accurate selectivity estimates on `(payload->>'merchant_id')`.

Two strict rules bite engineers who deploy expression indexes carelessly:

1. **Expressions must be `IMMUTABLE`:** Given identical input bytes, the function must return the exact same output forever, independent of database settings or wall-clock time. Indexing `DATE_TRUNC('day', created_at)` on a `TIMESTAMPTZ` column fails with `ERROR: functions in index expression must be marked IMMUTABLE`, because "midnight" depends on the session's `TimeZone` GUC. Fix it by pinning the zone explicitly: `DATE_TRUNC('day', created_at AT TIME ZONE 'UTC')` *is* `IMMUTABLE`.

2. **Query predicates must align syntactically:** PostgreSQL's planner does not do open-ended algebraic theorem proving across arbitrary string functions. If the index is defined on `LOWER(email)`, a query writing `WHERE email ILIKE 'ada@lovelace.org'` or `WHERE UPPER(email) = 'ADA@LOVELACE.ORG'` will walk right past the index and run a `Seq Scan`. Write `WHERE LOWER(email) = LOWER($1)`.

3.6 Beyond B-Trees: Hash, GIN, GiST, and BRIN

A Lehman–Yao B-link tree assumes one foundational property about your search predicate: *every row maps to a single scalar (or tuple) key drawn from a totally ordered 1D domain* ($<, \leq, =, \geq, >$).

The moment your data violates that assumption, B-trees hit a geometric wall:

- What if keys are 250-byte URLs queried *only* for exact equality `=`?
- What if a single row contains *50 distinct searchable tokens*—an array `tags TEXT[]`, a nested `JSONB` document, or a 300-word full-text `tsvector`?
- What if keys live in 2D space (polygons, GPS coordinates) or represent intervals (`tstzrange`) tested for *overlap* (`&&`), where no 1D total order preserves containment?
- What if an append-only ledger holds 10 billion rows (1 TB) and you cannot afford 215 GB of B-tree pages just to slice time windows?

Rather than hard-wiring B-trees into the executor, Stonebraker and Rowe designed Postgres around an extensible **Access Method API** (`pg_am` and `IndexAmRoutine`). Let us open the hood on the four companions sitting beside `nbtree` in `src/backend/access/`.

3.6.1 Hash Indexes (`USING hash`): 32-Bit Codes Only

A PostgreSQL Hash index (`src/backend/access/hash/`) organizes 8 KB pages into a dynamic linear-hashing directory of *primary bucket pages* (chained to overflow pages only when collisions or duplicates exceed one page). Its standout physical trick is what it leaves out: **a Hash index never stores the raw column value.** Each index tuple stores only the **4-byte (32-bit) hash code** plus the 6-byte heap `ctid` (16 bytes aligned, sorted by hash code within each bucket page for binary search).

Put numbers on that: indexing a 240-byte `webhook_url` or `idempotency_token` across 50,000,000 rows produces a $\sim$ 13.5 GB B-tree, whereas `CREATE INDEX ON deliveries USING hash (webhook_url)` takes just $\sim$ 1.1 GB! Because two distinct strings can collide on a 32-bit hash, a Hash scan *always* visits the heap page to recheck `webhook_url = $1` (so Hash indexes cannot perform Index-Only Scans, nor can they enforce `UNIQUE`, sort `ORDER BY`, or index multiple columns).

Reading note

Prior to PostgreSQL 10, manuals famously warned against Hash indexes because bucket splits were not WAL-logged—a crash required a manual `REINDEX`. Since PostgreSQL 10, Hash indexes are 100% WAL-logged, crash-safe, and replicated cleanly. Reach for them specifically when you have wide `TEXT`/`BYTEA` keys queried strictly with `=`.

3.6.2 GIN (Generalized Inverted Index): Multi-Element Containment at Scale

Suppose a `payments` table stores provider metadata in a `JSONB` column `meta`, and risk engineers filter with `meta @>` on `"card_brand": "visa"` and `"three_ds": "failed"`, or support engineers search ticket bodies via full-text search `WHERE body_tsv @@ plainto_tsquery('english', 'chargeback reversal')`. A B-tree on the whole `JSONB` blob is useless here: `'three_ds'` sits buried inside the document, not at byte 0.

A **GIN index** (`src/backend/access/gin/`) works like the subject index at the back of a textbook. Instead of mapping 1 row $\rightarrow$ 1 key, GIN invokes the operator class's `extractValue` function to shatter a single row into *dozens of atomic element keys* (lexemes like `'chargeback'` and `'revers'`, or JSONB keys/values), and maps each distinct element key to the sorted set of all heap `ctid`s containing it (left half of Figure 3.4):

1. **The Entry Tree:** A standard B-tree over *distinct extracted keys*. Even across 50 million documents, English vocabulary or JSONB field paths might have only 200,000 distinct keys, keeping the Entry Tree compact.
2. **Posting List vs. Posting Tree:** If a rare key like `'chargeback'` appears in 40 rows, GIN packs those 40 `ctid`s directly inside the Entry Tree leaf tuple as a **varbyte delta-compressed posting list** (often just 2–3 bytes per `ctid`). Once a popular key like `'visa'` accumulates more `ctid`s than fit comfortably inline ($\sim$ `GinMaxItemSize`), GIN spills that key's list into

a dedicated **Posting Tree**—a mini B-tree keyed by `(block, offset)` itself. Keeping `ctid`s physically sorted lets multi-term queries (`'visa' AND 'failed'`) intersect bitmaps in a single linear pass and read matching heap blocks sequentially (`Bitmap Index Scan` → `Bitmap Heap Scan`).

3. **The `fastupdate` Pending List:** Notice GIN's achilles heel on writes: inserting *one* JSONB row with 60 fields would otherwise descend and dirty up to 60 separate Entry Tree leaf pages. To prevent 60× write amplification on foreground `INSERT`s, PostgreSQL enables `fastupdate = on` by default: new key/`ctid` pairs append sequentially to an unsorted **pending list** of pages (capped by `gin_pending_list_limit`, default 4 MB). Every GIN query scans both the main Entry Tree and the tiny pending list, until `VACUUM` (or an `INSERT` crossing 4 MB) bulk-sorts and merges the pending list into the main tree in one batched pass.

In production

`jsonb_ops` vs. `jsonb_path_ops` and OLTP latency spikes. PostgreSQL ships two GIN operator classes for `JSONB`:

```
-- Default jsonb_ops: indexes every key AND value separately (supports @>, ?, ?|, ?&)
CREATE INDEX idx_pay_meta_default ON payments USING gin (meta);

-- jsonb_path_ops: hashes each full root-to-leaf path (e.g. hash('card_brand','visa'))
-- Supports @> containment ONLY, but is 2x-3x smaller and much faster when values repeat!
CREATE INDEX idx_pay_meta_paths   ON payments USING gin (meta jsonb_path_ops);

-- Full-text search via generated stored tsvector column + GIN
ALTER TABLE support_tickets
  ADD COLUMN search_tsv TSVECTOR
  GENERATED ALWAYS AS (to_tsvector('english', coalesce(subject,'') || ' ' ||
    coalesce(body,''))) STORED;
CREATE INDEX idx_tickets_fts ON support_tickets USING gin (search_tsv);
```

Watch out for one classic production tail-latency spike: when an unlucky user's `INSERT` pushes the `fastupdate` pending list past `gin_pending_list_limit` (4 MB), *that foreground backend* synchronously flushes all 4 MB into the Entry Tree, stalling one API call for 50–300 ms. On strict p99 latency tables, either tune aggressive `autovacuum` with a smaller `gin_pending_list_limit = '1MB'` on the index, or set `WITH (fastupdate = off)` if each row extracts only 2–4 keys.

3.6.3 GiST (Generalized Search Tree): Bounding Hierarchies and Exclusion Constraints

In 1995, Joseph Hellerstein, Jeffrey Naughton, and Avi Pfeffer asked: *can one single balanced-tree engine implement R-trees for 2D PostGIS maps, interval trees for calendar ranges, and nearest-neighbor k-NN priority search without rewriting buffer locking and WAL recovery every time?* Their answer, built straight into Berkeley Postgres and hardened in `src/backend/access/gist/`, is **GiST**.

Unlike a B-tree—which partitions a 1D line into strictly disjoint intervals $(-\infty, 60]$, $(60, 100]$,

$(100, +\infty)$—an internal node entry in GiST stores a **bounding predicate** P that encloses every entry in its child subtree (for example, a 2D minimum bounding rectangle around 300 delivery zones, or the span from the earliest start to the latest end across 300 `tstzrange` bookings). Notice the structural difference: **sibling bounding boxes in GiST are allowed to overlap.** Consequently, a GiST lookup calls the operator class's `consistent(entry, query)` method on every entry of an internal page and descends *all* child subtrees whose bounding boxes overlap the query, while inserts pick the child requiring minimal bounding-box expansion (`penalty`) and split full nodes via `picksplit`.

Beyond PostGIS geometry (`&&`, `ST_Contains`, nearest-neighbor `<->`), GiST enables a relational integrity guarantee that B-trees cannot express: **Exclusion Constraints**. Suppose you build a room-booking or ledger fee-schedule table where no two active rows for the same `room_id` may have overlapping `during` time intervals. A `UNIQUE` B-tree constraint can only forbid *identical* intervals (`=`); checking `SELECT COUNT(*) ... WHERE during && $new` in application code suffers classic race conditions when two concurrent transactions insert overlapping slots simultaneously under `READ COMMITTED`. With the built-in `btree_gist` contrib extension, PostgreSQL enforces non-overlap directly inside the physical GiST index:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE room_bookings (
    booking_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    room_id    INT       NOT NULL,
    during     TSTZRANGE NOT NULL,
    -- Forbid any two rows where (room_id = room_id) AND (during && during)!
    CONSTRAINT no_double_booking
        EXCLUDE USING gist (room_id WITH =, during WITH &&)
);
```

Just like `UNIQUE` in `nbtree`, `EXCLUDE USING gist` checks physical index pages across both committed and in-flight uncommitted tuples (blocking the second overlapping transaction until the first commits or aborts), eliminating double-booking anomalies at the storage layer. An overlapping insert fails with:

```
ERROR:  conflicting key value violates exclusion constraint "no_double_booking"
DETAIL:  Key (room_id, during)=(7, ["2026-11-02 10:00:00+00","2026-11-02 11:00:00+00"))
         conflicts with existing key (room_id, during)=(7, ["2026-11-02 09:30:00+00",
         "2026-11-02 10:30:00+00")).
```

3.6.4 BRIN (Block Range Index): Indexing a Terabyte in 30 Megabytes

Finally, consider an append-only `ledger_events` table recording 10,000,000,000 (10^{10}) immutable events at $\sim$ 100 bytes per row—roughly 134,217,728 pages, or 1.0 **TB of heap**. Compliance analysts query 15-minute slices by `recorded_at`. A standard B-tree on `recorded_at` costs $\sim$ 20 bytes $\times 10^{10}$ rows (plus split slack) $\approx$ **215** GB on disk.

Ask yourself: since rows are appended chronologically as wall-clock time advances, heap blocks 0..127 already hold events from 00:00:00 to 00:00:04 on January 1, blocks 128..255 hold 00:00:04 to 00:00:09, and so on. Why store ten billion individual row pointers when the heap is *already physically sorted by arrival time*?

Introduced in PostgreSQL 9.5 (`src/backend/access/brin/`), a **Block Range Index (BRIN)** abandons per-row `ctid`s completely. It slices the heap into fixed-size consecutive chunks of `pages_per_range` blocks (default 128 pages = 1 MB of heap). For each 128-page range, BRIN stores **one tiny summary tuple** containing only (`min_val`, `max_val`) across all rows anywhere in those 128 pages (right half of Figure 3.4):

$$\frac{134{,}217{,}728 \text{ heap pages}}{128 \text{ pages/range}} = 1{,}048{,}576 \text{ ranges} \times \sim 28 \text{ bytes/summary} \approx \mathbf{29.4} \text{ MB total.}$$

That is 215 **GB for a B-tree versus** ∼ 30 **MB for BRIN**, more than 7,000× smaller. When a query asks for `WHERE recorded_at BETWEEN '2026-10-20 14:00Z' AND '2026-10-20 14:15Z'`, PostgreSQL scans the 30 MB BRIN summary in milliseconds, skips 99.98% of the 128-page ranges whose [min, max] fails to overlap the 15-minute window, and runs a lossy `Bitmap Heap Scan` with `Recheck Cond` across only the ≈ 150 matching 1 MB ranges.

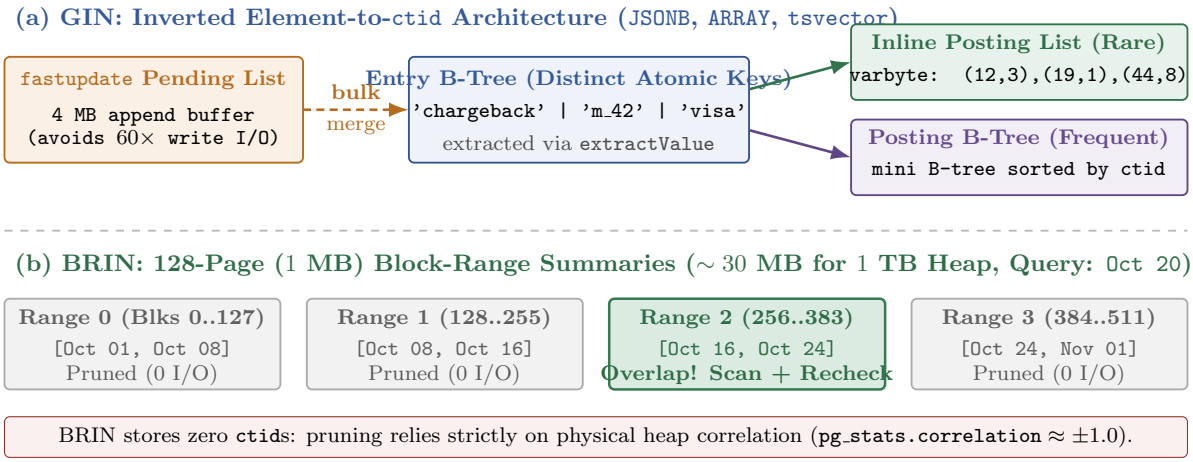


Figure 3.4: Two opposite poles of non-B-tree indexing. In (a), **GIN** inverts multi-element rows into exact `ctid` posting lists/trees cushioned by a `fastupdate` buffer. In (b), **BRIN** summarizes 128-page (1 MB) heap chunks by [min, max] to prune append-only streams at 0.003% space overhead.

Failure mode

The BRIN trap: physical correlation decay. BRIN stores no row-level pointers; its pruning power comes 100% from how tightly each 128-page physical block range clusters values on disk. Check `SELECT correlation FROM pg_stats WHERE tablename = 'ledger_events' AND attname = 'recorded_at';`:

- If `correlation` sits near +1.0 or −1.0 (pure append-only monotonic `TIMESTAMPTZ` or `BIGINT` sequence, never updated or deleted), each 128-page range has a narrow [min, max] window and BRIN skips > 99.9% of the table.
- Now suppose you build a BRIN index on a random `UUIDv4` or `account_id` (`correlation` ≈ 0.00), *or* suppose a retention job runs `DELETE` on January heap pages, `VACUUM` registers free space inside blocks 0..127 in `_fsm`, and November `INSERT`s reuse slots inside blocks 0..127! **One single November row landing inside block 12 stretches Range 0's summary from `[Jan 01, Jan 01]` to `[Jan 01, Nov 01]`.** Worse still, BRIN deliberately never shrinks a summary's [min, max] automatically when extreme rows are deleted (doing so requires `brin_desummarize_range()` or `VACUUM` / `REINDEX`).

Once a few scattered updates or _fsm reuses "poison" every 128-page range with both old and new timestamps, every range overlaps every query, and your 30 MB BRIN index silently reads 100% of the 1 TB heap.

Cost summary

PostgreSQL Access Method Decision Matrix:

Method	Key Structure	Operators	Typical Size	Sweet-Spot Workload
btree	1D total order + ctid	< <= = >= >	15–30% heap	OLTP point/range, ORDER BY, UNIQUE, INCLUDE
hash	32-bit hash + ctid	= only	5–12% heap	Exact lookups on wide URLs, tokens, BYTEA
gin	Inverted key → ctids	@> <@ ? @@ &&	20–80% heap	JSONB, ARRAY, tsvector full-text, trigrams
gist	Bounding hierarchy	&& @> <->	20–50% heap	PostGIS 2D shapes, tstzrange, EXCLUDE
brin	128-page [min, max]	< <= = >= >	∼ 0.003% heap	Multi-TB append-only streams ($\lvert r\rvert \approx 1$)

In Chapter 2 and Chapter 3, we treated buffer edits—inserting a heap tuple, updating P_HIKEY, linking btpo_next across a Lehman–Yao leaf split—as if writing bytes into shared_buffers made them immortal. What happens if the server loses power after Leaf P flushes to NVMe while Leaf P' is still sitting in volatile RAM, or when Linux tears an 8,192-byte page in half across two 4,096-byte sectors? That question brings us to Part II and Chapter 4: Write-Ahead Logging, checkpoints, and crash recovery.

3.7 Exercises

Exercise 3.1 (Exact B+Tree Fanout: BIGINT vs. UUID Keys)**.** Using the exact byte layout of an 8,192-byte nbtree page from Section 3.1 (24-byte PageHeaderData, 16-byte BTPageOpaqueData, 4-byte ItemIdData per slot, 8-byte IndexTupleData header, and MAXALIGN = 8 padding on 64-bit Linux):

1. Compute the exact per-entry byte cost (line pointer + aligned IndexTupleData) for an index on a single 16-byte UUID column, and compare it against the 20-byte cost of an 8-byte BIGINT.

2. Assuming internal pages average 80% occupancy after splits (reserving one slot for P_HIKEY) and leaf pages run at fillfactor = 90 (also reserving P_HIKEY), compute the effective internal branching factor B and leaf capacity L for both BIGINT and UUID.

3. For a table of 500,000,000 (5×10^8) rows, determine the exact number of tree levels (root down to leaf) and the total megabytes of shared_buffers required to keep *all non-leaf internal pages* pinned in RAM for both key types.

Exercise 3.2 (Tracing a Concurrent Lehman–Yao Split by Hand)**.** Leaf page Block 22 initially covers $(100, 200]$ (P_HIKEY = 200, btpo_prev = 9, btpo_next = 35) and holds keys $\{108, 115, 124, 130, 152, 168, 191\}$. Parent Block 5 routes $(100, 200] \to$ Block 22. Writer W inserts key 175, splits Block 22 at separator 130 into Block 22 and newly allocated Block 59, unlocks both leaf pages, and is paused by the OS scheduler *before* locking Parent Block 5.

1. Write down the exact `P_HIKEY`, data keys, `btpo_prev`, `btpo_next`, and `BTP_INCOMPLETE_SPLIT` flag state on both Block 22 and Block 59 during this pause.
2. Trace the exact sequence of page pins, shared latches acquired/released, and `P_HIKEY` comparisons performed by Reader A searching for $k = 124$, and Reader B searching for $k = 175$. At any instant, how many page latches does Reader B hold simultaneously during descent vs. during `_bt_moveright`?
3. Why must `_bt_moveright` acquire latches strictly left-to-right (Block 22 $\rightarrow$ Block 59), and what subtle deadlock hazard arises for *backward* scans (`ORDER BY k DESC` walking `btpo_prev` right-to-left) if a left neighbor splits concurrently? Explain how releasing and re-checking prevents a right-to-left latch cycle.

Exercise 3.3 (Deduplication Math and Bottom-Up Deletion)**.** An `orders` table holds 20,000,000 rows distributed evenly across 5 values of `status CHAR(4)` (`'OPEN'`, `'PAID'`, `'SHIP'`, `'DONE'`, `'VOID'`).

1. With `deduplicate_items = off`, calculate the byte size of one leaf entry (remembering the 1-byte short `varlena` header + 4 ASCII bytes padded to `MAXALIGN = 8` after the 8-byte `IndexTupleData` header, plus the 4-byte `ItemIdData`) and estimate the total leaf-level size in MB at `fillfactor = 90`.
2. With `deduplicate_items = on` (PG 13+), suppose `nbtree` packs up to 130 6-byte heap `ctid`s into each posting list tuple (16-byte header+key + $130 \times 6 = 780$ bytes $\rightarrow$ 796 bytes, padded to 800 bytes + 4-byte line pointer). Recompute the leaf-level size in MB and the compression factor.
3. Suppose `orders` also has an index on `updated_at`, and a worker updates `updated_at = NOW()` ten times on an `'OPEN'` order without changing `status`. Explain why Chapter 2/5's HOT optimization is blocked, why 10 index entries for `'OPEN'` accumulate on the leaf page, and how PostgreSQL 14's bottom-up index deletion prevents that leaf page from splitting.

Exercise 3.4 (Composite Ribbon Geometry and Sort Elimination)**.** A multi-tenant ledger table `entries(entry_id, tenant_id, account_id, status, posted_at, amount_cents)` serves three critical production queries:

- Q_1: `WHERE tenant_id = $1 AND account_id = $2 AND posted_at >= $3 ORDER BY posted_at DESC LIMIT 25`
- Q_2: `WHERE tenant_id = $1 AND status = 'SETTLED' AND posted_at BETWEEN $2 AND $3`
- Q_3: `WHERE tenant_id = $1 AND amount_cents > 100000 AND posted_at >= $2`

1. Design the minimal set of composite B-tree indexes so that Q_1 runs with **zero `Sort` node** and touches at most ~ 28 index tuples, and Q_2 reads a single contiguous leaf slice. Specify exact column order and `ASC`/`DESC` modifiers.
2. For Q_3, explain geometrically using Figure 3.2 why *no single 1D B-tree* can make both `amount_cents > 100000` and `posted_at >= $2` contiguous simultaneously. If 0.1% of a tenant's rows have `posted_at >= $2` while 15% have `amount_cents > 100000`, which column should come second after `tenant_id`, and why?

Exercise 3.5 (Visibility Map Churn and `Heap Fetches` Latency)**.** A 50,000,000-row `transactions` table occupies 600,000 heap pages ($\approx$ 4.69 GB) and has a covering index `(merchant_id, created_at) INCLUDE (net_cents)`.

1. Calculate the exact byte size (and number of 8 KB pages) of the relation's _vm fork at 2 bits per heap page.
2. Right after VACUUM, 100% of heap pages have VISIBILITYMAP_ALL_VISIBLE = 1. Over the next 20 minutes before autovacuum triggers, OLTP traffic performs 90,000 single-row updates distributed uniformly at random across the 600,000 heap pages. Using $1-(1-1/M)^K \approx 1-e^{-K/M}$, compute the fraction of heap pages whose _vm bit is now cleared to 0.
3. An analytics query runs SELECT SUM(net_cents) FROM transactions WHERE merchant_id = 77, matching 20,000 index entries scattered uniformly across cold heap pages (80 μs per random NVMe read; index pages and _vm are warm in RAM). Compute the expected Heap Fetches count and wall-clock query time right after VACUUM versus 20 minutes later.

Exercise 3.6 (Partial and Expression Index Debugging)**.** A teammate submits a migration containing three indexes on a 150,000,000-row invoices table (99.7% 'paid', 0.2% 'open', 0.1% 'overdue'):

```
-- Index 1
CREATE INDEX idx_inv_day ON invoices (DATE_TRUNC('day', issued_at));
-- Index 2
CREATE INDEX idx_inv_code ON invoices (LOWER(reference_code));
-- Index 3
CREATE INDEX idx_inv_unpaid ON invoices (due_at) WHERE status IN ('open', 'overdue');
```

1. issued_at has type TIMESTAMPTZ. Explain why Index 1 fails immediately with an immutability error, and rewrite both the CREATE INDEX and a fast full-day range query for UTC day 2026-10-31 (bonus: show how writing a half-open range on bare issued_at avoids needing an expression index at all).
2. Application code queries WHERE LOWER(TRIM(reference_code)) = 'inv-901'. Explain why PostgreSQL ignores Index 2, and fix the schema/index contract cleanly.
3. Estimate the storage reduction ratio of Index 3 compared to a full index on (status, due_at), and determine whether WHERE status = 'overdue' AND due_at < NOW() can use Index 3 (justify via predicate implication).

Exercise 3.7 (GIN Internals: jsonb_ops, jsonb_path_ops, and Pending Flushes)**.** Consider inserting the following JSONB document into a charges(charge_id, meta) table:

```
INSERT INTO charges (meta) VALUES
('{"gateway":"stripe","risk":{"score":92,"flags":["avs_mismatch","velocity"]}}');
```

1. List all distinct Entry Tree keys extracted under default jsonb_ops versus the root-to-leaf path hashes extracted under jsonb_path_ops.
2. Which of the following predicates can jsonb_path_ops accelerate, and which force a sequential scan: (a) meta @> matching velocity inside risk.flags; (b) meta ? 'gateway'; (c) meta->'risk'->>'score' = '92'?
3. Suppose each inserted charge appends $\sim$ 256 bytes to the GIN fastupdate pending list (gin_pending_list_limit = 4MB). After roughly how many INSERTs does a foreground transaction trigger a synchronous pending-list merge, and what two configuration knobs eliminate that p99 latency cliff?

Exercise 3.8 (Quantifying BRIN Scan Amplification Under `_fsm` Poisoning)**.** An append-only `telemetry` table holds 102,400,000 rows packed at 80 rows per 8 KB page across 1,280,000 heap pages ($\approx$ 10 GB), indexed by BRIN on `ts` with default `pages_per_range = 128` ($R = 10{,}000$ block ranges total, covering 100 consecutive days so 100 ranges = 1 day).

1. Initially rows are strictly chronological: a query for the latest 1 hour of Day 100 overlaps $\lceil 100/24 \rceil = 5$ block ranges. How many megabytes of heap does PostgreSQL read?
2. A cleanup job deletes 0.1% of rows scattered uniformly across Days 1..99 and runs `VACUUM` (without desummarizing or clustering). During Day 100, just $K = 30{,}000$ new Day-100 rows are directed by the Free Space Map (`_fsm`) uniformly at random into the 9,900 historical block ranges of Days 1..99. What is the probability $1 - (1 - 1/9900)^{30000}$ that a given historical 128-page range receives *at least one* Day-100 row?
3. Explain what happens to that historical range's BRIN $[\min, \max]$ tuple the moment a single Day-100 row lands inside it, and compute how many megabytes of heap the exact same "latest 1 hour of Day 100" query is now forced to scan and recheck.

Part II

Durability, Concurrency, and MVCC Internals

Chapter 4

Write-Ahead Logging, Checkpoints, and Crash Recovery

In Chapters 2 and 3 we built fast, compact 8,192-byte slotted heap pages and B-link tree index blocks inside PostgreSQL's `shared_buffers` pool. As long as electricity flows steadily into DRAM, that design flies: inserting a row stamps twenty-four bytes of tuple header, appends four bytes to the `ItemIdData` line-pointer array, updates a leaf page in each index, and flips a `BM_DIRTY` bit in memory in a few microseconds. Now pull the server's power cord out of the wall. Every byte in `shared_buffers` vanishes, every byte sitting in the Linux kernel page cache vanishes, and any 8 KB page caught halfway across the PCIe bus lands on the solid-state drive split into mismatched halves. How does a database engine report `COMMIT` in sub-millisecond time while guaranteeing that a sudden power outage neither loses a single committed row nor leaves a single B-tree pointer dangling?

Focus

This chapter opens Part II by tracing how PostgreSQL turns fragile 8 KB memory pages into crash-proof durable storage through `xlog.c` and `xlogrecovery.c`:

- **Hardware failure physics:** why `write(2)` is not durability, what `fdatasync(2)` asks of an NVMe controller, and how an 8 KB PostgreSQL page tears across two 4 KB hardware sectors.
- **Write-ahead logging and LSNs:** the sequential 16 MB segments in `$PGDATA/pg_wal/`, 64-bit `pg_lsn` byte offsets, `XLogRecord` internals, and the fundamental WAL Invariant binding `pd_lsn` to `XLogFlush()`.
- **Healing torn pages:** why physiological delta records fail on a fractured 8 KB page, how checkpoint-anchored `full_page_writes` fix the problem with hole-stripped Full Page Images, and why WAL volume spikes right after a checkpoint begins.
- **Group commit and durability modes:** leader-follower batching on `WALWriteLock`, the exact failure windows of `synchronous_commit`, and zero-WAL `UNLOGGED` tables.
- **Fuzzy checkpoints and single-pass recovery:** why recovery rewinds from `pg_control`'s checkpoint record back to its *Redo point*, why PostgreSQL needs no ARIES Undo phase, and an LSN-by-LSN crash trace inspected live with `pg_walinspect`.

4.1 The Physics of `write()`, `fsync()`, and Torn 8 KB Pages

Before looking at a single line of PostgreSQL recovery code, we need an accurate physical picture of what happens when a user-space process writes bytes to a file on Linux. Programmers casually say "we wrote the buffer to disk" when `write(fd, buf, 8192)` returns `8192`. Physically, nothing of the sort happened. Those 8,192 bytes merely crossed the CPU ring boundary from one region of volatile RAM into another region of volatile RAM.

Follow a dirty 8 KB page from the moment a backend modifies it inside `shared_buffers` until its bits sit safely inside non-volatile NAND flash charge traps:

1. **PostgreSQL `shared_buffers` (user-space shared memory):** When an `INSERT` or `UPDATE` modifies heap block 7, it mutates an 8,192-byte frame in POSIX/System V shared memory and sets `BM_DIRTY` in the buffer's `BufferDesc` header. If the `postgres` process crashes with `SIGSEGV` or `kill -9`, and the postmaster resets shared memory, every dirty frame here is wiped instantly.
2. **Linux kernel page cache (`pwrite(2)`):** When PostgreSQL calls `pwrite(fd, page, 8192, offset)`, the kernel copies those 8,192 bytes into two 4,096-byte kernel memory pages (`struct folio`) inside the operating system's page cache, marks those kernel pages dirty, and immediately returns `8192` to user space. Notice the difference between process death and machine death: if `postgres` dies via `kill -9` a microsecond after `pwrite()` returns, the Linux kernel is still alive and its background writeback threads will flush those dirty page-cache frames to the drive seconds later. If the kernel panics or building power fails, everything in the kernel page cache evaporates.
3. **NVMe controller volatile DRAM cache:** When kernel writeback (or an unforced block write) DMA-transfers 4 KB blocks across PCIe to an SSD, the drive does not immediately program slow NAND flash cells. Most SSDs contain 512 MB to 4 GB of onboard DDR4 DRAM sitting right on the drive PCB to absorb bursts and maintain Flash Translation Layer (FTL) mapping tables. Left in that onboard DRAM without backup power, the bytes still vanish if the power supply drops to zero volts.
4. **Non-volatile NAND flash cells (`fsync(2)` / `fdatasync(2)`):** Only when PostgreSQL invokes `fdatasync(fd)` (or opens a WAL file with `O_DSYNC`) does the Linux filesystem push dirty page-cache blocks to the block layer *and* emit an NVMe `Flush` command (or set the Force Unit Access, `FUA`, bit on the write command). A specification-compliant drive holds off completing `fdatasync()` until every buffered byte is either programmed into non-volatile NAND flash or protected by onboard hardware capacitors that guarantee NAND programming upon power loss.

Java 6 hides the same tiers behind `FileOutputStream.write()`, which returns as soon as the bytes reach tier 2, the kernel page cache. A Java program that needs durability must call `fos.getFD().sync()` or `FileChannel.force(true)`, and both end in `fsync(2)`. In C the pair is explicit:

```
ssize_t n = pwrite(fd, page, 8192, off); /* returns once bytes sit in the page cache */
if (n != 8192)
    report_error();
if (fdatasync(fd) != 0)                  /* returns once the device reports durable */
    abort();                             /* PostgreSQL PANICs here instead          */
```

Why abort instead of retrying? In 2018 PostgreSQL developers found that after a failed `fsync`, Linux may mark the dirty pages clean and discard them, so a second `fsync` reports success even though the data never reached the device. A failed WAL flush has always stopped the server. Since the February 2019 minor releases (9.4.21 through 11.2), a failed `fsync` of a data file during a checkpoint stops it too, and restart replays from the WAL (the subject of this chapter) instead of trusting the page cache. The log line you see is:

```
PANIC:  could not fsync file "base/16384/16391": Input/output error
```

Failure mode

Lying consumer drives vs. enterprise Power-Loss Protection (PLP). Enterprise NVMe drives carry a visible row of polymer tantalum or supercapacitors on their circuit boards. When the 12 V power rail drops, voltage-drop circuitry uses the stored joules in those capacitors to flush onboard DRAM and FTL metadata into NAND flash within a few milliseconds, allowing the drive to honor `NVMe Flush` rapidly and truthfully. Cheap consumer laptops and desktop SSDs often omit those capacitors to save two dollars on the bill of materials—and worse, some consumer firmware acknowledges `Flush` before the NAND cells are actually programmed in order to inflate benchmark scores. Pull the plug on such a drive mid-load (as tools like `fio` with hardware power relays test) and bytes that `fdatasync()` swore were durable vanish, silently corrupting any database engine on Earth.

4.1.1 Why Direct Heap Flushing on `COMMIT` Fails Twice

Suppose we had never heard of a write-ahead log and tried to build transaction durability the naive way: whenever a client sends `COMMIT`, find every 8 KB heap and index page modified by that transaction in `shared_buffers`, `pwrite()` each one to its table or index file, and call `fdatasync()` on every touched file before returning `COMMIT` to the client.

That design fails twice: first on *performance*, then on *correctness*.

First, count the random I/O cost. Inserting one row into an `orders` table with four B-tree indexes (`orders_pkey`, `idx_customer`, `idx_created_at`, `idx_status`) dirties five distinct 8 KB blocks scattered across five separate files in `$PGDATA/base/16384/`. Worse, a B-tree leaf split in Chapter 3 dirties three blocks in a single index (left sibling, new right sibling, parent internal node). Flushing five to eight random 8 KB blocks plus issuing multiple filesystem sync barriers on every single one-row transaction turns sequential disk bandwidth into random-write thrashing, amplifies NAND flash erase-block wear by an order of magnitude, and prevents you from keeping hot pages buffered in RAM across consecutive updates.

Second, and worse: even if you paid that 100× latency penalty and called `pwrite()` + `fdatasync()` on one 8,192-byte heap page, **a power cut during that flush destroys the page**. Look at the block sizes across the stack:

PostgreSQL page = 8,192 bytes vs. Linux VM page / NVMe 4Kn sector = 4,096 bytes.

Standard POSIX filesystems (`ext4`, `XFS`) and standard NVMe block devices guarantee sector atomicity at 512 bytes or 4,096 bytes—never 8,192 bytes. An 8 KB PostgreSQL page occupies two consecutive 4 KB sectors on the drive: bytes `0..4095` in sector $2k$ and bytes `4096..8191` in sector $2k+1$. Recall the slotted-page geometry from Chapter 2: the 24-byte `PageHeaderData` (`pd_lsn`,

`pd_checksum`, `pd_lower`, `pd_upper`) and the `ItemIdData` line-pointer array grow *downward* from offset `0` inside the first 4 KB sector, while the actual heap tuples grow *upward* from offset `8192` inside the second 4 KB sector.

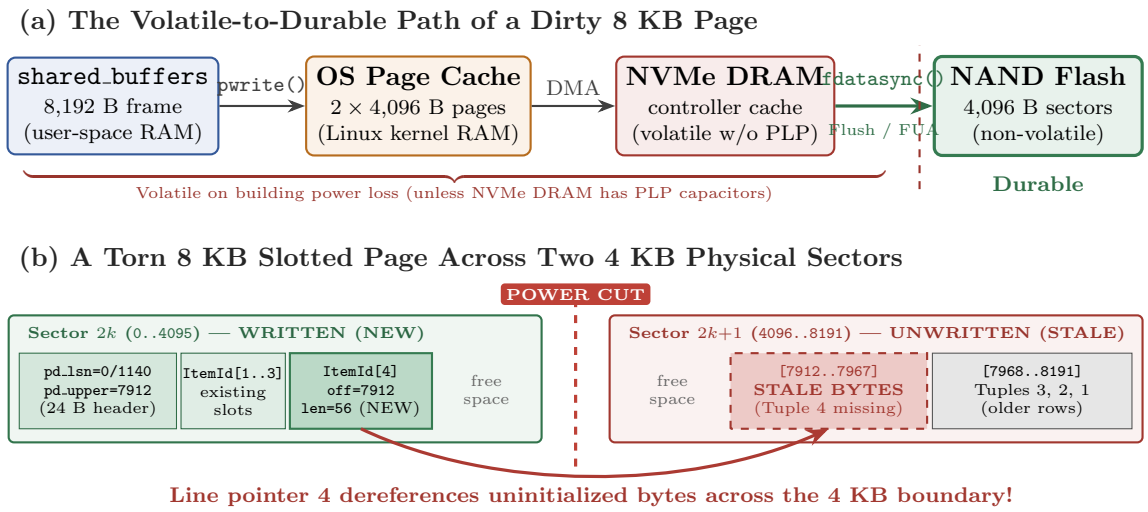


Figure 4.1: Top: the four memory tiers between a PostgreSQL backend and non-volatile NAND flash. Bottom: because an 8,192-byte PostgreSQL page spans two 4,096-byte hardware sectors, power loss mid-write produces a *torn page* whose updated header and line pointers in Sector $2k$ point into stale bytes in Sector $2k+1$.

Figure 4.1(b) shows the exact anatomy of this *torn page* (also called a *fractured write*). Before our insert, block 7 held three tuples occupying bytes `7968..8191` (`pd_upper = 7968`). Inserting a 56-byte fourth tuple lowers `pd_upper` to `7912`, writes the tuple into bytes `7912..7967` (inside Sector $2k+1$), and writes `ItemIdData[4] = (off=7912, len=56)` at byte offset `36` (inside Sector $2k$). Now power fails right after Sector $2k$ lands in NAND flash and right before Sector $2k+1$ arrives. When electricity returns, block 7 on disk is neither the old page nor the new page: `ItemIdData[4]` claims a live tuple sits at offset `7912`, while bytes `7912..7967` hold zeroes or leftover debris from a tuple vacuumed three weeks ago. If data checksums (`initdb --data-checksums`) are enabled, PostgreSQL's 16-bit FNV-1a `pd_checksum` catches the mismatch on the next read and aborts the query with `ERROR: invalid page in block 7`—detecting the corruption but unable to repair it. Without checksums, PostgreSQL silently decodes garbage bytes as customer balances.

Solving both problems at once—eliminating random multi-file `fdatasync()` calls at `COMMIT` *and* making every 8 KB page immune to mid-write tearing—is the job of Write-Ahead Logging.

4.2 Write-Ahead Logging (pg_wal), LSNs, and the Fundamental WAL Invariant

Intuition

Separate *recording* a change from *applying* it. Recording is one sequential append to a single file. Applying touches scattered 8 KB pages in many files. `COMMIT` waits only for the append to become durable; the scattered pages are written later, in batches. If power fails before they are written, restart reads the appended records and applies them again. One rule makes this safe: **a data page may not reach disk before the log records describing its changes are durable.**

PostgreSQL's Write-Ahead Log (historically called `XLOG`, implemented inside `src/backend/access/transam/xlog.c`) realizes that dock ledger as a single logical, append-only stream of bytes starting at offset `0` when `initdb` creates a cluster and growing monotonically for the lifetime of the database.

4.2.1 Segment Files in $PGDATA/pg_wal/ and pg_lsn Byte Arithmetic

Storing a multi-terabyte stream in a single filesystem file would make recycling old log entries painful, so PostgreSQL slices the logical byte stream into fixed-size **16 MB** (`0x01000000` = 16,777,216 byte) segment files inside `$PGDATA/pg_wal/` (configurable at `initdb --wal-segsize` in powers of two from 1 MB to 1,024 MB). Internally, each 16 MB segment is subdivided into 2,048 pages of 8,192 bytes each, headed by a compact `XLogPageHeaderData` struct so recovery can verify alignment every 8 KB.

Every position in this infinite logical stream is addressed by a 64-bit unsigned integer called a **Log Sequence Number** (`XLogRecPtr` in C, exposed in SQL as the `pg_lsn` type). By convention, PostgreSQL prints a 64-bit LSN as two hexadecimal numbers separated by a slash:

$$\texttt{X/YYYYYYYY} \iff (\texttt{0xX} \ll 32) \mid \texttt{0xYYYYYYYY}.$$

The left side `X` is the upper 32 bits (counting how many 4 GB $= 2^{32}$-byte logical epochs have elapsed), and the right side `YYYYYYYY` is the 32-bit byte offset within that 4 GB epoch (suppressing leading zeros on display, so `0/016B3748` prints as `0/16B3748`).

Definition 4.1 (Log Sequence Number and Segment Mapping)**.** A *Log Sequence Number* $L \in [0, 2^{64})$ is a monotonically increasing byte offset into PostgreSQL's logical WAL stream. Given segment size $S = 2^{24} = 16{,}777{,}216$ bytes (`0x01000000`) and a 32-bit timeline identifier $T \geq 1$ (initially $T = 1$, incremented upon point-in-time recovery promotion to prevent branching histories from overwriting old archives), L maps to:

$$\text{segno}(L) = \lfloor L/S \rfloor = L \gg 24, \qquad \text{offset}(L) = L \bmod S = L \,\&\, \texttt{0x00FFFFFF}.$$

Because each 4 GB epoch (2^{32} bytes) contains $2^{32}/2^{24} = 256 = \texttt{0x100}$ segments of 16 MB, PostgreSQL's `XLogFileName` macro formats the 24-character hexadecimal filename in `$PGDATA/pg_wal/` by concatenating three 8-hex-digit fields:

$$\underbrace{\texttt{TTTTTTTT}}_{\text{timeline } T}\ \underbrace{\texttt{XXXXXXXX}}_{\lfloor \text{segno}/256 \rfloor = (L \gg 32)}\ \underbrace{\texttt{000000SS}}_{\text{segno mod } 256}.$$

The third 8-character group `000000SS` explains a common puzzle: people staring at `ls $PGDATA/pg_wal/` for the first time are always puzzled when segment `000000010000000000000FF` wraps directly to `000000010000000100000000` instead of `...00000100`. Now you see why: the middle group counts 4 GB chunks, and 256×16 MB $= 4$ GB, so with default 16 MB segments the final eight hex digits only ever range from `00000000` through `000000FF`.

Example 4.1 (Concrete `pg_lsn` Arithmetic by Hand)**.** Suppose `pg_current_wal_lsn()` returns $L_1 =$ `0/16B3748` before a batch insert and $L_2 =$ `0/16B3A90` right after, on timeline $T = 1$:

1. **Which 16 MB file holds L_1, and at what byte offset?** Pad the lower half to eight hex digits: $L_1 =$ `0x00000000_016B3748`. Shifting right by 24 bits (dropping the last six hex digits) gives $\text{segno}(L_1) =$ `0x01` $= 1$, and masking the low six hex digits gives $\text{offset}(L_1) =$ `0x6B3748` $=$ 7,026,504 bytes (≈ 6.70 MB into file `000000010000000000000001`).

2. **How many WAL bytes did the batch generate?** Subtract directly in base 16:

$$L_2 - L_1 = \texttt{0x016B3A90} - \texttt{0x016B3748} = \texttt{0x348} = 3 \times 256 + 4 \times 16 + 8 = 840 \text{ bytes.}$$

3. **Cross-epoch subtraction:** Suppose $L_3 =$ `0/FFFFFC00` and $L_4 =$ `1/400` (which means `1/00000400`, i.e., `0x00000001_00000400`). Then

$$L_4 - L_3 = \texttt{0x100000400} - \texttt{0x0FFFFFC00} = \texttt{0x800} = 2{,}048 \text{ bytes,}$$

   which matches `SELECT pg_wal_lsn_diff('1/400', '0/FFFFFC00');` returning `2048`.

4.2.2 Inside an `XLogRecord` and Resource Managers

What actually sits at byte offset `0/16B3748` inside that segment file? Every change logged in PostgreSQL—inserting a heap tuple, splitting a B-tree page, committing a transaction, writing an online checkpoint—is wrapped in a binary envelope defined in `src/include/access/xlogrecord.h`:

```
/* Fixed 24-byte header at the front of every WAL record (xlogrecord.h) */
typedef struct XLogRecord
{
    uint32          xl_tot_len;   /* total byte length of entire record       */
    TransactionId   xl_xid;       /* 32-bit XID that wrote record (or 0)      */
    XLogRecPtr      xl_prev;      /* start LSN of previous record in WAL      */
    uint8           xl_info;      /* opcode flags specific to resource mgr    */
    RmgrId          xl_rmid;      /* resource manager ID (heap, btree, xact)  */
    /* 2 bytes of struct alignment padding here -> 20 bytes so far          */
    pg_crc32c       xl_crc;       /* CRC32C over payload + bytes 0..19        */
} XLogRecord;                     /* sizeof(XLogRecord) == 24 bytes           */
```

Immediately after those 24 header bytes come zero or more **block reference headers** (`XLogRecord BlockHeader`), each naming a physical target page via its tablespace, database, and relfilenode triple (`RelFileLocator`), fork number (main, FSM, visibility map), and 32-bit `BlockNumber`, followed by per-block delta bytes (or an optional Full Page Image) and any non-block main payload.

Three fields in `XLogRecord` deserve special attention:

- **`xl_rmid` and `xl_info` (Modular Resource Managers):** Rather than writing one giant `switch` statement for recovery, PostgreSQL partitions WAL ownership across roughly two

dozen subsystem **Resource Managers** (`RmgrTable` in `rmgr.c`), each providing an `rm_redo` function and an `rm_desc` human decoder:

  - `RM_XLOG_ID` (`XLOG`): checkpoints, parameter changes, standalone full-page writes.
  - `RM_XACT_ID` (`Transaction`): `COMMIT`, `ABORT`, and prepared-transaction records.
  - `RM_HEAP_ID` / `RM_HEAP2_ID` (`Heap` / `Heap2`): `INSERT`, `UPDATE`, `HOT_UPDATE`, `DELETE`, in-page pruning (`PRUNE`), and freeze maps.
  - `RM_BTREE_ID` (`Btree`): `INSERT_LEAF`, `SPLIT_L`, `SPLIT_R`, page deletion.

- `xl_prev` **(Back-pointer verification):** Every record stores the starting LSN of the immediately preceding record in the WAL stream. Because PostgreSQL recycles old 16 MB WAL segment files by renaming them to future segment numbers instead of zeroing 16 MB on disk, the bytes sitting *past* the crash point in a recycled file often look like plausible old `XLogRecord` structs from last Tuesday. Checking both the 8 KB WAL page header address and `xl_prev` prevents recovery from accidentally replaying stale records left over from an earlier segment incarnation.
- `xl_crc` **(Hardware CRC32C):** Computed with dedicated SSE4.2 (`crc32q`) or ARMv8 instructions over the entire record body and header. If power fails halfway through flushing a 200-byte WAL record, `xl_crc` fails to match, cleanly marking the exact end of valid WAL on restart.

4.2.3 `pd_lsn` and the Fundamental WAL Invariant

How does a physical 8 KB heap or index block on disk remember how much of the WAL stream has already been applied to it? Look back at the very first field of `PageHeaderData` from Chapter 2, sitting at byte offsets `0..7` of every single heap page, B-tree page, GIN page, and visibility map page in PostgreSQL:

```
typedef struct PageHeaderData
{
    PageXLogRecPtr pd_lsn;        /* bytes 0..7: LSN of last modifying record */
    uint16         pd_checksum;   /* bytes 8..9: 16-bit FNV-1a page checksum  */
    uint16         pd_flags;      /* bytes 10..11: header flag bits           */
    LocationIndex  pd_lower;      /* bytes 12..13: start of free space        */
    LocationIndex  pd_upper;      /* bytes 14..15: end of free space          */
    /* ... */
} PageHeaderData;
```

Whenever a backend modifies buffer P in `shared_buffers`—inside an atomic critical section while holding an exclusive content lock on P—it constructs the corresponding `XLogRecord`, appends that record into `wal_buffers` receiving back the record's *ending* LSN L, stamps `PageSetLSN(P, L)` into bytes `0..7` of P, and marks P dirty.

Notice that `wal_buffers` is still volatile shared memory. At that instant, neither the WAL record at L nor the dirty data page P has reached NAND flash. That brings us to the single invariant on which all crash recovery rests.

Theorem 4.1 (The Write-Ahead Logging Invariant)**.** *Let P be any dirty buffer frame in `shared_buffers` carrying header stamp `pd_lsn`$(P) = L$, and let* FlushLSN *denote the highest*

WAL byte offset durably synced (`fdatasync`) to non-volatile storage inside `$PGDATA/pg_wal/`. PostgreSQL's buffer manager (`FlushBuffer` in `bufmgr.c`) enforces:

$$P \text{ may be written to the OS via } \texttt{pwrite(2)} \implies \text{FlushLSN} \geq \texttt{pd_lsn}(P) = L.$$

If FlushLSN $< L$ *when P is chosen for eviction or background cleaning, `FlushBuffer` blocks and invokes `XLogFlush(L)` before issuing `pwrite(2)`. Under this rule (together with torn-page protection from Section 4.3 and commit-record flushing at `COMMIT`), every committed transaction is recoverable and no physical page can persist an unlogged state on disk.*

Proof. In Haerder and Reuter's classic buffer-management taxonomy, PostgreSQL is a **Steal / No-Force** engine:

- **No-Force:** At `COMMIT` of transaction T_1 with commit record at LSN L_{commit}, PostgreSQL forces only the sequential WAL stream up to FlushLSN $\geq L_{\text{commit}}$; dirty 8 KB pages modified by T_1 may remain only in volatile `shared_buffers`. Because every prior modification by T_1 occurred at some LSN $L_i < L_{\text{commit}} \leq$ FlushLSN, every log record needed to reconstruct T_1's heap and index edits is already durable in `pg_wal` before the client sees `COMMIT`.
- **Steal:** Conversely, if `shared_buffers` runs low on clean frames (or `bgwriter` sweeps), PostgreSQL may *steal* frame P modified by a still-uncommitted transaction T_2 at LSN L and write P out to the heap file before T_2 commits. Because `FlushBuffer` enforces FlushLSN $\geq L$ prior to `pwrite(2)`, if power fails after P lands on disk, the WAL record at L (plus its full-page image if P tore) is guaranteed to exist on non-volatile storage, so recovery can reconstruct a structurally valid page P and rely on transaction status (or ARIES undo in overwrite-in-place engines) to govern visibility of T_2's changes. □

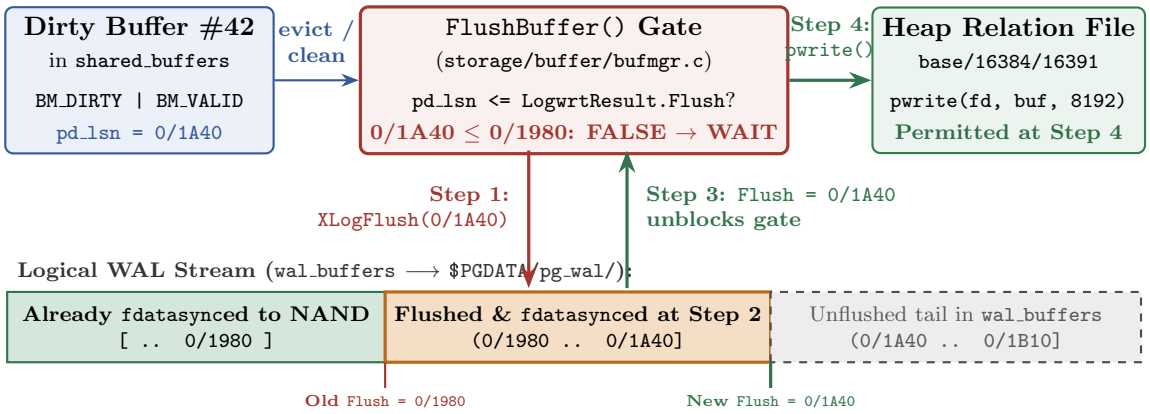


Figure 4.2: How `FlushBuffer()` enforces the WAL Invariant in shared memory. Dirty Buffer #42 carries `pd_lsn = 0/1A40`, while durable WAL (`XLogCtl->LogwrtResult.Flush`) has only reached `0/1980`. The gate blocks `pwrite()`, forces `wal_buffers` out through `fdatasync()` up to `0/1A40`, and only then lets the 8 KB heap page leave user-space RAM.

Failure mode

What breaks if `bgwriter` violates the WAL Invariant? Walk through a concrete violation. Suppose B-tree leaf block 19 is split by transaction T_1: a new right-sibling block 44 is

allocated, half of block 19's keys move to block 44 (logged at LSN `0/1A40`), and T_1 has not yet committed, so `0/1A40` sits only in volatile `wal_buffers` (`FlushLSN = 0/1980`). Suppose a broken background writer writes block 19 (now missing half its keys, with a right-link pointer to block 44) to the OS without calling `XLogFlush(0/1A40)`, the OS flushes block 19 to NAND, and power fails before `0/1A40` or block 44 ever reaches NAND. Upon reboot, recovery stops at `0/1980`. Block 44 does not exist on disk at all, while block 19 on disk permanently lost half its index entries *for previously committed rows* and holds a right-sibling pointer to a nonexistent page. Even in an MVCC database, pushing a data page ahead of its own WAL record destroys physical structural invariants across pages.

4.3 Torn Pages Solved: `full_page_writes` After a Checkpoint

Test your understanding with one question before reading on.

Walking question

Suppose PostgreSQL strictly obeys the WAL Invariant (Theorem 4.1): before dirty block 7 (`pd_lsn = 0/1140`) starts writing to the heap file, its WAL record at `0/1140` is already safely `fdatasync`ed in `pg_wal`. A millisecond later, `bgwriter` calls `pwrite()` on block 7, and building power dies halfway through, tearing block 7 across its two 4 KB sectors just like Figure 4.1(b). Since the WAL record at `0/1140` is safely on disk, why can't crash recovery simply replay record `0/1140` onto block 7 and fix the tear?

Work out what happens with actual byte offsets, and you will watch naive delta replay fall apart in *two independent ways*.

Normally, WAL records use **physiological logging**: *physical* to a specific 8 KB block ("relation 16384, block 7"), *logical* within that block's internal slotted structure ("decrement `pd_upper` by 56, write these 56 tuple bytes at `pd_upper`, and point `ItemId[4]` there"). Compact physiological deltas are tiny—often 55 to 80 bytes instead of 8,192 bytes—and replaying them requires knowing whether a given disk page already reflects the delta so we never insert the same tuple twice (idempotency). Standard recovery checks idempotency by comparing the page's `pd_lsn` against the record's end LSN:

$$\texttt{page.pd_lsn} \geq \texttt{record.end_lsn} \implies \text{skip replay (}\texttt{BLK_DONE}\text{);}$$
$$\texttt{page.pd_lsn} < \texttt{record.end_lsn} \implies \text{apply delta and set } \texttt{page.pd_lsn} \leftarrow \texttt{record.end_lsn}.$$

Now watch both halves of a torn 8 KB write defeat that rule:

1. **Failure Case A (Sector $2k$ lands, Sector $2k+1$ is lost):** Where does `pd_lsn` live inside an 8,192-byte page? At bytes `0..7`—right at the front of Sector $2k$! When Sector $2k$ lands on NAND flash and power cuts before Sector $2k+1$ arrives, the torn disk page carries the *new* `pd_lsn = 0/1140` in Sector $2k$, while Sector $2k+1$ still holds stale bytes missing the 56-byte tuple. When recovery restarts and reads block 7, it sees `page.pd_lsn` = `0/1140` $\geq$ `0/1140`, concludes "block 7 was already completely written before the crash," **skips replaying record** `0/1140`, and leaves a page whose header and tuples disagree, permanently.
2. **Failure Case B (Sector $2k+1$ lands first, Sector $2k$ is lost):** NVMe command queues freely reorder concurrent 4 KB sector writes when no barrier separates them. Suppose Sec-

tor $2k + 1$ (upper tuple bytes) hits NAND while Sector $2k$ (header + line pointers) stays at the old `pd_lsn = 0/1080`. Worse, suppose record `0/1140` performed an in-page defragmentation (`PageRepairFragmentation`) shifting twenty tuples upward in Sector $2k + 1$ and updating twenty `ItemIdData` offsets in Sector $2k$, followed by another delta at `0/1190`. Replaying physiological instructions that read current slot offsets or tuple headers off a half-shifted, half-unshifted 8 KB buffer reads misaligned bytes and scrambles the page.

Why it works

Physiological delta replay is only idempotent and well-defined when the 8 KB page on disk is a **clean snapshot from some single real point in time**—either completely before a WAL record or completely after it. A torn 8 KB page never existed in RAM at any instant in history; no sequence of deltas can safely build on a base image whose two 4 KB halves come from different points in time.

4.3.1 Hole-Stripped Full Page Images (FPIs) vs. Doublewrite Buffers

Database engineers have invented two main ways out of this trap. MySQL's InnoDB engine uses a **doublewrite buffer**: every time InnoDB flushes dirty 16 KB pages from its buffer pool, it first writes full 16 KB copies sequentially into a dedicated doublewrite region on disk, calls `fsync()`, and only then scatters the 16 KB pages to their real tablespace offsets—doubling every background data-page write.

PostgreSQL takes a leaner path controlled by the GUC `full_page_writes = on` (enabled by default and essential on every standard filesystem). Instead of writing every evicted buffer twice, PostgreSQL piggybacks torn-page protection directly onto its checkpoint cycle inside `XLogInsert` (`xloginsert.c`):

Formal

The `full_page_writes` Rule (`XLogCheckBufferNeedsBackup`). Let RedoRecPtr be the WAL LSN at which the most recently *started* checkpoint began. Whenever a backend modifies buffer P in `shared_buffers` and registers P in a WAL record, it inspects P's *prior* `pd_lsn` (before stamping the new LSN):

- If $\texttt{pd_lsn}_{\text{prior}}(P) \leq$ RedoRecPtr, this is the **very first modification to page P since the checkpoint began**. Instead of (or alongside) a physiological delta, PostgreSQL embeds a complete **Full Page Image (FPI)** of the modified 8 KB page directly inside this WAL record (`BKPBLOCK_HAS_IMAGE | BKPIMAGE_APPLY`).
- If $\texttt{pd_lsn}_{\text{prior}}(P) >$ RedoRecPtr, an earlier WAL record after RedoRecPtr already logged an FPI for page P! Every subsequent 2nd, 3rd, ..., 10,000th edit to page P until the next checkpoint begins emits **only a compact 55–80 byte physiological delta**.

Trace why this cures both Case A and Case B in one stroke. Suppose a checkpoint begins at RedoRecPtr = `0/1000`. Block 7 (previously at `pd_lsn = 0/0E20`) is touched first at `0/1080` (which embeds an FPI), then updated by deltas at `0/1140` and `0/1190`, and then tears on disk during a background flush right before power dies.

When PostgreSQL restarts, crash recovery always begins scanning forward from RedoRecPtr =

0/1000. What is the very first WAL record after 0/1000 that mentions block 7? Record 0/1080—carrying a Full Page Image. And when PostgreSQL's XLogReadBufferForRedoExtended encounters an active FPI (BKPIMAGE_APPLY), it **does not read or trust pd_lsn on the torn disk page at all**: it zeroes an 8 KB frame in shared_buffers, copies the intact FPI directly from WAL into that frame, stamps pd_lsn = 0/1080, and returns BLK_RESTORED. Even if Sector $2k$ on disk falsely claimed pd_lsn = 0/1190, the FPI unconditionally overwrites the entire 8 KB block in memory back to the clean, untorn state at 0/1080. Next, recovery reaches 0/1140, compares in-memory pd_lsn = 0/1080 < 0/1140, applies the 64-byte delta in memory, and then applies 0/1190's 58-byte delta on top.

Better still, an "8 KB" Full Page Image rarely writes 8,192 bytes to WAL. Look at a slotted page again: between pd_lower (end of the ItemIdData array) and pd_upper (start of the tuple region) sits a contiguous **free-space hole** containing zero live information. PostgreSQL strips that hole completely (BKPIMAGE_HAS_HOLE), recording only hole_offset = pd_lower and hole_length = pd_upper - pd_lower in a 4-byte image header:

$$\text{Raw FPI bytes} = 8{,}192 - (\texttt{pd_upper} - \texttt{pd_lower}).$$

If block 7 holds ten tuples totaling 2,000 bytes (pd_lower = 64, pd_upper = 6192), the hole is 6,128 bytes, so the raw FPI costs only 64+2,000 = 2,064 bytes—and enabling wal_compression = lz4 (or zstd in PostgreSQL 15+) typically shrinks those 2,064 bytes down to **650–900 bytes** at negligible CPU cost.

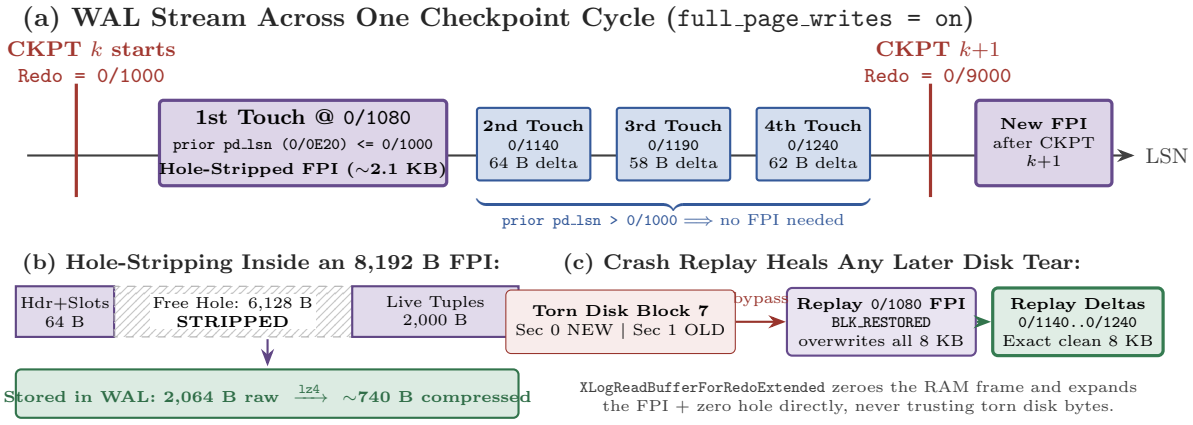


Figure 4.3: How full_page_writes eliminates torn pages without a doublewrite buffer. Only the *first* edit to Block 7 after each checkpoint writes a hole-stripped Full Page Image (0/1080); subsequent edits before the next checkpoint write tiny ~60-byte deltas. On crash recovery, the FPI unconditionally overwrites the entire 8 KB block in memory (BLK_RESTORED), healing any mid-write tear on disk before deltas replay.

In production

Diagnosing and taming the "Checkpoint WAL Storm" in production. Because every checkpoint advances RedoRecPtr, the instant a new checkpoint begins, *every hot page in shared_buffers* satisfies pd_lsn ≤ RedoRecPtr again. Over the next 30 to 90 seconds, the first update to each active heap and B-tree page emits a Full Page Image, causing WAL throughput (wal_fpi and wal_bytes in pg_stat_wal), disk sync load, and replication network

bandwidth to spike 3× to 10× in a sharp sawtooth wave. Two tuning knobs flatten that sawtooth immediately on busy OLTP clusters:

1. Increase `checkpoint_timeout` from the conservative `5min` default to `15min`–`30min` (raising `max_wal_size` to `16GB`–`64GB` to match), so hot buffers pay the FPI tax three to six times less often.
2. Set `wal_compression = lz4` (or `zstd`), which typically cuts FPI byte volume by 60%–75% while actually *increasing* commit throughput on NVMe and cloud network block volumes.

Never set `full_page_writes = off` on `ext4` or `XFS`; it is only safe on Copy-on-Write filesystems such as ZFS configured with `recordsize=8k` where the filesystem itself never overwrites an 8 KB block in place.

4.4 Commit Durability Tradeoffs, Group Commit, and `wal_writer`

Appending a 70-byte `XLogRecord` is sequential, which solves random heap seeks—yet a sharp systems engineer should immediately raise a hand and object: *"Wait. Even sequential `fdatasync()` takes* 80 μs *on a local enterprise NVMe drive and* 600–1,200 μs *on a cloud network block volume. If one backend holds a lock for* 800 μs *on every `COMMIT`, isn't our 64-core server capped at* $1/0.0008\ s = 1{,}250$ *commits per second?"*

If each committing backend serialized its own `fdatasync()` one at a time, that 1,250 TPS ceiling would be brick-solid. Breaking right through it requires separating *inserting bytes into shared WAL memory* from *flushing shared WAL memory to NAND*, and batching concurrent committers through **leader-follower group commit**.

4.4.1 `wal_buffers`, Parallel Insertion Slots, and Group Commit

Inside shared memory, PostgreSQL carves out a circular ring buffer of 8 KB WAL frames controlled by `wal_buffers` (auto-tuned to 1/32 of `shared_buffers`, clamped between 64 KB and 16 MB—one full WAL segment). Two distinct synchronization layers govern this ring:

1. **Concurrent insertion (`NUM_XLOGINSERT_LOCKS = 8`):** When forty backends execute `INSERT` and `UPDATE` statements simultaneously, each backend computes its `XLogRecord` CRC32C in private memory, claims one of eight `WALInsertLocks`, atomically reserves exact start/end byte offsets $[L_{\text{start}}, L_{\text{end}})$ via a tiny spinlock/atomic fetch-add on the global byte position, and copies its bytes into `wal_buffers` in parallel with seven other cores.
2. **Leader-follower `XLogFlush()` on `WALWriteLock`:** When backend B_1 executes `COMMIT`, it appends a tiny `XACT_COMMIT` record ending at L_1 = `0/1500` and calls `XLogFlush(0/1500)`. Only one process at a time may hold `WALWriteLock` to issue `write()` and `fdatasync()` on the active segment file. Watch what happens over the next 800 μs while B_1 is parked inside the kernel waiting for NVMe NAND programming to finish:
   - Backends $B_2, B_3, \ldots, B_{40}$ do *not* stall on insertion. Using the eight `WALInsertLocks`, all thirty-nine append their own `XACT_COMMIT` records into `wal_buffers`, advancing the

inserted tail all the way from `0/1500` to L_{40} = `0/4B20`, and then line up waiting on `WALWriteLock`.

- As soon as B_1's `fdatasync()` returns, B_1 updates shared memory (`LogwrtResult.Flush = 0/1500`) and releases `WALWriteLock`.
- Backend B_2 wakes up as the new **group leader**. Before touching the kernel, B_2 checks how far `wal_buffers` has filled—not merely up to its own commit LSN L_2 = `0/1590`, **all the way up to the latest completed insertion at** L_{40} = `0/4B20`**!** Backend B_2 writes the entire contiguous slice `(0/1500 .. 0/4B20]` in one sequential `write()` and issues **one single** `fdatasync()` for all thirty-nine transactions.
- When B_2 updates `LogwrtResult.Flush = 0/4B20` and drops `WALWriteLock`, followers $B_3 \dots B_{40}$ check shared memory before even acquiring (or right upon acquiring) the lock, see `Flush` = `0/4B20` $\geq L_i$, and return `COMMIT` to their clients immediately with **zero** system calls.

Under the hood

Here is the core leader-follower double-check pattern distilled from `XLogFlush()` in `src/backend/access/transam/xlog.c`:

```
void XLogFlush(XLogRecPtr record_end_lsn)
{
    /* Fast path: did an earlier group leader already sync past our LSN? */
    if (record_end_lsn <= LogwrtResult.Flush)
        return;

    /* Wait our turn (or wake up after the current leader finishes) */
    LWLockAcquire(WALWriteLock, LW_EXCLUSIVE);

    /* Refresh shared Flush pointer; if the previous leader swept our
     * commit record into its fdatasync batch, exit with ZERO syscalls! */
    RefreshLogwrtResult();
    if (record_end_lsn <= LogwrtResult.Flush)
    {
        LWLockRelease(WALWriteLock);
        return;
    }

    /* We are the new group leader: flush ALL ready WAL bytes in wal_buffers,
     * sweeping in every follower that appended while we waited! */
    XLogRecPtr insert_pos = WaitXLogInsertionsToFinish(GetXLogInsertRecPtr());
    XLogWrite(insert_pos);   /* sequential write() + single fdatasync()  */
    LWLockRelease(WALWriteLock);
}
```

Notice why forty concurrent clients on a 0.8 ms cloud disk are no longer stuck at 1,250 TPS: amortizing thirty-nine commits per 0.8 ms round-trip lifts sustained durable throughput above 45,000 TPS on the exact same storage volume. PostgreSQL also exposes `commit_delay` (microseconds a leader pauses before calling `XLogWrite` when at least `commit_siblings` transactions are active), though under genuine concurrency natural queueing during the previous `fdatasync()` already forms full batches without artificial sleep.

4.4.2 The `synchronous_commit` Spectrum and `wal_writer`

What if a workload writes millions of telemetry events, IoT sensor readings, or HTTP audit rows per minute where losing the last half-second of acknowledged inserts during a once-a-decade power outage is completely acceptable, provided the database itself **never corrupts**?

Most developers facing that tradeoff reach for the wrong knob—and occasionally blow up a company's database by setting `fsync = off`. The two settings differ in what they risk, so we separate them here.

Common trap

`synchronous_commit = off` is safe engineering; `fsync = off` is Russian roulette.

- Setting `fsync = off` strips `fdatasync()` out of PostgreSQL everywhere—commits, checkpoints, `FlushBuffer`, catalog updates, control-file updates. If the OS crashes or loses power, heap pages, B-tree pages, `pg_xact`, and `pg_control` land on NAND in arbitrary out-of-order fragments, violating the WAL Invariant and leaving the entire cluster unrecoverable (`PANIC: could not locate a valid checkpoint record` or silent index corruption). Use `fsync = off` *only* on throwaway CI test instances you intend to delete on failure.
- Setting `synchronous_commit = off` (even for a single transaction via `SET LOCAL synchronous_commit = off;`) changes *only one thing*: when a transaction commits, the backend appends `XACT_COMMIT` into `wal_buffers` and returns `COMMIT` to the client immediately without calling `XLogFlush()`. Better yet, `full_page_writes` **and the WAL Invariant inside** `FlushBuffer()` **remain** 100% **active.** No dirty heap or index page can ever reach the OS until its WAL record has been `fdatasync`ed.

Who flushes `wal_buffers` when backends commit asynchronously with `synchronous_commit = off`? A dedicated background daemon called `wal_writer` wakes up every `wal_writer_delay` (default `200ms`, or sooner once `wal_writer_flush_after = 1MB` of unwritten WAL accumulates) and calls `XLogBackgroundFlush()`. To avoid repeatedly overwriting partial 8 KB WAL sectors on quiet servers, `XLogBackgroundFlush()` flushes completed 8 KB WAL pages on normal ticks and forces out any lingering partial tail page once a full cycle elapses—bounding the maximum window of acknowledged-yet-unsynced commits on sudden power loss to at most $3 \times$ `wal_writer_delay` $\approx$ **600** ms. If power fails, PostgreSQL recovers cleanly to a 100% transactionally and structurally consistent state from ≤ 600 ms ago, with zero torn pages and zero broken indexes.

`synchronous_commit` can also be changed per transaction. Within the exact same database connection pool, a payment transfer runs at the default `synchronous_commit = on`, while an analytics click logger runs:

```
BEGIN;
SET LOCAL synchronous_commit = off;
INSERT INTO page_views (user_id, url, viewed_at) VALUES (4812, '/checkout', now());
COMMIT;  -- returns in ~15 microseconds without waiting on NVMe fdatasync!
```

When you add streaming replication standbys (which we wire up end-to-end in Chapter 8), `synchronous_commit` generalizes smoothly from two levels to five, letting each transaction choose exactly how many failure domains its `XACT_COMMIT` record must cross before the client receives `COMMIT`:

Setting	Primary WAL fdatasync?	Sync standby guarantee before client ACK	Power loss on primary host	Primary disk destroyed
off	Async ($\leq 3d$)	None	Lose $\leq$ 600 ms; 0 corruption	Lose unshipped WAL
local	**Yes**	None (local only)	**0 committed rows lost**	Lose unshipped WAL
remote_write	**Yes**	Standby OS page cache (write)	**0 committed rows lost**	0 lost unless dual power cut
on *(default)*	**Yes**	Standby NAND (fdatasync)*	**0 committed rows lost**	**0 committed rows lost**
remote_apply	**Yes**	Standby NAND + replayed in RAM	**0 committed rows lost**	**0 lost + read-your-writes**

Table 4.1: Exact durability guarantees across all five synchronous_commit modes (d = wal_writer_delay = 200 ms). *When synchronous_standby_names is empty, on behaves identically to local.

4.4.3 UNLOGGED Tables: Trading Crash Survival for Raw RAM-to-Disk Speed

Sometimes even compact WAL records are pure waste: think of an intermediate staging table in an idempotent nightly ETL job, a reconstructible materialized lookup cache, or ephemeral user-session state. Declaring a table UNLOGGED:

```
CREATE UNLOGGED TABLE etl_staging_events (
    batch_id    bigint NOT NULL,
    payload     jsonb  NOT NULL
);
```

instructs PostgreSQL (relpersistence = 'u' in pg_class) to **bypass WAL generation completely** on INSERT, UPDATE, DELETE, and index maintenance. Backends mutate 8 KB pages in shared_buffers without acquiring WALInsertLocks, without writing Full Page Images after checkpoints, and without streaming bytes to standbys—often doubling or tripling bulk write throughput.

How does PostgreSQL prevent torn or half-flushed UNLOGGED pages from corrupting queries after a crash? When CREATE UNLOGGED TABLE runs, PostgreSQL writes one tiny WAL-logged file on disk alongside the main relation file (say base/16384/24580): an empty initialization fork named 24580_init (and an initialized empty root page _init fork for each index on that table).

- On a **clean shutdown** (pg_ctl stop -m fast), the shutdown checkpoint flushes every dirty UNLOGGED buffer to disk cleanly, so all rows in etl_staging_events survive normal restarts intact.

- On **any unclean shutdown / crash recovery**, before opening the database for client connections, ResetUnloggedRelations() scans $PGDATA/base/ for every relation possessing an _init fork, unlinks the potentially torn main data files, and copies 24580_init over 24580—automatically **truncating every UNLOGGED table to zero rows** so broken pages can never be read.

4.5 Checkpoints, Background Writer, and Step-by-Step Crash Recovery

One question remains. If we append to `$PGDATA/pg_wal/` forever and never coordinate when dirty buffers reach their relation files, two bad things happen: `pg_wal` fills the entire NVMe drive, and recovering from a crash after six months of uptime requires replaying six months of WAL from LSN `0/0`.

A **checkpoint** draws a clean floor under history so PostgreSQL can recycle old 16 MB WAL segment files and bound crash-recovery time to seconds. Yet the way a production checkpoint actually runs contains a detail that surprises most people the first time they look inside `pg_control`.

4.5.1 `global/pg_control`, Fuzzy Checkpoints, and Why Recovery Rewinds to the Redo Point

When a crashed PostgreSQL server boots up, how does `StartupXLOG()` know where in a 50 GB sea of `pg_wal` segments to begin reading? It reads an 8,192-byte metadata file at `$PGDATA/global/pg_control`, holding a single CRC32C-protected C struct called `ControlFileData` (smaller than one 512-byte hardware sector in active payload so its update is atomic at the sector level):

```
/* Simplified excerpt from src/include/catalog/pg_control.h */
typedef struct CheckPoint
{
    XLogRecPtr      redo;           /* Redo start LSN when ckpt BEGAN!   */
    TimeLineID      ThisTimeLineID; /* current WAL timeline              */
    FullTransactionId nextXid;      /* next free 64-bit XID              */
    /* ... oldestActiveXid, oldestMulti, timestamp ... */
} CheckPoint;

typedef struct ControlFileData
{
    uint64          system_identifier;
    DBState         state;          /* DB_SHUTDOWNED vs DB_IN_PRODUCTION */
    XLogRecPtr      checkPoint;     /* LSN of latest Checkpoint Record   */
    CheckPoint      checkPointCopy; /* embedded copy including .redo!    */
    uint32          data_checksum_version;
    pg_crc32c       crc;
} ControlFileData;
```

Look at those two distinct LSN fields inside `ControlFileData`: `checkPoint` (say `0/1230`) and `checkPointCopy.redo` (say `0/1000`). Why aren't they equal?!

Imagine what would happen on a server with 64 GB of `shared_buffers` if the `checkpointer` process tried to take an instantaneous "sharp" checkpoint: freeze all user writes, blast 20 GB of dirty buffers to disk at maximum I/O queue depth, `fsync` three hundred relation files, write one checkpoint WAL record, and unfreeze clients. Every five minutes your application latency graph would flatline into a 25-second outage.

Instead, PostgreSQL performs **spread (fuzzy) online checkpoints** governed by three GUCs:

- `checkpoint_timeout` (default `5min`, commonly tuned to `15min`–`30min`): maximum wall-clock interval between checkpoints.

- `max_wal_size` (default `1GB`, commonly `16GB`–`64GB`): soft ceiling on `pg_wal` growth before triggering an early checkpoint.

- `checkpoint_completion_target` (default `0.9`): fraction of the interval over which `checkpointer` paces its dirty-buffer writes (0.9×300 s $= 270$ s).

Trace carefully what happens between $t = 0$ s and $t = 270$ s while online transactions keep running uninterrupted:

1. **At $t = 0$ s (LSN `0/1000`):** `checkpointer` wakes up, records the current WAL insert position as `RedoRecPtr = 0/1000` (publishing it immediately so `full_page_writes` logs FPIs for pages touched from `0/1000` onward), and scans all `BufferDescriptors`, setting the flag `BM_CHECKPOINT_NEEDED` on every buffer that is *already dirty right now at $t = 0$ s*.

2. **Across $t \in (0\text{ s}, 270\text{ s})$:** `checkpointer` trickles those `BM_CHECKPOINT_NEEDED` buffers out to the OS with gentle sleeps between batches and finally calls `fsync()` on every touched relation file. Meanwhile, user backends never pause. Suppose at $t = 40$ s (LSN `0/1080`), client T_1 modifies clean buffer B_7 (which was *not* dirty at $t = 0$, or which `checkpointer` already swept past at $t = 10$ s!). Buffer B_7 is now dirty in `shared_buffers` with `pd_lsn = 0/1080`, and `checkpointer` is **not** obligated to flush B_7 during this checkpoint cycle.

3. **At $t = 270$ s (LSN `0/1230`):** Having confirmed that *every dirty change from before `0/1000` is safely on NAND*, `checkpointer` appends an `XLOG_CHECKPOINT_ONLINE` record at LSN `0/1230` whose payload records `redo = 0/1000`, calls `XLogFlush(0/1230)`, atomically updates `global/pg_control` (`checkPoint = 0/1230`, `redo = 0/1000`), and recycles any 16 MB WAL segment strictly preceding `0/1000`.

Why it works

Now you see why recovery **must** rewind from `checkPoint = 0/1230` back to `redo = 0/1000`. At the moment `0/1230` hits disk, PostgreSQL only promises that changes *older than `0/1000`* have been flushed to relation files; changes logged between `0/1000` and `0/1230` (like T_1's edit to B_7 at `0/1080`) may still live solely in volatile `shared_buffers`. Starting Redo at `0/1230` would silently evaporate T_1's committed write.

That fuzzy window also yields the exact sizing formula inside `xlog.c` (`CalculateCheckpointSegments`): while Checkpoint 2 runs for $c \cdot T$ seconds (c = `checkpoint_completion_target` = 0.9), PostgreSQL cannot yet delete the WAL segments anchored at Checkpoint 1's Redo point. Just before Checkpoint 2 finishes, live unrecyclable WAL spans $(1 + c)$ checkpoint intervals:

$$\text{Peak live WAL} \approx (1 + \texttt{checkpoint_completion_target}) \times R_{\text{wal}} \times \texttt{checkpoint_timeout}.$$

With $c = 0.9$, if your workload writes $R_{\text{wal}} = 20$ MB/s and `checkpoint_timeout = 15min` (900 s, or 18 GB per interval), `max_wal_size` must be at least 1.9×18 GB $\approx$ **35** GB—otherwise PostgreSQL is forced to trigger early checkpoints before the 15-minute timer ever expires.

4.5.2 Three Buffer Writers, Three Distinct Jobs

Newcomers often confuse `checkpointer` with the background writer (`bgwriter`). In fact, dirty 8 KB frames leave `shared_buffers` via three different processes, visible side-by-side in `pg_stat_checkpointer`, `pg_stat_bgwriter`, and PostgreSQL 16+'s unified `pg_stat_io` view:

- **`checkpointer` (recovery bound & WAL reclamation):** Flushes every buffer dirtied before RedoRecPtr once per checkpoint cycle and issues relation `fsync()` calls. In a healthy cluster, `checkpointer` accounts for 70%–95% of all data-page writes (because keeping a hot page in RAM across 500 updates during a 15-minute checkpoint collapses 500 writes into 1 write).
- **`bgwriter` (eviction headroom for `shared_buffers`):** Wakes every `bgwriter_delay` (`200ms`) and scans just ahead of the clock-sweep victim hand, cleaning low-usage-count dirty buffers (up to `bgwriter_lru_maxpages`) *only* when buffer churn threatens to run out of clean frames. It never creates checkpoints.
- **Client backend direct writes (the distress signal):** If a user query needs to read a page from disk into a full `shared_buffers` pool and the clock-sweep hand hits a `BM_DIRTY` victim before `bgwriter` cleaned it, *your user's `SELECT` query has to call `FlushBuffer()` (and potentially `XLogFlush()`) inline.* Rising backend writes/evictions in `pg_stat_io` (`backend_type = 'client backend'`) tell you immediately that `shared_buffers` or `bgwriter_lru_maxpages` is starved.

4.5.3 Classic ARIES (3 Passes) vs. PostgreSQL's 1-Pass Redo-Only Recovery

Open almost any university database textbook and crash recovery is synonymous with IBM's classic **ARIES** algorithm (Mohan et al., 1992), used by engines that overwrite rows in place or maintain separate undo segments. ARIES makes **three distinct passes** across the log on startup:

1. **Analysis pass (forward):** Scans forward from the checkpoint to reconstruct the Dirty Page Table and identify "loser" transactions still active at the instant of the crash.
2. **Redo pass (forward, "repeating history"):** Replays all logged physical updates—both winners and losers—bringing disk pages back to the exact state at the moment power failed.
3. **Undo pass (backward):** Walks backward along `prevLSN` chains for every loser transaction, physically reversing every uncommitted row modification and appending Compensation Log Records (CLRs). If a batch job updated 50 million rows over two hours and power died at 99%, an eager ARIES Undo pass must physically roll back 50 million rows before recovery finishes (or run complex background rollback threads holding row locks).

Now open `src/backend/access/transam/xlogrecovery.c` in PostgreSQL and look for the Undo pass. **There isn't one.** PostgreSQL has zero Undo records, zero Compensation Log Records, and zero backward WAL scan.

Connection

How MVCC (Chapter 5) eliminates the Undo phase completely. Recall from Chapter 2 that a PostgreSQL `UPDATE` never overwrites an existing tuple's bytes in place: it stamps `t_xmax = 702` on the old tuple version and inserts a brand-new physical tuple version stamped

`t_xmin = 702`. During crash recovery, PostgreSQL executes **one single forward Redo pass** from RedoRecPtr to the end of valid WAL, faithfully replaying both committed transaction T_1 (`xid = 701`) *and* crashed uncommitted transaction T_2 (`xid = 702`) onto the heap and B-tree pages.
Why doesn't replaying T_2's uncommitted tuple corrupt query results when PostgreSQL opens for connections a second later? Because transaction commit status lives in `$PGDATA/pg_xact/` (two bits per XID: `00 = IN_PROGRESS`, `01 = COMMITTED`, `10 = ABORTED`, `11 = SUB_COMMITTED`). Transaction T_1 logged `XACT_COMMIT` at `0/1210`, so Redo sets `pg_xact[701] = 01`. Transaction T_2 crashed before committing, so `pg_xact[702]` stays `00` while recovery advances `nextXid` past `702`. As Chapter 5 proves, any past XID whose `pg_xact` status is `00` and is no longer running is **instantaneously treated as aborted by every MVCC snapshot**. Readers skip T_2's new tuple, treat the old tuple (`t_xmax = 702` aborted) as still live, set a one-bit hint flag on first visit, and let background `VACUUM` reclaim T_2's dead bytes lazily later. Whether T_2 modified ten rows or a hundred million rows before crashing, PostgreSQL's Undo time at startup is **0** ms.

Inside `xlogutils.h`, every block reference inspected during that single forward Redo pass resolves to one of four explicit actions returned by `XLogReadBufferForRedoExtended()`:

```
typedef enum
{
    BLK_NEEDS_REDO, /* page.pd_lsn < record.end_lsn -> apply delta & set LSN */
    BLK_DONE,       /* page.pd_lsn >= record.end_lsn -> already applied, skip */
    BLK_RESTORED,   /* record carried an FPI -> 8 KB block restored from WAL  */
    BLK_NOTFOUND    /* relation/block truncated & dropped later -> skip       */
} XLogRedoAction;
```

Algorithm 1 puts the entire `StartupXLOG()` recovery state machine onto one page.

Algorithm 1: PostgresCrashRecovery: Single-Pass Redo in `xlogrecovery.c`

Input: `global/pg_control` and segment files in `$PGDATA/pg_wal/`.
Output: Consistent `shared_buffers`, `pg_xact`, and relation files ready for clients.

```
1  Read ControlFileData from global/pg_control; verify CRC32C;
2  if state = DB_SHUTDOWNED then
3      return ;                                  // clean shutdown; zero recovery needed
4  Read Checkpoint Record at ControlFile.checkPoint; extract RedoLSN ← ckpt.redo;
5  Initialize WAL reader at RedoLSN;
6  while ReadNextXLogRecord() returns header + payload with valid xl_prev and
     CRC32C do
7      L_end ← ending byte offset of current XLogRecord;
8      nextXid ← max(nextXid, record.xl_xid + 1);
9      if record.xl_rmid = RM_XACT_ID then
10         Update pg_xact[record.xl_xid] to 01 (COMMITTED) or 10 (ABORTED);
11     else
12         foreach modified block reference B in record do
13             if B carries an active Full Page Image (BKPIMAGE_APPLY) then
14                 Zero buffer frame P in shared_buffers; expand hole-stripped FPI into P;
15                 P.pd_lsn ← L_end; mark P dirty ;        // BLK_RESTORED: heals any tear.
16             else
17                 Read disk block B into buffer frame P;
18                 if P.pd_lsn < L_end then
19                     Invoke RmgrTable[record.xl_rmid].rm_redo(record, P) ;
                        // BLK_NEEDS_REDO
20                     P.pd_lsn ← L_end; mark P dirty;
                   // else P.pd_lsn ≥ L_end ⟹ BLK_DONE (idempotent skip)
   // First invalid CRC32C or header marks exact end-of-WAL boundary
21 ResetUnloggedRelations() ;        // copy _init forks over all UNLOGGED tables
22 Write end-of-recovery checkpoint (or promote); set state ← DB_IN_PRODUCTION;
```

4.5.4 Worked LSN-by-LSN Crash Recovery Trace

Let us trace Algorithm 1 by hand on concrete hexadecimal LSNs so no moving part remains abstract. Suppose heap Block 7 of `accounts` initially sits on disk with `pd_lsn` = `0/0E20` holding one row `(id=1, bal=500)`. Watch six events unfold right before a power outage:

1. `0/1000` **(Online checkpoint begins):** `checkpointer` starts an online checkpoint and sets RedoRecPtr = `0/1000`.

2. `0/1080` **(T_1, `xid=701`, first touch to Block 7):** T_1 inserts `(id=2, bal=300)` onto Block 7. Because Block 7's prior `pd_lsn` = `0/0E20` ≤ `0/1000`, PostgreSQL logs `RM_HEAP_ID / INSERT` carrying a **Full Page Image** ending at `0/1080`, setting in-RAM `pd_lsn` = `0/1080`.

3. `0/1140` **(T_1, `xid=701`, delta update on Block 7):** T_1 runs `UPDATE accounts SET bal = 400 WHERE id = 1` (a same-page `HOT_UPDATE`). Because prior `pd_lsn` = `0/1080` > `0/1000`, PostgreSQL logs only a compact 64-byte delta ending at `0/1140`; in-RAM `pd_lsn` = `0/1140`.

4. `0/1190` (T_2, `xid=702`, **uncommitted insert on Block 7):** Concurrent transaction T_2 inserts `(id=3, bal=999)` onto Block 7 via a 58-byte delta ending at `0/1190`; in-RAM `pd_lsn` = `0/1190`. Notice T_2 has *not* committed yet.

5. `0/1210` (T_1 **commits):** T_1 executes `COMMIT`, writing `RM_XACT_ID / COMMIT` for `xid=701` and calling `XLogFlush(0/1210)`. Client T_1 receives `COMMIT`.

6. `0/1230` **(Checkpoint finishes) &** `0/1260` **(Torn write + power cut):** At `0/1230`, `checkpointer` finishes flushing pre-`0/1000` buffers, logs `XLOG_CHECKPOINT_ONLINE` (`redo = 0/1000`), and updates `global/pg_control` (`checkPoint = 0/1230`). Moments later, `bgwriter` flushes dirty Block 7 (`pd_lsn = 0/1190`) (permitted because WAL is synced past `0/1230`) and **power dies mid-write**:

   - On disk, **Block 7 is torn**: Sector 0 carries `pd_lsn = 0/1190` and three line pointers, while Sector 1 still holds stale pre-`0/1000` tuple bytes.
   - Meanwhile in `pg_wal`, T_2 was writing another record at `0/1260`; only its first 40 bytes reached NAND, leaving a broken `xl_crc`.

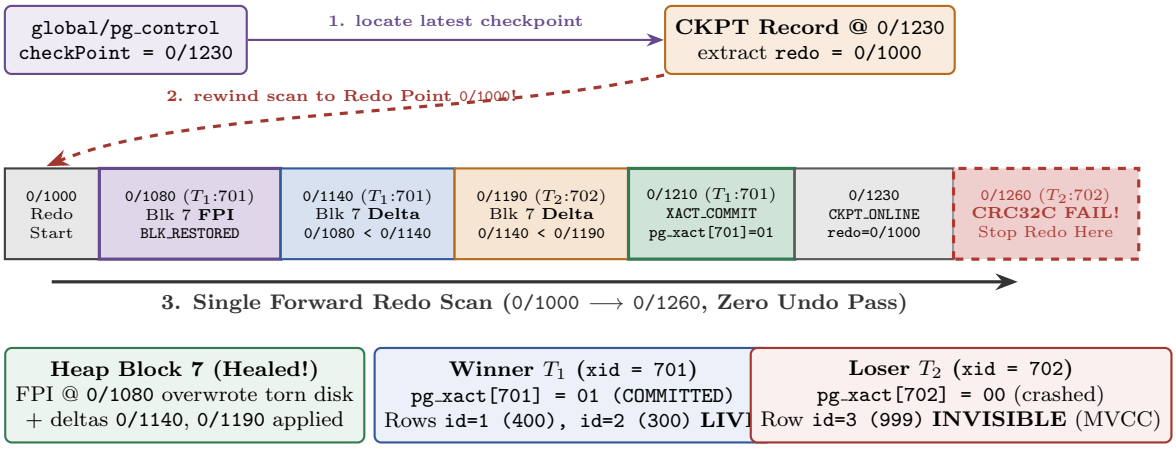


Figure 4.4: Complete crash-recovery trace from `global/pg_control` to database readiness. Recovery reads `0/1230`, jumps backward to `redo = 0/1000`, heals torn Block 7 via the `0/1080` FPI, replays both T_1 and T_2 forward, halts cleanly on the `0/1260` CRC failure, and relies on `pg_xact` + MVCC to hide T_2's uncommitted insert in $O(1)$ time.

Now step through Table 4.2 as electricity returns and `StartupXLOG()` boots:

WAL LSN	Record Type	Buffer pd_lsn Check	Action	Resulting State in Memory
pg_control	Bootstrap	state = IN_PRODUCTION	Crash detected	Jump to 0/1230, extract redo = 0/1000.
0/1080	HEAP INSERT+FPI	Ignored. (has FPI)	BLK_RESTORED	Torn disk pd_lsn=0/1190 discarded. Clean 8 KB Block 7 restored at pd_lsn = 0/1080.
0/1140	HEAP HOT_UPDATE	0/1080 < 0/1140	BLK_NEEDS_REDO	T_1 updates id=1 to bal=400; Block 7 pd_lsn ← 0/1140.
0/1190	HEAP INSERT	0/1140 < 0/1190	BLK_NEEDS_REDO	T_2 inserts (id=3, bal=999) (t_xmin=702); Block 7 pd_lsn ← 0/1190.
0/1210	XACT COMMIT	—	Update CLOG	pg_xact[701] ← 01 (COMMITTED).
0/1230	CKPT_ONLINE	—	Metadata sync	Update nextXid ≥ 703; keep scanning.
0/1260	Truncated tail	xl_crc mismatch.	**End of WAL**	Stop scan; pg_xact[702] stays 00 (never committed, so every later snapshot treats T_2 as aborted).

Table 4.2: Exact LSN-by-LSN state transitions executed by `StartupXLOG()` on our crashed cluster.

The server log for this restart shows the same steps:

```
LOG:  database system was interrupted; last known up at 2026-11-01 09:12:03 UTC
LOG:  database system was not properly shut down; automatic recovery in progress
LOG:  redo starts at 0/1000
LOG:  incorrect resource manager data checksum in record at 0/1260
LOG:  redo done at 0/1230
LOG:  checkpoint starting: end-of-recovery immediate wait
LOG:  database system is ready to accept connections
```

The checksum line is not an error: it is how recovery finds the end of valid WAL.

Row 2 (0/1080) of Table 4.2: if 0/1080 had been a delta instead of an FPI, recovery would have read torn Block 7 from disk, seen Sector 0's pd_lsn = 0/1190, returned BLK_DONE on 0/1080, 0/1140, *and* 0/1190, and left Sector 1 full of garbage. Instead, BLK_RESTORED wiped the torn disk block from memory at 0/1080, let 0/1140 and 0/1190 rebuild the exact byte-for-byte slotted page in RAM, and left T_2's uncommitted tuple (id=3, bal=999) harmlessly invisible behind t_xmin = 702.

Try it in PostgreSQL

In PostgreSQL 15 and later, the built-in pg_walinspect extension lets you watch this exact FPI-then-delta transition live inside SQL without even leaving psql:

```
CREATE EXTENSION IF NOT EXISTS pg_walinspect;

CREATE TABLE wal_demo (id int PRIMARY KEY, bal int);
INSERT INTO wal_demo VALUES (1, 500);

-- Force a checkpoint so RedoRecPtr advances past wal_demo's current pd_lsn:
CHECKPOINT;

```

```
-- Capture LSN right after the checkpoint, then touch Block 0 TWICE:
SELECT pg_current_wal_lsn() AS start_lsn \gset
BEGIN;
INSERT INTO wal_demo VALUES (2, 300);            -- 1st touch -> logs an FPI!
UPDATE wal_demo SET bal = 400 WHERE id = 1;      -- 2nd touch -> tiny delta!
COMMIT;
SELECT pg_current_wal_lsn() AS end_lsn \gset

SELECT start_lsn, end_lsn, xid, resource_manager AS rmgr,
       record_type, record_length AS rec_len, fpi_length AS fpi_len
FROM pg_get_wal_records_info(:'start_lsn', :'end_lsn')
WHERE resource_manager IN ('Heap', 'Transaction');
```

Running this snippet produces output structured right along our theoretical trace:

```
--  start_lsn |  end_lsn  | xid |    rmgr     | record_type | rec_len | fpi_len
-- -----------+-----------+-----+-------------+-------------+---------+---------
--  0/19420A8 | 0/1942150 | 745 | Heap        | INSERT      |     168 |     120
--  0/19421A0 | 0/19421E8 | 745 | Heap        | HOT_UPDATE  |      72 |       0
--  0/19421E8 | 0/1942210 | 745 | Transaction | COMMIT      |      40 |       0
```

Notice the `fpi_len` column: on the very first heap edit after `CHECKPOINT`, `fpi_len` is `120` bytes: the 24-byte page header, two 4-byte line pointers, and two 44-byte tuples, with the free-space hole between `pd_lower` and `pd_upper` left out. The record is 168 bytes because it also carries the 24-byte `XLogRecord` header, the block reference, and the insert's own data. Twenty microseconds later, `HOT_UPDATE` touches the exact same heap page and writes `fpi_len = 0`—just a 72-byte physiological delta, followed by a 40-byte `COMMIT` record. (You can also run `pg_waldump -p $PGDATA/pg_wal 000000010000000000000001` from the shell to inspect raw WAL offline when a server is stopped.)

Cost summary

Cost Summary of PostgreSQL WAL & Crash Recovery:

- `COMMIT` **I/O (`synchronous_commit = on`):** 1 sequential `write()` + 1 `fdatasync()` shared across K concurrent committers ($O(1/K)$ sync calls per transaction; zero random heap/index `fsyncs`).
- **Per-row WAL size:** $\sim$ 55–90 bytes per heap/B-tree delta; +(8,192 − hole) bytes (or $\sim 0.3\times$ compressed with `lz4`) on *only the first touch* per 8 KB block per checkpoint.
- **Crash recovery time:** O(WAL bytes since RedoRecPtr) in a single forward scan, plus $O(1) = 0$ **Undo work** regardless of how many rows crashed transactions modified.

Everything we built in this chapter gives PostgreSQL this guarantee: physical 8 KB pages never tear, committed transactions never vanish, and crashed uncommitted transactions leave physical tuple versions behind whose `pg_xact` bits remain `00`. That leaves one huge question hanging in the air: when twenty concurrent queries read Block 7 right now, how does each query decide in nanoseconds which tuple versions are visible to its snapshot, and who eventually sweeps away dead tuples like T_2's aborted `(id=3, bal=999)` without blocking readers? That is the story of Multi-Version Concurrency Control, snapshots, and `VACUUM` in Chapter 5.

4.6 Exercises

Exercise 4.1 (`pg_lsn` hex arithmetic and 24-character segment filenames)**.** Consider two WAL positions $L_a =$ `2/FE800028` and $L_b =$ `3/24001A8` recorded on timeline $T = 2$ in a cluster initialized with the default 16 MB segment size (`0x01000000` bytes):

1. Rewrite both L_a and L_b as full 64-bit hexadecimal integers and compute the exact byte difference `pg_wal_lsn_diff`(L_b, L_a) in both hexadecimal and decimal. How many 16 MB segment boundaries does the interval $[L_a, L_b]$ cross?
2. Using Definition 4.1, determine the exact 24-character filename inside `$PGDATA/pg_wal/` and the exact decimal byte offset inside that file for both L_a and L_b.
3. Repeat part (2) assuming the cluster was created with `initdb --wal-segsize=64` (64 MB segments = `0x04000000` bytes, so each 4 GB epoch holds 64 = `0x40` segments).

Exercise 4.2 (Hole-stripped FPI byte budget and checkpoint WAL amplification)**.** An 8,192-byte heap page holds 20 live tuples of 120 bytes each (padded) and zero dead slots. Recall that `PageHeaderData` is 24 bytes and each `ItemIdData` line pointer is 4 bytes.

1. Compute `pd_lower`, `pd_upper`, the byte length of the unused free-space hole, and the exact uncompressed byte size of the hole-stripped Full Page Image (plus 28 bytes of `XLogRecord` + image headers).
2. Suppose an application executes 50 consecutive single-row `HOT_UPDATE` transactions against rows on this page (assume for steady-state accounting that in-page pruning keeps live data around 2,400 bytes, and each non-FPI `HOT_UPDATE` + `COMMIT` pair writes $72 + 40 = 112$ bytes of WAL). Compare total WAL bytes emitted across those 50 transactions when (a) zero checkpoints start during the burst (only edit #1 writes an FPI) versus (b) an undersized `max_wal_size` triggers a new checkpoint every 10 updates. By what factor did WAL volume grow, and how much does `wal_compression = lz4` at a 3: 1 FPI compression ratio save in case (b)?

Exercise 4.3 (Proving both failure cases of delta replay without `full_page_writes`)**.** Suppose an administrator sets `full_page_writes = off` on an `ext4` filesystem backed by a standard 4Kn NVMe SSD. Heap Block 4 initially sits cleanly on disk at `pd_lsn` = `0/5000` with `pd_upper` = 8000.

1. Construct a concrete two-record WAL sequence (`0/5060` performing in-page tuple compaction `PageRepairFragmentation`, followed by `0/50C0` inserting a 64-byte tuple) and prove step-by-step why crash recovery produces a corrupted page *both* when Sector 0 (`0..4095`) hits NAND without Sector 1 (`4096..8191`) *and* when Sector 1 hits NAND without Sector 0.
2. Suppose data checksums (`initdb --data-checksums`) are enabled. Does `pd_checksum` allow PostgreSQL to *recover* Block 4 in either case without an FPI, or only to halt with an error? Explain why.

Exercise 4.4 (Violating the WAL Invariant)**.** Suppose you comment out the `XLogFlush(lsn)` check inside `FlushBuffer()` in `bufmgr.c`. Transaction T_9 (`xid = 880`, uncommitted) splits B-tree leaf Block 10 into Block 10 and newly allocated Block 25 at WAL LSN `0/7400` while `LogwrtResult.Flush` sits at `0/7100`. `bgwriter` writes modified Block 10 (but not Block 25) to the OS, the OS writes both 4 KB sectors of Block 10 cleanly (untorn) to NAND, and building power fails before `wal_buffers` flushes past `0/7100`.

1. Trace what `StartupXLOG()` does on reboot. What is the highest LSN replayed?
2. Even though Block 10 did *not* tear and T_9 never committed, prove that read queries scanning the B-tree for rows committed *hours before the crash* can now return wrong answers or crash with missing-block errors. Why didn't MVCC visibility save you here?

Exercise 4.5 (Hand-tracing a multi-page crash recovery table)**.** On startup after a power outage, `global/pg_control` points to an online checkpoint record at `0/9200` whose `redo` pointer is `0/9000` and `nextXid` was `500`. On disk right after the crash:

- Heap Block 3 is intact with `pd_lsn` = `0/8F00` holding `(k='a', v=10)`.
- Heap Block 8 is **torn**: Sector 0 has `pd_lsn` = `0/9180`, while Sector 1 holds stale bytes from `0/8D00`.

The WAL stream starting at `0/9000` contains the following eight records:

1. `0/9060` (`xid=501`): `HEAP INSERT` + **FPI** on Block 8 (restores `(k='b', v=20)`).
2. `0/90C0` (`xid=502`): `HEAP INSERT` + **FPI** on Block 3 (adds `(k='c', v=30)`).
3. `0/9120` (`xid=501`): `HEAP HOT_UPDATE` delta on Block 8 (`k='b'` updated to `v=25`).
4. `0/9160` (`xid=502`): `XACT_ABORT`.
5. `0/9180` (`xid=503`): `HEAP INSERT` delta on Block 8 (adds `(k='d', v=40)`).
6. `0/91C0` (`xid=501`): `XACT_COMMIT`.
7. `0/9200` (`xid=0`): `XLOG_CHECKPOINT_ONLINE` (`redo = 0/9000`).
8. `0/9250` (`xid=503`): `XACT_COMMIT` — **truncated mid-record (`xl_crc` mismatch)**.

Build a step-by-step trace table like Table 4.2 listing the `XLogRedoAction` for every record, the final `pd_lsn` of Blocks 3 and 8, the two-bit `pg_xact` entries for XIDs `501`, `502`, and `503`, and the exact `(k, v)` rows returned by `SELECT * FROM t ORDER BY k` once recovery completes. Did client `503` ever receive a `COMMIT` acknowledgment before the crash (assuming `synchronous_commit = on`)? Prove your answer.

Exercise 4.6 (Group commit throughput and `synchronous_commit` failure windows)**.** A PostgreSQL primary runs on a cloud block volume whose `fdatasync()` round-trip latency is $L_{\text{sync}} = 1.25$ ms. Preparing and copying a transaction's WAL records into `wal_buffers` takes 25 μs of CPU time, parallelized across 8 `WALInsertLocks`.

1. With a single sequential client thread ($C = 1$) and `synchronous_commit = on`, what is the maximum achievable `COMMIT/s`?
2. With $C = 64$ concurrent client connections continuously executing short one-row transactions under `synchronous_commit = on`, compute (a) how many `fdatasync()` calls per second `WALWriteLock` issues, (b) the average batch size swept per leader `XLogWrite()`, and (c) the new commit throughput ceiling (compare serialization on `fdatasync` against the 8-slot WAL insertion ceiling).
3. Suppose a synchronous standby lies 2.0 ms away (round-trip). For each of the five `synchronous_commit` settings in Table 4.1, state the client-perceived commit latency and whether acknowledged commits can be lost if (i) the primary kernel panics, (ii) building power

drops on the primary only, or (iii) a storage rack fire permanently destroys the primary's SSDs.

Exercise 4.7 (Sizing `max_wal_size` and checkpointer I/O bandwidth)**.** A high-throughput OLTP cluster has `shared_buffers = 32GB`, generates WAL at a steady average rate of $R_{\text{wal}} = 45$ MB/s, dirties 14.4 GB of distinct 8 KB buffers during each checkpoint window, and configures `checkpoint_timeout = 20min` with `checkpoint_completion_target = 0.9`.

1. Calculate the WAL volume generated during one 20-minute interval and use the (1 + `checkpoint_completion_target`) retention law to compute the minimum `max_wal_size` (rounded up to whole gigabytes, plus a 20% burst headroom) required so checkpoints are triggered cleanly by time rather than WAL-space pressure.
2. Compute the average write bandwidth (in MB/s) and 8 KB write rate (in IOPS) that `checkpointer` pushes to the OS over the spread window ($0.9 \times 1{,}200$ s).
3. Suppose power fails one second *before* Checkpoint $k + 1$ writes its `XLOG_CHECKPOINT_ONLINE` record. What is the exact worst-case gigabyte volume of WAL that `StartupXLOG()` must scan on reboot, and how long will recovery take if your NVMe drive + single-process Redo pipeline replays WAL at 600 MB/s?

Chapter 5

MVCC, Snapshots, `VACUUM`, and Transaction ID Wraparound

In Chapter 4, Write-Ahead Logging solved half of the database problem: if the power cord is yanked mid-transaction, the redo log and `pd_lsn` page headers guarantee that committed bytes survive and torn 8 KB pages heal cleanly on restart. Now imagine the server stays up, the disks are healthy, and five hundred clients connect at once.

Picture a payment platform at 4:00 PM. Four hundred checkout workers are executing tiny transfers—subtracting $50 from one row of `accounts` and adding $50 to another—while a treasury analyst runs an audit query across all ten million accounts:

```
SELECT SUM(balance_minor) FROM accounts;
```

What should happen when a transfer moves 5,000 cents from Account A (stored on heap block 12) to Account Z (stored on heap block 98,400) while the sequential scan is halfway through block 40,000? If the scan reads Account A *before* the debit and reaches block 98,400 *after* the credit commits, the audit counts those 5,000 cents twice. Reverse the blocks and 5,000 cents vanish from the company books, even though every individual transaction obeyed double-entry accounting.

Early relational engines prevented that miscount with two-phase read/write locks: the `SELECT` acquired shared read locks on every page it touched, blocking writers until the whole ten-million-row scan finished, while exclusive write locks blocked the scan right back. Throughput collapsed the moment analytics and OLTP shared a table.

PostgreSQL takes a different route: *Multi-Version Concurrency Control* (*MVCC*). Under MVCC, **readers never block writers, and writers never block readers**. Instead of overwriting a row in place while someone else might still be reading it, every modification creates a new physical version of the tuple alongside the old one, stamped with transaction identifiers that tell each reader which version belongs to its point in time.

If you have used Java's `java.util.concurrent.CopyOnWriteArrayList`, you already know the shape of this idea. A reader that called `iterator()` keeps walking the array it started with, even while a writer copies the array, modifies the copy, and swaps the list's internal reference to point at the new one. Nobody takes a lock to read. The old array is freed later by the garbage collector, once no iterator references it. MVCC does the same thing one row at a time: an `UPDATE` writes a new version instead of changing the old one, each query keeps reading the versions that existed when it

started, and `VACUUM` plays the garbage collector, reclaiming old versions once no running transaction can see them.

Keeping multiple tuple versions directly inside 8 KB heap pages makes `ROLLBACK` instantaneous and keeps snapshot reads fast. It also leaves obsolete tuple versions behind on disk, requires background `VACUUM` workers to reclaim dead space, and ties every table's survival to 32-bit modular arithmetic. This chapter builds all five pieces from the C structs upward: in-heap tuple versions, commit-log hint bits, the snapshot visibility rule, `HOT` pruning with `VACUUM`, and transaction ID wraparound.

5.1 Two Ways to Build a Time Machine: Undo Logs vs. In-Heap Versions

Once you decide that an `UPDATE` must not destroy the bytes an older reader still needs, there are only two places in the storage engine where the multiple versions of a row can live. Every major relational database picks one of these two designs, and nearly every operational quirk of Oracle, MySQL/InnoDB, and PostgreSQL flows from that single choice.

Design A: Overwrite in place and push old bytes to an undo log (Oracle, InnoDB). In InnoDB and Oracle, the main table page (the clustered B-tree leaf in InnoDB, or the data block in Oracle) holds *only the newest version* of each row. When transaction T_2 updates a balance from `10000` to `5000`, the engine writes `5000` directly into the main table page, and copies the displaced delta ("balance used to be `10000`") into a separate storage area called a *rollback segment* or *undo log*, linking the row header backward to that undo record.

Look at what Design A optimizes and what it penalizes:

- **Fast in-place updates for hot columns:** If an `UPDATE` does not change a secondary index column, secondary indexes do not need new entries because the row's primary key or physical slot did not move.
- **Expensive `ROLLBACK`:** If a bulk `UPDATE` modifies five million rows and then aborts at the last millisecond, the engine must walk five million undo records and copy the old bytes back into the table pages before the rollback completes.
- **Undo-chain reconstruction for analytical readers:** A reader running against a five-minute-old snapshot cannot read heap pages at face value; whenever it hits a recently modified row, it must chase a linked list of undo deltas backward until it reconstructs a version old enough for its snapshot. If the undo log fills up and recycles space first, Oracle raises its famous `ORA-01555: snapshot too old` error.

Design B: Append new tuple versions directly into the heap (PostgreSQL). Michael Stonebraker originally designed Postgres around a bold archival vision: never overwrite data at all, so queries could ask for a table "as of last Tuesday" without a separate undo store. Modern PostgreSQL dropped automatic infinite history in favor of garbage collection, yet kept the physical storage model. When transaction T_2 updates a row in PostgreSQL, the old tuple stays right where it sits on its 8 KB heap page. PostgreSQL stamps the old tuple's header with an expiration transaction

ID, allocates a fresh slot in an 8 KB heap page (the same page if space permits, or another page with free space), writes the complete new tuple there, and links the old tuple forward to the new one.

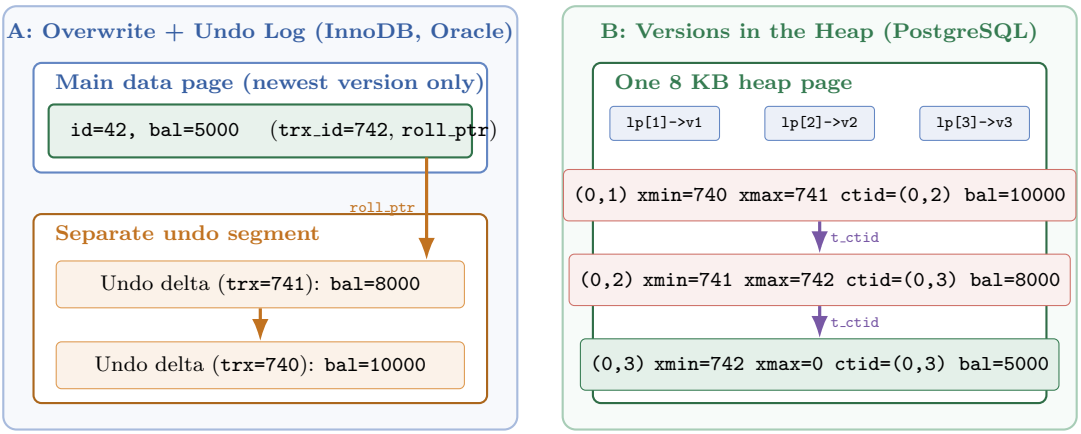


Figure 5.1: Two physical architectures for MVCC. Left: InnoDB and Oracle overwrite the main page with the newest value (`bal=5000`) and push historical deltas into a separate undo segment; older readers reconstruct history by chasing `roll_ptr` backward, and `ROLLBACK` must replay undo deltas onto data pages. Right: PostgreSQL leaves older versions untouched inside the 8 KB heap page, stamps `xmax`, appends a complete new tuple, and chains versions forward via `t_ctid`.

Intuition

In PostgreSQL, an `UPDATE` is physically a `DELETE` (stamping an expiration transaction ID onto the existing tuple) paired with an `INSERT` (appending a brand-new tuple version). Because the old bytes were never overwritten, a `ROLLBACK` of a ten-million-row update takes less than a millisecond: PostgreSQL flips two status bits in the commit log to `ABORTED` and walks away. Future readers see the aborted transaction ID on the new tuples and ignore them on sight.

Every strength and every headache of PostgreSQL administration comes back to Figure 5.1. Rollbacks are $O(1)$, historical reads inspect heap pages directly without reconstructing undo chains, and transactions never fail with "undo segment full." The price you pay is *dead tuples*: once every active transaction has moved past `742`, tuples `(0,1)` and `(0,2)` sit inside the 8 KB heap page consuming RAM and disk until pruning or `VACUUM` reclaims their slots.

5.2 Inside `HeapTupleHeaderData`: `xmin`, `xmax`, `cid`, and `ctid`

In Chapter 2 we saw that every heap tuple begins with a 23-byte fixed header (`HeapTupleHeaderData` in `src/include/access/htup_details.h`), padded to 24 bytes before user columns begin. Now we can open up those 23 bytes and read every field's concurrency job.

```
/* Simplified from src/include/access/htup_details.h */
typedef struct HeapTupleFields {
    TransactionId t_xmin;        /* 4 bytes: inserting XID */
    TransactionId t_xmax;        /* 4 bytes: deleting / locking XID, or 0 */
    union {
        CommandId t_cid;         /* 4 bytes: command ID within transaction */
```

```
        TransactionId t_xvac;    /* legacy VACUUM FULL xid */
    } t_field3;
} HeapTupleFields;               /* 12 bytes */

typedef struct HeapTupleHeaderData {
    union {
        HeapTupleFields t_heap;  /* 12 bytes: MVCC transaction fields */
        DatumTupleFields t_datum;/* used only for in-memory composite datums */
    } t_choice;
    ItemPointerData t_ctid;      /* 6 bytes: (block_hi, block_lo, offset) */
    uint16 t_infomask2;          /* 2 bytes: natts + HOT / keys-updated flags */
    uint16 t_infomask;           /* 2 bytes: hint bits, null-bitmap, lock flags */
    uint8  t_hoff;               /* 1 byte: offset to user data */
    /* optional null bitmap follows, then aligned user columns */
} HeapTupleHeaderData;           /* 23 bytes fixed */
```

Let us trace each of the four MVCC fields, because PostgreSQL packs a lot of state into these 20 bytes.

5.2.1 `t_xmin` and Lazy Transaction ID Assignment

Every writing transaction receives an increasing 32-bit XID (`TransactionId`), such as `740`. When transaction `740` runs an `INSERT`, it stamps `t_xmin = 740` and `t_xmax = 0` (signaling "no deleter yet") into the new tuple's header.

Notice the qualifier: *every transaction that writes*. What if your application runs five thousand read-only `SELECT` queries per second? Burning a 32-bit `xid` for every read query would exhaust four billion IDs in days. PostgreSQL therefore assigns a real 32-bit `TransactionId` *lazily*: when `BEGIN` executes, the backend receives only a lightweight, backend-local virtual XID (combining its backend slot number and a local counter). Only at the exact moment the transaction executes its first `INSERT`, `UPDATE`, `DELETE`, `SELECT FOR UPDATE`, or `pg_current_xact_id()` does PostgreSQL allocate a permanent 32-bit `xid` from shared memory.

5.2.2 `t_xmax`: Deletion, Update Chains, and Row Locks

When a tuple is freshly inserted and untouched, `t_xmax = 0`. Three different events write a non-zero number into `t_xmax`:

1. **A `DELETE`:** Transaction `741` deletes the row by setting `t_xmax = 741` on the existing tuple. Its `t_ctid` continues pointing to its own slot, say `(0,1)`, signaling "no newer version follows me; the row ends here."

2. **An `UPDATE`:** Transaction `741` updates the row by setting `t_xmax = 741` on old tuple `(0,1)`, inserting a brand-new tuple at slot `(0,2)` with `t_xmin = 741, t_xmax = 0`, and rewriting the old tuple's `t_ctid` from `(0,1)` to `(0,2)`.

3. **A row-level lock (`SELECT ... FOR SHARE / FOR UPDATE`):** Suppose transaction `741` locks the row without modifying it. Allocating a separate shared-memory hash table entry for every locked row in a million-row batch would blow up RAM. Instead, PostgreSQL stores the locker's `xid` directly in `t_xmax = 741` and sets the `HEAP_XMAX_LOCK_ONLY` bit (`0x0080`) in `t_infomask`.

Readers see `HEAP_XMAX_LOCK_ONLY` and immediately know the tuple was *not* deleted. Even better, if transactions `741` and `743` both acquire a shared lock (`FOR KEY SHARE`) on the same row simultaneously, PostgreSQL allocates a single *MultiXactId* representing the set $\{741, 743\}$, stores that ID in `t_xmax`, and sets `HEAP_XMAX_IS_MULTI` (`0x1000`) in `t_infomask`.

5.2.3 `t_cid` and the Halloween Problem

Why does the header spend 4 precious bytes on `t_cid` (Command ID)? Imagine a single transaction runs two SQL statements in sequence:

```
BEGIN;
-- Command 0: give every underpaid account a 1,000-cent bonus
UPDATE accounts SET balance_minor = balance_minor + 1000
 WHERE balance_minor < 50000;

-- Command 1: inspect our own uncommitted change
SELECT balance_minor FROM accounts WHERE id = 42;
COMMIT;
```

Both commands run under the *exact same* `xid`, say `740`. Look at the two competing requirements inside transaction `740`:

- In Command 1 (`SELECT`), transaction `740` *must* see the new tuple created by Command 0, and *must not* see the old tuple retired by Command 0.
- During Command 0 (`UPDATE`) itself, as the sequential scan walks from block 0 to block 500, a newly inserted tuple with `balance_minor + 1000` might land on block 420 ahead of the scan cursor. If Command 0 treated its own newly inserted tuples as visible input rows, it would match that row a second time, add *another* 1,000 cents, append yet another tuple further ahead, and keep going until every account reached 50,000 cents. Database engineers call this the **Halloween Problem**. IBM's System R team found it on Halloween 1976, when a query meant to give a 10% raise to every employee earning under $25,000 kept re-reading the rows it had just raised and stopped only when every salary had passed $25,000.

PostgreSQL solves the Halloween Problem with a 32-bit `CommandId` counter inside each transaction that starts at `0` and increments before each SQL command that can modify rows. Command 0 stamps `cmin = 0` on every tuple it inserts. While Command 0 is executing, its snapshot requires `cmin < 0` for tuples written by its own `xid`, so any tuple inserted during Command 0 is completely invisible to Command 0's own scan. As soon as Command 0 finishes, the counter advances to `1`, so Command 1 checks `cmin < 1` and sees Command 0's updates cleanly. (When the same transaction both inserts and later deletes a tuple, PostgreSQL combines `cmin` and `cmax` into a backend-private lookup index called a *combo CID* so both fit inside the single 4-byte `t_cid` slot.)

Try it in PostgreSQL

With PostgreSQL's `pageinspect` extension you can inspect the live 8 KB page and check every claim in this section yourself. Run this sequence on a scratch database:

```
CREATE EXTENSION IF NOT EXISTS pageinspect;
CREATE TABLE demo_mvcc (id INT PRIMARY KEY, bal BIGINT);

```

```
INSERT INTO demo_mvcc VALUES (1, 10000);
UPDATE demo_mvcc SET bal = 8000 WHERE id = 1;

BEGIN;
UPDATE demo_mvcc SET bal = 5000 WHERE id = 1;
ROLLBACK;

SELECT lp, t_xmin, t_xmax, t_ctid,
       t_infomask::bit(16) AS infomask
  FROM heap_page_items(get_raw_page('demo_mvcc', 0));
```

You will see *three* physical tuples on page 0 (the XID numbers on your machine will differ; we use 740, 741, 742):

- `lp = 1`: `t_xmin = 740`, `t_xmax = 741`, `t_ctid = (0,2)` (retired by the committed `UPDATE`).
- `lp = 2`: `t_xmin = 741`, `t_xmax = 742`, `t_ctid = (0,3)` (still the live row. Its `t_xmax = 742` belongs to an *aborted* transaction, so readers ignore `t_xmax` and keep `bal = 8000`).
- `lp = 3`: `t_xmin = 742`, `t_xmax = 0`, `t_ctid = (0,3)` (dead on arrival because transaction `742` rolled back).

5.3 Commit Status (`pg_xact`) and Why a `SELECT` Can Dirty Disk Pages

Pause on `lp = 2` in the lab above: its header says `t_xmin = 741` and `t_xmax = 742`. Nowhere inside those 23 bytes did transaction `742` come back during `ROLLBACK` to erase `t_xmax = 742`; doing so across a million touched pages would make `ROLLBACK` slow again. So how does a subsequent `SELECT` discover that `741` committed whereas `742` aborted?

5.3.1 The Commit Log (`pg_xact`): Two Bits per Transaction

PostgreSQL maintains a dedicated commit-status array on disk under `$PGDATA/pg_xact/` (called `pg_clog` prior to PostgreSQL 10), cached in shared memory via a Simple LRU (*SLRU*) buffer pool. Because every transaction ends in one of four states, `pg_xact` needs only **2 bits per `TransactionId`**:

Bits	C constant	Meaning
00	`STATUS_IN_PROGRESS`	Started (or unassigned); neither committed nor aborted
01	`STATUS_COMMITTED`	Committed (either synchronously or asynchronously)
10	`STATUS_ABORTED`	Rolled back explicitly or crashed before commit
11	`STATUS_SUB_COMMITTED`	Subtransaction (`SAVEPOINT`) committed; check parent XID

Check the arithmetic of that packing. Four transactions fit into 1 byte ($4 \times 2 = 8$ bits). A single

8,192-byte SLRU page of `pg_xact` therefore stores the commit status of

$$8{,}192 \text{ bytes} \times 4\frac{\text{transactions}}{\text{byte}} = 32{,}768 \text{ transactions.}$$

Even ten million transactions consume barely 2.5 MB of `pg_xact` cache. Better still, locating the exact byte and bit-pair for `xid` requires zero tree lookups—just integer division by 32,768 for the page number, modulo 32,768 divided by 4 for the byte offset, and modulo 4 times 2 for the bit shift. When a transaction commits or rolls back, PostgreSQL writes its WAL commit/abort record and flips those 2 bits in `pg_xact`.

5.3.2 Tuple Hint Bits: Caching Commit Status on the Heap Tuple

If `pg_xact` is tiny and $O(1)$, why not look up `t_xmin` and `t_xmax` in `pg_xact` every single time a query inspects a tuple?

Because `pg_xact` lives in shared memory shared across dozens of CPU cores. Suppose sixty-four CPU cores run analytical sequential scans over 100 million tuples per second. Even if every `pg_xact` page sits in RAM, sixty-four cores hammering shared SLRU locks and cache lines two hundred million times per second would stall the CPU interconnect.

PostgreSQL avoids repeated lookups by reserving four **hint bits** inside every heap tuple's 16-bit `t_infomask` field:

- `HEAP_XMIN_COMMITTED` (`0x0100`): `t_xmin` is known to have committed.
- `HEAP_XMIN_INVALID` (`0x0200`): `t_xmin` is known to have aborted (the tuple never existed).
- `HEAP_XMAX_COMMITTED` (`0x0400`): `t_xmax` is known to have committed.
- `HEAP_XMAX_INVALID` (`0x0800`): `t_xmax` is either `0` or known to have aborted.

Watch what happens during the lifetime of a newly inserted row:

1. Transaction `740` inserts the row on page 0. At the moment of insertion, transaction `740` has *not* committed yet. So the tuple on page 0 is written with `t_xmin = 740` and neither `HEAP_XMIN_COMMITTED` nor `HEAP_XMIN_INVALID` set.
2. Transaction `740` runs `COMMIT`. PostgreSQL flushes the commit WAL record and sets `740`'s 2 bits in `pg_xact` to `01` (`COMMITTED`). It does *not* revisit page 0.
3. Ten seconds later, the very first reader (say, a plain `SELECT`) visits page 0 and sees `t_xmin = 740` with no hint bit set. That reader consults `pg_xact`, learns that `740` committed (and checks that `740`'s commit WAL LSN has already been flushed to disk), ORs `HEAP_XMIN_COMMITTED` directly into `t_infomask` inside the in-memory 8 KB buffer, and **marks the buffer dirty**.
4. Every subsequent `SELECT`—millions of them—reads `t_infomask & HEAP_XMIN_COMMITTED != 0` in a single CPU register instruction and never touches `pg_xact` at all.

Common trap

New PostgreSQL operators are often startled when a 100% read-only `SELECT COUNT(*) FROM ledger_events;` right after a bulk data load produces gigabytes of disk writes and `EXPLAIN (ANALYZE, BUFFERS)` reports `Buffers: shared hit=120000 dirtied=120000`.

Nothing is broken: your `SELECT` is the first visitor after the bulk insert committed, so it is stamping `HEAP_XMIN_COMMITTED` hint bits onto every page and dirtying those buffers. Even more subtly, if data checksums are enabled (`initdb --data-checksums`) or `wal_log_hints = on`, setting the very first hint bit on a page after a checkpoint must emit an `FPI_FOR_HINT` full-page WAL image (Chapter 4) so a torn 4 KB write cannot corrupt the page checksum. In batch ETL pipelines, running `VACUUM ANALYZE table_name;` immediately after a bulk `COPY` stamps hint bits, sets Visibility Map bits, and refreshes planner statistics in one predictable pass instead of surprising your first user query.

5.4 Snapshots and the Exact Visibility Formula

Hint bits tell us *whether* transaction `740` eventually committed, yet knowing that `740` committed is still not enough. Remember our 4:00 PM audit query from the opening page: if our audit started at 4:00:00 PM and a transfer transaction `746` committed at 4:00:01 PM, `746` is genuinely committed, yet our 4:00:00 PM audit *must not see it*.

To freeze a consistent view of the database at an instant, a query captures a **snapshot** (`SnapshotData` in `src/include/utils/snapshot.h`) by scanning shared memory's `ProcArray`—the tiny array listing the active `xid` of each currently connected backend. You can print your own snapshot at any moment with `SELECT pg_current_snapshot();`, which returns a compact string in the format

`xmin:xmax:xip_list`, for example: `740:748:742,745`.

Read those three components carefully, because their names in `SnapshotData` mean the *exact opposite* of `t_xmin`/`t_xmax` on a tuple, and mixing the two up is the classic stumbling block when reading PostgreSQL source code:

Definition 5.1 (MVCC Snapshot Triple)**.** A PostgreSQL MVCC snapshot $S = (\text{xmin}_S,\ \text{xmax}_S,\ \text{xip}_S)$ captured from `ProcArray` consists of:

1. xmin_S: the **lowest still-active** transaction ID across the system at snapshot creation (or xmax_S if none are active). Every transaction $x < \text{xmin}_S$ has already finished—it either committed before S was taken or aborted.
2. xmax_S: **one past the highest completed** (or allocated) transaction ID at snapshot creation. Every transaction $x \geq \text{xmax}_S$ had not even finished (or started) when S was taken, so its effects are unconditionally *invisible* to S.
3. xip_S (*transactions in progress*): the sorted list of transaction IDs in the half-open interval $[\text{xmin}_S, \text{xmax}_S)$ that were still actively running at the instant S was captured. Even if $x \in \text{xip}_S$ commits one microsecond later, x remains *invisible* to S.

Look how cleanly $(\text{xmin}_S, \text{xmax}_S, \text{xip}_S) = (740, 748, \{742, 745\})$ partitions the entire timeline of transactions (Figure 5.2):

- Any $x < 740$ (such as `738`, `739`): finished before our snapshot. Visible if committed; invisible if aborted.
- $x \in [740, 748)$:

  - $x \in \{742, 745\}$: was still running when our snapshot snapped the shutter. **Invisible**, even if it commits later.
  - $x \in \{740, 741, 743, 744, 746, 747\}$: already finished before our snapshot. Visible if committed; invisible if aborted.
- Any $x \geq 748$ (such as `748`, `749`): started or finished after our snapshot. **Invisible**, even if already committed.

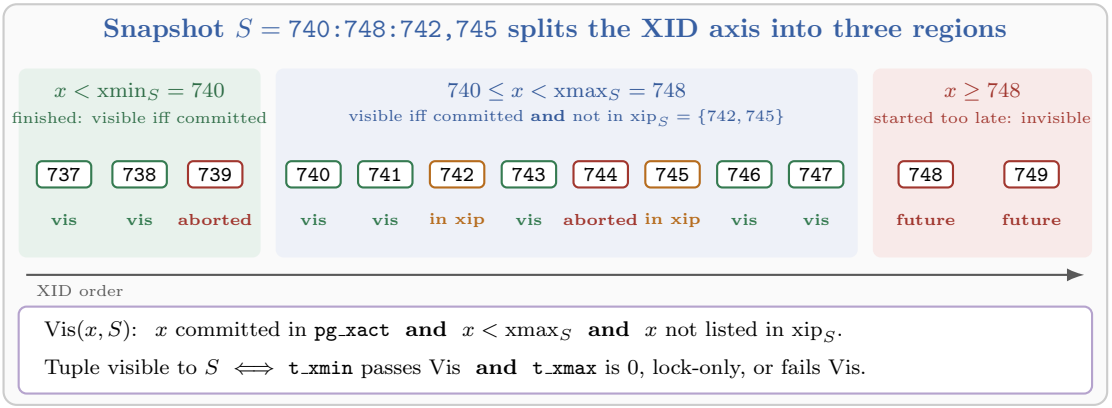


Figure 5.2: Geometry of a PostgreSQL MVCC snapshot S = `740:748:742,745`. Transactions `742` and `745` were running when S was captured (xip_S), so even if `742` commits moments later, Vis(742, S) stays false for the lifetime of snapshot S. Transaction `748` had not finished when S was taken, so it is invisible even after it commits.

Now we can state the exact visibility theorem implemented by `HeapTupleSatisfiesMVCC()` in `src/backend/access/heap/heapam_visibility.c`. Let x_{self} be the current transaction's own XID (if assigned) and c_{cur} its current command counter.

Formal

Exact MVCC Tuple Visibility Rule (`HeapTupleSatisfiesMVCC`). First define when an arbitrary transaction ID x has effects visible to snapshot $S = (\text{xmin}_S, \text{xmax}_S, \text{xip}_S)$ at command c_{cur}:

- For another transaction ($x \neq x_{\text{self}}$), define

$$\text{Vis}(x, S) \iff \big(\text{Status}(x) = \texttt{COMMITTED}\big) \wedge \big(x < \text{xmax}_S\big) \wedge \big(x \notin \text{xip}_S\big).$$

  (Note: if $x < \text{xmin}_S$, $x \notin \text{xip}_S$ holds automatically without binary-searching xip_S!)

Then a physical heap tuple T with header (`t_xmin`, `t_xmax`, c_{min}, c_{max}, `t_infomask`) is **visible** to snapshot S if and only if *both* its birth condition (**B**) and its non-death condition (**D**) hold:

1. **Birth condition (B):** Either (`t_xmin` = x_{self} and $c_{\text{min}} < c_{\text{cur}}$), or (`t_xmin` $\neq x_{\text{self}}$ and Vis(`t_xmin`, S) is true, or `HEAP_XMIN_FROZEN` is set).
2. **Non-death condition (D):** At least one of the following holds:
   (a) `t_xmax` = 0 (`InvalidTransactionId`), or `HEAP_XMAX_INVALID` is set (deleter aborted), or `HEAP_XMAX_LOCK_ONLY` is set (row is merely locked, not deleted);
   (b) `t_xmax` = x_{self} and $c_{\text{max}} \geq c_{\text{cur}}$ (our own transaction deleted it, but in the current

command or a later one, so the current command still sees the version it started with);

(c) `t_xmax` $\neq x_{\text{self}}$ and $\neg\,\text{Vis}(\texttt{t_xmax}, S)$ (the deleting transaction is still in progress, aborted, belonged to xip_S, or has `t_xmax` $\geq \text{xmax}_S$).

Example 5.1 (Evaluating Tuple Visibility by Hand)**.** Suppose an analytical read-only query captures snapshot $S =$ `740:748:742,745` (so x_{self} is unassigned). Assume every completed transaction committed except `744`, which rolled back, and suppose `742` commits *after* our snapshot was captured. Which of these five tuples on page 0 does our query emit?

Slot	t_xmin	t_xmax	Note	Birth (B)?	Alive (D)? $\Longrightarrow$ Emit?
(0,1)	735	741	both committed	Yes ($735 < 740$)	No ($\text{Vis}(741, S)$) $\Longrightarrow$ **Hide**
(0,2)	741	742	742 commits later	Yes ($741 \notin \text{xip}_S$)	Yes ($742 \in \text{xip}_S$) $\Longrightarrow$ **Emit.**
(0,3)	742	0	742 commits later	No ($742 \in \text{xip}_S$)	— $\Longrightarrow$ **Hide**
(0,4)	743	744	744 aborted	Yes ($743 \notin \text{xip}_S$)	Yes (744 aborted) $\Longrightarrow$ **Emit.**
(0,5)	746	749	749 lock-only	Yes ($746 < 748$)	Yes ($749 \geq 748$) $\Longrightarrow$ **Emit.**

Look closely at `(0,1)`, `(0,2)`, and `(0,3)`—they are three versions of a *single* logical row updated first by `741` and then by `742`. Across that three-version chain, `(0,1)` died before our snapshot, `(0,3)` was born in a transaction (`742`) that was still in flight when our snapshot was taken, and **exactly one version**—`(0,2)`—passes both (B) and (D). That exact-one-version invariant is why our 4:00 PM audit query can scan ten million rows while payments fly past and still compute a balance sheet accurate to the cent.

Walking question

When is a snapshot taken? That single timing choice is the sole difference between PostgreSQL's first two transaction isolation levels (Chapter 6):

- Under `READ COMMITTED` (PostgreSQL's default), PostgreSQL takes a **fresh snapshot at the start of every individual SQL statement** inside the transaction.
- Under `REPEATABLE READ` (and `SERIALIZABLE`), PostgreSQL takes **one snapshot at the start of the first non-control SQL statement** in the transaction and reuses that identical ($\text{xmin}_S, \text{xmax}_S, \text{xip}_S$) triple until `COMMIT` or `ROLLBACK`.

5.5 Write Amplification and HOT (Heap-Only Tuple) Updates

Now flip from the reader's perspective to the B-tree indexes we studied in Chapter 3, and watch a painful consequence of Design B emerge.

Suppose your `accounts` table has four B-tree indexes:

1. `accounts_pkey` on `(id)`,
2. `idx_accounts_owner` on `(owner_id)`,
3. `idx_accounts_currency` on `(currency)`,

4. `idx_accounts_updated` on `(updated_at)`.

Remember from Chapter 3 that PostgreSQL secondary indexes do *not* point to primary keys (unlike InnoDB); every PostgreSQL B-tree leaf entry points directly to a physical heap `ctid = (block, offset)`. Now run a tiny balance update that also bumps `updated_at`:

```
UPDATE accounts
   SET balance_minor = balance_minor - 5000,
       updated_at    = now()
 WHERE id = 42;
```

Old tuple `(0,1)` stays where it is; new tuple `(0,2)` lands on the heap. Because `updated_at` is indexed, `idx_accounts_updated` clearly needs a new entry pointing to `(0,2)`. What about the other three indexes—`id`, `owner_id`, and `currency`—whose column values *did not change at all*? In naive in-heap MVCC, because those three indexes currently point only to `(0,1)`, any future index scan looking up `owner_id = 99` under a new snapshot would land on `(0,1)`, see that `(0,1)` is dead, and miss the row completely unless **every single index on the table** also gets a brand-new index tuple pointing to `(0,2)`.

Updating one 8-byte integer just forced PostgreSQL to dirty one heap page *plus four separate B-tree leaf pages*, emit five WAL records, and double index bloat across the board. On write-heavy financial or telemetry tables with six or eight indexes, that write amplification used to cripple PostgreSQL prior to version 8.3.

5.5.1 How `HOT` Decouples B-Trees from Unindexed Updates

PostgreSQL 8.3 introduced **Heap-Only Tuples** (**HOT**) to remove this cost for the common case. Suppose we drop `idx_accounts_updated` (or update *only* `balance_minor`, which is not part of any index), and suppose the 8 KB heap page holding `(0,1)` still has enough free space to fit the new tuple `(0,2)`.

Look at what PostgreSQL already has inside that single 8 KB page: a forward pointer `t_ctid = (0,2)` from `(0,1)` to `(0,2)`. Following `t_ctid` across *different* heap blocks during an index scan would cost extra random disk reads and buffer pins, which is too slow and creates subtle vacuum hazards. Following `t_ctid` **inside the very same 8 KB page that the backend already has pinned in** `shared_buffers`, however, costs almost zero CPU nanoseconds and zero extra disk I/O.

Therefore, PostgreSQL performs a **HOT update** whenever *both* conditions hold:

1. **Zero indexed columns are modified** (checked against a bitmap of columns referenced by any index, including partial index `WHERE` predicates and expression indexes); and
2. **The new tuple fits on the exact same 8 KB heap page** as the old tuple.

When a HOT update fires:

- **Zero B-tree indexes are touched.** Every index on `accounts` keeps its single existing leaf entry pointing to the root line pointer `(0,1)`.
- Old tuple `(0,1)` gets `HEAP_HOT_UPDATED` (`0x4000`) set in `t_infomask2`.
- New tuple `(0,2)` gets `HEAP_ONLY_TUPLE` (`0x8000`) set in `t_infomask2` (signaling that *no* index anywhere points directly to `(0,2)`).

- When an index scan visits `(0,1)`, it sees `HEAP_HOT_UPDATED` and walks the intra-page `t_ctid` chain `(0,1)` → `(0,2)` → `(0,3)` right inside the pinned buffer until it finds the version matching its snapshot.

5.5.2 Single-Page Pruning (On-the-Fly Defragmentation Without `VACUUM`)

HOT gives you a second major advantage that is just as important as skipping index writes: **single-page HOT pruning**.

Ask yourself why normal `VACUUM` has to scan every index on a 500 GB table before it dares free a dead heap slot `(0,1)`: if any B-tree still holds a pointer to `(0,1)` and you reuse slot 1 for an unrelated customer's row, an index lookup for Customer A will suddenly return Customer B's bank account.

Now look at an intermediate tuple `(0,2)` in a HOT chain `(0,1)` → `(0,2)` → `(0,3)` once `(0,1)` and `(0,2)` are both older than every active snapshot in the system (`OldestXmin`):

- *No index anywhere* points to `(0,2)` or `(0,3)` (both are marked `HEAP_ONLY_TUPLE`).
- Every index points *only* to root line pointer `ItemId[1]`.

Consequently, the very next backend that touches this single 8 KB page—even an ordinary `SELECT` or the next `UPDATE`, when free space runs low—can clean up the entire chain **locally inside this one page, in microseconds, without touching a single index**:

1. It removes the dead tuple bytes of `v1` and `v2`, sliding live tuple `v3` upward toward the top of the page (`pd_upper`) to compact free space.
2. It converts root line pointer `ItemId[1]` from `LP_NORMAL` into a 4-byte `LP_REDIRECT` pointing directly to slot 3.
3. It marks `ItemId[2]` as `LP_UNUSED`, making slot 2 immediately reusable for the very next `UPDATE`.

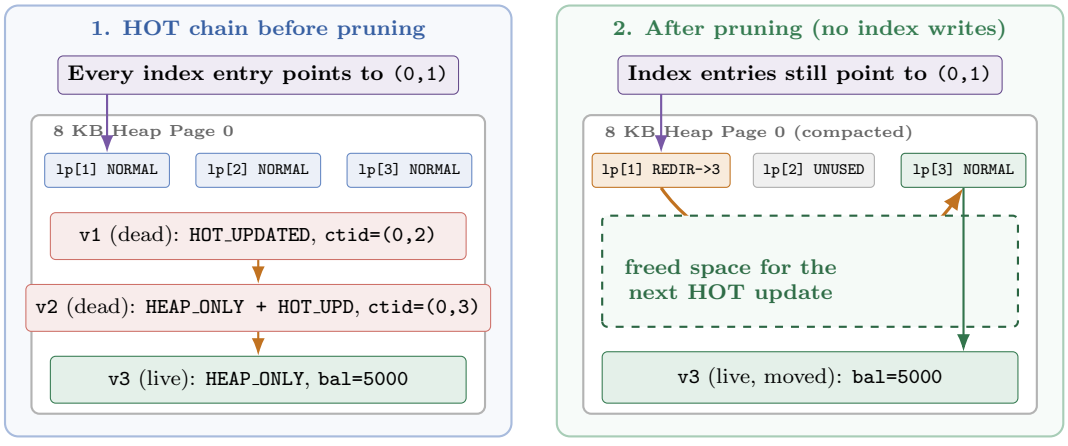


Figure 5.3: Single-page HOT pruning inside an 8 KB heap page. Left: Two HOT updates left dead tuples `v1` and `v2` inside Page 0 while every B-tree index continued pointing to root slot `lp[1]`. Right: Any query visiting Page 0 after `v1` and `v2` pass the global `OldestXmin` horizon compacts the page in place, turns `lp[1]` into an `LP_REDIRECT` to `lp[3]`, and frees `lp[2]` without touching any index.

In production

On a high-throughput `accounts` table receiving 15,000 balance updates per second, two tiny DDL choices separate steady-state bliss from runaway disk bloat:

1. **Keep rapidly mutating counters out of secondary indexes** unless queries genuinely filter or sort on them. Indexing `updated_at` or `balance_minor` disables HOT on every single balance update. You can verify your live HOT ratio immediately in `pg_stat_user_tables`: compare `n_tup_hot_upd` against `n_tup_upd`. On a well-tuned account-balance table, $> 95\%$ of updates should be HOT.

2. **Leave headroom on each 8 KB page with `fillfactor`:** By default, heap tables use `fillfactor = 100`, packing newly inserted pages 100% full so the *very first* `UPDATE` has nowhere to land on the same page and spills to a distant block. Setting `ALTER TABLE accounts SET (fillfactor = 85);` reserves 15% ($\approx$ 1.2 KB) of every page during `INSERT`s so HOT chains have room to breathe and self-prune in place forever.

5.6 Table Bloat, `VACUUM`, `autovacuum`, and `VACUUM FULL`

Whenever a row is `DELETE`d, or whenever an `UPDATE` cannot use HOT (either an indexed column changed or the target 8 KB page was full), single-page pruning can free the tuple's payload bytes, yet it **must leave a 4-byte `LP_DEAD` stub in the heap's line-pointer array** and **must leave dead entries inside every B-tree index**. Only full `VACUUM` can clear both.

5.6.1 The Three Phases of Concurrent ("Lazy") `VACUUM`

Plain `VACUUM` (called "lazy vacuum" in `src/backend/access/heap/vacuumlazy.c`) acquires only a `SHARE UPDATE EXCLUSIVE` lock—it **never blocks `SELECT`, `INSERT`, `UPDATE`, or `DELETE`** traffic. Here is what it actually executes:

1. **Phase 1 — Heap Scan:** `VACUUM` computes `OldestXmin` (the oldest `xmin` among all currently active snapshots across the whole database, including replication slots and prepared transactions). Walking the table sequentially—while *skipping* any 8 KB page already marked `ALL_VISIBLE` in the **Visibility Map** (`_vm` fork)—it prunes dead tuple payloads and records the 6-byte `(block, offset)` TIDs of dead line pointers in an in-memory store limited by `autovacuum_work_mem` (or `maintenance_work_mem`). Before PostgreSQL 17 this store was a plain sorted array of TIDs, capped at 1 GB; PostgreSQL 17 replaced it with a compact radix tree that needs far less memory per TID and removed the cap.

2. **Phase 2 — Index Vacuuming:** Whenever that dead-TID store fills up (or the heap scan finishes), `VACUUM` scans every index on the table and removes every index entry whose `ctid` is in the dead-TID store. (A manual `VACUUM (PARALLEL n)` can hand different indexes to different worker processes; autovacuum processes them one at a time.) This phase is where memory settings matter: if the dead-TID store holds only a tenth of the dead TIDs on a bloated 500 GB table with 8 indexes, `VACUUM` must scan all 8 indexes *ten full times*. Giving vacuum enough memory to hold all the dead TIDs at once collapses those ten index passes into one.

3. **Phase 3 — Heap Line-Pointer Cleanup, FSM, and VM:** Having removed every index pointer to those TIDs, `VACUUM` revisits the affected heap pages, flips those `LP_DEAD` line pointers

to LP_UNUSED (now 100% reusable), records the newly freed bytes in the **Free Space Map** (_fsm fork) so future INSERTs and cross-page UPDATEs reuse interior space instead of extending the file, and sets ALL_VISIBLE (and ALL_FROZEN) bits in the Visibility Map.

5.6.2 Why VACUUM Does Not Shrink Disk Usage (vs. VACUUM FULL)

Suppose you mistakenly insert 50 million test rows (consuming 8 GB on disk), run DELETE FROM ledger_test;, run VACUUM ledger_test;, and check pg_relation_size('ledger_test')—only to see it still sitting at 8 GB if a single live row remains near the end of the file.

Why? Because VACUUM reclaims free space *inside* existing 8 KB pages and advertises them in the Free Space Map for future inserts into ledger_test; it only truncates OS file blocks from the *trailing tail* of the relation if the final pages are 100% empty. Moving a live tuple from block 999,999 down to block 0 would change its ctid, invalidating open cursors and index entries unless the entire table were locked.

VACUUM FULL, by contrast, behaves like CREATE TABLE new_copy AS SELECT * FROM old_table plus rebuilding every index from scratch into brand-new relfilenode files, and then unlinking the old files. It shrinks an 8 GB bloated table with 1,000 live rows right back down to 64 KB—at a brutal operational cost:

- VACUUM FULL holds an ACCESS EXCLUSIVE **lock** for the entire duration of the table and index rewrite, freezing *every single SELECT and write* on that table.
- Because both the old relfilenode and the newly written copy exist on disk simultaneously until commit, VACUUM FULL temporarily requires enough free disk space to hold a second copy of the table and its indexes.

In 24/7 production systems, if severe bloat ever occurs, engineers reach for online rewrite tools such as pg_repack (which builds the clean shadow table concurrently, captures concurrent writes via a trigger log, and acquires ACCESS EXCLUSIVE only for a sub-second catalog pointer swap at the very end) rather than running VACUUM FULL during business hours.

Common trap

Two production traps that silently paralyze autovacuum:

1. **A single idle in transaction session:** Suppose a developer opens psql, runs BEGIN; SELECT * FROM accounts LIMIT 1;, and goes to lunch for four hours. Under REPEATABLE READ—or if that session holds an open snapshot or uncommitted write—the global OldestXmin horizon freezes at 12:00 PM. Even if autovacuum runs every minute, it **cannot reclaim a single dead tuple created anywhere in the database after 12:00 PM**, because that idle session might theoretically read those versions. Always set idle_in_transaction_session_timeout = '60s' in postgresql.conf (and monitor abandoned logical replication slots in pg_replication_slots, which pin catalog_xmin / xmin across restarts).
2. **The 20% default scale factor on giant tables:** PostgreSQL triggers autovacuum when dead tuples exceed threshold $\tau = 50$ plus scale factor $s = 0.20$ times row count:

$$\texttt{n_dead_tup} > \tau + s \times \texttt{reltuples}.$$

With defaults $50 + 0.20 \times N$, a 200-million-row `postings` table will accumulate **40,000,050 dead tuples** before `autovacuum` even wakes up. Override scale factors per hot large table:

```
ALTER TABLE accounts SET (
    autovacuum_vacuum_scale_factor = 0.01,
    autovacuum_vacuum_cost_delay   = 2,
    autovacuum_vacuum_cost_limit   = 2000
);
```

5.7 32-Bit Transaction ID Wraparound and Tuple Freezing

The last consequence of PostgreSQL's in-heap MVCC design is the one that can take a whole cluster offline. Look back at `HeapTupleHeaderData` in Section 5.2: `t_xmin` and `t_xmax` are 32-bit unsigned integers (`uint32`). Why didn't PostgreSQL widen them to 64 bits years ago?

Because of the byte budget. Widening both `t_xmin` and `t_xmax` from 4 bytes to 8 bytes adds **8 bytes of fixed header overhead to every row in every table**. On a narrow 10-billion-row ledger-postings table where each row is 48 bytes total, 8 extra bytes per row is 80 GB more disk and cache ($\approx 17\%$). PostgreSQL does track a full 64-bit `FullTransactionId` in memory, the value returned by `pg_current_xact_id()` (a 32-bit epoch alongside the 32-bit XID), but on-disk heap tuple headers store only the low 32 bits.

With 32 bits there are $2^{32} = 4{,}294{,}967{,}296$ distinct values. PostgreSQL reserves the first three:

- `0` = `InvalidTransactionId` (used for empty `t_xmax`),
- `1` = `BootstrapTransactionId` (used during `initdb` catalog creation; always visible),
- `2` = `FrozenTransactionId` (legacy sentinel for frozen tuples; always visible),
- `3..`$(2^{32} - 1)$ = normal circular `TransactionId` space (`FirstNormalTransactionId = 3`).

Whenever the 32-bit counter increments past 4,294,967,295 (`0xFFFFFFFF`), it wraps right around to `3` (skipping `0`, `1`, `2`).

5.7.1 Modular Circle Comparison (`TransactionIdPrecedes`)

If `xid` wraps around a finite circle of size $\approx 2^{32}$, you cannot compare two transaction IDs a and b with ordinary integer inequality `a < b`. Right after wraparound from `4,294,967,295` to `3`, transaction `3` happened *after* `4,294,967,295`, even though $3 < 4{,}294{,}967{,}295$ as unsigned integers.

Instead, PostgreSQL defines ordering on the modular circle $\mathbb{Z}/2^{32}\mathbb{Z}$ using signed 32-bit modular subtraction (`src/backend/access/transam/transam.c`):

```
/* Normal XID comparison in PostgreSQL: true if id1 logically precedes id2 */
bool NormalTransactionIdPrecedes(TransactionId id1, TransactionId id2) {
    int32 diff = (int32) (id1 - id2);
    return (diff < 0);
}
```

Look at the geometry of `(int32)(id1 - id2) < 0` on the 2^{32} clock dial (Figure 5.4):

- Stand at the current transaction counter x_{now}.
- Looking counterclockwise up to $2^{31} \approx 2.147$ billion steps behind x_{now}, `(int32)(t_xmin - x_now) < 0`: all of those 2^{31} XIDs lie in the **visible Past**.
- Looking clockwise up to $2^{31} - 1 \approx 2.147$ billion steps ahead of x_{now}, `(int32)(t_xmin - x_now) > 0`: all of those 2^{31} XIDs lie in the **invisible Future**.

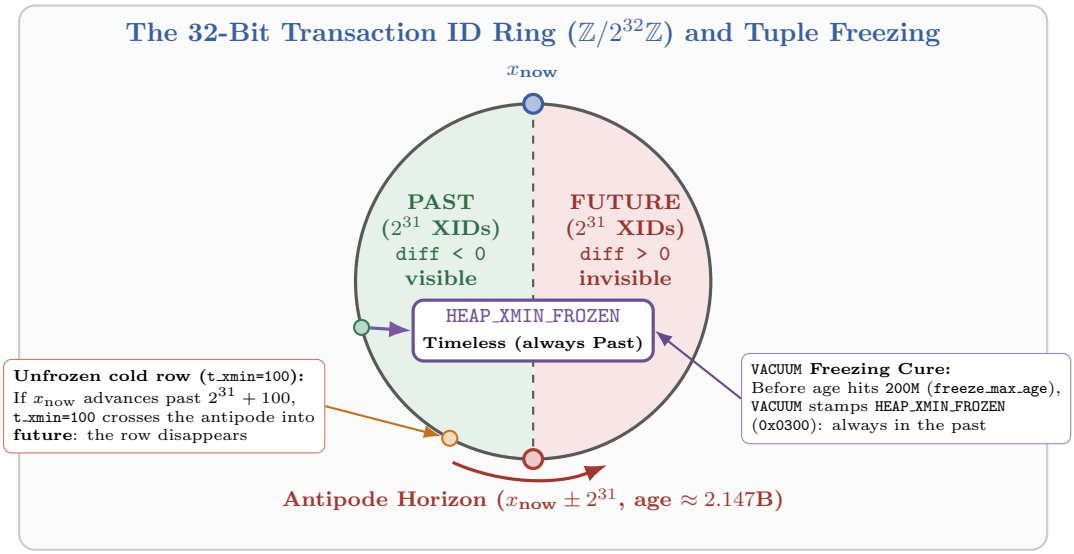


Figure 5.4: Modular 32-bit `TransactionId` arithmetic. Half the 2^{32} circle behind x_{now} is the visible Past; the half ahead is the invisible Future. As x_{now} marches clockwise, an untouched row's `t_xmin` drifts relatively toward the antipode (2^{31} transactions old). Freezing stamps `HEAP_XMIN_FROZEN` onto the tuple header, detaching the row from modular comparison forever.

Here is what would happen if a row were never frozen:

1. In January, transaction `100` inserts a founding-capital row into `journal_entries`, so the tuple has `t_xmin = 100`. It is an immutable historical record; nobody ever updates or deletes it again.
2. Over the next eight months or so, your payment service runs 100 write transactions per second, advancing x_{now} past 2,147,483,748 ($= 2^{31} + 100$).
3. Now an auditor runs `SELECT * FROM journal_entries WHERE id = 1;`. PostgreSQL compares `t_xmin = 100` against $x_{\text{now}} = 2{,}147{,}483{,}749$:

$$\texttt{(int32)(100 - 2147483749)} = +2{,}147{,}483{,}647 > 0 \quad \text{(so 100 is in the future).}$$

4. No byte on disk has changed, yet **that committed row disappears from every query**, because `t_xmin = 100` moved across the antipode from "2.1 billion transactions ago" to "2.1 billion transactions in the future."

5.7.2 The Cure: Tuple Freezing, `relfrozenxid`, and the Safety Shutdown

PostgreSQL prevents wraparound loss through **tuple freezing**. Once a committed tuple's `t_xmin` is older than `OldestXmin` and older than `vacuum_freeze_min_age` (default 50 million XIDs), `VACUUM` marks it frozen by setting *both* `xmin` hint bits at once:

`HEAP_XMIN_FROZEN = HEAP_XMIN_COMMITTED | HEAP_XMIN_INVALID = 0x0300.`

(Since PostgreSQL 16, `VACUUM` also freezes younger tuples opportunistically when it is already writing a full-page image of that page, because the extra freezing then costs almost nothing.) Before PostgreSQL 9.4, freezing overwrote `t_xmin` with `2`, which destroyed forensic history. Setting `0x0300` instead keeps the original `t_xmin` readable in `pageinspect` while telling every visibility check to treat the tuple as committed infinitely long ago.

Once every tuple on an 8 KB page is frozen, `VACUUM` also sets the page's **all-frozen bit** in the Visibility Map (`_vm` fork). Later anti-wraparound vacuums skip those pages entirely, so a 50 TB archive table whose old pages are all frozen costs almost nothing to keep safe.

PostgreSQL tracks the oldest unfrozen XID per table in `pg_class.relfrozenxid`, and the minimum across all tables in a database in `pg_database.datfrozenxid`. Three guardrails protect your cluster:

1. **Routine freeze threshold (`autovacuum_freeze_max_age = 200,000,000`, $\approx 9.3\%$ of 2^{31}):** Even if `autovacuum = off` on a table, once `age(relfrozenxid)` exceeds 200 million XIDs, PostgreSQL launches a mandatory anti-wraparound `autovacuum` worker (using the Visibility Map's all-frozen bits to skip already-frozen pages) and afterwards truncates unneeded old `pg_xact` files.
2. **Client warning zone ($\approx$ 40 million XIDs before 2^{31}):** Every command that assigns an XID starts returning `WARNING: database "prod" must be vacuumed within N transactions`.
3. **Hard safety stop (3 million XIDs before 2^{31}):** Before wraparound can hide a committed row, PostgreSQL **refuses to assign any new XIDs**, so every write in that database fails with (PostgreSQL 16 and later):

```
ERROR: database is not accepting commands that assign new
       transaction IDs to avoid wraparound data loss in
       database "prod"
HINT:  Execute a database-wide VACUUM in that database.
```

   Read-only queries keep working, and `VACUUM` itself does not need an XID, so a superuser can run it directly; `VACUUM (FREEZE, INDEX_CLEANUP OFF, TRUNCATE OFF) worst_table;` advances `relfrozenxid` fastest because it skips the index passes. (Older releases told you to restart in single-user mode instead; current documentation advises against that.)

Try it in PostgreSQL

Monitor XID wraparound health in every production database before it ever reaches 50% of the 2^{31} ceiling. Add this exact query to your operational runbook and Prometheus exporter:

```
SELECT c.oid::regclass                    AS table_name,
       age(c.relfrozenxid)                AS xid_age,
       ROUND(100.0 * age(c.relfrozenxid)
```

```
             / 2147483647.0, 2)              AS pct_toward_wraparound,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS total_size
  FROM pg_class c
  JOIN pg_namespace n ON n.oid = c.relnamespace
 WHERE c.relkind IN ('r', 'm', 't')
   AND n.nspname NOT IN ('pg_catalog', 'information_schema')
 ORDER BY age(c.relfrozenxid) DESC
 LIMIT 10;
```

Notice `'t'` in `relkind IN ('r', 'm', 't')`: TOAST tables (Chapter 2) have their *own* `relfrozenxid` independent of their parent heap table, and a bloated TOAST table stuck behind a slow vacuum has caught many experienced teams off guard.

MVCC snapshots let readers and writers fly past one another without read locks, yet a subtle question remains: if two concurrent transactions both read a consistent snapshot and both write back based on what they read, what happens when their writes collide—or worse, when they update *two different rows* whose sum was supposed to stay positive? That is the domain of isolation levels, concurrency anomalies, row locks, and Serializable Snapshot Isolation in Chapter 6.

5.8 Exercises

Exercise 5.1 (Storage design tradeoffs)**.** Suppose a batch job runs `UPDATE ledger_staging SET fee_minor = fee_minor + 10;` across 8,000,000 rows, hits a constraint violation on the 8,000,000th row, and executes `ROLLBACK`. Compare step by step what physical disk and memory operations InnoDB (in-place update + undo log) vs. PostgreSQL (in-heap tuple versions + `pg_xact`) perform during (a) the 7,999,999 successful row updates, (b) the `ROLLBACK` command itself, and (c) the next hour after the rollback.

Exercise 5.2 (Commit log byte and bit offsets)**.** Recall that `pg_xact` packs 2 status bits per `TransactionId` into 8,192-byte SLRU pages. (a) For `xid = 1,000,005`, compute the exact 0-indexed `pg_xact` page number, the byte offset within that page, and the bit-shift offset (`0`, `2`, `4`, or `6`) within that byte. (b) Suppose your system executes 5,000 write transactions per second continuously for 24 hours. How many megabytes of `pg_xact` data do those 24 hours occupy before truncation?

Exercise 5.3 (Tracing snapshot visibility by hand)**.** A transaction running at `REPEATABLE READ` has assigned XID $x_{\text{self}} = 912$, current command ID $c_{\text{cur}} = 2$, and snapshot $S =$ `905:915:907,910`. Every transaction not in $\{907, 909, 910, 912\}$ with XID < 915 committed; `909` aborted; `907` committed two seconds *after* S was taken; `910` is still running; and `916` committed five seconds after S was taken. Determine whether each of the following six heap tuples is **visible** or **invisible** to this query, citing which clause of `HeapTupleSatisfiesMVCC` decides the outcome:

1. `t_xmin = 902, t_xmax = 907` (normal delete).
2. `t_xmin = 907, t_xmax = 0`.
3. `t_xmin = 908, t_xmax = 909` (normal delete).
4. `t_xmin = 908, t_xmax = 911, infomask = HEAP_XMAX_LOCK_ONLY`.
5. `t_xmin = 912, cmin = 1, t_xmax = 0`.

6. `t_xmin = 912, cmin = 2, t_xmax = 0`.

Exercise 5.4 (The Halloween Problem)**.** Consider `CREATE TABLE nums (x INT);` populated with rows `1`, `2`, `3`, and an index on `(x)`. Suppose PostgreSQL forgot to check `cmin < curcid` during `UPDATE nums SET x = x + 1 WHERE x > 0;`. Trace how an index scan ordered by `x ASC` behaves step by step, and explain why `t_cid` stops the infinite loop without hiding the updated rows from the *next* SQL statement in the same transaction.

Exercise 5.5 (Why a read-only query dirties buffers and emits WAL)**.** Right after a checkpoint completes, a bulk loader `INSERT`s 500,000 rows into `events` across 4,000 heap pages and runs `COMMIT`. Immediately afterward, on a server with data checksums enabled, an analyst runs `SELECT COUNT(*) FROM events;`. (a) Explain why `EXPLAIN (ANALYZE, BUFFERS)` on that `SELECT` reports roughly `dirtied=4000`. (b) Explain why that `SELECT` also generates roughly $4{,}000 \times 8$ KB ≈ 32 MB of WAL records, whereas running the *exact same* `SELECT COUNT(*) FROM events;` a second time one second later reports `dirtied=0` and generates zero bytes of WAL.

Exercise 5.6 (Designing for HOT updates)**.** Suppose a `sessions` table has four columns—`id`, `user_id`, `seen_at`, and `hits`—with secondary B-tree indexes on both `(user_id)` and `(seen_at)`. Every HTTP request executes:

```
UPDATE sessions
   SET seen_at = now(), hits = hits + 1
 WHERE id = $1;
```

(a) What fraction of these updates can use HOT today, and how many index leaf insertions occur per 1,000 HTTP requests? (b) Product only uses `seen_at` once per hour to purge expired sessions older than 30 days. Redesign the indexing or update threshold strategy (plus `fillfactor`) so $> 98\%$ of HTTP requests become single-page HOT updates while keeping the hourly purge fast.

Exercise 5.7 (Autovacuum trigger math)**.** A `ledger_postings` table holds $N = 150{,}000{,}000$ rows and receives 2,000 non-HOT updates per second. (a) Under PostgreSQL's default threshold `50` and scale factor `0.20` (`autovacuum_vacuum_scale_factor`), how many hours pass between automatic vacuums, and how many dead tuples accumulate before each run? (b) Choose explicit per-table `autovacuum` scale factor (or fixed threshold with scale factor `0.0`) settings so `autovacuum` triggers every $\approx 500{,}000$ dead tuples (≈ 4 minutes).

Exercise 5.8 (Wraparound modular arithmetic)**.** Test PostgreSQL's circular XID ordering rule—checking whether signed 32-bit modular subtraction satisfies $(\text{int32})(\text{id}_1 - \text{id}_2) < 0$—on each pair of unsigned 32-bit XIDs below:

1. `id1 = 4294967290` (`0xFFFFFFFA`) and `id2 = 10` (`0x0000000A`).
2. `id1 = 100` and `id2 = 2147483747` ($= 100 + 2^{31} - 1$).
3. `id1 = 100` and `id2 = 2147483749` ($= 100 + 2^{31} + 1$). Explain what happens to an unfrozen committed row with `t_xmin = 100` between (2) and (3).

Exercise 5.9 (Emergency wraparound triage)**.** You are paged at 2:00 AM because a production database has 8 million XIDs remaining before wraparound shutdown (`age(datfrozenxid) = 2,139,000,000`) and write traffic is burning 1,500 XIDs/second ($\approx$ 88 minutes left). (a) Write the diagnostic queries against `pg_stat_activity`, `pg_prepared_xacts`, and `pg_replication_slots` to check whether a blocker is pinning `OldestXmin` and preventing `VACUUM` from advancing

`relfrozenxid`. (b) Once any blocker is cleared, why should you run `VACUUM` with `(FREEZE, INDEX_CLEANUP OFF, TRUNCATE OFF)` on the worst table rather than `VACUUM FULL` during those remaining 88 minutes?

Chapter 6

Isolation Anomalies, Row Locking, and Serializable Snapshot Isolation

In Chapter 5 we built PostgreSQL's snapshot engine: every query receives a frozen list of which transactions had already finished when its snapshot was captured, so readers and writers glide past each other without read locks. At first glance, snapshots look like the complete answer to concurrency. Every query sees a clean, self-consistent photograph of the database; nobody reads half-written bytes; rollbacks vanish cleanly.

Now try running a bank on snapshots alone.

Suppose Alice holds two sub-accounts under a single prime brokerage margin agreement: `Checking = $80` and `Savings = $80`, subject to the firm-wide rule that *her combined balance across both sub-accounts may never drop below $0.* At 10:00:00.000 AM, Alice submits two $100 withdrawals from two browser tabs at the exact same millisecond—one from Checking, one from Savings. Both backend workers start transactions at PostgreSQL's `REPEATABLE READ` isolation level:

- Worker 1 reads its snapshot: `Checking = $80, Savings = $80` (total $160). Withdrawing $100 leaves $60 $\geq$ $0, so Worker 1 updates `Checking` to −$20 and commits.

- Worker 2 reads its identical 10:00:00.000 AM snapshot: `Checking = $80, Savings = $80` (total $160). Withdrawing $100 leaves $60 $\geq$ $0, so Worker 2 updates `Savings` to −$20 and commits.

Neither worker touched a single uncommitted byte. Neither worker overwrote the row the other worker modified. Both ran under `REPEATABLE READ` without a single error or lock wait—and Alice just walked out of the bank with $200 in cash from $160 of collateral, leaving a −$40 combined balance that your code swore was impossible.

Understanding *why* that bug happens, which exact anomalies each SQL isolation level permits or forbids, how PostgreSQL's `EvalPlanQual` machinery behaves under `READ COMMITTED`, how *Serializable Snapshot Isolation* (*SSI*) catches multi-row anomalies without blocking readers, and when to reach for explicit row or advisory locks instead is one of the defining skills of a backend systems engineer. We will trace every anomaly on concrete two-transaction timelines so none of them ever catches you by surprise in production.

6.1 What "Isolation" Actually Promises (and Why ANSI SQL-92 Missed Half the Story)

The "I" in ACID stands for *Isolation*: even though hundreds of transactions execute concurrently on interleaved CPU cores, each transaction should appear to execute *as if it had the entire database to itself.* Database theory calls this property **serializability**.

Definition 6.1 (Serializable Execution)**.** A concurrent execution (schedule) of committed transactions $T_1, T_2, \ldots, T_k$ is *serializable* if its final database state and every query result returned to each T_i are identical to running those same transactions **strictly one after another with zero overlap** in some permutation $T_{\pi(1)} \to T_{\pi(2)} \to \cdots \to T_{\pi(k)}$.

Look back at Alice's two withdrawals above under Definition 6.1. There are only two serial orders of T_1 (withdraw \$100 from Checking) and T_2 (withdraw \$100 from Savings):

- T_1 **then** T_2**:** T_1 sees $\$80 + \$80 = \$160$, sets Checking to $-\$20$, and commits. Then T_2 starts, sees $-\$20 + \$80 = \$60 < \$100$, and *rejects* the second withdrawal. Final combined balance: $+\$60$.
- T_2 **then** T_1**:** Symmetrically, T_2 sets Savings to $-\$20$, and T_1 sees $\$80 + (-\$20) = \$60 < \$100$ and *rejects* the first withdrawal. Final combined balance: $+\$60$.

In *every* possible one-at-a-time execution, at most one withdrawal succeeds and the combined balance is $+\$60$. Any concurrent schedule that lets both commit and leaves $-\$40$ has violated serializability: no sequential universe could ever have produced that database state.

6.1.1 The ANSI SQL-92 Table vs. PostgreSQL Reality

When the ANSI SQL-92 committee standardized transaction isolation levels (`READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ`, and `SERIALIZABLE`), the specification had two famous flaws uncovered in 1995 by Hal Berenson, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O'Neil, and Patrick O'Neil:

1. The committee implicitly assumed every database used 1980s lock-based concurrency rather than MVCC snapshots; and
2. They defined isolation levels solely by listing *three read phenomena* (Dirty Reads, Non-Repeatable Reads, and Phantom Reads), completely omitting **Lost Updates** and **Write Skew**.

In PostgreSQL, MVCC snapshots change the table in ways every SQL developer needs to memorize:

PostgreSQL Level	Dirty	Non-Repeat.	Phantom	Lost Upd. / Write Skew
`READ UNCOMMITTED`[†]	Impossible	Possible	Possible	Possible / Possible
`READ COMMITTED` (default)	Impossible	Possible	Possible	**Possible.** / Possible
`REPEATABLE READ` (SI)	Impossible	Impossible	**Impossible.**	Impossible[‡] / **Possible.**
`SERIALIZABLE` (SSI)	Impossible	Impossible	Impossible	Impossible / Impossible

[†]PostgreSQL accepts `READ UNCOMMITTED` syntax for SQL compatibility, mapping it identically to `READ COMMITTED`.

[‡]Prevented via automatic `40001` conflict abort.

Notice two immediate surprises in that table:

- **Dirty reads never happen in PostgreSQL** at any isolation level, because our Chapter 5 visibility check rejects uncommitted `t_xmin` values unconditionally.

- **PostgreSQL's `REPEATABLE READ` already blocks phantom reads** (unlike lock-based engines that lack gap locks), yet it is *not* serializable because it implements pure *Snapshot Isolation* (*SI*), which still allows **Write Skew**.

Let us put each anomaly under the microscope on concrete timelines.

6.2 Read Anomalies: Dirty Reads, Read Skew, and Phantoms

6.2.1 Dirty Read (P_1): Reading Uncommitted Bytes

A *dirty read* occurs when transaction T_2 reads a value written by concurrent transaction T_1 *before* T_1 commits. If T_1 subsequently executes `ROLLBACK`, T_2 has acted on a balance that never existed in committed history:

```
-- T1                                    -- T2
BEGIN;                                   BEGIN;
UPDATE accounts SET bal = 99999900
 WHERE id = 1;
                                         -- Under true READ UNCOMMITTED:
                                         SELECT bal FROM accounts WHERE id = 1;
                                         -- Sees 99999900 and wires cash out!
ROLLBACK; -- T1 aborts!
```

In PostgreSQL, when T_1 runs its `UPDATE`, old tuple `(0,1)` gets `t_xmax = xid(T1)` and new tuple `(0,2)` gets `t_xmin = xid(T1)`. Because `xid(T1)` is still `IN_PROGRESS` in `pg_xact`, T_2's visibility check sees that `(0,1)`'s deleter has not committed (so `(0,1)` stays visible.) and `(0,2)`'s inserter has not committed (so `(0,2)` stays hidden.). Dirty reads are ruled out by the storage format itself.

6.2.2 Non-Repeatable Read (P_2) and Read Skew Under `READ COMMITTED`

Remember from Chapter 5 how PostgreSQL's default isolation level—`READ COMMITTED`—chooses snapshots: **it captures a new snapshot at the start of every SQL statement inside the transaction**.

Suppose Alice has `id = 1` (Checking, `5000` cents) and `id = 2` (Savings, `5000` cents). Transaction T_1 transfers `4000` cents from Checking to Savings while Transaction T_2 (a dashboard handler running at default `READ COMMITTED`) queries Alice's two accounts in two separate `SELECT` statements (Figure 6.1):

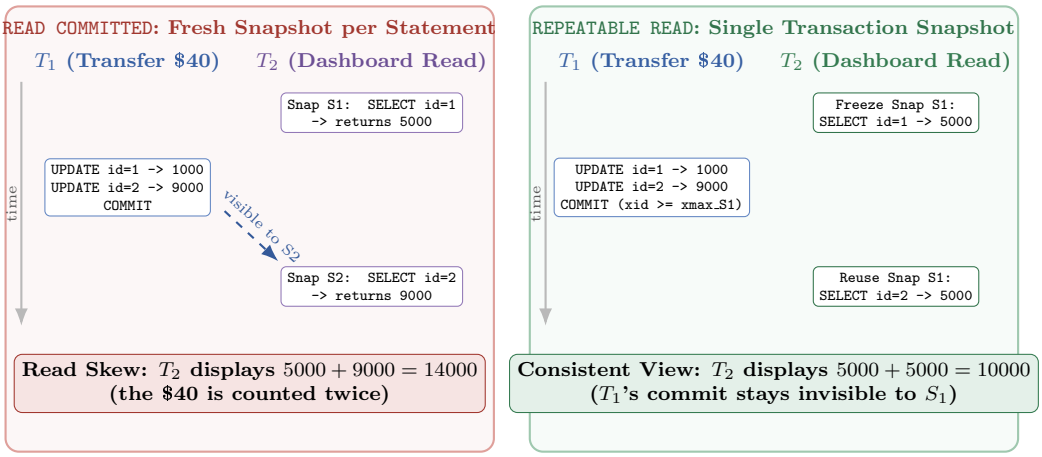


Figure 6.1: Read skew across two statements inside one transaction. Left: Under default `READ COMMITTED`, T_2's second `SELECT` takes a new snapshot S_2 after T_1 commits, seeing Checking *before* the transfer (`5000`) and Savings *after* the transfer (`9000`). Right: Under `REPEATABLE READ`, T_2 reuses S_1 for every statement until `COMMIT`, reading a consistent pre-transfer total of `10000`.

If T_2 instead re-read `id = 1` in its second statement, it would see `5000` first and `1000` second—the classic *non-repeatable read*. Reading two related rows across two statements and seeing an inconsistent cut between them is its multi-row twin, **read skew**.

6.2.3 Phantom Reads (P_3)

A *phantom read* is the set-level version of a non-repeatable read: instead of modifying an existing row, concurrent transaction T_1 `INSERT`**s a new row** (or `DELETE`s one) matching T_2's `WHERE` predicate between two queries in T_2. Under `READ COMMITTED`:

1. T_2 runs `SELECT COUNT(*) FROM postings WHERE entry_id = 500;` and gets `2`.
2. T_1 inserts a third fee leg into `entry_id = 500` and commits.
3. T_2 runs `SELECT amount_minor FROM postings WHERE entry_id = 500;` to render those "two" rows into a fixed-size PDF layout—and receives **three** rows.

Switch T_2 to `BEGIN ISOLATION LEVEL REPEATABLE READ;` and both non-repeatable reads and phantom reads both disappear: the third posting's `t_xmin` belongs to T_1, which was either in xip_{S_1} or $\geq \text{xmax}_{S_1}$, so S_1 hides the newly inserted tuple just as cleanly as an updated tuple.

Common trap

Many engineers believe wrapping two `SELECT` queries in `BEGIN; ... COMMIT;` automatically gives them a consistent multi-query snapshot. **At PostgreSQL's default `READ COMMITTED` level, `BEGIN ... COMMIT` around pure `SELECT`s does nothing to freeze your snapshot across statements.** Every `SELECT` inside a `READ COMMITTED` transaction still grabs a brand-new snapshot. Any multi-query report, export, or reconciliation job *must* explicitly start with `BEGIN ISOLATION LEVEL REPEATABLE READ READ ONLY;` (or collapse the two queries into a single SQL statement, which shares one snapshot).

6.3 Lost Updates and `EvalPlanQual` Re-Evaluation

Now let both concurrent transactions *write* to the same account row `id = 1`, which starts with `bal = 10000` ($100.00). Suppose two payment workers simultaneously process two separate $80.00 (`8000`-cent) ATM withdrawals. The outcome depends on *how* you write the SQL and *which* isolation level you run.

6.3.1 Case 1: Read-Modify-Write in Application Code Under `READ COMMITTED`

In ORM-heavy codebases (Java Hibernate, naive ActiveRecord, or hand-rolled Rust structs), developers frequently load a row into application memory, subtract in application code, and write back the absolute number:

```
-- Worker 1 (READ COMMITTED)           -- Worker 2 (READ COMMITTED)
BEGIN;                                 BEGIN;
SELECT bal FROM accounts WHERE id = 1; SELECT bal FROM accounts WHERE id = 1;
-- App computes: 10000 - 8000 = 2000   -- App computes: 10000 - 8000 = 2000

UPDATE accounts SET bal = 2000
 WHERE id = 1;
                                       UPDATE accounts SET bal = 2000
                                        WHERE id = 1;
                                       -- Blocks waiting on Worker 1's XID!
COMMIT;
                                       -- Unblocks, writes 2000, and COMMITS!
```

The ATM dispensed $80.00 *twice* ($160.00 cash handed out), yet `accounts.bal` sits at `2000` ($20.00). Worker 2 overwrote Worker 1's $80 debit with a value computed from a stale read: a **Lost Update**. This is exactly the bug you get in Java when two threads run `balance = balance - 80` on a shared field without `synchronized`: each thread reads, subtracts, and writes back, and one write erases the other.

6.3.2 Case 2: Atomic SQL `SET bal = bal - 8000` and `EvalPlanQual` (EPQ)

Now move the arithmetic into the database and writes a single atomic SQL `UPDATE` with a guard predicate under `READ COMMITTED`:

```
UPDATE accounts
   SET bal = bal - 8000
 WHERE id = 1 AND bal >= 8000
RETURNING bal;
```

Trace carefully what happens inside PostgreSQL's update executor (`ExecUpdate` in the source file `nodeModifyTable.c`) when Worker 1 and Worker 2 execute this exact statement at the same microsecond under `READ COMMITTED`:

1. Both workers capture a statement snapshot seeing tuple `(0,1)` with `bal = 10000`. For both workers, `10000 >= 8000` evaluates to `true`.

2. Worker 1 reaches `heap_update()` first, stamps its XID into `(0,1)`'s `t_xmax`, inserts `(0,2)` with `bal = 2000`, and prepares to commit.
3. Worker 2 reaches `heap_update()` on `(0,1)` a microsecond later, sees an active `t_xmax` belonging to Worker 1, and **sleeps on Worker 1's transaction lock** (`XactLockTableWait`).
4. Worker 1 executes `COMMIT`. Worker 2 wakes up.
5. What does Worker 2 do now? Remember that Worker 2's original statement snapshot predated Worker 1. If Worker 2 evaluated `bal - 8000` against `(0,1)` (`bal = 10000`), it would write 2000 and lose Worker 1's update. If Worker 2 blindly updated `(0,2)` (`bal = 2000`) *without* re-checking the `WHERE` clause, it would write $2000 - 8000 = -6000$, violating `WHERE bal >= 8000`.

Instead, PostgreSQL invokes a specialized executor subsystem called `EvalPlanQual` (`EPQ`):

- Worker 2 follows `(0,1)`'s forward `t_ctid` chain to the latest committed version `(0,2)` (`bal = 2000`) and locks `(0,2)`.
- `EvalPlanQual` substitutes `(0,2)` into the plan sub-tree and **re-evaluates the `WHERE` quals and `SET` expressions against `(0,2)`.**
- Testing `bal >= 8000` on `bal = 2000` yields **false**. Worker 2 therefore skips the row cleanly, returns zero rows (`UPDATE 0`), and the second ATM withdrawal is safely declined. (Had there been no `bal >= 8000` guard, `EPQ` would have recomputed `bal - 8000` using `2000 - 8000 = -6000`, preserving both debits.)



Figure 6.2: Three outcomes when T_1 and T_2 concurrently withdraw \$80 from a \$100 row. (1) Application read-modify-write under `READ COMMITTED` silently overwrites T_1's debit because `SET bal = 2000` contains a literal constant. (2) Self-relative `SET bal = bal - 8000 WHERE bal >= 8000` triggers `EvalPlanQual` to walk `t_ctid` to `(0,2)` and re-evaluate the `WHERE` predicate. (3) `REPEATABLE READ` rejects any concurrent update to the target row with `SQLSTATE 40001`.

Common trap

Even `EvalPlanQual` has a sharp edge on multi-table `UPDATE ... FROM` queries. When `EvalPlanQual` wakes up under `READ COMMITTED`, it fetches the newest version *only for locked/target rows*, while non-locked joined tables in `FROM` or subqueries keep the rows seen

by the statement's *original* snapshot. For single-row updates, atomic `SET bal = bal - $1 WHERE bal >= $1` (or `SELECT ... FOR NO KEY UPDATE` before reading) is safe; for multi-statement read-modify-write flows without explicit locks, `REPEATABLE READ` catches every single-row collision automatically via **First-Updater-Wins** (`SQLSTATE 40001: could not serialize access due to concurrent update`).

6.4 Write Skew: Why `REPEATABLE READ` Still Breaks Multi-Row Invariants

Look back at Panel 3 of Figure 6.2. Why did `REPEATABLE READ` catch the lost update? Because T_1 and T_2 tried to update **the exact same physical row** (`id = 1`). When T_2 tried to stamp `t_xmax` on `id = 1`, it saw T_1's committed `t_xmax` and raised `40001`.

Now return to Alice's two sub-accounts from the opening page of this chapter, or its classic sibling in hospital scheduling ("at least one doctor on the night shift must stay `on_call = true`"). Look at the SQL executed by Worker 1 and Worker 2 under `REPEATABLE READ`:

```
-- Worker 1 (REPEATABLE READ)            -- Worker 2 (REPEATABLE READ)
BEGIN ISOLATION LEVEL REPEATABLE READ;  BEGIN ISOLATION LEVEL REPEATABLE READ;

SELECT SUM(bal) FROM sub_accounts       SELECT SUM(bal) FROM sub_accounts
 WHERE user_id = 7;  -- returns 16000    WHERE user_id = 7;  -- returns 16000

-- 16000 - 10000 = 6000 >= 0 (OK!)      -- 16000 - 10000 = 6000 >= 0 (OK!)
UPDATE sub_accounts                     UPDATE sub_accounts
   SET bal = bal - 10000                   SET bal = bal - 10000
 WHERE user_id = 7 AND kind = 'chk';     WHERE user_id = 7 AND kind = 'sav';

COMMIT; -- Succeeds!                    COMMIT; -- ALSO SUCCEEDS!
-- Combined balance is now -2000 + (-2000) = -4000 (< 0)!
```

Database researchers call this anomaly **Write Skew**:

1. T_1 and T_2 read *overlapping* sets of rows ($R(T_1) = \{\texttt{chk}, \texttt{sav}\}$ and $R(T_2) = \{\texttt{chk}, \texttt{sav}\}$).
2. Based on that shared premise, T_1 and T_2 write to *disjoint* sets of rows ($W(T_1) = \{\texttt{chk}\}$ and $W(T_2) = \{\texttt{sav}\}$, so $W(T_1) \cap W(T_2) = \emptyset$!).
3. Because their write sets do not intersect, neither transaction ever touches a tuple `t_xmax` modified by the other. Snapshot Isolation's First-Updater-Wins check never fires, and the multi-row invariant ($\texttt{chk} + \texttt{sav} \geq 0$) shatters silently.

Write skew also appears with `INSERT`s (*insert write skew*):

- **Meeting room double-booking** (without an exclusion constraint): Worker 1 checks `SELECT COUNT(*) FROM bookings WHERE room = 4 AND ts && '[2pm, 3pm)'` (`0`) and `INSERT`s row A; Worker 2 runs the identical check (`0`) and `INSERT`s row B. Neither updated an existing row, so both `INSERT`s commit under `REPEATABLE READ` and Room 4 is double-booked.
- **Ledger batch limit**: Two concurrent payouts check that today's outgoing wires for Merchant 99 total $< \$50{,}000$, and each `INSERT`s a new $40,000 wire posting. Both commit under

`REPEATABLE READ`, wiring $80,000.

6.5 Serializable Snapshot Isolation (SSI): True `SERIALIZABLE` Without Read Locks

For thirty years after System R, database textbooks taught a hard rule: if you wanted true `SERIALIZABLE` isolation that prevented write skew, you had to abandon lock-free MVCC reads and use **Strict Two-Phase Locking (S2PL)**—forcing every `SELECT` to acquire shared read locks and index-gap locks that blocked concurrent writers. Because S2PL destroyed concurrency, early PostgreSQL releases didn't even implement real `SERIALIZABLE` (prior to PostgreSQL 9.1, asking for `SERIALIZABLE` quietly gave you `REPEATABLE READ` Snapshot Isolation).

In 2008, Michael Cahill, Uwe Röhm, and Alan Fekete proved (implemented in PostgreSQL 9.1 by Kevin Grittner and Dan Ports): you can keep **100% non-blocking MVCC reads**, never stall a single reader or writer on read locks, and still achieve full mathematical serializability by tracking lightweight conflict edges called rw-**antidependencies**. PostgreSQL calls this engine **Serializable Snapshot Isolation** (**SSI**).

6.5.1 Conflict Edges and the "Dangerous Structure" Theorem

Between any two transactions T_1 and T_2 touching a common datum x (or predicate gap), three kinds of ordering dependencies can arise:

1. **Write-Read ($T_1 \xrightarrow{\text{wr}} T_2$):** T_1 writes a version of x and commits; T_2's snapshot sees that version. Clearly T_1 must precede T_2 in any equivalent serial schedule.
2. **Write-Write ($T_1 \xrightarrow{\text{ww}} T_2$):** T_1 writes x and commits; T_2 overwrites T_1's version of x. Again T_1 must precede T_2 (and Snapshot Isolation already forbids concurrent T_1, T_2 from both overwriting x).
3. **Read-Write Antidependency ($T_1 \xrightarrow{\text{rw}} T_2$):** T_1's snapshot reads an *older* state of x (before T_2's write), while concurrent transaction T_2 writes a *newer* version of x (or inserts a tuple matching T_1's `WHERE` predicate). Think carefully about which transaction must come first in an equivalent serial world: T_1 **MUST come before** T_2. Why? Because if T_2 had executed completely before T_1, then T_1's read would have seen T_2's new value instead of the old one.

Now draw the dependency graph for Alice's Checking/Savings write skew:

- T_1 read the old value of `sav` (`8000`), while concurrent T_2 updated `sav` to `-2000`: that creates an antidependency edge $T_1 \xrightarrow{\text{rw}} T_2$ (T_1 must precede T_2).
- Symmetrically, T_2 read the old value of `chk` (`8000`), while concurrent T_1 updated `chk` to `-2000`: that creates an antidependency edge $T_2 \xrightarrow{\text{rw}} T_1$ (T_2 must precede T_1).

Put the two edges together and you get a cycle:

$$T_1 \xrightarrow{\text{rw}} T_2 \xrightarrow{\text{rw}} T_1.$$

T_1 must come before T_2, and T_2 must come before T_1—no serial order exists. More generally, Fekete et al. proved the structural theorem that makes SSI fast:

Theorem 6.1 (Dangerous Structure Under Snapshot Isolation)**.** *In any non-serializable schedule produced by Snapshot Isolation, the serialization dependency graph contains a cycle, and* ***every minimal cycle contains a "dangerous structure" of two consecutive*** rw***-antidependency edges****:*

$$T_{\text{in}} \xrightarrow{\text{rw}} T_{\text{pivot}} \xrightarrow{\text{rw}} T_{\text{out}},$$

where T_{in} *and* T_{pivot} *ran concurrently,* T_{pivot} *and* T_{out} *ran concurrently, and* T_{out} *is the* ***first transaction among the cycle to commit*** *(note that in 2-party write skew,* T_{in} *and* T_{out} *are the very same transaction).*

Intuition

Why does Theorem 6.1 matter for performance? Checking arbitrary cycles across a giant distributed graph on every read is expensive. Theorem 6.1 says PostgreSQL does *not* even have to run full general graph search. It only has to watch for any **pivot transaction** T_{pivot} that acquires *both* an incoming rw-conflict ($T_{\text{in}} \xrightarrow{\text{rw}} T_{\text{pivot}}$) and an outgoing rw-conflict ($T_{\text{pivot}} \xrightarrow{\text{rw}} T_{\text{out}}$) where T_{out} commits before T_{pivot}! One local two-hop check in shared memory catches every Snapshot Isolation anomaly.

6.5.2 How `SIRead` Predicate Locks Work Inside PostgreSQL

How does PostgreSQL detect when T_2 writes to something T_1 already read (including an empty index gap where no row existed yet)? Through **`SIRead` predicate locks** managed inside `predicate.c` in PostgreSQL's lock manager:

1. **`SIRead` locks never block anyone:** Despite the word "lock," an `SIRead` tag is purely a passive sticky note in shared memory saying "serializable transaction T_1 inspected this tuple, index page, or table." Readers never wait on writers, and writers never wait on `SIRead` tags.

2. **Index gaps prevent phantom write skew:** When T_1 runs an index scan `WHERE room = 4 AND ...` and finds zero rows, PostgreSQL places an `SIRead` lock on the **B-tree leaf page** where `room = 4` keys belong. When concurrent T_2 inserts a new booking into that exact B-tree leaf page, T_2's index insertion trips T_1's page-level `SIRead` lock and records $T_1 \xrightarrow{\text{rw}} T_2$! (If T_1 instead runs a full table `Seq Scan` because no index exists, PostgreSQL places a relation-level `SIRead` lock on the whole table—a huge practical reason to index predicates queried inside `SERIALIZABLE` transactions so unrelated writes do not trigger false-positive retries.)

3. **Bounded memory via lock promotion:** If one transaction touches thousands of tuples on a heap page, PostgreSQL automatically coarsens those tuple-level `SIRead` tags into a single page-level (or relation-level) `SIRead` tag controlled by `max_pred_locks_per_transaction`. Coarsening never misses a real bug (zero false negatives); at worst it causes an occasional conservative `40001` retry.

4. **Keep `SIRead` tags slightly past `COMMIT`:** Notice in Alice's example that T_1 commits *first*, and only when T_2 tries to commit afterward does the dangerous structure $T_2 \xrightarrow{\text{rw}} T_1 \xrightarrow{\text{rw}} T_2$ complete with T_1 already committed. Therefore PostgreSQL retains a committed serializable transaction's `SIRead` summary in shared memory until all overlapping concurrent serializable transactions finish.

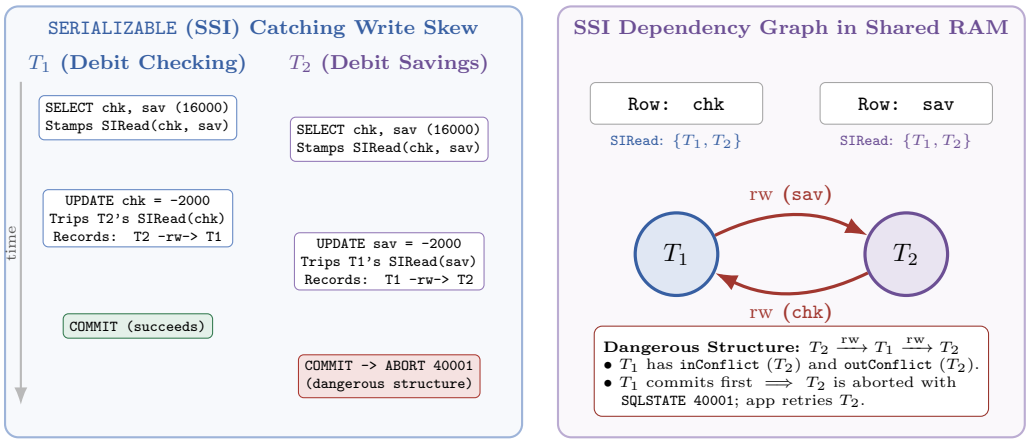


Figure 6.3: How PostgreSQL's Serializable Snapshot Isolation (SSI) defeats write skew without blocking readers. Left: Both T_1 and T_2 stamp passive `SIRead` predicate locks on `chk` and `sav`. Neither reader blocks. Right: When T_1 writes `chk` and T_2 writes `sav`, two consecutive rw-antidependency edges form a dangerous structure; once T_1 commits, T_2 is aborted with `SQLSTATE 40001` and retries against the post-T_1 balance (`6000`), declining the overdraft.

In production

Two rules every `SERIALIZABLE` production service lives by:

1. **Every `REPEATABLE READ` or `SERIALIZABLE` transaction MUST sit inside an idempotent retry loop** that catches `SQLSTATE 40001` (`serialization_failure`) and `40P01` (`deadlock_detected`), rolls back, and replays the unit of work from `BEGIN` (with small jittered backoff). Never perform irreversible external side effects (charging a Stripe webhook or sending an email) inside a retryable database transaction—write an `outbox` row inside the transaction instead.

2. **Mark long read-only analytics `SERIALIZABLE READ ONLY DEFERRABLE`:** If a 30-second reporting query runs at plain `SERIALIZABLE`, its `SIRead` locks can force concurrent OLTP writers or the report itself into `40001` retries. Starting the report with

```
BEGIN ISOLATION LEVEL SERIALIZABLE READ ONLY DEFERRABLE;
```

   waits briefly at `BEGIN` (if needed) until PostgreSQL can hand it a provably "safe" snapshot isolated from in-flight writers, and then runs the entire 30-second report with **zero `SIRead` locks and zero risk of `40001`.**

6.6 Explicit Row Locks, Work Queues, Deadlocks, and Advisory Locks

SSI is optimistic: transactions race ahead without waiting and retry if a `40001` conflict fires. When contention on a single hot row is fierce—say, 200 concurrent checkout threads debiting the same flash-sale inventory SKU or settlement account—optimistic retries waste CPU re-running 199 aborted transactions. In high-contention hotspots, **pessimistic locking** (queuing writers cleanly in line on the first pass) is cheaper.

6.6.1 PostgreSQL's Four Row-Level Lock Strengths

Why does PostgreSQL have *four* row-level `SELECT ... FOR ...` lock modes instead of just "shared" and "exclusive"? Because of **foreign keys**.

Suppose `postings.account_id` has a foreign key `REFERENCES accounts(id)`. When a worker inserts a new posting for Account 42, PostgreSQL must ensure nobody concurrently `DELETE`s Account 42 (or changes `accounts.id = 42` to `99`) before the posting commits. Prior to PostgreSQL 9.3, inserting that child row acquired `FOR SHARE` on `accounts(id = 42)`, which **blocked every concurrent `UPDATE` to `accounts.balance_minor`**—serializing the entire ledger even though `accounts.id` never changed.

PostgreSQL solved that bottleneck permanently by splitting row locks into four strengths:

1. `FOR KEY SHARE`**:** Weakest lock. Placed automatically on the parent row during foreign-key checks. Conflicts *only* with `DELETE` and updates that modify key columns (`FOR UPDATE`).
2. `FOR SHARE`**:** Shared read lock ensuring the entire row does not change until commit. Conflicts with both `FOR NO KEY UPDATE` and `FOR UPDATE`, while allowing concurrent `FOR SHARE` readers.
3. `FOR NO KEY UPDATE`**:** Acquired automatically by any `UPDATE` that does *not* modify a `PRIMARY KEY` or `UNIQUE` index column referenced by foreign keys. **Compatible with `FOR KEY SHARE`**, so child inserts and non-key parent updates run concurrently without blocking.
4. `FOR UPDATE`**:** Strongest exclusive row lock (acquired automatically by `DELETE` or key-changing `UPDATE`s). Blocks all four row-lock modes (while *still* never blocking plain MVCC `SELECT`s).

To prevent Alice's Checking/Savings write skew pessimistically even under default `READ COMMITTED`, lock **both** sub-account rows (or lock the parent `users` row) before summing:

```
BEGIN;
-- Locks BOTH sub-accounts in deterministic ID order before summing!
SELECT id, kind, bal
  FROM sub_accounts
 WHERE user_id = 7
 ORDER BY id ASC
   FOR NO KEY UPDATE;
-- Worker 2 sleeps right here until Worker 1 commits, then sees bal = 6000!
```

6.6.2 Non-Blocking Modifiers: `NOWAIT` and `SKIP LOCKED` Job Queues

By default, `FOR UPDATE` blocks until the lock holder finishes (or `lock_timeout` expires). Two modifiers change what happens when the row is already locked:

- `FOR UPDATE NOWAIT`**:** If the row is already locked by another transaction, fail *immediately* in < 0.1 ms with `SQLSTATE 55P03` (`could not obtain lock on row`) instead of parking an HTTP worker thread in a lock queue. Always set `SET LOCAL lock_timeout = '2s';` or use `NOWAIT` in latency-sensitive APIs so a stuck lock cannot exhaust your entire connection pool.
- `FOR UPDATE SKIP LOCKED`**:** Any row currently locked by another active worker is *quietly skipped* during the scan. With six lines of SQL, `SKIP LOCKED` turns an ordinary PostgreSQL table into a high-throughput, ACID-compliant transactional work queue or outbox consumer where

fifty concurrent Rust workers claim disjoint batches with zero lock waits and zero duplicate processing:

```
WITH claimed AS (
    SELECT id
      FROM webhook_outbox
     WHERE status = 'pending' AND next_attempt_at <= now()
     ORDER BY next_attempt_at ASC
     LIMIT 25
       FOR UPDATE SKIP LOCKED
)
UPDATE webhook_outbox w
   SET status = 'processing', attempts = attempts + 1
  FROM claimed c
 WHERE w.id = c.id
RETURNING w.id, w.endpoint_url, w.payload_json;
```

6.6.3 Deadlocks (`deadlock_timeout`) and Deterministic Lock Ordering

Whenever transactions lock more than one row one at a time, lock waits can form a cycle:

- T_1 transfers $10 from Account 10 to Account 20: locks row `10`, then tries to lock row `20`.

- At the same millisecond, T_2 transfers $15 from Account 20 to Account 10: locks row `20`, then tries to lock row `10`!

Now T_1 waits for T_2 and T_2 waits for T_1; neither can ever advance on its own.

PostgreSQL detects deadlocks lazily. Scanning the global wait-for graph on every microsecond lock wait would waste CPU when 99.99% of row locks clear in under a millisecond. Instead, a sleeping backend sets a timer for `deadlock_timeout` (default `1s`). Only if the backend is *still* asleep after 1,000 ms does it wake up, build the wait-for graph in shared memory, detect the cycle $T_1 \to T_2 \to T_1$, and abort one transaction with `SQLSTATE 40P01: deadlock detected` so the survivor finishes immediately.

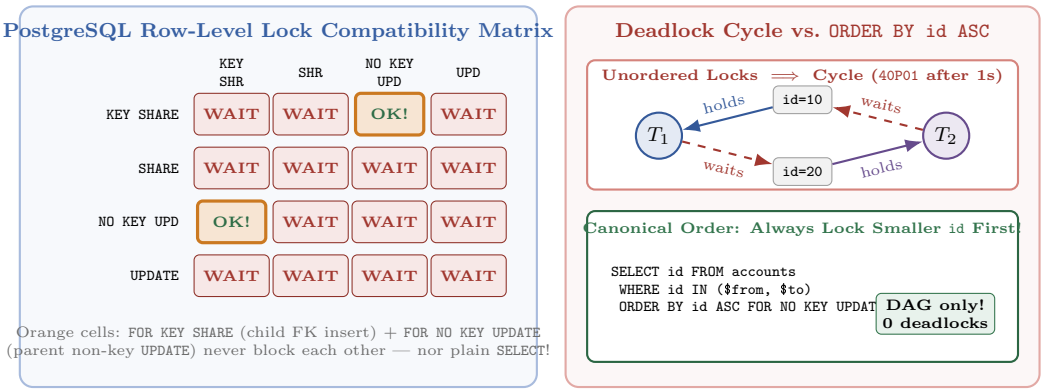


Figure 6.4: Left: Compatibility among PostgreSQL's four row-level lock modes; highlighted orange cells show why child foreign-key inserts (`FOR KEY SHARE`) do not block non-key parent `UPDATE`s (`FOR NO KEY UPDATE`). Right: Acquiring row locks in arbitrary caller order permits a 2-cycle (T_1 holds 10 waiting for 20; T_2 holds 20 waiting for 10), whereas locking keys in strictly increasing order (`ORDER BY id ASC`) forces every wait edge ($u \to v$) to point upward ($u < v$), making cycles mathematically impossible.

Here is the short proof behind Figure 6.4: if every transaction acquires locks in strictly increasing order of `account_id`, then any transaction waiting for lock k_2 while holding lock k_1 satisfies $k_1 < k_2$. Around any hypothetical cycle $k_1 < k_2 < \cdots < k_m < k_1$ we would get $k_1 < k_1$, a contradiction. In our Chapter 8 financial ledger, sorting account IDs before locking eliminates transfer deadlocks completely.

6.6.4 Transactional Advisory Locks (`pg_advisory_xact_lock`)

One puzzle remains: how do you pessimistically lock a resource **whose row does not exist yet**? Suppose two concurrent requests try to onboard a new merchant `tax_id = 'US-98-7654321'` and run complex validation queries before inserting the first row. `SELECT ... FOR UPDATE WHERE tax_id = ...` returns zero rows and locks nothing.

PostgreSQL provides **advisory locks**—application-defined 64-bit integer locks stored purely in PostgreSQL's fast shared-memory lock table (zero heap writes, zero dead tuples, zero WAL):

```
BEGIN;
-- Hash a namespace + business key to a 64-bit BIGINT lock ID;
-- automatically released at COMMIT or ROLLBACK!
SELECT pg_advisory_xact_lock(
    hashtextextended('onboard_tax_id:' || $1, 0)
);
-- Now strictly one worker per tax_id executes this critical section at a time!
COMMIT;
```

Always prefer transaction-scoped advisory locks (`pg_advisory_xact_lock`) over session-scoped ones (`pg_advisory_lock`): if an application panics or sits behind a transaction-mode connection pooler like `PgBouncer` (Chapter 8), a session-scoped advisory lock stays leaked on the pooled backend connection and deadlocks unrelated tenants.

We now understand how PostgreSQL keeps concurrent transactions both fast and correct. Yet before

any transaction can lock or read a tuple, PostgreSQL has to decide *how* to find that tuple among billions of rows and dozens of tables—sequential scan, B-tree index scan, bitmap heap scan, nested loop, hash join, or merge join. In Chapter 7 we step inside the PostgreSQL query planner and learn to read `EXPLAIN (ANALYZE, BUFFERS)` output line by line.

6.7 Exercises

Exercise 6.1 (Testing serializability by hand)**.** Initially accounts `A = 100` and `B = 100`. Consider two committed transactions:

- T_1: reads `A`, then updates `B = A + 50`.
- T_2: reads `B`, then updates `A = B * 2`.

(a) Compute the final (`A`, `B`) pair produced by the two valid serial schedules ($T_1 \to T_2$ and $T_2 \to T_1$). (b) Compute the final (`A`, `B`) pair if T_1 and T_2 run concurrently from the same initial snapshot under PostgreSQL `REPEATABLE READ`. Does `REPEATABLE READ` raise an error? Is the outcome serializable? (c) Trace what happens when you change both transactions to `ISOLATION LEVEL SERIALIZABLE`.

Exercise 6.2 (Read skew in pagination and reporting)**.** An API endpoint running at default `READ COMMITTED` runs two queries inside `BEGIN; ... COMMIT;`: first counting rows via `SELECT COUNT(*) FROM invoices WHERE user_id = 99 AND status = 'unpaid';` for a header badge, and 5 ms later fetching rows via `SELECT id, amount_minor FROM invoices` with the same `WHERE` clause. Show a concrete interleaving with a concurrent payment transaction where the header badge says "1 unpaid invoice" while the table lists zero rows, and give two ways to eliminate the mismatch without blocking the payment writer.

Exercise 6.3 (Tracing `EvalPlanQual` step by step)**.** Table `wallets(id INT PRIMARY KEY, bal BIGINT)` holds a single row `(1, 10000)`. Transactions T_1 and T_2 both run at `READ COMMITTED` and simultaneously start executing:

- T_1: `UPDATE wallets SET bal = bal - 7000 WHERE id = 1 AND bal >= 7000;`
- T_2: `UPDATE wallets SET bal = bal - 5000 WHERE id = 1 AND bal >= 5000;`

Suppose T_1 acquires the tuple lock on `(0,1)` first. (a) Trace what happens to T_2 if T_1 commits after 20 ms: which physical tuple does `EvalPlanQual` inspect, what does `bal >= 5000` evaluate to, and what is the final balance? (b) Now repeat (a) if T_2's query was `UPDATE wallets SET bal = bal - 2000 WHERE id = 1 AND bal >= 2000;`. (c) Contrast (b) with what happens if T_2 ran at `REPEATABLE READ`.

Exercise 6.4 (Insert write skew and SSI index gaps)**.** A table `on_call_shifts(shift_date DATE, doctor_id INT, PRIMARY KEY (shift_date, doctor_id))` starts with two doctors scheduled on `'2026-11-05'`. A hospital rule states at most *three* doctors may claim overtime on the same `shift_date`. Doctors 3 and 4 concurrently run `SELECT COUNT(*) FROM on_call_shifts WHERE shift_date = '2026-11-05';` (both see `2 < 3`) and each executes `INSERT INTO on_call_shifts VALUES ('2026-11-05', ...)`. (a) Why do both `INSERT`s commit under `REPEATABLE READ` without violating the primary key? (b) Explain which B-tree structure receives an `SIRead` lock under `SERIALIZABLE` and why SSI aborts the second committer with `40001`. (c) Suppose `on_call_shifts` had *no* index starting with `shift_date`, forcing a `Seq Scan`. What happens under `SERIALIZABLE` when Doctor 5 concurrently books an unrelated shift on `'2026-12-25'`?

Exercise 6.5 (Drawing the dangerous structure)**.** Three concurrent transactions run under Snapshot Isolation (`REPEATABLE READ`) over variables $x = 0, y = 0, z = 0$:

- T_1 reads y (sees 0), writes $x \leftarrow 1$, and commits at $t = 3$.
- T_2 reads z (sees 0), writes $y \leftarrow 1$, and commits at $t = 2$.
- T_3 reads x (sees 0), writes $z \leftarrow 1$, and commits at $t = 1$.

Draw all rw-antidependency edges among $\{T_1, T_2, T_3\}$, verify that no two transactions wrote the same variable, prove the resulting state $(x, y, z) = (1, 1, 1)$ cannot arise from any of the $3! = 6$ serial orders, and identify $T_{\text{in}}, T_{\text{pivot}}, T_{\text{out}}$ in Theorem 6.1.

Exercise 6.6 (`FOR UPDATE` vs. `FOR NO KEY UPDATE`)**.** Consider parent table `accounts(id, bal)` and child table `transfers(id, account_id, amount)`, where `account_id` is a foreign key referencing `accounts(id)`. Explain precisely what lock contention occurs when Connection A runs `SELECT bal FROM accounts WHERE id = 42 FOR UPDATE;` (holding the transaction open for 50 ms while computing fees) while 20 concurrent connections insert rows into `transfers` for `account_id = 42`. What changes the instant you replace `FOR UPDATE` with `FOR NO KEY UPDATE` in Connection A?

Exercise 6.7 (Transactional outbox edge cases with `SKIP LOCKED`)**.** Examine the `FOR UPDATE SKIP LOCKED` webhook queue query in Section 6.6. (a) Suppose Worker A claims 25 rows, marks them `status = 'processing'`, commits that claim transaction, makes HTTP calls, and **crashes with `SIGKILL`** after delivering 10 of the 25 webhooks. What happens to the remaining 15 rows stuck in `'processing'` unless you add a lease column such as `leased_until TIMESTAMPTZ`? (b) Write the complete crash-resilient claim query using `leased_until` so dead workers' unfinished items automatically re-enter the pool after 60 seconds while live workers never double-claim active leases.

Exercise 6.8 (Multi-row deadlock construction and prevention)**.** A batch payroll endpoint transfers bonuses from one funding account f to k recipient accounts $r_1, \ldots, r_k$ by looping in application code and running k individual `UPDATE accounts SET bal = bal + $1 WHERE id = $2;` statements inside one transaction, followed by debiting f. (a) Construct a two-transaction deadlock between two concurrent batch runs that share funding account $f = 1$ and one recipient $r = 50$. (b) Rewrite the locking preamble as a single `SELECT ... WHERE id = ANY($1::bigint[]) ORDER BY id ASC FOR NO KEY UPDATE;` statement and prove why deadlocks between any number of concurrent batch runs become impossible.

Exercise 6.9 (Advisory lock collision probability and pooling)**.** (a) Suppose 100,000 concurrent tenants use `pg_advisory_xact_lock(hashtext('tenant:' || id))` (which takes a **32-bit** `INT4` hash) vs. `pg_advisory_xact_lock(hashtextextended('tenant:' || id, 0))` (which takes a **64-bit** `BIGINT` hash). Using the birthday paradox approximation $p \approx k^2/(2M)$ for $k = 1{,}000$ simultaneously active locks, estimate the probability that two unrelated tenants block each other under 32-bit vs. 64-bit advisory keys. (b) Explain concretely what goes wrong if a developer calls `pg_advisory_lock(42)` (session level) through a transaction-mode connection pooler and forgets `pg_advisory_unlock(42)` on an early error return.

Part III

Query Execution and Production PostgreSQL

Chapter 7

The Query Planner, Join Algorithms, and `EXPLAIN ANALYZE`

Here is a query that a payments dashboard runs every few seconds:

```
SELECT a.id, a.owner_name, SUM(p.amount_minor) AS total
  FROM accounts a
  JOIN postings p ON p.account_id = a.id
 WHERE a.region = 'EU'
   AND p.created_at >= '2026-10-01'
 GROUP BY a.id, a.owner_name
 ORDER BY total DESC
 LIMIT 10;
```

The query says *what* answer we want. It says nothing about *how* to get it. Should PostgreSQL read all 20 million postings and throw most of them away? Should it find the EU accounts first through an index, and then look up each account's postings through a second index? Should it build a hash table of EU accounts in memory and stream the postings past it? Each of those is a correct way to compute the same ten rows. On one real table, the first plan took 45 seconds and the second took 0.2 seconds. Same SQL, same data, same hardware: a factor of two hundred.

The piece of PostgreSQL that chooses among those plans is the *query planner* (also called the optimizer). It never runs the candidates. It *predicts* what each would cost using statistics about your data and a small set of cost constants, and then hands the cheapest prediction to the executor. When the prediction is right, you never think about it. When it is wrong, a query that ran in 3 ms yesterday runs for 40 seconds today, and the only way to see why is to read `EXPLAIN (ANALYZE, BUFFERS)` output.

This chapter builds the planner's reasoning from the ground up: the statistics it collects, the units it measures cost in, the four ways it can read one table, the three ways it can join two tables, how it searches through join orders, and how to read its output line by line so you can tell exactly where a prediction went wrong.

7.1 From SQL Text to a Running Plan

Every statement passes through four stages inside the backend process that owns your connection:

1. **Parse.** The SQL text becomes a parse tree. Syntax errors are caught here. Nothing about tables has been checked yet.
2. **Analyze and rewrite.** Names are resolved against the system catalogs (`a.region` becomes "column 4 of relation OID 16402"), types are checked, and views are expanded into their underlying queries. The result is a *query tree*.
3. **Plan.** The planner turns the query tree into a *plan tree*: a tree of physical operators such as `Seq Scan`, `Index Scan`, `Hash Join`, and `Sort`. This is the only stage that makes choices, and it is the subject of this chapter.
4. **Execute.** The executor runs the plan tree. Each node is an iterator: a parent asks its child for the next row, the child asks *its* child, and rows flow upward one at a time. (This is the "Volcano" iterator model; PostgreSQL also batches some work, such as building hash tables, but the pull-one-row interface is the mental model to keep.)

The relational algebra from Chapter 1 lives in stage 3. A join is commutative and associative, a selection σ can be pushed below a join, and a projection π can drop columns early. Those algebraic laws generate many logically equivalent trees. The planner's job is to attach a physical algorithm to every node of every reasonable tree, estimate the cost, and keep the cheapest.

Intuition

Think of the planner as a travel agent who has never visited the city. It cannot drive each route to time it. Instead it owns a map (table and index sizes), a census (statistics about the values in each column), and a price list (cost constants per page read and per row processed). Every route it proposes is only as good as that map and census. Most bad plans are not caused by a stupid search; they are caused by a census that is stale or blind to some pattern in your data.

7.2 What the Planner Knows: `ANALYZE` and `pg_stats`

7.2.1 Sampling a Table of Any Size

The `ANALYZE` command (run automatically by autovacuum after enough rows change) reads a *random sample* of each table and stores a summary per column in the catalog `pg_statistic`. The human-readable view of that catalog is `pg_stats`.

How big is the sample? PostgreSQL samples 300 × `default_statistics_target` rows. With the default target of 100, that is **30,000 rows**, whether the table holds 50,000 rows or 5 billion. This can feel wrong at first. Why would 30,000 rows describe 5 billion?

Because the questions the planner asks are about *fractions*, not totals. "What fraction of postings have `status = 'settled'`?" is a proportion, and the error of an estimated proportion from a random sample of size n shrinks like $1/\sqrt{n}$, regardless of population size. With $n = 30{,}000$, a value

that occurs in 10% of rows is estimated to within about ± 0.35 percentage points (two standard errors of $\sqrt{0.1 \times 0.9 / 30000} \approx 0.0017$). That is plenty for choosing between a sequential scan and an index scan.

What a small sample *cannot* do well is count rare things. If a column has 40 million distinct values in 5 billion rows, the sample sees most values exactly once, and the estimate of `n_distinct` is a statistical extrapolation that can be off by a factor of several. We will see how to fix that when it matters.

7.2.2 Reading One Row of `pg_stats`

Here is what `pg_stats` reports for two columns of a 20-million-row `postings` table:

```
SELECT attname, null_frac, avg_width, n_distinct,
       most_common_vals, most_common_freqs,
       histogram_bounds, correlation
  FROM pg_stats
 WHERE tablename = 'postings'
   AND attname IN ('status', 'amount_minor');
```

```
-[ RECORD 1 ]-----+-----------------------------------
attname           | status
null_frac         | 0
avg_width         | 8
n_distinct        | 5
most_common_vals  | {settled,pending,failed}
most_common_freqs | {0.82,0.12,0.04}
histogram_bounds  |
correlation       | 0.31
-[ RECORD 2 ]-----+-----------------------------------
attname           | amount_minor
null_frac         | 0.002
avg_width         | 8
n_distinct        | -0.21
most_common_vals  |
most_common_freqs |
histogram_bounds  | {0,1000,2000,5000,20000}
correlation       | 0.02
```

(The real histogram has 101 bounds, giving 100 buckets; we use five bounds and four buckets so we can do the arithmetic by hand.) Each field answers one question the planner will ask:

- `null_frac`: the fraction of rows where the column is `NULL`. A predicate like `amount_minor > 0` can never match those rows, so the planner removes them first.
- `avg_width`: average stored bytes per value. The planner uses it to estimate how much memory a sort or hash table will need.
- `n_distinct`: the number of distinct non-null values. A *positive* number is an absolute count: `status` has 5 distinct values. A *negative* number is a fraction of the row count: -0.21 means "about 21% of rows have a distinct value," so the planner computes $0.21 \times 20{,}000{,}000 = 4.2$ million distinct amounts. The negative form is used when the number of distinct values seems

to grow with the table, so the estimate stays sensible as the table grows. A value of -1 means every row is distinct, as for a primary key.

- `most_common_vals` and `most_common_freqs` (the MCV list): the most frequent values and their frequencies, as observed in the sample. Only values that are noticeably more common than average make the list.
- `histogram_bounds`: boundaries of an *equi-depth* histogram over the values that are *not* in the MCV list. Equi-depth means each bucket holds the same number of rows, so buckets are narrow where values are dense and wide where they are sparse.
- `correlation`: how closely the physical order of rows on disk follows the sorted order of this column, from -1 (perfectly reversed) through 0 (random) to $+1$ (perfectly sorted). We will see in Section 7.4 why this one number can swing an index scan's cost by a factor of forty.

7.2.3 Turning Statistics into Selectivity

The planner's central quantity is *selectivity*: the fraction of a table's rows that a predicate keeps. Multiply selectivity by the row count and you get the estimated `rows` you see in `EXPLAIN`. Let us compute several by hand from the statistics above.

Example 7.1 (Equality on a value in the MCV list)**.** For `WHERE status = 'pending'`, the value is in the MCV list with frequency 0.12. The selectivity is simply 0.12, and the estimate is $0.12 \times 20{,}000{,}000 = 2{,}400{,}000$ rows.

Example 7.2 (Equality on a value not in the MCV list)**.** For `WHERE status = 'reversed'`, the value is not in the MCV list. The planner reasons: the MCVs account for $0.82 + 0.12 + 0.04 = 0.98$ of rows, and there are no `NULL`s, so $1 - 0.98 = 0.02$ of rows hold the *other* values. There are $5 - 3 = 2$ other distinct values. Assume they split the remainder evenly:

$$\text{sel} \;=\; \frac{1 - \text{null_frac} - \sum \text{mcv_freqs}}{\text{n_distinct} - |\text{MCV}|} \;=\; \frac{1 - 0 - 0.98}{5 - 3} \;=\; 0.01.$$

Estimate: $0.01 \times 20{,}000{,}000 = 200{,}000$ rows.

Example 7.3 (A range predicate using the histogram)**.** For `WHERE amount_minor < 3500`, there is no MCV list for this column, so every non-null row is covered by the histogram with bounds $\{0, 1000, 2000, 5000, 20000\}$. Each of the four buckets holds $\frac{1}{4}$ of the non-null rows.

The value 3500 falls inside the third bucket $[2000, 5000)$. Buckets one and two lie entirely below 3500: that is $\frac{2}{4}$. Within bucket three, PostgreSQL assumes values are spread uniformly and interpolates linearly: 3500 sits $\frac{3500-2000}{5000-2000} = 0.5$ of the way across, contributing $0.5 \times \frac{1}{4} = \frac{1}{8}$. So the histogram fraction is $\frac{2}{4} + \frac{1}{8} = 0.625$. Finally, remove the 0.2% of `NULL`s:

$$\text{sel} = 0.625 \times (1 - 0.002) \approx 0.624, \qquad \text{rows} \approx 12{,}475{,}000.$$

A `BETWEEN 1500 AND 3500` predicate works the same way: compute the fraction below 3500 (0.625), subtract the fraction below 1500 ($\frac{1}{4} + 0.5 \times \frac{1}{4} = 0.375$), and get 0.25 of non-null rows.

Figure 7.1 shows these three pictures together, along with what `correlation` measures.

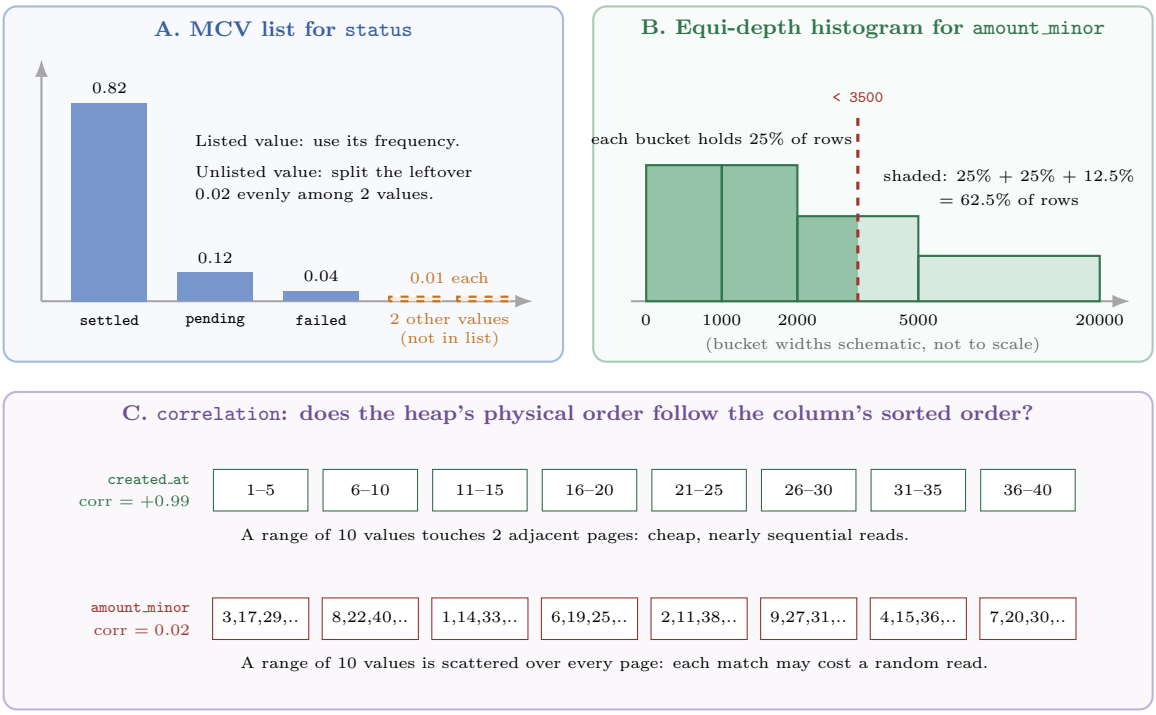


Figure 7.1: The three statistics that drive most row estimates. A: the MCV list gives exact frequencies for common values and an even split of the leftover for rare ones. B: an equi-depth histogram puts the same number of rows in every bucket, and the planner interpolates linearly inside the bucket containing the boundary value. C: `correlation` measures whether rows that are adjacent in sorted order are also adjacent on disk.

7.2.4 When Two Predicates Are Not Independent

With two predicates joined by `AND`, PostgreSQL by default assumes they are statistically independent and multiplies their selectivities:

$$\mathrm{sel}(A \wedge B) = \mathrm{sel}(A) \times \mathrm{sel}(B).$$

That is often fine. It is badly wrong when the columns are related. Consider a `transfers` table where 60% of rows have `currency = 'USD'` and 55% have `country = 'US'`. Independence predicts

$$0.60 \times 0.55 = 0.33,$$

but almost every US transfer is in dollars, so the true fraction with both is about 0.54. The planner underestimates by about 40%. That is survivable. Now take `city = 'Lyon' AND postal_code = '69001'`: if each has selectivity 0.001, independence predicts 10^{-6}, yet every row in that postal code is in Lyon, so the truth is close to 10^{-3}. The estimate is *a thousand times* too small, and a thousand-times-too-small row count is exactly what makes the planner pick a nested loop that runs for minutes (we will see why in Section 7.5).

PostgreSQL cannot discover such relationships on its own, because `pg_stats` describes each column separately. You can tell it to look at a group of columns together with *extended statistics*:

```
CREATE STATISTICS addr_stats (dependencies, ndistinct, mcv)
```

```
    ON city, postal_code FROM customers;
ANALYZE customers;
```

The three kinds each fix a different mistake:

- `dependencies` measures how strongly one column determines another. If `postal_code` determines `city` with degree f (from 0 to 1), the planner estimates $\text{sel}(A \wedge B) = \text{sel}(A) \times \big(f + (1-f)\,\text{sel}(B)\big)$. With $f = 1$, the second predicate costs nothing extra, which is right: knowing the postal code already tells you the city.
- `ndistinct` records the number of distinct *combinations*, which fixes `GROUP BY city, postal_code` estimates (otherwise the planner multiplies the two distinct counts and predicts far too many groups).
- `mcv` keeps a most-common-values list over the *pairs*, which gives exact frequencies for common combinations, including ones where the columns are anti-correlated.

Common trap

Statistics only exist for plain columns, and for expressions that have an index or an expression statistic built on them. If you write `WHERE date_trunc('day', created_at) = '2026-10-31'`, the planner has no histogram for that expression and falls back to a hard-coded default selectivity (0.5% for equality). On a 20-million-row table that predicts 100,000 rows no matter whether the true answer is 60 or 6 million. Either rewrite the predicate as a range on the raw column (`created_at >= '2026-10-31' AND created_at < '2026-11-01'`), which also lets an ordinary index on `created_at` be used, or create an expression index or `CREATE STATISTICS ... ON (date_trunc('day', created_at))` so `ANALYZE` collects a histogram for it.

7.3 The Planner's Currency: Cost Units

Once the planner knows how many rows each step touches, it needs a common unit to compare a plan that reads many pages sequentially against a plan that reads few pages randomly but does more CPU work. PostgreSQL measures everything in arbitrary *cost units*, anchored so that reading one 8 KB page sequentially costs 1.0:

Parameter	Default	What it prices
`seq_page_cost`	1.0	Reading one page as part of a sequential run
`random_page_cost`	4.0	Reading one page at an arbitrary location
`cpu_tuple_cost`	0.01	Processing one heap row
`cpu_index_tuple_cost`	0.005	Processing one index entry
`cpu_operator_cost`	0.0025	Evaluating one operator or function call
`effective_cache_size`	4GB	How much of the data the OS and `shared_buffers` likely cache

Two of these deserve a comment.

`random_page_cost = 4.0` dates from spinning disks, where a random read cost a seek of several milliseconds while sequential reads streamed. The default is already a compromise: on a real hard

drive a random read is closer to 40 times the cost of a sequential one, but most random reads hit cache. On NVMe SSDs, random and sequential reads cost nearly the same, and most teams set `random_page_cost = 1.1`. That single change makes the planner far more willing to use indexes.

`effective_cache_size` allocates nothing. It is only a hint: "assume about this much data is cached somewhere." The planner uses it in the Mackert–Lohman formula (below) to predict how many repeated page fetches will be cache hits.

7.3.1 Pricing a Sequential Scan by Hand

Take a `ledger_lines` table that occupies 10,000 pages and holds 500,000 rows (50 rows per 8 KB page). The cost of reading every row and testing one `WHERE` predicate is

$$\underbrace{10{,}000 \times 1.0}_{\text{pages}} + \underbrace{500{,}000 \times 0.01}_{\text{rows}} + \underbrace{500{,}000 \times 0.0025}_{\text{one predicate per row}} = 10{,}000 + 5{,}000 + 1{,}250 = 16{,}250.$$

You can check this against PostgreSQL directly:

```
EXPLAIN SELECT * FROM ledger_lines WHERE amount_minor > 900000;
```

```
Seq Scan on ledger_lines  (cost=0.00..16250.00 rows=4975 width=48)
  Filter: (amount_minor > 900000)
```

Every node in `EXPLAIN` prints `cost=`*startup*`..`*total*:

- **Startup cost** is the work done before the node can return its *first* row. A sequential scan can return a row as soon as it reads page 0, so its startup cost is 0. A `Sort` must read its entire input before emitting anything, so its startup cost is nearly its total cost.
- **Total cost** is the work to return *all* rows.
- `rows` is the estimated output row count and `width` is the estimated average row size in bytes.

The distinction matters because of `LIMIT`. If a plan's total cost is 50,000 for 100,000 rows but you only want 10 rows, the planner charges roughly startup + (total − startup) × $\frac{10}{100{,}000}$. A plan with near-zero startup cost (such as walking an index that is already in the requested `ORDER BY` order) can beat a plan with a lower total cost but a large startup cost (such as sorting everything first). This is the reason `ORDER BY created_at DESC LIMIT 20` on a table with an index on `created_at` returns in a millisecond.

7.4 Four Ways to Read One Table

For a single table with a predicate such as `WHERE merchant_id = 4242`, PostgreSQL considers up to four *access paths*.

7.4.1 Seq Scan

Read every page in physical order and test every row. The cost is almost independent of how many rows match. Sequential reads are the cheapest kind of I/O, and the operating system's read-ahead makes them faster still. For a predicate that keeps a large fraction of rows, nothing beats it.

7.4.2 Index Scan

Descend the B-tree (Chapter 3) to the first matching key, then walk the leaf pages. For each index entry, follow its `ctid` to the heap page, fetch the row, and check visibility against the snapshot (Chapter 5). Rows come out in index order, which can satisfy an `ORDER BY` for free.

The catch is the heap fetches. Each matching row may live on a different heap page, and each of those page reads may be a random read. How many distinct heap pages does an index scan touch? That depends on `correlation`, and the planner handles it by computing two extremes and interpolating between them.

Formal

How PostgreSQL prices the heap side of an Index Scan (`cost_index` in `costsize.c`). Let the table have T pages, and let the scan fetch N rows.

- **Worst case (correlation 0):** the rows are scattered. The number of distinct pages fetched is estimated by the Mackert–Lohman formula; when the table fits in cache it is

$$P_{\max} = \min\Big(\frac{2TN}{2T+N},\ T\Big),$$

  and each page costs `random_page_cost`: $\mathrm{IO}_{\max} = P_{\max} \times 4.0$.

- **Best case (correlation ± 1):** the matching rows sit on consecutive pages. That is $P_{\min} = \lceil \mathrm{sel} \times T \rceil$ pages, the first a random read and the rest sequential: $\mathrm{IO}_{\min} = 4.0 + (P_{\min} - 1) \times 1.0$.

- **Interpolate with the square of the correlation c:**

$$\mathrm{IO} = \mathrm{IO}_{\max} + c^2\,(\mathrm{IO}_{\min} - \mathrm{IO}_{\max}).$$

CPU cost is added on top: `cpu_index_tuple_cost` plus one `cpu_operator_cost` per index entry, and `cpu_tuple_cost` per heap row.

Example 7.4 (The same 1% query on two different columns)**.** Use the `ledger_lines` table again: $T = 10{,}000$ pages, 500,000 rows. A predicate keeps 1%, so $N = 5{,}000$ rows.

Uncorrelated column ($c = 0.02$, effectively 0). Mackert–Lohman gives

$$P_{\max} = \frac{2 \times 10{,}000 \times 5{,}000}{20{,}000 + 5{,}000} = \frac{10^8}{25{,}000} = 4{,}000 \text{ pages}, \qquad \mathrm{IO} \approx 4{,}000 \times 4.0 = 16{,}000.$$

Add about $5{,}000 \times (0.005 + 0.0025) = 37.5$ for index entries, $5{,}000 \times 0.01 = 50$ for heap rows, and a few dozen for index pages, and the total lands near 16,150. The sequential scan costs 16,250. At 1%, on an uncorrelated column, the two plans are essentially tied.

Correlated column ($c = 0.99$, such as `created_at` on an append-only table). Now $P_{\min} = 100$ pages and $\mathrm{IO}_{\min} = 4 + 99 = 103$. With $c^2 = 0.98$:

$$\mathrm{IO} = 16{,}000 + 0.98 \times (103 - 16{,}000) \approx 420.$$

The total is about 520, *thirty times* cheaper than the sequential scan. Same selectivity, same table size; the only difference is whether the matching rows sit together on disk.

That example explains a pattern every PostgreSQL user eventually notices: range queries on `id` or `created_at` of an append-only table use indexes even for large ranges, while the same range size on a randomly distributed column flips to a sequential scan early.

7.4.3 Bitmap Index Scan plus Bitmap Heap Scan

Between "a handful of rows" (Index Scan) and "a large fraction of the table" (Seq Scan) there is a wide middle zone where neither is good. Suppose 2% of `ledger_lines` match an uncorrelated predicate: $N = 10{,}000$ rows. An Index Scan would visit about 6,667 distinct pages (Mackert–Lohman again), in index-key order, which means jumping randomly back and forth across the file and possibly re-reading pages that fell out of cache. That costs about 26,700, well above the sequential scan.

PostgreSQL's answer is to split the work into two nodes:

1. **Bitmap Index Scan.** Walk the index and collect every matching `ctid`, but do *not* visit the heap yet. Instead, record each `ctid` in an in-memory bitmap keyed by heap page number: "page 17, tuples 1 and 9; page 812, tuples 3 and 7."
2. **Bitmap Heap Scan.** Visit the heap pages listed in the bitmap *in increasing page order*. Each page is read once, no matter how many matching tuples it holds, and the reads move steadily forward through the file, so the operating system's read-ahead helps and the drive sees something close to a sequential pattern.

The planner prices the per-page cost on a sliding scale between `random_page_cost` and `seq_page_cost`: the larger the fraction of the table being read, the closer to sequential each page is. For our 2% case, about two thirds of the pages are touched, the per-page cost comes out near 1.55, and the whole plan costs about 10,600, comfortably below both the Index Scan (26,700) and the Seq Scan (16,250). This is why you so often see bitmap scans for predicates matching a few percent of a table.

Exact versus lossy bitmaps. The bitmap must fit in `work_mem`. While it does, every page entry records exactly which tuple offsets matched: an *exact* page. If the bitmap would grow past `work_mem`, PostgreSQL starts converting entries to *lossy* form: it keeps only "some tuple on this page matched" and forgets which ones. A lossy page costs one bit of information instead of a list of offsets, so memory stays bounded no matter how many rows match. The price is paid at the heap: for a lossy page, the Bitmap Heap Scan must re-evaluate the original condition against *every* tuple on that page. That re-evaluation is the `Recheck Cond` line you see in the plan, and `EXPLAIN ANALYZE` tells you how much it cost:

```
Bitmap Heap Scan on postings
    (cost=4120.55..98211.31 rows=201004 width=40)
    (actual time=38.412..402.118 rows=198734 loops=1)
  Recheck Cond: (merchant_id = 4242)
  Rows Removed by Index Recheck: 1210554
  Heap Blocks: exact=21004 lossy=61311
  Buffers: shared hit=1080 read=81785
  ->  Bitmap Index Scan on postings_merchant_idx
        (cost=0.00..4070.30 rows=201004 width=0)
        (actual time=31.552..31.552 rows=198734 loops=1)
        Index Cond: (merchant_id = 4242)
```

```
        Buffers: shared hit=550
```

(PostgreSQL prints each node's estimate and actuals on one long line; we wrap lines in this chapter so they fit the page.) Read the middle lines. Of the 82,315 heap pages visited, 61,311 were lossy, and rechecking every tuple on them discarded 1.2 million non-matching rows. Raising `work_mem` for this query (for example, `SET LOCAL work_mem = '64MB'`) keeps every page exact; the `Rows Removed by Index Recheck` line disappears and the heap scan does far less CPU work.

Combining indexes. Bitmaps can be combined before touching the heap. For `WHERE merchant_id = 4242 AND status = 'failed'` with separate indexes on each column, PostgreSQL can run two Bitmap Index Scans and intersect them with a `BitmapAnd` node (or union them with `BitmapOr` for an `OR`), then read only the heap pages present in both. A plain Index Scan can use only one index per table.

7.4.4 Index-Only Scan

If every column the query needs is stored in the index, the heap fetch could be skipped entirely. The obstacle is visibility: index entries carry no `xmin`/`xmax`, so the index alone cannot tell whether an entry points to a row your snapshot should see.

PostgreSQL solves this with the *visibility map* (Chapter 3 and Chapter 5): one bit per heap page meaning "every tuple on this page is visible to every transaction." For each index entry, an Index-Only Scan checks the visibility-map bit for its heap page. If the bit is set, the index value is returned directly. If not, the scan falls back to fetching the heap tuple. `EXPLAIN ANALYZE` counts those fallbacks:

```
Index Only Scan using postings_acct_amt_idx on postings
    (actual time=0.031..9.880 rows=48200 loops=1)
  Index Cond: (account_id = 77)
  Heap Fetches: 48200
```

`Heap Fetches: 48200` out of `rows=48200` means the visibility map was useless and this "index-only" scan visited the heap for every row. The usual cause is a recently bulk-loaded or heavily updated table that `VACUUM` has not yet processed. After `VACUUM postings;` the same query reports `Heap Fetches: 0`. Covering indexes with `INCLUDE` columns, such as `CREATE INDEX ON postings (account_id) INCLUDE (amount_minor)`, exist to make Index-Only Scans possible; vacuum is what makes them actually index-only.

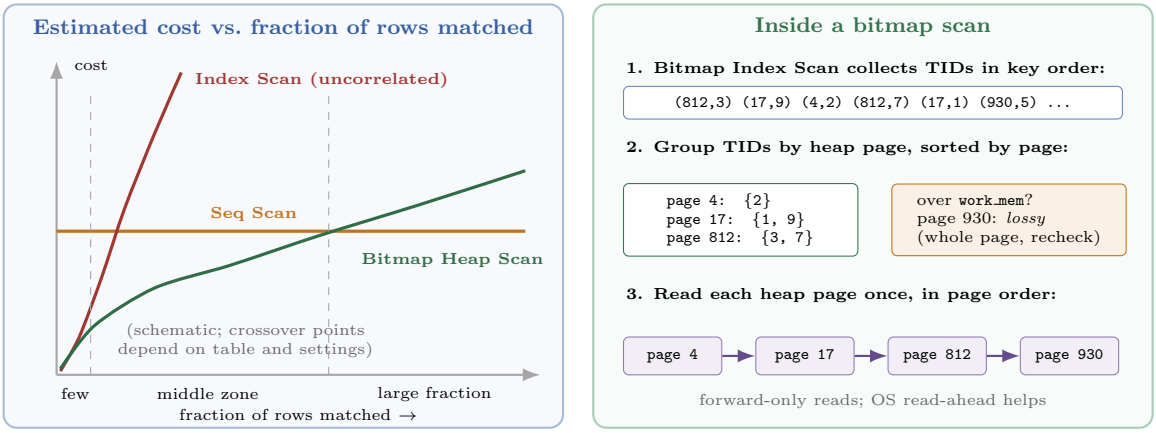


Figure 7.2: Left: as the fraction of matching rows grows, a plain Index Scan on an uncorrelated column becomes expensive quickly because each row may cost a random page read; a sequential scan costs the same regardless; a bitmap scan fills the middle zone. Right: a bitmap scan first collects all matching TIDs, groups them by heap page, and then visits each page once in physical order. If the bitmap outgrows `work_mem`, some pages become lossy and every tuple on them must be rechecked.

7.5 Three Ways to Join Two Tables

A join of two inputs, call them *outer* (left) and *inner* (right), can be computed by three physical algorithms. Each wins in a different situation, and the planner prices all three for every pair of inputs it considers.

You have written all three in Java. Given two `ArrayLists` to match up, you can write a double `for` loop (nested loop join), or replace the inner loop with a lookup in a `TreeMap` you already have (nested loop with an index probe). You can build a `HashMap` from the smaller list and look up each element of the larger one (hash join). Or, if both lists are sorted, you can walk them together with two indexes, as in the merge step of mergesort (merge join). The database versions are the same algorithms, with two extra concerns: the data may not fit in memory, and the planner must choose *before* it knows the real sizes.

7.5.1 Nested Loop

The simplest algorithm: for each outer row, scan the inner input for matches.

```
for each row o in outer:
    for each row i in inner where i.key = o.key:
        emit (o, i)
```

If the inner side is a full scan, the cost is proportional to $N_{\text{outer}} \times N_{\text{inner}}$, which is disastrous for large inputs: a million outer rows against a million inner rows is 10^{12} comparisons. But if the inner side is an *index lookup parameterized by the outer row* (an Index Scan with `Index Cond: (account_id = a.id)`), each inner probe costs one B-tree descent, about $\log M$ page touches, often all cache hits. Then a nested loop costs about $N_{\text{outer}} \times \log N_{\text{inner}}$, and it has a property neither of the other algorithms has: **zero startup cost**. It can return its first joined row after one outer row and one probe. Paired with `LIMIT`, that makes it unbeatable.

The rule of thumb: *a nested loop is the best join when the outer side is small*, and the worst when the outer side is large and the planner thought it was small. That second case is the most common cause of a sudden, catastrophic plan regression.

Since PostgreSQL 14, the planner can insert a `Memoize` node between a nested loop and its inner side. If the same outer key value repeats (many postings for the same account, for example), `Memoize` caches the inner results for that key, so a repeat costs a hash lookup instead of an index probe. `EXPLAIN ANALYZE` reports `Hits`, `Misses`, and `Evictions` for it.

7.5.2 Hash Join

A hash join works in two phases:

1. **Build.** Read the *inner* input (the planner chooses the smaller side) and insert every row into an in-memory hash table keyed on the join column.
2. **Probe.** Stream the *outer* input. For each row, hash its join key and look in one bucket for matches.

Each input is read once, so the cost is proportional to $N_{\text{outer}} + N_{\text{inner}}$. There is no need for any index or any sort order. The cost is memory and startup: no row can be emitted until the entire build side is hashed. The hash join only works for equality conditions (`=`), since hashing destroys order.

The hash table must fit in `work_mem` × `hash_mem_multiplier` (defaults `4MB` and `2.0`, so 8 MB). What if the build side is 400 MB?

Multi-batch (Grace) hash join. PostgreSQL splits the problem into *batches* using some bits of the hash value. Suppose it needs 64 batches. During the build phase, rows whose hash falls in batch 0 go into the in-memory table, and rows belonging to batches 1 through 63 are written to 63 temporary files. During the probe phase, outer rows for batch 0 are probed immediately; outer rows for other batches are written to 63 more temporary files. Then, for each batch $k = 1, \ldots, 63$, PostgreSQL loads inner batch k into memory, streams outer batch k past it, and moves on. Because matching rows have equal hashes, they always land in the same batch, so no match is missed.

The result is still linear in the input sizes, but almost all of both inputs gets written to disk and read back once. That roughly triples the I/O compared with an in-memory hash join, and you can see it in the plan:

```
Hash Join  (actual time=812.4..6411.9 rows=9840211 loops=1)
  Hash Cond: (p.account_id = a.id)
  Buffers: shared hit=2210 read=310118, temp read=96120 written=96120
  ->  Seq Scan on postings p  (actual rows=20000000 loops=1)
  ->  Hash  (actual time=811.9..811.9 rows=3000000 loops=1)
        Buckets: 131072  Batches: 64  Memory Usage: 7937kB
        ->  Seq Scan on accounts a  (actual rows=3000000 loops=1)
```

`Batches: 64` and the matching `temp read`/`written` counts tell you the hash spilled. If the planner *expected* one batch but the build side turned out much larger, you will also see `(originally 1)` next to the batch count, which is a sign of an underestimated row count.

7.5.3 Merge Join

If both inputs arrive sorted on the join key, they can be joined like a zipper. Keep a cursor on each input. Compare the two current keys; advance whichever is smaller; when the keys are equal, emit the matching rows (handling runs of duplicates on both sides) and advance. Each input is read once, so the cost is proportional to $N_{\text{outer}} + N_{\text{inner}}$, with very little memory.

Merge join is attractive when sorted input is free: both sides come from B-tree indexes on the join key, or the query needs the result ordered by that key anyway. Otherwise each side needs a `Sort` node. Sorts report how they ran:

- `Sort Method: quicksort Memory: 3120kB`: the input fit in `work_mem`.
- `Sort Method: top-N heapsort Memory: 26kB`: a `LIMIT` above the sort needs only the best N rows, so PostgreSQL keeps a heap of size N instead of sorting everything.
- `Sort Method: external merge Disk: 412000kB`: the input did not fit; PostgreSQL sorted chunks that fit in memory, wrote them to temporary files as sorted runs, and merged the runs.

Merge join also handles range and inequality join conditions that a hash join cannot, though in practice those are rare.

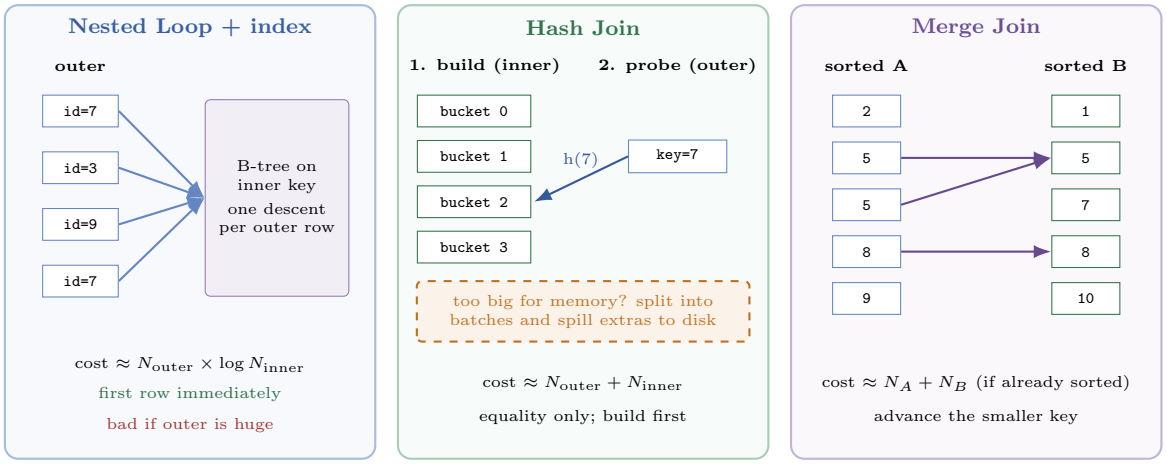


Figure 7.3: The three physical join algorithms. A nested loop with an index probe is ideal for a small outer input and returns rows immediately. A hash join reads each input once and needs memory for the build side, spilling to batches on disk when that memory runs out. A merge join walks two sorted inputs in step and is cheapest when an index or the query's `ORDER BY` already provides the sort.

Focus

Which join will the planner pick? A rough guide that matches most real plans:

- Small outer side (up to a few thousand rows) and an index on the inner join key: **Nested Loop**.
- Two large inputs, equality join, no useful order: **Hash Join**.
- Both inputs already ordered on the join key, or the output must be ordered by it: **Merge Join**.

Every one of these decisions depends on the estimated row counts. If the estimate for the outer side is 10 and the truth is 500,000, the planner will choose a nested loop that does 500,000 index probes. That is why reading a plan starts with comparing `rows` estimates against actual rows.

7.6 Searching Join Orders

A query that joins n tables can be evaluated in many orders. How many? Count the binary join trees whose leaves are the n tables. The shapes of binary trees with n leaves are counted by the Catalan number C_{n-1}, and the tables can be assigned to the leaves in $n!$ ways, so there are $n! \cdot C_{n-1}$ trees:

n (tables)	$n! \cdot C_{n-1}$ (join trees)	2^n (table subsets)
3	12	8
4	120	16
8	17,297,280	256
12	about 2.8×10^{13}	4,096

Each tree also needs a join algorithm at every node and an access path at every leaf. Enumerating trees one at a time is hopeless past a handful of tables. The third column shows the way out.

7.6.1 Dynamic Programming over Subsets

PostgreSQL's standard join search, inherited from IBM's System R optimizer (Selinger et al., 1979), builds plans bottom-up over *sets of tables*:

1. **Level 1:** for each single table, find the cheapest access path (Section 7.4).
2. **Level 2:** for each pair of tables $\{A, B\}$ that are connected by a join condition, consider joining the best plan for $\{A\}$ with the best plan for $\{B\}$, in both orders and with all three join algorithms. Keep the cheapest plan for the set $\{A, B\}$.
3. **Level k:** for each set S of k tables, consider every way to split S into two smaller sets $S_1 \cup S_2$ whose best plans are already known. Keep the cheapest plan for S.

The key insight is that the best plan for $\{A, B, C\}$ only needs the best plans for its subsets, not every tree that produces them. So the work grows with the number of subsets, 2^n, instead of the number of trees. (A refinement: the planner keeps more than one plan per set when a more expensive plan produces a useful sort order, because a sorted result might save a `Sort` or enable a merge join higher up. These are called *interesting orders*.)

7.6.2 Limits and the Genetic Optimizer

2^n is still exponential. Two settings keep planning time in check:

- `join_collapse_limit` and `from_collapse_limit` (default 8): the planner only reorders up to 8 items freely. Beyond that, explicit `JOIN` syntax order and subquery boundaries are partially respected as written.
- `geqo_threshold` (default 12): when a query has 12 or more items in `FROM`, PostgreSQL switches to the *Genetic Query Optimizer* (GEQO). GEQO treats a join order as a chromosome, generates a population of random orders, and repeatedly combines the cheaper ones. It finds a good plan quickly but not necessarily the best one, and because it is randomized, plans for very large joins can vary between runs (set `geqo_seed` to make them reproducible).

In production

ORMs and reporting tools sometimes generate queries joining 15 or 20 tables, many of them small lookup tables. If such a query plans slowly or chooses erratic plans, check the `Planning Time` line in `EXPLAIN ANALYZE`. Splitting the query, or writing explicit `JOIN`s in a sensible order and lowering `join_collapse_limit` for that session, gives you a stable plan and a planning time measured in milliseconds rather than seconds.

7.7 Reading `EXPLAIN (ANALYZE, BUFFERS)` Line by Line

Plain `EXPLAIN` shows the plan the planner chose and its predictions. `EXPLAIN ANALYZE` *runs* the query and adds what actually happened at every node. `BUFFERS` adds how many pages each node touched and where they came from. Use all three together.

Common trap

`EXPLAIN ANALYZE` really executes the statement. `EXPLAIN ANALYZE DELETE FROM postings WHERE ...` deletes the rows. Wrap data-modifying statements in a transaction you roll back:

```
BEGIN;
EXPLAIN (ANALYZE, BUFFERS) DELETE FROM postings WHERE ...;
ROLLBACK;
```

7.7.1 A Complete Plan

Here is the plan for the dashboard query from the start of the chapter:

```
Limit  (cost=48211.20..48211.23 rows=10 width=48)
       (actual time=212.441..212.446 rows=10 loops=1)
  Buffers: shared hit=40213 read=6120
  ->  Sort  (cost=48211.20..48236.20 rows=10000 width=48)
            (actual time=212.439..212.442 rows=10 loops=1)
        Sort Key: (sum(p.amount_minor)) DESC
        Sort Method: top-N heapsort  Memory: 26kB
        Buffers: shared hit=40213 read=6120
        ->  HashAggregate
              (cost=47745.10..47995.10 rows=10000 width=48)
              (actual time=205.118..209.870 rows=11842 loops=1)
              Group Key: a.id
```

```
               Batches: 1  Memory Usage: 2577kB
               Buffers: shared hit=40213 read=6120
               ->  Nested Loop
                     (cost=0.86..46120.40 rows=320000 width=24)
                     (actual time=0.071..151.904 rows=296050 loops=1)
                     Buffers: shared hit=40213 read=6120
                     ->  Index Scan using accounts_region_idx on accounts a
                           (cost=0.42..1210.40 rows=10000 width=24)
                           (actual time=0.031..6.211 rows=11842 loops=1)
                           Index Cond: (region = 'EU'::text)
                           Buffers: shared hit=310 read=95
                     ->  Index Scan using postings_acct_created_idx
                           on postings p
                           (cost=0.44..4.17 rows=32 width=16)
                           (actual time=0.004..0.011 rows=25 loops=11842)
                           Index Cond: ((account_id = a.id) AND
                                        (created_at >= '2026-10-01'))
                           Buffers: shared hit=39903 read=6025
Planning Time: 0.912 ms
Execution Time: 212.533 ms
```

Read from the inside out. Execution starts at the most indented nodes and flows upward. The two `Index Scan` nodes are the leaves; the `Nested Loop` joins them; `HashAggregate` groups; `Sort` orders; `Limit` stops after 10 rows. Figure 7.4 draws the same tree.

Estimated versus actual. Every node prints two parentheses. The first, `cost=... rows... width=...`, is the planner's prediction. The second, `actual time=... rows=... loops=...`, is the measurement. Compare `rows` in each pair. For the EU accounts the planner predicted 10,000 and got 11,842: an error of 18%, which is excellent. For the postings per account it predicted 32 and got 25. These estimates are good, so the plan is probably good.

`actual time` is in milliseconds, per loop. `actual time=0.031..6.211` means the node returned its first row after 0.031 ms and its last row after 6.211 ms. Unlike `cost`, these are real milliseconds.

The `loops` multiplier. Look at the inner index scan on `postings`: `actual time=0.004..0.011 rows=25 loops=11842`. This node ran 11,842 times, once for each EU account from the outer side. **Both `actual time` and `rows` are averages per loop**, so the totals are

$$\text{rows} = 25 \times 11{,}842 = 296{,}050, \qquad \text{time} \approx 0.011 \text{ ms} \times 11{,}842 \approx 130 \text{ ms}.$$

That 130 ms is most of the query's 212 ms. If you read only the per-loop number, you would conclude this node costs 0.011 ms and is irrelevant. It is actually the most expensive node in the plan. Whenever you see `loops` greater than 1, multiply.

Buffers. `Buffers: shared hit=39903 read=6025` on the inner scan means it touched 45,928 pages in total: 39,903 were already in `shared_buffers` (`hit`), and 6,025 had to be requested from

the operating system (`read`). A `read` may still be served from the OS page cache rather than the disk; `BUFFERS` cannot tell the difference, but `track_io_timing = on` adds an `I/O Timings` line that can. Other counters you will meet:

- `dirtied`: pages this node modified in shared buffers. On a `SELECT`, this usually means hint bits were set (Chapter 5).
- `written`: dirty pages this backend had to write out itself to make room, a sign that `shared_buffers` or the background writer is struggling.
- `temp read` / `temp written`: pages spilled to temporary files by sorts, hashes, and materialization. Any non-zero value here is worth a look at `work_mem`.

Buffer counts are cumulative: a parent's count includes its children's. Here every node above the nested loop shows the same totals because none of them reads any pages itself.

Planning and execution time. `Execution Time` covers running the plan, but not sending rows to the client. `Planning Time` covers the planner's search. For most OLTP queries, planning should be well under a millisecond; if it rivals execution time, prepared statements (Chapter 8) or simpler queries will help.

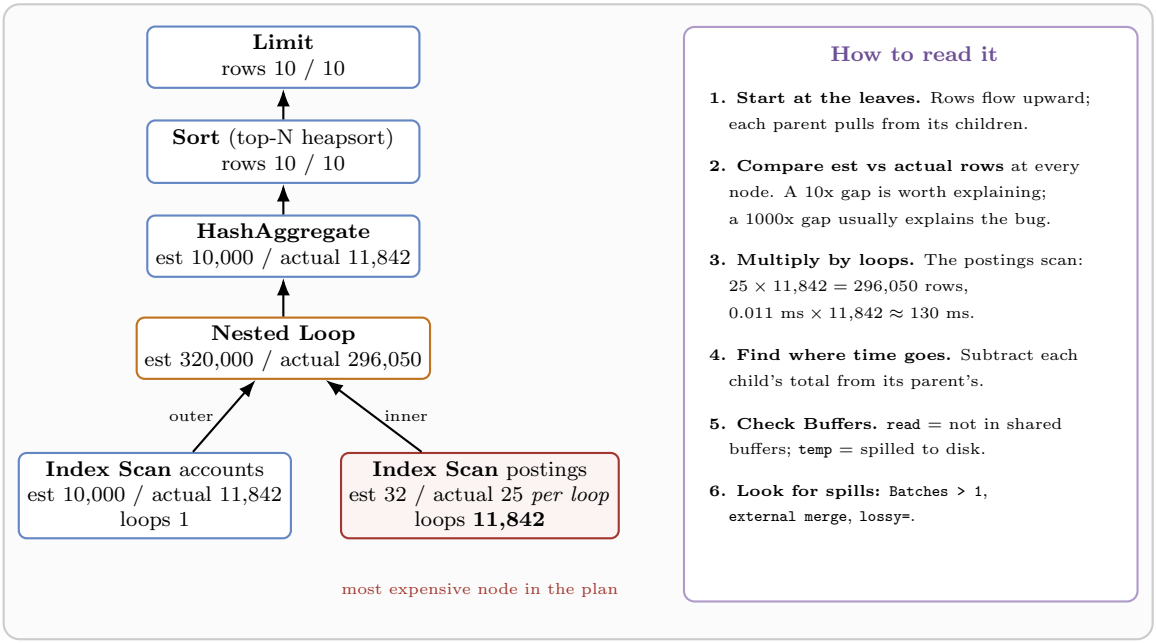


Figure 7.4: The dashboard plan as a tree. Each box shows estimated and actual rows. The inner index scan on `postings` looks cheap per loop but runs 11,842 times and accounts for most of the execution time.

7.7.2 Diagnosing a Bad Plan

Almost every slow plan comes down to one of a few causes. Here is how each shows up and how to fix it.

1. Stale statistics. A table received a large batch of rows, but autovacuum has not run `ANALYZE` yet. Typical symptom: a node estimates `rows=1` or a few hundred and actually produces hundreds of thousands, often on a predicate over recent dates that lie beyond the last histogram bound. Fix: run `ANALYZE table;` immediately after bulk loads, and lower `autovacuum_analyze_scale_factor` for large, fast-growing tables.

2. Correlated predicates. Two predicates on related columns are multiplied as if independent, and the estimate comes out orders of magnitude too low (Section 7.2). Symptom: the estimate is far too small exactly at the node that applies both filters. Fix: `CREATE STATISTICS ... (dependencies, mcv)` on those columns, then `ANALYZE`.

3. Opaque expressions. A function wraps an indexed column (`date_trunc`, `lower`, a cast), so the planner falls back to default selectivities and cannot use a plain index. Symptom: `Seq Scan` with a `Filter:` line containing the function, and a round-number estimate such as 0.5% of the table. Fix: rewrite as a range on the raw column, or add an expression index.

4. Too little memory. Sorts, hashes, and bitmaps spill. Symptom: `Sort Method: external merge`, `Batches:` above 1, `lossy=` in `Heap Blocks`, and non-zero `temp read/written`. Fix: raise `work_mem` for that query or role with `SET LOCAL`. Be careful raising it globally: each sort or hash node in each concurrent query may use that much, so 200 connections times 3 nodes times 256 MB is 150 GB.

5. Wrong cost constants for the hardware. The planner avoids index scans on NVMe storage that would serve random reads nearly as fast as sequential ones. Symptom: a sequential scan is chosen, but `SET enable_seqscan = off` in a test session produces a much faster index plan. Fix: set `random_page_cost = 1.1` and `effective_cache_size` to roughly the memory available for caching (often 50–75% of RAM). Use `enable_*` settings only for diagnosis, never as a permanent fix.

Example 7.5 (A thousand-fold misestimate)**.** A fraud report filters `transfers` by `city = 'Lyon' AND postal_code = '69001'` and joins to `accounts`:

```
Nested Loop  (cost=0.85..412.66 rows=20 width=64)
             (actual time=0.090..48711.302 rows=19874 loops=1)
  ->  Seq Scan on transfers t  (cost=0.00..250.10 rows=20 width=40)
        (actual time=0.041..911.402 rows=19874 loops=1)
        Filter: ((city = 'Lyon') AND (postal_code = '69001'))
        Rows Removed by Filter: 19980126
  ->  Index Scan using accounts_pkey on accounts a
        (cost=0.43..8.12 rows=1 width=24)
        (actual time=2.401..2.405 rows=1 loops=19874)
        Index Cond: (id = t.account_id)
```

The scan on `transfers` estimated 20 rows and produced 19,874: almost exactly the factor of 1,000 that independence predicts when one predicate implies the other. On a 20-row estimate, a nested loop with 20 index probes is a sensible choice. With 19,874 probes, each a cold random read of about 2.4 ms (multiply by `loops`: $2.4 \times 19{,}874 \approx 47.7$ seconds), it is not. After `CREATE STATISTICS addr_stats (dependencies) ON city, postal_code FROM transfers; ANALYZE transfers;`

the estimate becomes about 20,000, the planner switches to a hash join over `accounts`, and the query finishes in under two seconds.

Try it in PostgreSQL

Build intuition with a table you control:

```
CREATE TABLE t AS
SELECT g AS id,
       (random() * 1000)::int AS rnd,
       g / 1000 AS seq
  FROM generate_series(1, 2000000) g;
CREATE INDEX ON t (rnd);
CREATE INDEX ON t (seq);
ANALYZE t;

SELECT attname, n_distinct, correlation
  FROM pg_stats WHERE tablename = 't';

EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM t WHERE rnd < 5;
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM t WHERE seq < 10;
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM t WHERE rnd < 300;
```

Both of the first two predicates match about 10,000 rows. Notice how differently they are planned and how many buffers each touches, and connect what you see to the `correlation` values. Then set `work_mem = '64kB'` and rerun the `rnd < 300` query to make the bitmap go lossy.

The planner turns SQL into a plan; MVCC and locking keep concurrent plans correct; WAL keeps their results durable. In Chapter 8 we put all of it to work: designing financial schemas whose constraints make invalid states unrepresentable, replicating them across machines, and serving them to thousands of application tasks through a connection pooler.

7.8 Exercises

Exercise 7.1 (Selectivity from `pg_stats`)**.** A column `country` on a 4,000,000-row table has `null_frac = 0.01`, `n_distinct = 60`, `most_common_vals = {US,GB,DE,FR}`, and `most_common_freqs = {0.40,0.15,0.10,0.08}`. Estimate the rows returned by (a) `country = 'DE'`, (b) `country = 'NZ'`, and (c) `country IN ('GB','NZ')`. Show each step.

Exercise 7.2 (Histogram interpolation)**.** A column has `null_frac = 0`, no MCV list, and `histogram_bounds = {10,20,40,80,160,320}` on a 1,000,000-row table. Estimate the rows for (a) `x < 60`, (b) `x > 200`, and (c) `x BETWEEN 15 AND 100`. Why does an equi-depth histogram give better estimates than an equi-width one when the data is skewed?

Exercise 7.3 (Pricing scans by hand)**.** A table occupies 25,000 pages and holds 2,000,000 rows. Using the default cost constants: (a) Compute the total cost of a sequential scan with a single `WHERE` predicate. (b) A predicate keeps 0.5% of rows on a column with correlation 0. Use the Mackert–Lohman formula to estimate the heap I/O cost of an Index Scan, and compare it with (a). (c) Repeat (b) for correlation 0.95. (d) Repeat (b) and (c) with `random_page_cost = 1.1`. Which decisions change?

Exercise 7.4 (Startup cost and `LIMIT`)**.** Plan A for a query has `cost=0.43..90000.00 rows=1000000`. Plan B has `cost=21000.00..23500.00 rows=1000000`. Which plan does the planner prefer with no `LIMIT`? With `LIMIT 50`? Find the `LIMIT` value at which the two plans cost the same, and describe what kind of physical operators would produce each cost profile.

Exercise 7.5 (Reading a lossy bitmap)**.** A plan reports `Heap Blocks: exact=4000 lossy=36000` and `Rows Removed by Index Recheck: 2100000` for a query that returns 90,000 rows. (a) Explain what happened in memory. (b) Roughly how many tuples per page does this table hold? (c) Predict how the `Heap Blocks` line and the `Rows Removed` line change after `SET LOCAL work_mem = '256MB'`, and explain why the number of heap pages read does not change.

Exercise 7.6 (Choosing a join algorithm)**.** For each case, say which join algorithm you expect and why: (a) 50 rows from `orders` joined to `customers` on its primary key; (b) 30 million `postings` joined to 3 million `accounts` for a monthly report, `work_mem = 64MB`; (c) the same as (b) with `ORDER BY account_id` and B-tree indexes on both join columns; (d) the same as (a), but the planner estimated 50 rows and the true count is 2 million.

Exercise 7.7 (Multi-batch hash join arithmetic)**.** The build side of a hash join has 6,000,000 rows of 56 bytes each, and the hash memory limit is `work_mem` $\times$ `hash_mem_multiplier` $= 16$ MB. (a) Estimate the number of batches (a power of two). (b) What fraction of the build and probe inputs is written to temporary files? (c) If the probe side is 40,000,000 rows of 48 bytes, estimate the total temporary-file bytes written. (d) How large must `work_mem` be for a single batch?

Exercise 7.8 (Counting join orders)**.** (a) Verify the table in Section 7.6 for $n = 3$ by listing all 12 join trees of tables A, B, C. (b) If only *left-deep* trees are allowed (every right child is a single table), how many trees are there for n tables? (c) Explain why dynamic programming over subsets needs to examine only 2^n sets, and estimate the number of (set, split) pairs it considers for $n = 10$.

Exercise 7.9 (Find the slow node)**.** In the plan below, compute the total rows and total time contributed by the inner node, identify which node is responsible for most of the runtime, and propose two different fixes.

```
Nested Loop  (actual time=0.05..8120.4 rows=420000 loops=1)
  ->  Index Scan using invoices_due_idx on invoices i
        (cost=0.43..95.12 rows=40 width=16)
        (actual time=0.02..58.1 rows=60000 loops=1)
        Index Cond: (due_date < '2026-11-01')
  ->  Index Scan using lines_invoice_idx on invoice_lines l
        (cost=0.43..12.30 rows=7 width=24)
        (actual time=0.010..0.133 rows=7 loops=60000)
        Index Cond: (invoice_id = i.id)
```

Exercise 7.10 (Index-only scans that are not)**.** A query on `(account_id) INCLUDE (amount_minor)` reports `Index Only Scan ... rows=12000` with `Heap Fetches: 11800`. (a) Explain what the visibility map has to do with this. (b) Give two operational changes that would bring `Heap Fetches` near zero. (c) Why can an Index-Only Scan never skip the visibility check entirely, even when the index contains every column the query needs?

Chapter 8

Financial Schemas, Replication, and Connection Pooling

For seven chapters we have worked inside the PostgreSQL engine room. We walked 8 KB slotted heap pages and buffer descriptors in Chapter 2, split Lehman–Yao B-link trees and generalized search trees in Chapter 3, flushed write-ahead log records and full-page images in Chapter 4, traced multiversion snapshots and `VACUUM` in Chapter 5, reasoned about row-level locks and serializable anomalies in Chapter 6, and read cost-based execution plans in Chapter 7. Now we sit at the architect's desk and build a system where every one of those low-level mechanisms has a dollar sign attached to it: a production double-entry financial ledger served by an asynchronous Rust (`Axum` + `SQLx`) API over PostgreSQL.

Financial systems punish vague thinking because every shortcut fails in a specific, observable way. Store balances in binary floating point and sums of fee postings stop matching the books to the cent. Store wall-clock timestamps without a UTC reference and the autumn daylight-saving fallback folds two distinct hours onto the same nominal minute. Choose random `UUIDv4` primary keys and a sixty-gigabyte B-tree turns small inserts into an 8 KB full-page-write storm across cold NVMe blocks. Validate invariants only in Rust handlers and a single concurrent retry or backfill script creates money out of thin air. Leave replication asynchronous and a primary disk failure erases transfers you already confirmed to customers; point five thousand Tokio tasks directly at PostgreSQL and shared-memory snapshot contention brings a thirty-two-core server to its knees.

In this final chapter, we state every architectural rule by starting with the exact production failure it prevents, then trace how PostgreSQL's storage, indexing, WAL, locking, and wire-protocol internals enforce the guarantee.

Focus

By the end of this chapter you will be able to:

- Select exact storage types for money (`BIGINT` minor units versus base-10,000 `NUMERIC`), instants in time (`TIMESTAMPTZ`), and high-throughput keys (time-ordered `UUIDv7` versus `UUIDv4`).
- Encode domain invariants directly in the schema with `NOT NULL`, `CHECK`, idempotency keys, and GiST temporal exclusion constraints (`EXCLUDE USING gist`).

- Implement an append-only double-entry ledger enforced at `COMMIT` time by a `DEFERRABLE INITIALLY DEFERRED` constraint trigger, paired with deadlock-free Rust (`Axum` + `SQLx`) transfer code.
- Partition multi-billion-row posting tables by time range for warm buffer locality and zero-bloat retention archival.
- Tune physical streaming replication across the `synchronous_commit` durability ladder and stream decoded row events with logical replication.
- Multiplex thousands of async Rust tasks onto dozens of PostgreSQL backend processes through `PgBouncer` transaction pooling without breaking prepared statements or session state.

8.1 Data Types That Never Lie About Money, Time, or Identity

A relational schema is a physical contract about the bit patterns allowed onto 8 KB heap pages. If a column's type rounds fractions, discards UTC context, or scatters keys uniformly across disk, no amount of application-layer discipline can undo the damage downstream.

8.1.1 Money: Why Binary Floats Fail, and `BIGINT` vs. `NUMERIC`

Suppose a payment platform stores account adjustments in PostgreSQL's 8-byte `DOUBLE PRECISION` type (`float8`, matching `f64` in Rust and `double` in Java or C++). A merchant collects a \$0.10 platform fee and a \$0.20 network fee, expecting a \$0.30 total. Ask PostgreSQL whether the two sides match:

```
SELECT (0.1::float8 + 0.2::float8) = 0.3::float8 AS exact_match,
       (0.1::float8 + 0.2::float8) - 0.3::float8 AS drift;
-- exact_match: false | drift: 5.551115123125783e-17
```

The comparison returns `false` because IEEE 754 binary floating point represents numbers as $(-1)^s \times m \times 2^e$ with a 53-bit binary significand m. In base 2, one tenth is an infinite repeating fraction, $0.1_{10} = 0.0001100110011\overline{0011}_2$, just as $1/3 = 0.333\ldots_{10}$ never terminates in base 10. Rounding at bit 53 introduces an error of rough magnitude 10^{-17} on a single addition. Sum ten million fee postings across parallel `HashAggregate` workers (Chapter 7), where summation order varies from run to run, and two identical reconciliation queries can disagree by cents. Binary floats (`REAL` and `DOUBLE PRECISION`) are therefore strictly forbidden for monetary quantities.

PostgreSQL gives you two exact alternatives, each suited to a clear regime:

1. **`BIGINT` (`int8`) minor units.** Instead of storing dollars as fractions, store integer counts of the asset's smallest indivisible unit: cents (10^{-2} USD), micro-dollars (10^{-6} USD) for sub-cent API billing, or satoshis (10^{-8} BTC). On disk, `BIGINT` occupies a fixed 8 bytes on the heap page, maps directly to `i64` in Rust or `long` in Java, and adds or compares in a single CPU integer instruction. Its signed range reaches $\pm(2^{63} - 1) \approx \pm 9.22 \times 10^{18}$: in USD cents that exceeds \$92 quadrillion; in micro-dollars it exceeds \$9.22 trillion. Whenever you store minor

units, store the asset's ISO code or integer exponent alongside it so nobody adds Japanese yen (10^0) to US cents (10^{-2}) or Kuwaiti dinar fils (10^{-3}).

2. `NUMERIC(p, s)` **(equivalently** `DECIMAL(p, s)`**).** Foreign-exchange rates (`1.08437525`), fractional equity shares (`0.00418275` shares), and multi-asset bond accruals often combine precision p up to dozens of digits with varying decimal scales s, or multiply two high-precision figures where an intermediate 64-bit integer would overflow. Internally (`NumericVar` in PostgreSQL's `numeric.c`), `NUMERIC` is a variable-length (`varlena`) structure that packs *four base-10 digits into each 16-bit integer word* (base 10,000), paired with a compact header storing the base-10,000 weight, sign, and display scale. Because every power of 10^{-1} terminates in base 10,000, `0.1::numeric + 0.2::numeric = 0.3::numeric` is exact to every declared digit. The tradeoff is CPU and space: `NUMERIC` carries a variable-length header (typically 8–16 bytes on disk for moderate values) and executes digit loops in software rather than a single ALU instruction, making heavy `SUM(amount)` scans roughly 5× to 15× more CPU-intensive than `SUM(amount_minor)` on `BIGINT`. In Rust `SQLx`, `NUMERIC` binds cleanly to `rust_decimal::Decimal` (128-bit fixed decimal) or `bigdecimal::BigDecimal`.

For single-scale ledger postings and account balances, `BIGINT` minor units give unbeatable speed and simplicity; for FX rates, unit prices, and tax or interest multipliers, `NUMERIC(p, s)` prevents premature rounding and 64-bit intermediate overflow.

8.1.2 Time: `TIMESTAMPTZ` and the Daylight-Saving Trap

At 01:59:59 Eastern Daylight Time on the first Sunday of November, clocks in New York jump backward to 01:00:00 Eastern Standard Time. Suppose a ledger table uses `TIMESTAMP WITHOUT TIME ZONE` (which SQL abbreviates as plain `TIMESTAMP`). A customer wires funds at 01:15 EDT (`05:15 UTC`), and fifty minutes later wires a second payment at 01:05 EST (`06:05 UTC`). Stored as wall-clock New York `TIMESTAMP` values, the second transfer appears to happen *ten minutes before* the first: `'2026-11-01 01:05:00'` sorts ahead of `'2026-11-01 01:15:00'`. Worse, if one API node runs with `TimeZone = 'UTC'` and an admin tool connects with `TimeZone = 'America/Chicago'`, `TIMESTAMP WITHOUT TIME ZONE` strips and ignores timezone offsets on input, mixing incompatible local clocks in the same column with zero warning.

Intuition

Despite its verbose SQL name, `TIMESTAMP WITH TIME ZONE` (`TIMESTAMPTZ`) stores **zero timezone string or UTC offset on disk**. Both `TIMESTAMP` and `TIMESTAMPTZ` occupy the exact same 8 bytes in a heap tuple: a signed 64-bit integer counting microseconds since PostgreSQL's epoch, `2000-01-01 00:00:00 UTC`. The sole difference happens at the input and output boundary. When you insert `'2026-11-01 01:15:00-04'` into a `TIMESTAMPTZ` column, PostgreSQL immediately converts that instant into UTC microseconds and stores that single universal point on the physical timeline. On `SELECT`, it formats those UTC microseconds into whatever session `TimeZone` the reader requested.

The rule for financial schemas has only two cases:

- Every physical instant when an event occurred (`created_at`, `posted_at`, `expires_at`) must use `TIMESTAMPTZ`, mapped in Rust `SQLx` to `DateTime<Utc>` (from `chrono`) or `OffsetDateTime` (from `time`) with session `TimeZone` set to `'UTC'`.

- Calendar dates defined by legal contract rather than a physical instant (such as a wire's settlement `value_date = '2026-11-02'` or a statement billing date) should use 4-byte `DATE`, never a midnight timestamp that shifts to the previous calendar day when viewed in another timezone.

8.1.3 Primary Keys at Scale: Why `UUIDv4` Keys Thrash B-Trees and `UUIDv7` Flies

An 8-byte `BIGINT GENERATED ALWAYS AS IDENTITY` sequence appends strictly to the right edge of its primary-key B-tree, keeping inserts fast and compact. Yet distributed financial systems rarely expose sequential integers as external IDs: sequential IDs leak business volume to competitors (`entry_id = 1042` versus `1089` an hour later) and force Rust services or mobile clients to make a database round-trip before knowing a new resource's identifier. That pushes teams toward 128-bit `UUID` keys (stored natively in PostgreSQL's 16-byte `uuid` type, never as 36-byte ASCII `TEXT`).

Here is the failure mode that catches growing ledgers six months after launch. You choose random `UUIDv4` (122 cryptographically random bits). At five million rows, the primary-key B-tree is roughly 150 MB, sits comfortably inside an 8 GB `shared_buffers` pool (Chapter 2), and sustains 35,000 inserts per second. A year later, `postings` reaches two billion rows and its primary B-tree spans 65 GB on a server with 32 GB of RAM. Insert throughput drops by an order of magnitude while NVMe write bandwidth pins at 100%, even though each new posting row is barely a hundred bytes. Three mechanisms from Chapters 2–4 combine to cause this collapse:

1. **Buffer-pool cache misses.** Every `UUIDv4` lands uniformly at random across the entire $[0, 2^{128})$ keyspace. When a 65 GB leaf layer exceeds `shared_buffers`, nearly every single `INSERT` probes a cold leaf page not in memory, stalling on a random 8 KB disk read and evicting another page.

2. **Midpoint page-split bloat.** As we saw in Chapter 3, random B-tree inserts split full leaf pages 50/50, leaving average leaf occupancy near 70% instead of the 90% fill PostgreSQL keeps when it detects monotonically increasing right-edge inserts.

3. **WAL `full_page_writes` amplification.** Recall from Chapter 4 that the first modification to *any* 8 KB heap or index page after a `CHECKPOINT` must write the entire 8 KB Full Page Image (FPI) into the WAL stream to survive torn writes. After a checkpoint, ten thousand random `UUIDv4` inserts dirty roughly ten thousand distinct B-tree leaf pages, turning a few hundred kilobytes of new index entries into about $10{,}000 \times 8$ KB ≈ 80 MB of WAL.

RFC 9562 `UUIDv7` (available natively as `uuidv7()` in PostgreSQL 18 and via `uuid::Uuid::now_v7()` in Rust) eliminates all three penalties without reintroducing central sequence contention. Its first 48 bits encode the big-endian Unix timestamp in milliseconds, followed by a 4-bit version tag (`0111`), 12 bits of sub-millisecond monotonic or random data (`rand_a`), a 2-bit variant (`10`), and 62 bits of cryptographic randomness (`rand_b`). Because PostgreSQL compares 16-byte `uuid` values lexicographically from most-significant byte to least, every `UUIDv7` minted during the current millisecond sorts to the *far right* of the entire historical B-tree (Figure 8.1). Concurrent Rust pods mint collision-free IDs offline, yet all inserts converge onto a handful of hot rightmost leaf pages pinned permanently in `shared_buffers`.

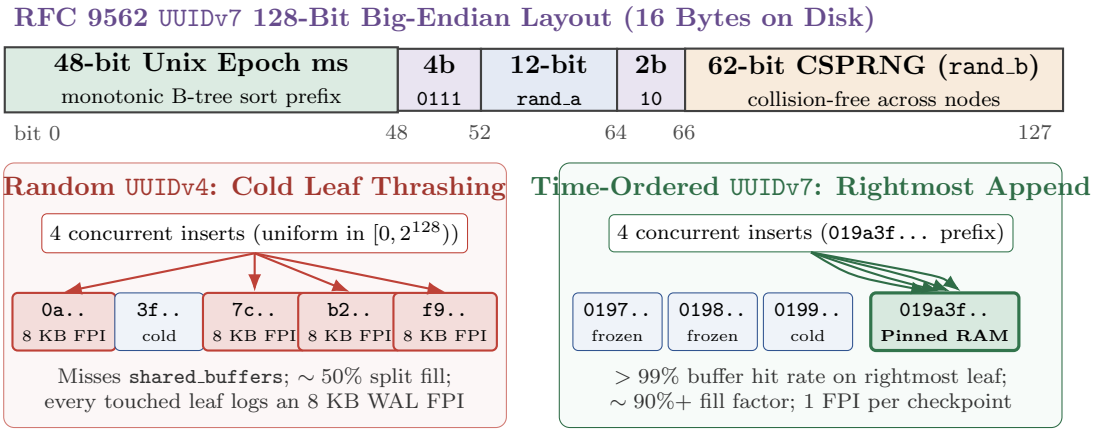


Figure 8.1: Bit layout of RFC 9562 `UUIDv7` (top) and its effect on a multi-gigabyte B-tree (bottom). Random `UUIDv4` keys scatter writes across cold disk pages and amplify post-checkpoint WAL full-page images, whereas `UUIDv7` funnels concurrent inserts onto the hot rightmost leaf in `shared_buffers`.

8.2 Encoding Domain Invariants Directly in the Schema

"Our Rust service validates every request struct before calling SQL, so the database only needs plain columns." That sentence precedes nearly every silent data-corruption postmortem. Application code is only one doorway into a database: settlement batch jobs, emergency `psql` backfills, schema migrations, and concurrent network retries all write to the same tables. Worse, under `READ COMMITTED` isolation (Chapter 6), two Rust handlers that validate against separate snapshots can each see valid preconditions and still commit a combined state that violates business law. Pushing invariants into PostgreSQL turns application bugs from silent ledger corruption into loud, rolled-back `ERROR` responses.

8.2.1 `NOT NULL`, `CHECK`, Domains, and Atomic Idempotency Keys

Recall from Chapter 1 how three-valued logic treats `NULL` inside a `CHECK` constraint: a row passes `CHECK (expr)` whenever `expr` evaluates to `TRUE` *or* `UNKNOWN`. If you write `amount_minor BIGINT CHECK (amount_minor > 0)` without `NOT NULL`, inserting `NULL` evaluates `NULL > 0` to `UNKNOWN` and lands cleanly on the heap page. Always pair `NOT NULL` with `CHECK`.

For constrained text categories, choose deliberately between three tools:

- `CREATE TYPE ... AS ENUM` stores each label as a compact 4-byte OID on disk and excels for closed state machines whose values are tied to code branches (`'pending'`, `'posted'`, `'voided'`). However, PostgreSQL enums cannot drop obsolete labels without recreating the type.
- **`DOMAIN`s or `CHECK`-constrained `TEXT`** (such as `CHAR(3)` checked against `'^[A-Z]{3}$'`) validate format uniformly across dozens of tables while allowing new currencies without DDL catalog locks. When currencies carry per-asset metadata such as minor-unit scale (2 for `USD`, 0 for `JPY`), a foreign key to a small `assets` table (`currency`, `minor_scale`) stays permanently cached in `shared_buffers`.

- **Storage-enforced idempotency keys.** When a mobile app or upstream gateway sends `POST /v1/transfers` and its TCP socket times out while waiting for the HTTP response, the client cannot tell whether the commit succeeded; it *must* retry with the same client-generated `idempotency_key`. Checking `SELECT 1 FROM journal_entries WHERE client_id = $1 AND idempotency_key = $2` in Rust before inserting suffers a classic Time-of-Check to Time-of-Use race if two retries arrive simultaneously. Declaring `UNIQUE (client_id, idempotency_key)` makes PostgreSQL's B-tree uniqueness machinery (`_bt_check_unique`) inspect even uncommitted in-flight index tuples, block the second retry on the first transaction's lock, and resolve atomically via `ON CONFLICT DO NOTHING`.

8.2.2 Temporal Non-Overlapping Invariants with `EXCLUDE USING gist`

Suppose each savings or margin account has a tiered interest-rate schedule valid over a time interval. Two administrative jobs concurrently insert schedules for account A: one sets 4.50% over $[\text{Jun } 1, \text{Dec } 1)$ and another sets 5.00% over $[\text{Sep } 1, \text{Mar } 1)$. A nightly accrual query joins `accounts` to `account_rate_schedules WHERE now() <@ effective_during`, matches *both* rows throughout autumn, and pays 9.50% interest. No `UNIQUE` constraint on `(account_id, effective_from)` can catch partial interval overlap because the start timestamps differ.

Chapter 3 introduced Generalized Search Trees (GiST) as balanced trees over arbitrary bounding predicates rather than total orders. Paired with the bundled `btree_gist` extension (which teaches GiST to test equality `=` on ordinary scalar types alongside range overlap `&&`), PostgreSQL's **exclusion constraint** rejects any two rows where *all* listed operator comparisons return true simultaneously:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE account_rate_schedules (
    schedule_id      UUID PRIMARY KEY DEFAULT uuidv7(),
    account_id       UUID NOT NULL,
    annual_bps       INTEGER NOT NULL CHECK (annual_bps BETWEEN 0 AND 10000),
    effective_during TSTZRANGE NOT NULL,
    CONSTRAINT non_empty_window CHECK (NOT isempty(effective_during)),
    CONSTRAINT no_overlapping_rates EXCLUDE USING gist (
        account_id       WITH =,
        effective_during WITH &&
    )
);
```

Notice the range type `TSTZRANGE` and PostgreSQL's canonical half-open convention `'[)'` (closed lower bound, open upper bound). When an old rate schedule `[2026-01-01Z, 2026-06-01Z)` (constructed via `tstzrange`) ends at midnight June 1 and its replacement `[2026-06-01Z, infinity)` begins at that exact microsecond, the two intervals touch with zero gap yet share zero instants, so `&&` evaluates to `false`. Even under `READ COMMITTED`, two concurrent transactions inserting overlapping windows for the same `account_id` collide inside the GiST index just like a duplicate primary key: one commits and the other aborts with SQLSTATE `23P01` (`exclusion_violation`).

8.3 A Double-Entry Financial Ledger in PostgreSQL and Rust

Beginners often model money with a single mutable column: `UPDATE accounts SET balance = balance - 5000 WHERE id = $1` followed by `UPDATE accounts SET balance = balance + 5000 WHERE id = $2`. Three production hazards make that design unacceptable for a real ledger:

1. **Destroyed history.** Overwriting `balance` in place erases *why* the balance changed, making regulatory audits, dispute reversals, and point-in-time historical reconstructions impossible once `VACUUM` reclaims old tuple versions (Chapter 5).
2. **Unchecked money creation.** Nothing in the schema ties the two `UPDATE` statements together. A bug in a new promo-credit handler that runs one `UPDATE` without its counterparty silently alters the total supply of customer funds.
3. **Hot-row contention without audit legs.** Separating immutable event recording from balance caching lets us enforce conservation of money at the database kernel level while keeping point balance reads $O(1)$.

8.3.1 Conservation of Value and `DEFERRABLE INITIALLY DEFERRED` Triggers

Luca Pacioli codified the cure in Venice in 1494: **double-entry bookkeeping**. Every financial event is recorded as an immutable `journal_entry` header accompanied by two or more immutable `postings` (legs) in the same currency whose signed minor-unit amounts sum to zero:

$$\forall\, e \in \text{journal_entries}: \qquad |P(e)| \geq 2 \quad \text{and} \quad \sum_{p \in P(e)} p.\texttt{signed_amount_minor} = 0.$$

When Alice sends $50.00 (5,000 cents) to Bob, we do not invent 5,000 cents in Bob's row; we append one leg of -5000 against Alice's liability account and one leg of $+5000$ against Bob's liability account under the same `entry_id`. If a 1% platform fee applies, the same entry has three legs: -5000 (Alice), $+4950$ (Bob), and $+50$ (Platform Fee Revenue), summing cleanly to $-5000 + 4950 + 50 = 0$.

How do we force PostgreSQL itself to reject any transaction that violates this zero-sum balance law? A table-level `CHECK` constraint can inspect only the columns of a *single* row. And an ordinary `AFTER INSERT` trigger fails immediately: the moment you insert Alice's -5000 leg (before inserting Bob's $+5000$ leg), the running sum for `entry_id` is $-5000 \neq 0$!

PostgreSQL solves multi-row invariants across statement boundaries with a `CONSTRAINT TRIGGER` declared `DEFERRABLE INITIALLY DEFERRED`. Instead of firing mid-transaction when only half the legs exist, PostgreSQL queues a lightweight trigger event in backend memory and executes the verification function *when the client issues `COMMIT`*, right before flushing the commit record to WAL (Figure 8.2). Let us write out the complete ledger schema, including an immutability trigger on `postings` and a materialized `cached_balance_minor` on `accounts` protected against negative overdrafts:

```
CREATE TABLE accounts (
    account_id           UUID PRIMARY KEY DEFAULT uuidv7(),
    currency             CHAR(3) NOT NULL CHECK (currency ~ '^[A-Z]{3}$'),
    cached_balance_minor BIGINT NOT NULL DEFAULT 0,
    allow_overdraft      BOOLEAN NOT NULL DEFAULT FALSE,
    created_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT non_negative_unless_overdraft
```

```
        CHECK (allow_overdraft OR cached_balance_minor >= 0)
);

CREATE TABLE journal_entries (
    entry_id        UUID PRIMARY KEY DEFAULT uuidv7(),
    client_id       UUID NOT NULL,
    idempotency_key TEXT NOT NULL CHECK (length(idempotency_key) <= 128),
    currency        CHAR(3) NOT NULL CHECK (currency ~ '^[A-Z]{3}$'),
    description     TEXT NOT NULL,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT uq_client_idempotency UNIQUE (client_id, idempotency_key)
);

CREATE TABLE postings (
    posting_id          UUID PRIMARY KEY DEFAULT uuidv7(),
    entry_id            UUID NOT NULL REFERENCES journal_entries(entry_id),
    account_id          UUID NOT NULL REFERENCES accounts(account_id),
    signed_amount_minor BIGINT NOT NULL CHECK (signed_amount_minor <> 0),
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_postings_entry   ON postings (entry_id);
CREATE INDEX idx_postings_account ON postings (account_id, created_at DESC);

CREATE OR REPLACE FUNCTION forbid_posting_mutation()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    RAISE EXCEPTION 'postings are append-only; post a reversing entry';
END;
$$;

CREATE TRIGGER trg_postings_immutable
BEFORE UPDATE OR DELETE ON postings
FOR EACH ROW EXECUTE FUNCTION forbid_posting_mutation();

CREATE OR REPLACE FUNCTION verify_entry_zero_sum()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_entry_id UUID := COALESCE(NEW.entry_id, OLD.entry_id);
    v_legs     BIGINT;
    v_net_sum  BIGINT;
    v_bad_ccy  BIGINT;
BEGIN
    SELECT COUNT(*), COALESCE(SUM(p.signed_amount_minor), 0),
           COUNT(*) FILTER (WHERE a.currency <> je.currency)
      INTO v_legs, v_net_sum, v_bad_ccy
      FROM journal_entries je
      LEFT JOIN postings p ON p.entry_id = je.entry_id
      LEFT JOIN accounts a ON a.account_id = p.account_id
     WHERE je.entry_id = v_entry_id;

    IF v_legs < 2 OR v_net_sum <> 0 OR v_bad_ccy > 0 THEN
        RAISE EXCEPTION USING
            ERRCODE = '23514',
```

```
            MESSAGE = 'unbalanced entry ' || v_entry_id
                   || ': legs=' || v_legs || ', sum=' || v_net_sum
                   || ', ccy_mismatch=' || v_bad_ccy;
    END IF;
    RETURN NULL;
END;
$$;

CREATE CONSTRAINT TRIGGER trg_verify_entry_zero_sum
AFTER INSERT OR UPDATE OR DELETE ON postings
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION verify_entry_zero_sum();
```

Common trap

`verify_entry_zero_sum` checks *three* things together: `v_legs < 2`, `v_net_sum <> 0`, and `v_bad_ccy > 0`. Without `LEFT JOIN` and `v_legs < 2`, an entry whose postings were somehow deleted (if someone disabled the immutability trigger during maintenance) would return `SUM(...) = NULL`, which naive `IF v_net_sum <> 0` evaluates to `UNKNOWN` and lets slip past. And without checking `a.currency <> je.currency`, a buggy handler could debit 5000 JPY from a yen account and credit 5000 USD cents to a dollar account while still summing to zero numerically. Finally, attach a second deferred check on `journal_entries` (or always insert postings in the same helper) so an empty header with zero postings cannot commit alone.

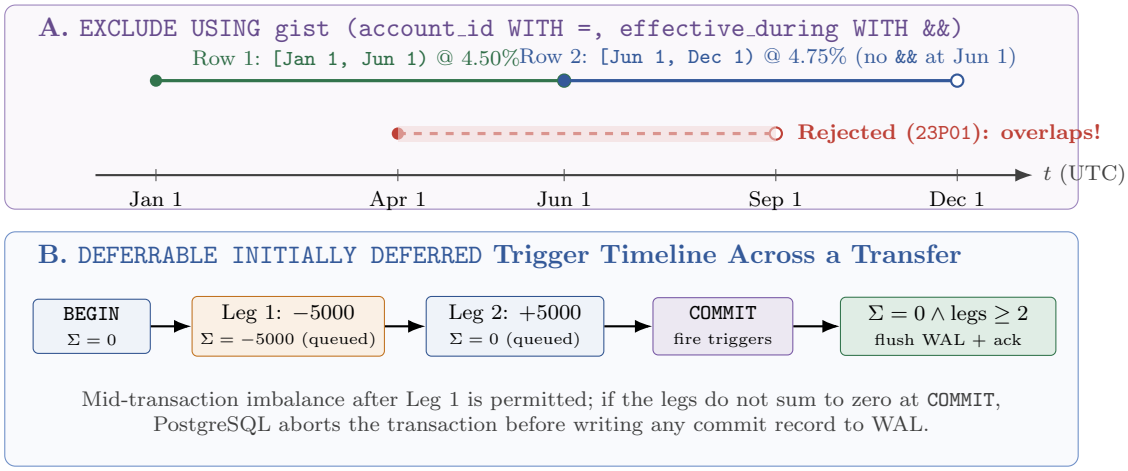


Figure 8.2: Two declarative PostgreSQL invariants that replace fragile application-side checks: (A) half-open `TSTZRANGE` intervals enforced by `EXCLUDE USING gist`, and (B) commit-time conservation verification via a deferred constraint trigger.

8.3.2 Deadlock-Free Transfers in Rust (`Axum` + `SQLx`)

Now let us wire our schema to a Rust `Axum` service using `SQLx`. Two subtle concurrency failures await careless transfer code even when the schema is correct:

1. **Opposite-order deadlocks (Chapter 6).** Suppose Task 1 transfers $10 from Alice to Bob while Task 2 simultaneously transfers $5 from Bob to Alice. If each handler locks `from_account`

first and `to_account` second, Task 1 holds Alice's row lock and waits on Bob, while Task 2 holds Bob's row lock and waits on Alice until PostgreSQL's `deadlock_timeout` kills one transaction. Sorting the account UUIDs in canonical ascending order (`ORDER BY account_id`) before acquiring locks makes deadlocks across account pairs mathematically impossible, because a strict total order has no cycles.

2. **Over-strong row locks blocking foreign keys.** Recall from Chapter 6 that `INSERT INTO postings` automatically acquires a `FOR KEY SHARE` tuple lock on the referenced `accounts` row to make sure nobody deletes that account mid-transaction. If you lock the account rows with `SELECT ... FOR UPDATE`, PostgreSQL assumes you might change the primary key `account_id` itself, which conflicts with `FOR KEY SHARE`. Instead, use `FOR NO KEY UPDATE`: it still serializes concurrent `UPDATE accounts SET cached_balance_minor = ...` mutations on the same account, while remaining compatible with `FOR KEY SHARE` foreign-key checks.

Listing 8.1 puts every piece together: input validation, atomic idempotency deduplication, canonical `FOR NO KEY UPDATE` lock ordering, double-entry leg insertion, cached balance updates, and deferred-trigger verification on `tx.commit().await`.

Listing 8.1: Idempotent, deadlock-free double-entry transfer in Rust with `SQLx`.

```rust
use sqlx::{PgPool, Postgres, Transaction};
use uuid::Uuid;

#[derive(Debug)]
pub struct TransferCmd {
    pub client_id: Uuid,
    pub idempotency_key: String,
    pub from_account: Uuid,
    pub to_account: Uuid,
    pub currency: String,
    pub amount_minor: i64,
    pub description: String,
}

#[derive(Debug, thiserror::Error)]
pub enum LedgerError {
    #[error("invalid transfer parameters")]
    InvalidInput,
    #[error("account missing or currency mismatch")]
    AccountMismatch,
    #[error("insufficient funds")]
    InsufficientFunds,
    #[error("database error: {0}")]
    Db(#[from] sqlx::Error),
}

#[derive(sqlx::FromRow)]
struct LockedAccount {
    account_id: Uuid,
    currency: String,
    cached_balance_minor: i64,
    allow_overdraft: bool,
}

```

```rust
pub async fn execute_transfer(
    pool: &PgPool,
    cmd: &TransferCmd,
) -> Result<Uuid, LedgerError> {
    if cmd.amount_minor <= 0 || cmd.from_account == cmd.to_account {
        return Err(LedgerError::InvalidInput);
    }

    let mut tx: Transaction<'_, Postgres> = pool.begin().await?;

    // 1. Atomic idempotency claim via UNIQUE (client_id, idempotency_key).
    let claimed: Option<Uuid> = sqlx::query_scalar(
        "INSERT INTO journal_entries \
            (client_id, idempotency_key, currency, description) \
         VALUES ($1, $2, $3, $4) \
         ON CONFLICT (client_id, idempotency_key) DO NOTHING \
         RETURNING entry_id",
    )
    .bind(cmd.client_id)
    .bind(&cmd.idempotency_key)
    .bind(&cmd.currency)
    .bind(&cmd.description)
    .fetch_optional(&mut *tx)
    .await?;

    let Some(entry_id) = claimed else {
        // Retry detected: return the committed entry_id cleanly.
        let existing: Uuid = sqlx::query_scalar(
            "SELECT entry_id FROM journal_entries \
             WHERE client_id = $1 AND idempotency_key = $2",
        )
        .bind(cmd.client_id)
        .bind(&cmd.idempotency_key)
        .fetch_one(&mut *tx)
        .await?;
        tx.rollback().await?;
        return Ok(existing);
    };

    // 2. Canonical lock order prevents A->B vs B->A deadlocks.
    let mut ids = [cmd.from_account, cmd.to_account];
    ids.sort_unstable();

    let locked: Vec<LockedAccount> = sqlx::query_as(
        "SELECT account_id, currency, cached_balance_minor, \
                allow_overdraft \
         FROM accounts \
         WHERE account_id = ANY($1) \
         ORDER BY account_id \
         FOR NO KEY UPDATE",
    )
    .bind(&ids[..])
    .fetch_all(&mut *tx)
    .await?;
```

```
89
90      if locked.len() != 2
91          || locked.iter().any(|a| a.currency.trim() != cmd.currency)
92      {
93          return Err(LedgerError::AccountMismatch);
94      }
95
96      let sender = locked
97          .iter()
98          .find(|a| a.account_id == cmd.from_account)
99          .ok_or(LedgerError::AccountMismatch)?;
100
101     if !sender.allow_overdraft
102         && sender.cached_balance_minor < cmd.amount_minor
103     {
104         return Err(LedgerError::InsufficientFunds);
105     }
106
107     // 3. Append both immutable postings in a single multi-row INSERT.
108     sqlx::query(
109         "INSERT INTO postings \
110             (entry_id, account_id, signed_amount_minor) \
111          VALUES ($1, $2, $3), ($1, $4, $5)",
112     )
113     .bind(entry_id)
114     .bind(cmd.from_account)
115     .bind(-cmd.amount_minor)
116     .bind(cmd.to_account)
117     .bind(cmd.amount_minor)
118     .execute(&mut *tx)
119     .await?;
120
121     // 4. Update cached balances while holding FOR NO KEY UPDATE locks.
122     sqlx::query(
123         "UPDATE accounts \
124          SET cached_balance_minor = cached_balance_minor + CASE \
125              WHEN account_id = $1 THEN -$3 \
126              WHEN account_id = $2 THEN  $3 \
127          END \
128          WHERE account_id IN ($1, $2)",
129     )
130     .bind(cmd.from_account)
131     .bind(cmd.to_account)
132     .bind(cmd.amount_minor)
133     .execute(&mut *tx)
134     .await?;
135
136     // 5. Commit fires trg_verify_entry_zero_sum inside PostgreSQL!
137     tx.commit().await?;
138     Ok(entry_id)
139 }
```

8.4 Declarative Partitioning for Billion-Row Ledgers

Imagine our ledger sustains 150 transfers (300 postings) per second around the clock: $300 \times 86{,}400 \times 365 \approx 9.5$ billion new `postings` rows a year. At roughly 100 bytes per row (tuple header, line pointer, and five columns), that is close to a terabyte of heap pages a year before any index. Keeping years of that inside a single physical relation causes three operational failures:

1. **Cold secondary indexes.** Even with `UUIDv7` keeping primary key inserts localized, secondary indexes such as `(account_id, created_at DESC)` span years of history across hundreds of gigabytes, evicting hot pages from `shared_buffers`.
2. **Multi-hour `VACUUM` freeze passes.** As we learned in Chapter 5, even an append-only table that never runs `UPDATE` or `DELETE` still requires periodic `autovacuum` freeze scans to mark heap pages all-frozen in the Visibility Map before the 32-bit transaction ID horizon wraps around. Scanning a half-terabyte single heap file saturates storage I/O for hours.
3. **Retention archival via `DELETE` bloats disk.** When compliance policy moves postings older than seven years to cold object storage, running `DELETE FROM postings WHERE created_at < '2019-11-01'` on 250 million rows does *not* return bytes to the filesystem: it marks 250 million dead tuples, writes tens of gigabytes of WAL, and leaves fragmented pages behind.

PostgreSQL's **declarative range partitioning** solves all three problems by presenting one logical parent table (`postings`) backed by independent monthly physical child tables (`postings_2026_10`, `postings_2026_11`, and so on), each owning its own heap files, Visibility Map, and local B-tree indexes:

```
CREATE TABLE postings (
    posting_id          UUID NOT NULL DEFAULT uuidv7(),
    entry_id            UUID NOT NULL,
    account_id          UUID NOT NULL,
    signed_amount_minor BIGINT NOT NULL CHECK (signed_amount_minor <> 0),
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (posting_id, created_at)
) PARTITION BY RANGE (created_at);

CREATE TABLE postings_2026_11 PARTITION OF postings
    FOR VALUES FROM ('2026-11-01 00:00:00+00')
                TO ('2026-12-01 00:00:00+00');

CREATE TABLE postings_2026_12 PARTITION OF postings
    FOR VALUES FROM ('2026-12-01 00:00:00+00')
                TO ('2027-01-01 00:00:00+00');

CREATE INDEX idx_postings_acct_time
    ON postings (account_id, created_at DESC);
```

Look at what changes inside the engine:

- **Plan-time and execution-time partition pruning.** When a customer loads their November statement (`WHERE account_id = $1` with November `created_at` bounds), the query planner (Chapter 7) compares those predicates against each child's `FOR VALUES` range and prunes 83 out of 84 monthly partitions before touching a single index page. Even when bounds come

from runtime parameters or join keys, PostgreSQL's `Append` executor prunes non-matching partitions dynamically at execution startup. Only the current month's B-trees need to stay warm in `shared_buffers`.

- **Localized `autovacuum`.** Once November ends, all pages in `postings_2026_11` are frozen once in the Visibility Map and never scanned again by `VACUUM`.
- **Instantaneous zero-bloat archival.** Expiring the oldest month never runs `DELETE`:

```
ALTER TABLE postings DETACH PARTITION postings_2019_10 CONCURRENTLY;
-- Export standalone table postings_2019_10 to Parquet/archive, then:
DROP TABLE postings_2019_10;
```

`DETACH PARTITION CONCURRENTLY` updates system catalogs without blocking concurrent reads or inserts on live months, and `DROP TABLE` unlinks the underlying 1 GB OS segment files in milliseconds with zero dead tuples.

Common trap

Notice that our partitioned table declared `PRIMARY KEY (posting_id, created_at)` rather than `PRIMARY KEY (posting_id)`. PostgreSQL rejects any `UNIQUE` or `PRIMARY KEY` constraint on a partitioned table unless *every column of the partition key appears inside the constraint.* From Chapter 3 you can see why: each monthly partition owns a separate local B-tree, and enforcing global uniqueness on `posting_id` alone would force every single `INSERT` to probe all 84 monthly B-trees. Keep `journal_entries` unpartitioned (or partitioned with `created_at` included in the idempotency window) if you need a single global B-tree on `(client_id, idempotency_key)`.

8.5 High Availability: Physical Streaming and Logical Replication

A RAID array or cloud block volume protects one server against a single drive failure, not against a kernel panic, motherboard fault, or availability-zone power loss. Yet simply standing up a standby server is not enough: depending on two configuration parameters, an automatic failover can still silently erase the last few milliseconds of committed customer deposits, or a read-balanced API endpoint can report a stale balance right after a transfer succeeds.

8.5.1 Physical Streaming WAL Replication and Standby Conflicts

Recall from Chapter 4 that PostgreSQL's Write-Ahead Log records physical changes keyed by relation file node, 8 KB block number, and page offset, ordered globally by a monotonically increasing 64-bit Log Sequence Number (`XLogRecPtr`, or LSN). **Physical streaming replication** turns that crash-recovery log into a continuous network stream:

1. On the primary, a `walsender` backend process reads freshly generated WAL bytes straight from WAL buffers or `pg_wal` and streams them over TCP to each connected standby. Attaching a **physical replication slot** records how far each standby has consumed so the primary never recycles a 16 MB WAL segment a disconnected replica still needs (always alert on `pg_replication_slots` lag: an orphaned slot on a dead standby will retain WAL indefinitely until the primary's disk fills 100%).

2. On the standby, a `walreceiver` process writes incoming WAL bytes to the standby's own `pg_wal` directory and `fsync`s them, while the standby's `startup` process continuously replays those records onto the standby's 8 KB heap and index pages.

Because physical replication ships exact page modifications, a physical standby is a **byte-for-byte clone** of the primary: identical 8 KB slotted pages, identical `xmin`/`xmax` transaction IDs, and identical B-tree leaves across the entire cluster. With `hot_standby = on`, read-only queries can execute directly against the standby while replay runs underneath them.

That concurrency creates a subtle clash with Chapter 5's `VACUUM`. Suppose a reporting query runs for forty seconds on the hot standby using snapshot `xmin = 50000`. Meanwhile on the primary, no old query is running, so `VACUUM` reclaims dead tuples from transaction `50010` and emits a `HEAP2_PRUNE` WAL record. When the standby's `startup` process reaches that WAL record, replaying it immediately would delete tuples still visible to the standby's running query. PostgreSQL resolves this **recovery conflict** via `max_standby_streaming_delay` (default `30s`): the standby pauses WAL replay up to that threshold to let the read query finish, and if the query is still running when the timer expires, cancels the read query with `ERROR: canceling statement due to conflict with recovery`. Enabling `hot_standby_feedback = on` tells the standby to advertise its oldest active `xmin` back to the primary's `ProcArray`, preventing query cancellations at the cost of holding back `VACUUM` cleanup on the primary whenever a replica runs multi-minute analytics.

8.5.2 The `synchronous_commit` Durability and Visibility Ladder

When a Rust handler calls `tx.commit().await`, how far through the replication pipeline must the commit LSN travel before the primary returns `CommandComplete` to the client? PostgreSQL lets you choose the exact point on the latency-versus-durability frontier, either globally in `postgresql.conf` or for one transaction at a time with `SET LOCAL`. Two settings work as a pair: `synchronous_commit` says how far the commit must travel, and `synchronous_standby_names` says which standbys count (for example, `ANY 1 (az2_standby, az3_standby)` across three availability zones).

Each standby continuously sends feedback messages reporting three LSN watermarks, all visible in the `pg_stat_replication` view:

- `write_lsn`: WAL copied into the standby's OS page cache;
- `flush_lsn`: WAL `fsync`ed to the standby's NVMe drive;
- `replay_lsn`: WAL applied to the standby's buffer pages, and therefore visible to standby snapshots.

Figure 8.3 and Table 8.1 compare all five levels.

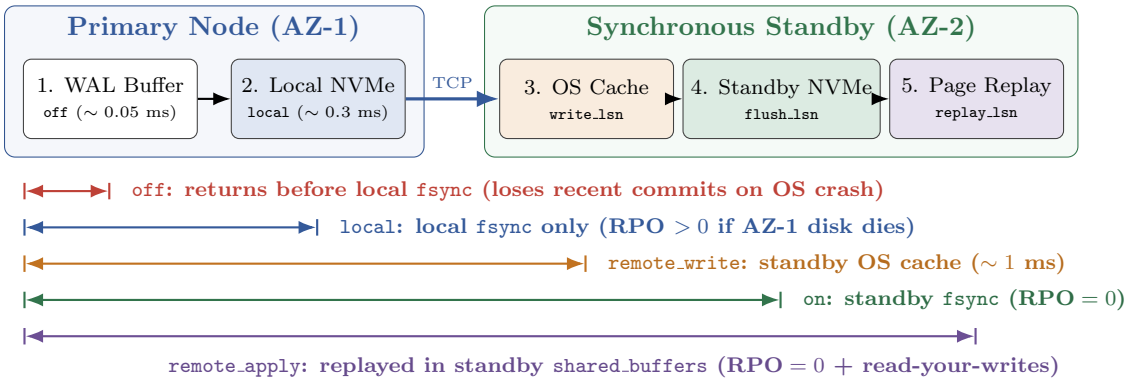


Figure 8.3: The five commit-wait watermarks of `synchronous_commit` when `synchronous_standby_names` is active.

Mode	Wait Condition	Commit Latency	AZ-1 Loss RPO / Standby Reads
`off`	WAL buffer copy	∼ 0.05 ms	Loses ∼ 200–600 ms on reboot
`local`	Primary `fsync`	∼ 0.3 ms	RPO > 0 on AZ loss; stale reads
`remote_write`	Standby `write()`	∼ 1.0 ms	RPO = 0 unless dual crash; brief lag
`on`	Standby `fsync`	∼ 1.8 ms	**RPO = 0 on AZ loss**; brief lag
`remote_apply`	Standby WAL replay	∼ 2.4 ms	**RPO = 0 + causal standby reads**

Table 8.1: Durability (Recovery Point Objective, RPO) and intra-region cross-AZ latency across `synchronous_commit` modes.

Two production lessons follow from Table 8.1. First, if you leave `synchronous_standby_names = ''` (empty, the PostgreSQL default), `synchronous_commit = on` silently degrades to `local`: PostgreSQL flushes only the primary's disk and ships WAL asynchronously. Second, in a three-AZ cluster using `ANY 1 (az2_standby, az3_standby)`, setting `synchronous_commit = on` gives zero-data-loss (`RPO = 0`) survival of an entire availability zone while tolerating one slow or rebooting standby without halting writes; handlers that immediately redirect a customer to a read-replica endpoint can elevate just that transaction via `SET LOCAL synchronous_commit = 'remote_apply'`.

8.5.3 Logical Replication for Zero-Downtime Upgrades and Event Streaming

Physical replication requires the primary and standby to share the *exact* same major PostgreSQL version and CPU architecture because 8 KB page layouts and catalog formats evolve across major releases. How, then, do you upgrade a live financial ledger from PostgreSQL 16 to PostgreSQL 18 without taking a maintenance outage, or stream committed `postings` rows into an analytics warehouse or transactional outbox consumer?

Setting `wal_level = logical` enables **logical decoding**. Instead of shipping raw 8 KB block deltas, a logical `walsender` feeds WAL into the **Reorder Buffer**, which reassembles interleaved physical WAL records per `TransactionId` (spilling large transactions to disk if needed), discards rolled-back transactions, and invokes an output plugin (`pgoutput`) the moment a transaction commits to emit structured tuple-level `INSERT`, `UPDATE`, and `DELETE` messages:

```
-- On the primary (or PG 16 source cluster):
```

```
CREATE PUBLICATION ledger_pub
    FOR TABLE accounts, journal_entries, postings
    WITH (publish_via_partition_root = true);

-- On the PG 18 target cluster:
CREATE SUBSCRIPTION ledger_sub
    CONNECTION 'host=pg16-primary dbname=ledger user=repl_role'
    PUBLICATION ledger_pub;
```

Notice `publish_via_partition_root = true`: logical subscribers receive changes tagged under the parent table `postings`, allowing the target cluster to use a completely different partitioning scheme or index set. Keep two operational constraints in mind during logical upgrades: DDL statements (`ALTER TABLE`) are not replicated automatically, and `SEQUENCE` counters are not synchronized in real time—another reason time-ordered `UUIDv7` primary keys simplify zero-downtime cutovers.

8.6 Connection Pooling at Scale: Process Architecture and PgBouncer

Our Rust `Axum` service runs on Tokio's work-stealing async runtime, where a single pod handles ten thousand concurrent HTTP sockets in a few dozen megabytes of RAM. Suppose you autoscale to 50 Kubernetes pods, set each pod's `sqlx::PgPool` `max_connections` to 100, and raise PostgreSQL's `max_connections = 5500` on a 32-core database server so all 5,000 pooled sockets can stay open directly against PostgreSQL. Everything looks idle in staging. Then Monday morning traffic arrives: database CPU spikes to 100% inside kernel spinlocks, free RAM vanishes, and P99 transfer latency climbs from 3 ms to four seconds.

8.6.1 Why Five Thousand Backend Processes Melt PostgreSQL

Unlike Tokio's cooperative async tasks, PostgreSQL uses a classic **one-OS-process-per-connection** architecture: when a client opens a TCP socket, the `postmaster` supervisor forks a dedicated `postgres` backend process. Holding 5,000 backend processes open degrades throughput through three physical bottlenecks:

1. **Per-backend private memory.** While `shared_buffers` is shared across all processes, every `postgres` backend maintains its own private relation and system-catalog caches (`relcache` and `catcache`, which grow substantially on partitioned schemas), cached execution plans, and per-node `work_mem` allocations (Chapter 7). At a realistic 10–16 MB RSS per active backend, 5,000 processes consume 50–80 GB of RAM before caching a single heap page.

2. `ProcArray` **MVCC snapshot scans.** Recall from Chapter 5 that every statement under `READ COMMITTED` calls `GetSnapshotData()`, iterating across the shared-memory `ProcArray` of connected backends to record in-flight transaction IDs. Scanning 5,000 backend slots hundreds of thousands of times per second causes heavy CPU cache-line bouncing around `ProcArrayLock` whenever commits update those same slots.

3. **OS scheduler preemption inside locks.** When 600 processes become runnable simultaneously on 32 CPU cores, the Linux scheduler preempts backends every few milliseconds, flushing

TLBs and L1/L2 caches, and stalling dozens of peers whenever a backend is context-switched off CPU *while holding a lightweight buffer lock or spinlock.*

A common starting point, which you should confirm by benchmarking your own workload, is that peak transactional throughput on a server with N_{cores} CPU cores and NVMe storage occurs with roughly $1.5 \times N_{\text{cores}}$ to $3 \times N_{\text{cores}}$ *active* backends: about 48 **to** 96 **backends on a 32-core primary**. Beyond that, extra connections add contention without adding throughput.

8.6.2 `PgBouncer` Transaction Multiplexing and the Prepared Statement Trap

How do 50 Rust pods holding 2,500 client connections share just 48 physical `postgres` processes? You place `PgBouncer`—a lightweight, event-driven C proxy that speaks the PostgreSQL v3 wire protocol at roughly 2 KB of memory per client socket—in front of the database (Figure 8.4). `PgBouncer` offers three pooling modes:

- **`session` mode** pins a physical `postgres` backend to a client socket from connect until disconnect. Because `sqlx::PgPool` holds long-lived TCP connections open, `session` mode achieves *zero* multiplexing (2,500 client sockets still demand 2,500 backends).
- **`transaction` mode** (the production standard) leases a physical backend to a client *only from `BEGIN` (or a standalone query) until `COMMIT` or `ROLLBACK`*. The instant `COMMIT` completes, `PgBouncer` returns that backend to the warm server pool for the next waiting request. While an `Axum` handler parses JSON, checks cryptographic signatures, or streams HTTP responses, it occupies *zero* PostgreSQL processes.
- **`statement` mode** reclaims the backend after every single SQL statement and immediately aborts any multi-statement `BEGIN ... COMMIT` block—unusable for double-entry transactions.

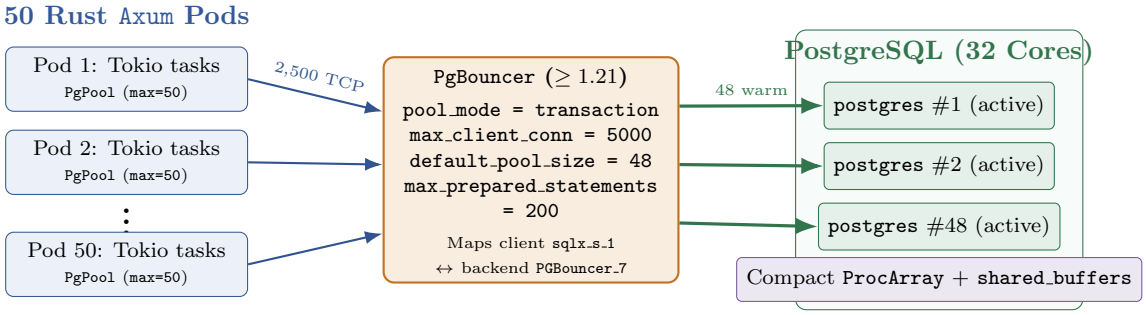


Figure 8.4: Multiplexing 2,500 `SQLx` client connections across 48 warm `postgres` backends via `PgBouncer` in `transaction` mode, keeping `ProcArray` compact and CPU caches hot.

Switching `pool_mode = transaction` without understanding the PostgreSQL wire protocol triggers a classic production outage within seconds: `ERROR: prepared statement "sqlx_s_1" does not exist` (or `already exists`). Here is the exact packet-level reason:

1. By default, `SQLx` uses PostgreSQL's extended query protocol and caches up to 100 named prepared statements per TCP socket. In Transaction A, Client 1 is assigned `postgres` Backend #4

and sends a `Parse` message creating statement `sqlx_s_1` inside Backend #4's private process memory, then commits.

2. One millisecond later, Client 1 starts Transaction B and `PgBouncer` routes it to Backend #19. Assuming its TCP socket still talks to the same process, `SQLx` skips `Parse` and sends a `Bind` packet naming `sqlx_s_1`. Backend #19 has never heard of `sqlx_s_1` and aborts the transaction.

The same backend-hopping mechanism breaks any state stored at *session* scope rather than *transaction* scope:

- Never run bare `SET synchronous_commit = 'remote_apply'` or `SET statement_timeout = '2s'` in transaction mode—the setting leaks onto the next unrelated request that lands on that backend. Always write `SET LOCAL ...`, which PostgreSQL automatically reverts at `COMMIT` or `ROLLBACK`.
- Never call session-scoped `pg_advisory_lock`, which leaves the lock held on a pooled backend after your handler returns; always use the transaction-scoped `pg_advisory_xact_lock` function instead.
- `LISTEN / NOTIFY` pub-sub channels require a dedicated direct or `session`-mode connection outside the transaction pool.

For prepared statements, you have two clean fixes. Best of all, **modern `PgBouncer` ($\geq$ 1.21)** supports protocol-level prepared-statement multiplexing when you set `max_prepared_statements = 200` in `pgbouncer.ini`: `PgBouncer` tracks each client's `Parse` text, rewrites client names (`sqlx_s_1`) into deduplicated server-side names (`PGBOUNCER_7`), and transparently prepares any missing statement on whichever physical backend leases the transaction—preserving Chapter 7's plan-cache speedups. When deploying behind older proxies or cloud poolers without protocol tracking, disable client-side named statement caching on `PgConnectOptions` so `SQLx` sends unnamed single-use statements:

```rust
use sqlx::postgres::{PgConnectOptions, PgPoolOptions};
use std::str::FromStr;
use std::time::Duration;

pub async fn build_ledger_pool(
    db_url: &str,
    pgbouncer_tracks_stmts: bool,
) -> Result<sqlx::PgPool, sqlx::Error> {
    let mut opts = PgConnectOptions::from_str(db_url)?;
    if !pgbouncer_tracks_stmts {
        // Fallback for stateless transaction poolers: unnamed statements.
        opts = opts.statement_cache_capacity(0);
    }

    PgPoolOptions::new()
        .max_connections(50)
        .acquire_timeout(Duration::from_secs(2))
        .idle_timeout(Duration::from_secs(300))
        .connect_with(opts)
        .await
}
```

Here is the whole path of one $50.00 transfer through the machinery of this book. An `Axum` task

borrows a pooled connection through PgBouncer and lands on one of 48 backends, so snapshot scans of ProcArray stay short (Chapter 5). It inserts rows with UUIDv7 keys, which append to the rightmost B-tree leaf already in shared_buffers (Chapters 2 and 3). It claims its idempotency key through a UNIQUE index, locks both accounts in ascending order with FOR NO KEY UPDATE (Chapter 6), and inserts two BIGINT postings into the current month's partition. At COMMIT, the deferred trigger checks that the legs sum to zero, and with synchronous_commit = on the commit record is flushed to WAL on two machines before the client hears success (Chapter 4).

8.7 Exercises

Exercise 8.1 (IEEE 754 Drift vs. Exact Ledger Representations)**.** Execute the following query in PostgreSQL:

```
SELECT SUM(val::float8) AS float_sum, SUM(val::numeric) AS num_sum
FROM generate_series(1, 1000000) AS g(i),
     LATERAL (VALUES (0.10), (0.20), (-0.30)) AS v(val);
```

Explain why float_sum is nonzero while num_sum is exact. Next, design the columns for an FX conversion table that converts JPY (scale 0), USD (scale 2), and BTC (scale 8) using BIGINT minor units for posted legs and NUMERIC(24, 12) for the exchange rate, specifying where rounding happens and which rule it uses so the converted journal entry still obeys conservation within each currency. (Note that PostgreSQL's round() on NUMERIC rounds halves away from zero, not to even; banker's rounding needs an explicit expression.)

Exercise 8.2 (The Autumn Daylight-Saving Collision)**.** In a psql session with TimeZone set to 'America/New_York', compare storing '2026-11-01 01:30:00-04' and '2026-11-01 01:30:00-05' in a TIMESTAMP column versus a TIMESTAMPTZ column. Compute the exact difference between the two 64-bit integer values stored on the 8 KB heap page for TIMESTAMPTZ (in microseconds since 2000-01-01 00:00:00 UTC), and show how TIMESTAMP corrupts an hourly TWAP (time-weighted average price) query across that hour.

Exercise 8.3 (Quantifying UUIDv4 WAL Amplification vs. UUIDv7)**.** Suppose a primary-key B-tree on postings has 8,000,000 leaf pages (≈ 64 GB). Immediately after a CHECKPOINT completes (with full_page_writes = on), a burst inserts 40,000 new rows within two seconds. Estimate how many distinct 8 KB leaf pages are dirtied—and how many megabytes of 8 KB Full Page Images are written to WAL—under random UUIDv4 versus time-ordered UUIDv7 (assuming $\sim$ 120 index tuples fit per new rightmost leaf page).

Exercise 8.4 (Non-Overlapping Credit Limits with EXCLUDE USING gist)**.** Write the DDL for a credit_limits table where each account_id has an overdraft limit valid over an effective_during column of type TSTZRANGE, ensuring that *active* schedules (status = 'active') for the same account can never overlap in time while allowing historical status = 'superseded' rows to remain in the table for audit purposes. (Hint: inspect PostgreSQL's partial exclusion constraint syntax EXCLUDE USING gist (...) WHERE (...).) Test what happens at the boundary when two intervals use closed bounds '[]' instead of half-open '[)'.

Exercise 8.5 (Auditing and Hardening the Double-Entry Deferred Trigger)**.** Suppose a transaction inserts four postings for the same entry_id (sender debit, recipient credit, platform fee, partner rebate). How many times does a FOR EACH ROW deferred constraint trigger fire at COMMIT? Show

how to deduplicate redundant checks within the same transaction (for instance, skipping if a per-transaction temporary table or session variable already verified `v_entry_id` at the final command counter), and write the companion deferred constraint trigger on `journal_entries` that rejects a header committed with zero rows in `postings`.

Exercise 8.6 (Deadlock Cycles and `FOR NO KEY UPDATE` Internals)**.** Construct a two-session `psql` interleaving (T_1 transferring $A \to B$ and T_2 transferring $B \to A$) that deterministically triggers `ERROR: deadlock detected` when canonical UUID sorting is omitted from Listing 8.1. Then, using Chapter 6's tuple header lock modes, demonstrate in two `psql` sessions why locking `accounts` with `FOR UPDATE` blocks a concurrent `INSERT INTO postings` foreign-key check, whereas `FOR NO KEY UPDATE` lets the foreign-key check proceed until the cached-balance `UPDATE` step.

Exercise 8.7 (Partitioning and Retention on a Multi-Billion-Row Table)**.** Write a SQL migration or PL/pgSQL helper procedure `ensure_monthly_partition` taking `target_month DATE` that safely creates next month's `postings_YYYY_MM` range partition ahead of time. Explain what happens to live `INSERT` traffic at `2027-01-01 00:00:00Z` if the cron job fails to create January's partition and no `DEFAULT` partition exists—and conversely, why adding a bloated `DEFAULT` partition can turn a future `ATTACH PARTITION` into a long table-scan lock.

Exercise 8.8 (Sizing `synchronous_commit` Across Three Availability Zones)**.** A ledger cluster spans `az1` (primary), `az2` (standby), and `az3` (standby) with local NVMe `fsync` latency 0.25 ms, round-trip network latency 1.1 ms (`az1`–`az2`) and 1.6 ms (`az1`–`az3`), and standby WAL replay overhead 0.35 ms. With `synchronous_standby_names = 'ANY 1 (az2, az3)'`, calculate the expected commit latency under normal operation and during an `az2` outage for `local`, `remote_write`, `on`, and `remote_apply`. Which mode should `POST /v1/transfers` use if `GET /v1/accounts/:id/balance` is load-balanced across `az2` and `az3`?

Exercise 8.9 (Fixing a `PgBouncer` Transaction-Mode Rollout)**.** An engineering team flips `PgBouncer` from `pool_mode = session` to `pool_mode = transaction` in front of 40 Rust `Axum` pods. Within two minutes, monitoring lights up with three errors: (a) `prepared statement ''sqlx_s_4'' does not exist`, (b) fast transfers mysteriously timing out at 150 ms after an admin endpoint ran `SET statement_timeout = '150ms'`, and (c) a nightly settlement job holding `pg_advisory_lock(99)` never releasing the lock after commit. Trace the exact backend-multiplexing cause of all three bugs and write the complete configuration and SQL/Rust fixes.